

Advanced Data Management



Federico Segala

Anno Accademico: 2025-2026

Appunti del corso di Advanced Data Management
prof. Claudio Silvestri

Sommario

1. Introduzione	6
1.1. Proprietà di una Base di Dati	6
1.2. Componenti di un Database	7
1.3. Database Management System Relazionali	9
1.3.1. Progettazione di una Base di Dati	11
1.3.2. Query Relazionali	12
1.3.3. Transazioni e Gestione della Concorrenza	13
1.3.4. Problematiche del Modello Relazionale	14
1.4. Nuovi Requisiti	14
2. Key-Value Stores e Document Databases	16
2.1. Key-Value Stores	16
2.1.1. Map-Reduce	16
2.2. Document Database	17
2.2.1. JavaScript Object Notation	18
2.2.2. MongoDB	19
2.2.3. Caso d'uso: Location-Based Application	23
2.2.4. Operazioni di Aggregazione in MongoDB	28
2.2.5. Todo Maybe: forse torneremo qui per parlare di sharded deployments ...	35
3. Column Stores	36
3.1. Architettura di un R-DBMS	36
3.2. Column Stores	38
3.2.1. Vantaggi e Svantaggi dei Column Stores	39
3.2.2. Compressione delle colonne	39
3.2.3. Alcuni Prodotti Commerciali	44
4. Extensible Record Stores	45
4.1. Modello Logico dei Dati	45
4.2. Storage Fisico dei Dati	46
4.2.1. Flusso delle operazioni di scrittura	46
4.2.2. Modifica e Cancellazione dei Dati	47
4.2.3. Flusso di Lettura dei Dati	48
4.2.4. Ulteriori Accorgimenti	49
4.3. Alcune Implementazioni	53
5. Graph Databases	54
5.1. Teoria dei Grafi	54
5.1.1. Navigazione in un Grafo	55
5.1.2. Problemi sui Grafi	56
5.2. Basi di Dati a Grafo	56
5.2.1. Modello dei Dati	57
5.2.2. Memorizzazione di un Grafo	59
5.2.3. Graph Database Management Systems & API	64
5.3. Neo4j	66
5.3.1. Property Graph Model	66
5.3.2. Graph Querying: Qualche Esempio	66

5.3.3.	Introduzione a Cypher	68
5.3.4.	Indici e vincoli	70
5.3.5.	Scrittura di Query in Cypher: Utilizzi Avanzati	71
5.3.6.	Data Ingestion in Neo4j	73
6.	Gestione di Dati Distribuiti	75
6.1.	Concetti di Base	75
6.2.	Guasti nei Sistemi Distribuiti	76
6.3.	Protocolli Epidemici	77
6.3.1.	Background	77
6.3.2.	Varianti di Protocolli Epidemici	78
6.3.3.	Hash Trees	78
6.4.	Frammentazione («Sharding»)	79
6.4.1.	Allocazione	80
6.5.	Replicazione dei Dati	83
6.5.1.	Replicazione Master-Slave	83
6.5.2.	Replicazione Multi-record	84
6.5.3.	Replicazione Multi-Master	84
6.5.4.	Guasti e Recovery	85
6.6.	Gestione della Concorrenza	86
6.7.	Proprietà dei Database Distribuiti	87
6.7.1.	Congettura di Brewer	88
7.	Architetture per Basi di Dati	89
7.1.	Persistent Memory Manager	90
7.1.1.	Geometria di un Hard Disk	90
7.1.2.	Lettura da Disco Magnetico	91
7.2.	Buffer Manager	92
7.2.1.	Bozza di Interfaccia del Buffer Manager	92
7.2.2.	Buffer Pool	93
7.2.3.	Tabella delle Pagine Residenti	94
7.2.4.	Struttura di una Pagina di Memoria	95
7.3.	Strutture per File Management	97
7.3.1.	Modello Seriale e Sequenziale	98
7.3.2.	Algoritmi di Ricerca e Costi	101
7.4.	Problemi di Ordinamento	103
8.	Organizzazioni per la Ricerca di Chiavi	106
8.1.	Hashing Statico	106
8.1.1.	Funzione di Hashing	107
8.1.2.	Gestione dell'overflow	108
8.1.3.	Loading Factor	109
8.1.4.	Costi dell'approccio	109
8.1.5.	Riorganizzazione della struttura	110
8.2.	Hashing Dinamico	110
8.2.1.	Hashing virtuale	110
8.2.2.	Hashing Lineare	112
8.3.	Strutture basate su B+ Tree	114

8.3.1.	Ricerca di una chiave	114
8.3.2.	Inserimento di una chiave	115
8.3.3.	Cancellazione di una chiave	117
8.3.4.	Profondità di un B+ Tree	117
8.4.	Indici	117
8.4.1.	Modalità di Costruzione di un Indice	118
8.4.2.	Costi degli Indici	119
8.5.	Indici su Attributi Non Chiave	121
8.5.1.	Indici a Lista Invertita	121
8.5.2.	Indici Bitmap	124
9.	Organizzazioni e Indici Multidimensionali	126
9.1.	Tipologie di Organizzazioni Multidimensionali	127
9.2.	B+ Tree Multidimensionali	129
9.2.1.	Vantaggi e Svantaggi	129
9.2.2.	Space Filling Curves	129
9.3.	Tecniche di Space Partitioning	133
9.3.1.	Point Region Quadtree	133
9.3.2.	KDB - Tree	134
9.4.	Alberi G - Partizionamento Non-Overlapping	135
9.4.1.	Codifica delle Partizioni - Creazione del Partition Tree	135
9.4.2.	Creazione del G Tree	136
9.4.3.	Ricerca di un Punto	136
9.4.4.	Ricerca di un Range Spaziale	137
9.4.5.	Inserimento di un Punto	137
9.4.6.	Cancellazione di un Punto	139
9.5.	Alberi R^* - Partizionamento con Overlapping	139
9.5.1.	Struttura degli Alberi R^*	139
9.5.2.	Ricerca di Regioni sovrapposte	140
9.5.3.	Inserimento nell'Albero	140
10.	Metodi di accesso e Operatori Fisici	143
10.1.	Gestione dei Metodi di Accesso	143
10.1.1.	Gestione della Base di Dati	143
10.1.2.	Operazioni sugli Heap File	144
10.1.3.	Operazioni sugli Indici	144
10.1.4.	Operatori dei Metodi di Accesso	145
10.1.5.	Esecuzione delle Query	146
10.1.6.	Da un Operatore Logico a un Piano di Accesso Fisico	148
10.2.	Operatori Fisici	149
10.2.1.	Physical Query Plan	149
10.2.2.	Algebra Relazionale Estesa	150
10.2.3.	Operatori Fisici per le relazioni	151
10.2.4.	Operatori di Proiezione e Deduplicazione	152
10.2.5.	Operatori di Selezione	153
10.2.6.	Operatori Fisici per il Raggruppamento	155
10.2.7.	Operatori Fisici per il JOIN	156

10.2.8. Operatori Fisici per le Operazioni su Set e Multi-Set	159
10.3. Ottimizzazione delle Query Relazionali	160
10.3.1. Analisi della Query	160
10.3.2. Generazione del Piano Fisico	163
11. Gestione di Transazione e Concorrenza	164
11.1. Gestore delle Transazioni	164
11.1.1. Requisiti di funzionamento	164
11.1.2. Ciclo di vita di una transazione	165
11.1.3. Compiti del modulo	166
11.1.4. Prevenzione in caso di Failure	168
11.1.5. Procedure per Undo e Redo	169
11.1.6. Algoritmo di Ripristino	172

1. Introduzione

Le basi di dati sono elementi fondamentali in molti aspetti della tecnologia quotidiana. Ogni giorno infatti, è prodotta, memorizzata e elaborata una **immensa quantità di dati**.

Un **database management system** che funzioni in maniera corretta è dunque cruciale per rendere queste attività il più semplici ed efficaci possibili. Questo capitolo si occupa di introdurre *principi e proprietà* che un database dovrebbe soddisfare.

1.1. Proprietà di una Base di Dati

Dal momento che lo storage di dati è cruciale, un database dovrebbe garantire le proprietà che andiamo di seguito a definire.

- **Gestione dei dati:** Una base di dati non si occupa solo del salvataggio, ma deve supportare operazioni che permettano recupero, ricerca, e aggiornamento dei dati. Per fare ciò spesso è necessario avere delle interfacce tramite le quali sia possibile comunicare con la base di dati. Un altro aspetto importante è quello del supporto alle transazioni, ossia insiemi di operazioni atomici, non interrompibili.
- **Scalabilità:** La quantità di dati processati è di solito enorme. Elaborare questa mole di dati è fattibile solamente **distribuendoli** in una rete e garantendo un alto livello di parallelismo. La necessità in questo caso è di *adattarsi al workload corrente* del sistema e allocare risorse di conseguenza.
- **Eterogeneità:** Nel momento in cui andiamo a memorizzare dati questi non sono tipicamente nella forma corretta per essere memorizzati in forma relazionale; I dati possono essere memorizzati in maniera **strutturata** ma non solo. Possono infatti essere **semi-strutturati**, altre forme tipiche sono strutture ad **albero** (XML) o a **grafo**, nel peggiore dei casi, si può avere un dato che è completamente non strutturato.
- **Efficienza:** La maggioranza delle applicazioni hanno bisogno di sistemi molto veloci in modo da riflettere nel minor tempo possibile i cambiamenti (real time applications).
- **Persistenza:** Lo scopo principale di una base di dati è quello di fornire storage a lungo termine dei dati. Ci sono delle eccezioni a questo, casi in cui solo parti dei dati necessitano permanenza a lungo termine, mentre altri sono più *volatili*; questo comportamento è chiamato **persistenza selettiva**.
- **Affidabilità:** Un buon sistema è tipicamente in grado di prevenire la perdita di dati o l'avvenimento di distorsioni degli stessi. In pratica ciò cui ci si concentra è l'**integrità** dei dati. Ciò avviene tipicamente tramite *ridondanza fisica e replicazione*.
- **Consistenza:** È importante che la base di dati garantisca che non siano presenti dati contraddittori o errati nel sistema. Ciò si ottiene tipicamente tramite chiavi primarie, integrità referenziale e aggiornamento automatico delle repliche dei dati.
- **Non Ridondanza:** La ridondanza fisica è cruciale per garantire affidabilità, la duplicazione di valori (*ridondanza logica*) è invece da evitare per quanto possibile. Ciò aumenta inutilmente il consumo di spazio e la possibilità di *anomalie*.

- **Supporto multi-utente:** Nei sistemi moderni è spesso richiesto il supporto all'accesso concorrente alle risorse da parte di più utenti o applicazioni.

Tutte queste caratteristiche ci consentono di formalizzare nel modo più completo possibile cosa sia una database management system (DBMS) come riporta Definizione 1.1°

Definizione 1.1 (Base di Dati)

Una base di dati è un sistema che consente di gestire grandi quantità di dati eterogenei, in maniera efficiente, persistente, affidabile, consistente e non ridondante. È inoltre in grado di supportare accesso concorrente da parte di più utenti.

Tipicamente è complicato che una base di dati supporti tutte le caratteristiche elencate di sopra; si rivela dunque fondamentale un'analisi dettagliata dei **requisiti** di ogni caso d'uso per rendere il più ponderata possibile la scelta del sistema da utilizzare.

1.2. Componenti di un Database

Il componente software che si fa carico di tutte le operazioni sulla base di dati è il **database management system** (DBMS). Figura 1.1 illustra come molti altri componenti nel sistema operativo vadano a interagire tra di loro e con questo importante elemento.

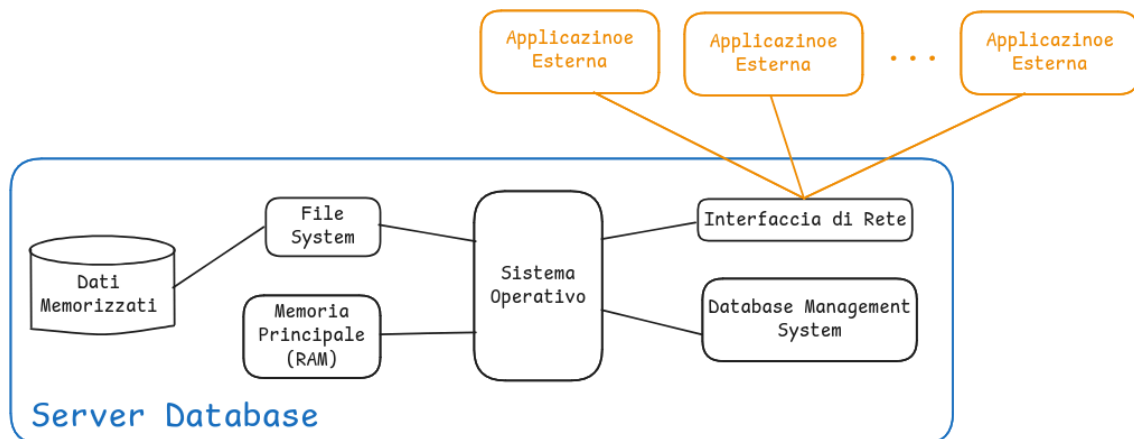


Figura 1.1: Interazioni del DBMS

A seconda delle necessità che vengono specificate dai requisiti, è possibile andare a concentrarsi su una specifica implementazione. Andando oltre alle peculiarità associate ad ogni diverso prodotto, è possibile identificare i seguenti elementi comuni a tutti i DBMS:

- **Gestione della memoria persistente:** si occupa di salvare i dati nel file system
- **Gestione del buffer:** si occupa, collaborando con il gestore della memoria, di effettuare *swap in/out* di varie pagine della memoria persistente in base a ciò che è richiesto dall'operazione che si sta eseguendo

Il gestore del buffer potrebbe sembrare molto simile al *sistema* di paginazione utilizzato sui comuni sistemi operativi. La differenza consiste nel fatto che il gestore del buffer è a conoscenza del tipo di operazione che è in esecuzione all'interno del database.

- **Strutture dati** per la memoria secondaria: si usano tipicamente per ottenere maggiore efficienza rispetto allo swap in/out delle pagine
- **Metodi di accesso:** consistono in un modo per accedere ai dati in memoria secondaria seguendo specifici *pattern* in base all'operazione che è necessario eseguire
- **Gestione delle transazioni:** si occupa di eseguire sequenze di operazioni in maniera atomica, mantenendo la consistenza del sistema
- **Gestione della concorrenza:** si occupa di garantire che molti processi siano abilitati ad accedere al database nello stesso momento. In linea di massima garantisce che le richieste parallele seguano uno schedule sequenziale

Per fare un esempio di ciò, immaginiamo che sia necessario gestire in maniera concorrente le seguenti transazioni: T_1, T_2, T_3 ; ci sono (3!) 6 possibili scheduling sequenziali che possiamo scegliere di seguire per soddisfarle:

- | | |
|-------------------|-------------------|
| • T_1, T_2, T_3 | • T_1, T_3, T_2 |
| • T_2, T_1, T_3 | • T_2, T_3, T_1 |
| • T_3, T_2, T_1 | • T_3, T_1, T_2 |

compito del gestore della concorrenza è quello di fare in modo che il risultato del sistema dopo aver eseguito in concorrenza le tre transazioni sia equivalente al risultato di uno casuale degli scheduling sequenziali di sopra riportati.

- **Operatori fisici:** ne esistono alcuni che sono comuni a più famiglie di basi di dati, altri che sono specifici di singole famiglie. Si tratta di operazioni rese disponibili tramite API che quando eseguite su un insieme di dati garantiscono un certo risultato

La maggior parte di questi operatori fisici, nel caso di database relazionali, sono in diretta corrispondenza con operatori dell'*algebra relazionale*; ad esempio, operazioni di *filtering* corrispondono alla clausola `WHERE`, mentre alcune operazioni di *proiezione* corrispondono alla clausola `SELECT`.

- **Ottimizzatore delle query:** ipotizziamo di voler effettuare un'operazione di `JOIN`. Sappiamo che esistono molte variante di questa operazione, il compito di questa componente è quello di scegliere il tipo di implementazione da utilizzare per rendere il più efficiente possibile la valutazione della query in esame
- **Frammentazione dei dati:** esistono casi in cui i dati non sono salvati in un'unica posizione (potrebbero essere salvati su macchine diverse, o sulla stessa macchina ma in file system separati, oppure sullo stesso file system ma non sullo stesso disco). In questi casi è necessario decidere come *partizionare* i dati
- **Replicazione e sharding:** nel caso in cui i dati siano memorizzati su **sistemi diversi** è necessario decidere cosa e dove *replicare* i dati, questo è un aspetto comune a praticamente tutte le diverse famiglie di database
- **Gestore della consistenza:** si occupa di fare in modo che tutte le repliche di uno stesso dato salvate su uno o diversi sistemi siano tra loro consistenti

- **Esecuzione distribuita:** per consentire ai sistemi di essere il più efficienti possibili, nel momento in cui le informazioni sono salvate in maniera frammentaria, è possibile sfruttare il calcolo distribuito in parallelo, in modo da abbattere i costi di esecuzione
- **Gestione dei dati in streaming**
- **Code di messaggi:** si tratta di un meccanismo pensato ad hoc per la gestione di esecuzione distribuita e dei dati in streaming

1.3. Database Management System Relazionali

Tutti i database relazionali sono basati sul **modello dei dati relazionale**. Andiamo di seguito a definirne le varie peculiarità:

- Un database è un *insieme di tabelle* ognuna delle quali è caratterizzata da un **nome** che nello specifico è chiamato **relation symbol**
- L'intestazione della relazione va ad identificare i nomi delle *colonne* che nello specifico sono chiamati **attribute names** insieme al *dominio* dal quale ciascun attributo può prendere valore
- L'insieme delle *righe* (**tuple**) di una tabella ne vanno a definire il contenuto

Di seguito vedremo le definizioni di **schema relazionale** e **schema della base di dati** con alcuni esempi per meglio comprendere questi concetti fondamentali.

Definizione 1.2 (Schema Relazionale)

È possibile definire lo **schema relazionale** di una base di dati tramite i seguenti oggetti:

- Simbolo di relazione R
- Insieme degli attributi $A_1, \dots, A_n$
- Insieme delle dipendenze locali Σ_R

Possiamo unire gli oggetti di cui sopra tramite la seguente formula:

$$R = (\{A_1, \dots, A_n\}, \Sigma_R) \quad (1.1)$$



Di seguito possiamo osservare la come lo schema relazionale possa essere visto dal punto di vista di una tabella:

SIMBOLO DI RELAZIONE R	ATTRIBUTO A_1	ATTRIBUTO A_2	ATTRIBUTO A_3
tupla t_1	v_{11}	v_{12}	v_{13}
tupla t_2	v_{21}	v_{22}	v_{23}

Esempio: Modello relazionale

Di seguito mostriamo un'istanza del modello relazionale precedentemente illustrato:

BOOKLENDING	BOOKID	READERID	RETURNDATE
	123	225	25-10-2016
	234	347	31-10-2016

Definizione 1.3 (Schema della Base di Dati)

È possibile andare a definire lo **schema della base di dati** tramite i seguenti oggetti:

- Simbolo della base di dati D
- Insieme di schemi relazionali $R_1, \dots, R_m$
- Insieme di dipendenze globali Σ

Possiamo unire gli oggetti di cui sopra tramite la seguente formula:

$$D = (\{R_1, \dots, R_m\}, \Sigma) \quad (1.2)$$



Esempio: Schema di una Base di Dati

- Schema relazionale 1:

$$\text{Book} = (\{\text{BookID}, \text{Author}, \text{Title}\}, \Sigma_{\text{Book}})$$

- Schema relazionale 2:

$$\text{BookLending} = (\{\text{BookID}, \text{ReaderID}, \text{ReturnDate}\}, \Sigma_{\text{Book}})$$

- Schema relazionale 3:

$$\text{Reader} = (\{\text{ReaderID}, \text{Name}\}, \Sigma_{\text{Reader}})$$

- Schema della base di dati:

$$\text{Library} = (\{\text{Book}, \text{BookLending}, \text{Reader}\}, \Sigma)$$

Sia in Definizione 1.2° che in Definizione 1.3° compare la nozione di **dipendenza**; è però necessario chiarire le differenze tra queste:

- quando il concetto di dipendenza compare con un pedice, per esempio Σ_R , ci stiamo riferendo a **dipendenze intra-relazionali**; cioè che si verificano all'interno di una tabella.

Un esempio di dipendenze intra-relazionali sono le *dipendenze funzionali*, più in particolare le dipendenze indotte da attributi *chiave*: $\text{BookID} \rightarrow \text{BookID}, \text{Author}, \text{Title}$ è una dipendenza funzionale all'interno della relazione Book

- quando le dipendenze compaiono senza pedice, significa che sono **globali**, anche dette **inter-relazionali**

Un esempio di queste dipendenze sono *dipendenze di inclusione* su particolari chiavi esterne: $\text{BookLending.BookID} \subseteq \text{Book.BookID}$ o ancora $\text{BookLending.ReaderID} \subseteq \text{Reader.ReaderID}$.

1.3.1. Progettazione di una Base di Dati

Una volta presi in esame la situazione da rappresentare e i requisiti da questa richiesti, è possibile iniziare con la progettazione della base di dati. Dal momento che molti contesti presentano molte complicazioni e requisiti specifici, è bene avere un quadro il più generale possibile di ciò che si renderà necessario implementare; per questo motivo possiamo dividere la progettazione di un database in tre fasi fondamentali:

- definizione di un **modello concettuale**: serve a modellare ad alto livello la situazione presa in esame; tipicamente vengono impiegati i diagrammi entità-relazione.

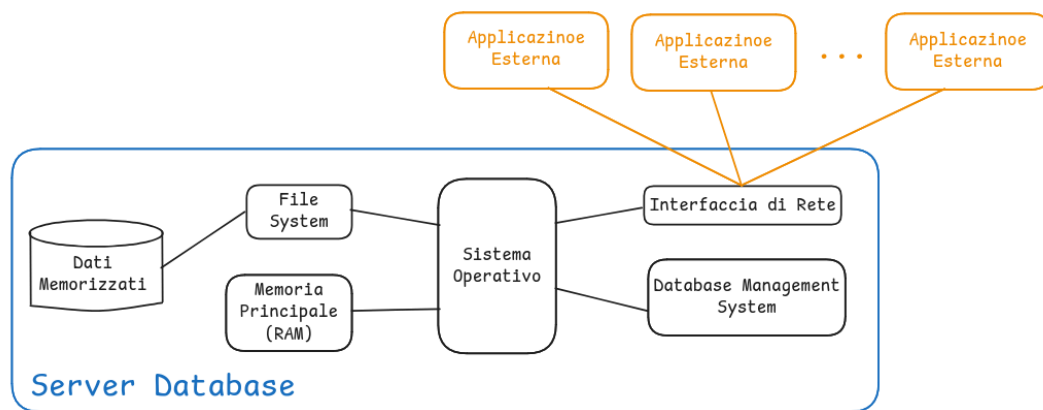


Figura 1.2: Diagramma Entità-Relazione di una libreria

- *traduzione* dello schema relazionale in un **modello logico**: il processo da seguire per la traduzione è piuttosto semplice e standard; infatti tipicamente ogni entità viene di solito mappata in una relazione, così come ogni relazione, in base alla sua cardinalità può essere tradotta in maniera differente

Alcuni design per una base di dati possono essere problematici. Di seguito andiamo ad indicare alcune forme di inconsistenza che possono presentarsi:

- Alcune tabelle potrebbero contenere troppi valori, o addirittura valori **duplicati**
- Potrebbero verificarsi anomalie nel momento in cui andiamo a manipolare i dati:
 - **Anomalie di inserimento**: nel momento in cui andiamo ad inserire i dati abbiamo bisogno di tutti i valori ma alcuni potrebbero essere ancora sconosciuti
 - **Anomalie di cancellazione**: nel momento in cui andiamo a cancellare una tupla, potremmo andare a cancellare informazioni di cui abbiamo ancora bisogno in altri record di altre relazioni
 - **Anomalie di aggiornamento**: quando i dati sono memorizzati in maniera ridondante, questi devono essere modificati per tutte le loro occorrenze

Le problematiche e le anomalie sopra elencate sono tipicamente ammortizzate applicando tecniche di **normalizzazione** della base di dati. L'obiettivo della normalizzazione è quello di distribuire i dati in maniera omogenea tra le tabelle. In base al tipo di normalizzazione che andiamo a garantire riusciamo a prevenire diversi tipi di anomalia:

- **1° Forma Normale:** non vengono ammessi attributi multivalore o composti
- **2° Forma Normale:** tutti gli attributi non chiave sono completamente dipendenti dagli attributi chiave, in altre parole non deve esistere un sottoinsieme degli attributi chiave che può essere usato per derivare attributo non chiave
- **3° Forma Normale:** tutti gli attributi non chiave sono direttamente dipendenti dagli attributi chiave, in altre parole si dice che non sono ammesse dipendenze transitive
- **4°, 5° Forma Normale e Forma Normale di Backus-Naur** sono altri tipi di forma normale ma non così comuni e comunque fuori dallo scopo di questo corso

1.3.2. Query Relazionali

Una volta che i dati sono memorizzati all'interno della nostra base di dati, possiamo chiederci in quale modo ottenere informazioni da questi. A questo scopo abbiamo a disposizione degli strumenti che sono noti con il nome di **query**.

Le query sono strumenti che ci consentono di effettuare diversi tipi di operazioni sui dati:

- Specificare condizioni per selezionare solo tuple rilevanti
- Restringere tabelle ad un sottoinsieme di attributi
- Combinare valori provenienti da diverse tabelle

Esistono diversi linguaggi per effettuare query a una base di dati: *calcolo relazionale*, *algebra relazionale*, *SQL*. Di seguito andiamo ad elencare alcuni operatori dell'algebra relazionale:

- **Proiezione π :** utilizzata per restringere una tabella ad un sottoinsieme di attributi
- **Selezione σ :** utilizzata per selezionare solo alcune tuple di una tabella
- **Rinominaione ρ :** utilizzata per cambiare nomi ad un attributo
- **Operazioni insiemistiche:** unione $\cup$, differenza $-$, intersezione $\cap$
- **Join naturale $\bowtie$:** utilizzato per combinare due tabelle sulla base di attributi comuni
- **Operatori di join avanzati** come θ - join, equi-join, ...

Le query relazionali, che si possono vedere come una catena di applicazioni di operatori relazionali, possono essere visualizzate in strutture ad albero, questo tipo di visualizzazione porta con sé alcuni vantaggi:

- Mostra l'ordine di valutazione di ogni operazione
- È utile nel contesto dell'ottimizzazione delle query

Esempio: Alberi equivalenti per una query che lista il nome di tutti i lettori dei libri prestati che abbiano $\text{ReturnDate} < 20/10/2016$

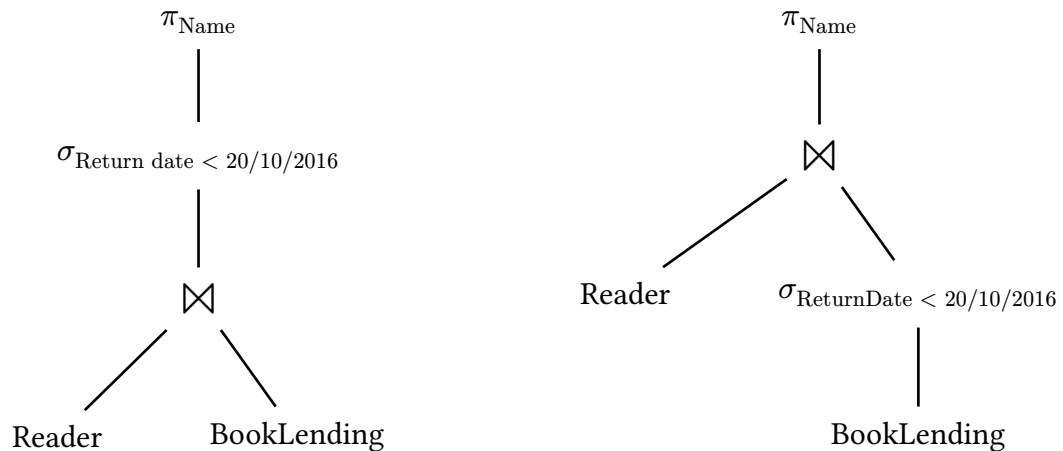


Figura 1.3: Due diversi piani di query per la stessa richiesta in algebra relazionale.

Possiamo osservare come Figura 1.3 illustri due modi alternativi di eseguire la stessa query relazionale. Osservando attentamente l'ordine tra *join* e *selezione*, è possibile notare che nell'albero di destra otteniamo una query più efficiente, dal momento che ci troveremo a effettuare un join dove una delle due tabelle da unire è stata prima filtrata. In questo modo ci troveremo a dover effettuare un confronto con molti meno record rispetto alla rappresentazione di sinistra.

1.3.3. Transazioni e Gestione della Concorrenza

Quando andiamo a modificare i dati all'interno di una base di dati, possiamo andare incontro a diverse tipologie di problematiche. Di seguito andiamo ad elencarne alcune:

- **Integrità logica dei dati:** dobbiamo assicurarci che tutti i valori scritti siano corretti e che siano effettivamente il risultato atteso dall'esecuzione di un'operazione
- **Integrità fisica e recovery:** dobbiamo garantire la persistenza dei dati assieme alla possibilità di recuperarli nel caso in cui si verificano dei crash di sistema
- **Gestione di più utenti:** dobbiamo permettere agli utenti di operare in maniera concorrente sulla stessa base di dati senza che ci siano interferenze

Tutte le questioni sopra elencate possono essere indirizzate tramite l'impiego di **transazioni**.

Definizione 1.4 (Transazione)

Una **transazione** è una sequenza di operazioni di *lettura e scrittura* su una base di dati con le seguenti proprietà:

- Deve essere trattata come **entità atomica** di esecuzione
- Deve portare la base di dati da uno stato consistente ad un nuovo stato anch'esso consistente



L'esempio più tipico di una transazione è quello di un trasferimento di denaro da un conto bancario ad un altro. Di seguito specifichiamo alcune delle proprietà che le basi di dati devono rispettare affinché possiamo affermare che gestiscano le transazioni correttamente:

- **Atomicità:** data una transazione possiamo eseguire tutte le operazioni di questa, oppure nessuna, non è possibile eseguire una transazione in modo *parziale*
- **Consistenza:** dopo l'esecuzione di una transazione tutti i valori nella base di dati devono essere corretti rispetto ai vincoli e alle dipendenze intra-inter relazionali
- **Isolamento:** transazioni concorrenti di utenti differenti non devono interferire tra loro
- **Durabilità:** è necessario che i risultati di una transazione rimangano persistenti anche a seguito di possibili crash di sistema. Per garantire questa proprietà di solito si utilizza un **transaction log**, nel quale tutte le transazioni vengono registrate

Le proprietà sopra elencate sono anche dette **ACID**, utilizzando le loro iniziali per formare l'acronimo.

1.3.4. Problematiche del Modello Relazionale

L'impiego di modelli relazionali può portare con sé alcune problematiche dovute intrinsecamente a come vengono impiegate tabelle e relazioni. Andiamo ad elencare alcune problematiche di seguito:

- **Overloading semantico:** il modello relazionale rappresenta sia entità che relazione tramite l'utilizzo di *tabelle*, non esiste infatti in modo per rappresentare separatamente i due concetti
- **Struttura dati omogenea:** il modello relazionale assume omogeneità sia orizzontale che verticale. L'omogeneità orizzontale implica che tutte le tuple hanno valori per gli stessi attributi; quella verticale si riferisce al fatto che, data una colonna, i suoi valori provengono tutti dallo stesso dominio. Inoltre ogni cella può contenere soltanto valori atomici, il che potrebbe risultare limitante in alcuni contesti
- **Supporto limitato alla ricorsione:** è molto complicato andare a definire query ricorsive in SQL; nel caso ad esempio in cui ci trovassimo a dover lavorare su strutture a grafo sarebbe veramente complicato utilizzare un modello relazionale

1.4. Nuovi Requisiti

Dopo aver descritto in maniera approfondita tutte le peculiarità di SQL e del modello relazionale che va ad implementare, spostiamo la nostra attenzione su dei *nuovi requisiti* che situazioni odierne ci portano a dover soddisfare.

Il primo di questi è sicuramente legato a dati che hanno bisogno di essere organizzati in **strutture** sempre più **complesse** come ad esempio nei social network.

Se un tempo le operazioni di lettura erano molto più frequenti rispetto alle operazioni di scrittura, il mondo moderno e l'utilizzo di nuove tecnologie ci pongono davanti ad un cambio di paradigma dove spesso è anche necessario andare a **scrivere in maniera frequente** sulle nostre basi di dati.

Dal momento che il mondo contemporaneo è sempre più *data-centric*, è necessario dover gestire una quantità sempre maggiore di dati, il che sarebbe impossibile utilizzando una singola macchina, rendendo necessario **distribuire i dati** su molti server interconnessi (*cloud storage*).

Tutte le esigenze di sopra aprono la strada all'utilizzo di un nuovo approccio, quello **NoSQL**, nel quale alcune proprietà e garanzie del modello relazionale vengono trascurate al fine di provare a soddisfare in maniera migliore i nuovi requisiti. Di seguito elenchiamo alcune delle differenze tra gli approcci NoSQL e il tradizionale SQL:

- Il modello dei dati potrebbe essere diverso da quello tradizionale basato sulle tabelle
- Accesso programmatico alla base di dati o con strumenti diversi da SQL
- Capacità di gestire modifiche allo schema dei dati
- Capacità di gestire dato senza uno schema specifico
- Supporto alla distribuzione dei dati
- Requisito di aderenza alle proprietà ACID che viene alleggerito, specialmente in termini di consistenza, rispetto ai DBMS tradizionali

2. Key-Value Stores e Document Databases

Nell'ultimo decennio i database relazionali sono stati particolarmente apprezzati grazie alla loro flessibilità; purtroppo non sono noti per le loro **performance**. Come accennato nell'ultima parte del capitolo precedente, gli importanti avanzamenti tecnologici degli ultimi anni hanno portato alla luce delle forti limitazioni legate alle caratteristiche dei sistemi relazionali.

L'idea è dunque quella di passare a dei sistemi che siano in generale meno flessibili rispetto a sistemi relazionali sotto alcuni punti di vista, ma che possano adattarsi meglio e con maggiore efficienza ai casi d'uso nei quali è necessaria la loro applicazione.

2.1. Key-Value Stores

L'idea alla base di questo approccio è molto semplice: viene costruito un **array associativo permanente**. Come il nome suggerisce, gli elementi chiave di un array associativo sono una **chiave** e **valori** associati alla chiave; volendo fare un paragone con i linguaggi di programmazione, possiamo associare questo concetto a quello di *dizionario in Python* e a quello di *hashMap in Java*. Di seguito andiamo ad elencare alcune delle proprietà fondamentali:

- È possibile accedere a valori (o cancellarli) tramite l'utilizzo delle *chiavi*
- È possibile inserire coppie chiave-valore arbitrarie senza che queste aderiscano necessariamente ad uno schema (**schemaless**)
- I valori possono avere tipi di dato molteplici (liste, stringhe, valori atomici, array, ...)
- È un sistema molto semplice ma veloce: grazie alla semplicità della struttura dati, non abbiamo bisogno di un query language molto avanzato per accedere ai dati. Si tratta di un'alternativa ottimale nel caso di applicazioni *data intensive*.

Proprio riguardo all'ultimo punto, è necessario specificare che il compito di combinare più coppie chiave-valore in oggetti complessi è tipicamente responsabilità dell'applicazione che si interfaccia con il sistema. Alcuni esempi molto comuni di questo approccio sono Amazon Dynamo, Riak e Redis

2.1.1. Map-Reduce

In generale, ci si riferisce a MapReduce come un approccio di programmazione che consente di processare enormi quantità di dati in parallelo sfruttando diversi cluster di calcolo dividendo grandi operazioni in piccoli passi di map e reduce. Nell'ambito dei key-value stores si può vedere la procedura divisa nei seguenti passaggi:

- **Splittare** l'input e iterare sulle coppie chiave-valore in sottoinsiemi disgiunti
- Calcolare la funzione di **map** su ognuno dei sottoinsiemi splittati
- Raggruppare tutti i valori intermedi per chiave (**shuffle**)
- Iterare su tutti i gruppi applicare **reduce** in modo da riunire i vari gruppi

Figura 2.1 illustra chiaramente il funzionamento dei vari passaggi necessari al funzionamento dell'algoritmo MapReduce.

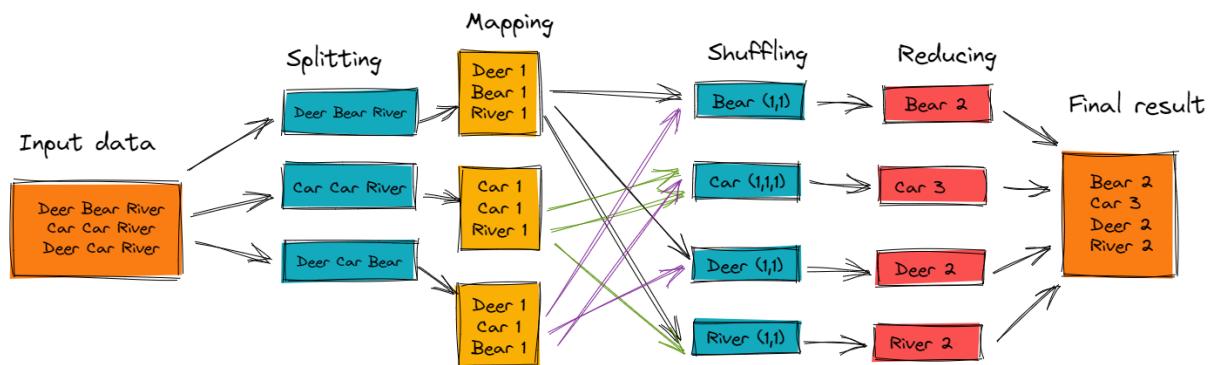


Figura 2.1: Esempio di applicazione dell'algoritmo MapReduce per contare le occorrenze di ogni parola all'interno di un input

È possibile notare i seguenti aspetti:

- L'approccio è estremamente adatto alla **parallelizzazione**, infatti i passaggi di mapping e di riduzione possono essere eseguiti in parallelo, ad esempio lanciando un processo di map per ogni «frase» o, nel caso dell'esempio, ogni n parole e un processo di *reduce* per ogni parola
- È possibile sfruttare la **località** dei dati in modo da processare i dati direttamente sulla macchina che li sta «ospitando» in modo da ridurre il più possibile il traffico sulla rete.
- È possibile migliorare ulteriormente quanto presente in Figura 2.1 applicando una procedura di **combinazione**, che consenta di combinare i risultati intermedi invece di mandarli alla procedura di riduzione in formato grezzo, tuttavia non è sempre garantito che questa operazione sia implementata sui sistemi che scegliamo di utilizzare

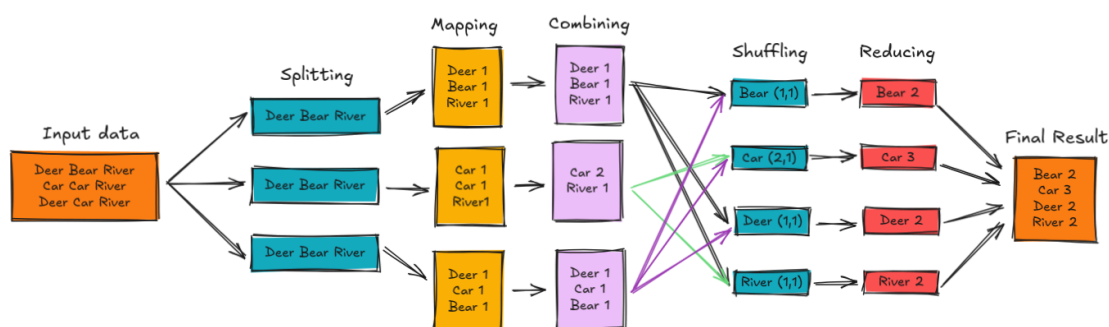


Figura 2.2: Applicazione della funzionalità di combine al metodo MapReduce

2.2. Document Database

Se i key-value store sono metodi molto semplici per memorizzare dati, questo approccio a volte può risultare fin troppo semplice: memorizzare soltanto dati primitivi a volte è fin troppo riduttivo per quelle che potrebbero essere le necessità di un'applicazione.

Per questo motivo nascono i **document database**, i quali permettono di memorizzare documenti in un formato di **testo strutturato** (JSON, XML, YAML, ...). Anche in questo caso ogni documento è identificato da una *chiave univoca*, mostrando quindi una forte correlazione al concetto di key-value store, ma i dati memorizzati hanno un requisito in più sulla loro struttura. Questo è spesso molto utile in quanto ci consente di effettuare validazione dei dati, per esempio tramite XML o JSON schema.

2.2.1. JavaScript Object Notation

Per lo scopo di questo corso non andremo a soffermarci su database di documenti che utilizzano XML per dare struttura ai propri documenti, per semplicità sceglieremo di concentrare la nostra attenzione su quelli che memorizzano oggetti JSON. Per fare ciò è necessario andare a comprendere quali siano le peculiarità di questo linguaggio:

- Si tratta di un **formato testuale** di facile lettura per rappresentazione di strutture dati
- Ogni documento JSON è sostanzialmente un annidamento di coppie **chiave-valore** separate da un simbolo « : »
- Per dare struttura al documento vengono utilizzate le parentesi graffe « {} »
- La chiave è sempre definita tramite una **stringa**, mentre i valori possono essere vari:
 - floating point
 - stringhe unicode
 - booleani
 - array
 - oggetti

Esempio: Semplice Descrizione JSON di un oggetto Person

```
1 {
2   "firstName": "Alice",
3   "lastName" : "Smith",
4   "age"      : 31,
5 }
```

JSON

Esempio: Oggetto complesso con figli composti e array di valori

```
1 {
2   "firstName" : "Alice",
3   "lastName"  : "Smith",
4   "age"       : 31,
5   "address"   : {
6     "street"   : "Main Street",
7     "number"   : 12,
8     "city"     : "Newtown",
9     "zip"      : 31141,
10  },
11  "telephone" : [123456, 908077, 2782783],
12 }
```

JSON

Il linguaggio JSON presenta tuttavia alcune limitazioni:

- Non supporta referenze da un documento all'altro, dunque non è possibile adottare un meccanismo di foreign key come in SQL
- Non supporta referenze all'interno di uno stesso documento JSON

Esistono tuttavia alcuni strumenti per il processing di file JSON che supportano referenze basate su ID: per esempio, è possibile aggiungere una chiave `id` per l'oggetto persona e impostare un valore univoco per questo, per fare in modo di utilizzare tale `id` per riferirci all'oggetto di tipo persona appena costruito in altri oggetti.

2.2.2. MongoDB

Uno degli esempi più noti di document database è sicuramente **MongoDB**. Se volessimo andare a fare un confronto con un DBMS relazionale avremmo le seguenti differenze:

- Capacità di **scalare orizzontalmente** su più macchine
- Rispetto a un DBMS relazionale abbiamo una **miglior località dei dati**; questa proprietà è garantita proprio dal fatto che ogni oggetto contiene tutti i dati di cui ha bisogno, senza necessità di dover effettuare «join» con altre tabelle
- Mancanza di possibilità di far rispettare ai dati memorizzati uno **schema** con conseguente mancata possibilità di validare i dati in ingresso
- Mancata possibilità di eseguire operazioni di **join** per unire risultati
- Mancata possibilità di supportare **transazioni**

Similmente ad un database relazionale, è invece possibile operare query per il recupero di dati o la costruzione di indici sia primari che secondari per migliorare l'efficienza. Per semplicità andiamo di seguito a stabilire un mapping tra un DBMS relazionale e MongoDB:

RDBMS		MongoDB
Database		Database
Table, View		Collection
Row		Document (JSON, BSON)
Column		Field
Index		Index
Join		Embedded Document
Foreign Key		Reference
Partition		Shard

De-normalizzazione

L'aspetto più rilevante nell'utilizzo di questo modello risiede nella **semplicità** con cui è possibile rappresentare e gestire diverse strutture dati. Il concetto chiave a cui si deve questa immediatezza è la **de-normalizzazione**. Come mostrato in Figura 2.3, la de-normalizzazione semplifica la struttura della base di dati accorpondo le informazioni che altrimenti sarebbero distribuite su più tabelle o entità.

Questa scelta, tuttavia, introduce un importante svantaggio: la **gestione delle modifiche ai dati**. Se, ad esempio, un utente compare in più contesti e si rende necessario aggiornare i dati relativi agli ordini a lui associati, sarà necessario applicare la modifica in **ogni copia** presente nel sistema. In caso contrario, la base di dati rischierebbe di diventare incoerente.

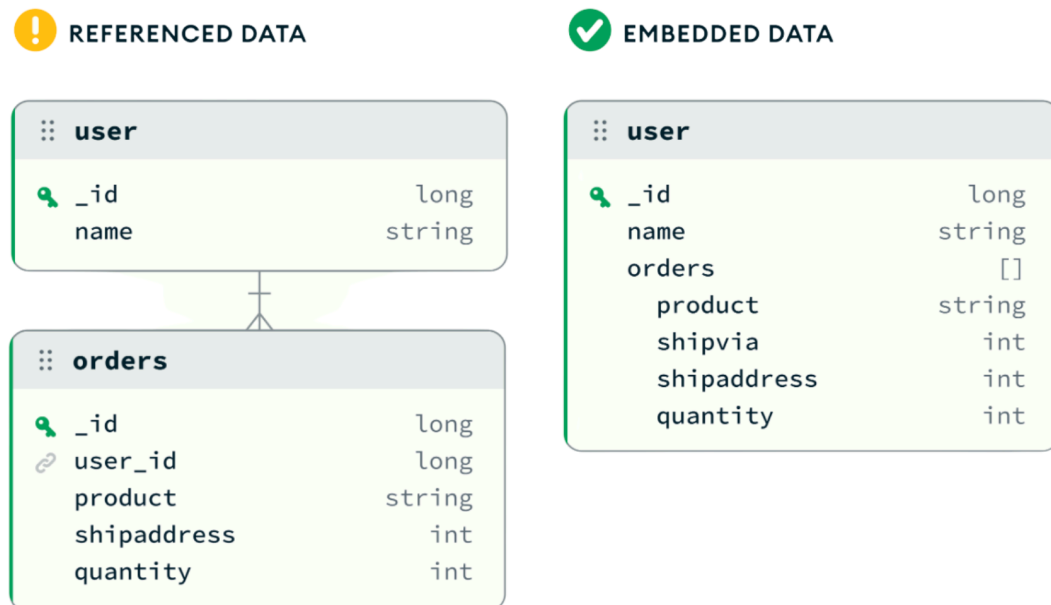


Figura 2.3: Esempio di de-normalizzazione: a sinistra una struttura normalizzata basata su due entità distinte, a destra una rappresentazione de-normalizzata della stessa relazione.

Salvataggio Atomico

Per quanto abbiamo citato che sia stato abbandonato il concetto di transazioni e delle loro proprietà «ACID», è comunque necessario anche in questo contesto andare a garantire consistenza, specialmente nel momento in cui più processi concorrenti vanno ad interrogare la base di dati. Per questo il paradigma adottato è quello del **salvataggio atomico**, che risulta comunque più semplice e rapido da effettuare rispetto all'esecuzione di una transazione.

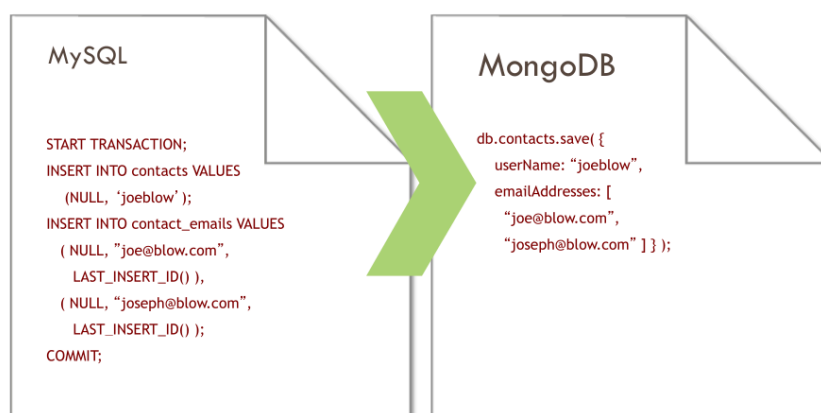


Figura 2.4: Esempio di Salvataggio atomico

Altre caratteristiche di MongoDB

Di seguito andiamo ad elencare altre caratteristiche peculiari di MongoDB che lo differenziano rispetto ad altri sistemi:

- Ogni documento JSON memorizzato deve essere provvisto di un campo identificativo con nome `_id`, se non fornito, questo campo viene creato in automatico dal sistema
- I dati vengono in realtà memorizzati in formato **BSON** che consiste in una rappresentazione binaria dei dati JSON per garantire maggiore efficienza e semplicità di manipolazione dei dati
- MongoDB è in grado di capire quali sono i dati ai quali sono richiesti accessi più frequenti e di **cachare** in memoria principale i loro valori, così da garantirne un accesso più rapido ed efficiente

CRUD

Questa sezione andrà ad illustrare i vari comandi che è possibile utilizzare per effettuare le operazioni CRUD su MongoDB:

Per quanto riguarda l'operazione di **create** abbiamo a disposizione i seguenti comandi:

- `db.collection.insert(<document>)`
- `db.collection.save(<document>)`
- `db.collection.update(<query>, <update>, {upsert: true})` : prova ad aggiornare un record ma se non esiste nulla che corrisponda alla `query` allora va ad inserire il valore che avrebbe dovuto aggiornare

Esempio: Inserimento di un documento

```
1 > db.user.insert({  
2   firstName : "John",  
3   lastName  : "Doe",  
4   age       : 39  
5 })
```

JS JavaScript

Per quanto riguarda l'operazione di **read**, in modo simile a come facciamo in SQL andiamo a leggere tutti i dati che soddisfano una certa condizione:

- `db.collection.find(<query>, <projection>)`
- `db.collection.findOne(<query>, <projection>)`

Esempio: Lettura di tutti gli elementi

```
1 > db.user.find()  
2  
3 > result : {  
4   "_id"      : ObjectId("51.."), // assegnato automaticamente  
5   "firstName" : "John",
```

JS JavaScript

```

6   "lastName" : "Doe",
7   "age"      : 39
8 }

```

Si può notare come in questa occasione, dal momento che non sono stati passati parametri per `<query>`, il database abbia restituito tutti gli elementi della collection

Per quello che riguarda le operazioni di **update**:

- `db.collection.update(<query>, <update>, <options>)`

Esempio: Aggiornamento di un documento

```

1 > db.user.update(
2   {"_id": ObjectId("51..")}, // query per modificare specifico objectId
3   {
4     $set: { // aggiornamento che si intende fare
5       age: 40, salary: 7000
6     }
7   }
8 )

```

Per ciò che invece riguarda le operazioni di **delete** abbiamo la seguente funzionalità:

- `db.collection.remove(<query>, <justOne>)`

Esempio: Eliminazione di un documento in base ad una query

```

1 > db.user.remove({
2   "firstName": /^J/ // regexp per identificare tutti i documenti
3   dove firstName inizia per "J"
4 })

```

Proprietà ACID vs. BASE

Se abbiamo visto che nei database relazionali vengono tenute in forte considerazione proprietà *ACID* (atomicità, consistenza, isolamento, durevolezza); in sistemi come MongoDB è stato preferito richiedere l'aderenza ad un nuovo tipo di paradigma, più lasco, ma che consente maggior efficienza e meglio si adatta ai casi d'uso:

- **Basically Available:** il sistema rimane operativo anche durante possibili crash parziali di sistema. Anche in questi casi dovrebbe essere possibile l'accesso alla base di dati, assicurando che il servizio continui ad essere disponibile. Si tratta di una proprietà fondamentale in sistemi che richiedono «constant uptime», come ad esempio applicazioni di e-commerce o applicazioni social media
- **Soft State:** questo concetto si riferisce all'idea che lo stato del database potrebbe cambiare nel corso del tempo, anche se non dovessero essere stati aggiunti o modificati

dati memorizzati. Queste modifiche sono tipicamente atte al raggiungimento di uno stato che sia consistente per tutti i nodi della base di dati

- **Eventually Consistent:** questo significa che dopo un aggiornamento non è necessario che le modifiche apportate alla base di dati siano rese note ad ogni istanza. Questo è particolarmente utile in un *contesto distribuito*, dove sarebbe altrimenti necessario aggiornare tutte le possibili istanze del dato aggiornato, che si trovano potenzialmente in più località fisiche

2.2.3. Caso d'uso: Location-Based Application

Vogliamo costruire un'applicazione con le seguenti caratteristiche:

- Gli utenti devono avere la possibilità di effettuare il *check-in*
- Gli utenti possono lasciare *note* o *commenti* riguardo una location

Per ogni **location** vogliamo le seguenti possibilità:

- Salvare il *nome*, l'*indirizzo* e dei *tag*
- Possibilità di memorizzare contenuti generati dagli utenti (*tips*, *note*)
- Possibilità di trovare altre location nei paraggi

Per quanto riguarda invece i **check-in** abbiamo i seguenti requisiti:

- Gli utenti dovrebbero essere in grado di effettuare il check-in
- Possibilità di generare *statistiche* sui check-in per ogni location

In primo luogo è necessario andare a definire le **collection** (tabelle) che andranno in qualche modo a rappresentare le entità coinvolte all'interno del sistema.

Collection **locations**

Per prima cosa andiamo a dare un rudimentale schema per la collection `locations`, che andrà a rappresentare le varie location del nostro sistema:

Esempio: Locations v1 - Possibilità di filtrare per zipcode e per tags

```
1 location = {
2   name: "10gen East Coast",
3   address: "134 5th Avenue 3rd Floor",
4   city: "New York",
5   zip: "10011",
6
7   tags: ["business", "offices"]
8 }
```

JSON

Di seguito andiamo a mostrare alcune query che sarà possibile effettuare su questa collection per andarne a visualizzarne i valori:

```
1 // 1.trova le prime 10 location con zip code 10011
2 db.locations.find({zip:"10011"}).limit(10)
```

JavaScript

```

3
4 // 2. trova le prime 10 location con tag business
5 db.locations.find({tag: "business"}).limit(10)
6
7 // 3. trova le location con zip code 10011 e tag business
8 db.locations.find({zip: "10011", tags: "business"})

```

Si noti come nell'esempio sopra le query 2. e 3. differiscano dalla query 1. , infatti nella prima query andiamo ad effettuare un controllo di uguaglianza con un valore unico, mentre nelle altre due il tag « business » si trova inserito all'interno di una lista, per cui il controllo sarà effettuato sugli elementi della lista e basterà trovare un un elemento della lista che corrisponda al valore che stiamo cercando.

Ci piacerebbe andare a memorizzare anche le *coordinate* di una posizione, in modo tale da andare in seguito a ricercare locations vicine ad alcune coordinate.

Esempio: Locations v2 - Implementazione di un semplice sistema di coordinate

```

1 location = {
2   name: "10gen East Coast",
3   address: "134 5th Avenue 3rd Floor",
4   city: "New York",
5   zip: "10011",
6
7   tags: ["business", "offices"],
8   latlong: [40.0, 72.0]
9 }

```

Per quanto noi siamo consapevoli che il campo `latlong` corrisponda a delle coordinate, quel campo per MongoDB non è altro che una lista. Anche se MongoDB è nativamente un sistema **schema-less**, si rende a volte necessario aggiungere degli schemi parziali per garantire più efficienza. A questo scopo vengono creati degli **indici**:

```

1 db.locations.ensureIndex({latlong: "2d"})

```

Questo comando ci permette non solo di rendere le nostre query più efficienti, ma anche di andare a forzare il fatto che i valori per il campo `latlong` siano bidimensionali ("2d").

Per andare ora ad effettuare query che ci permettano di ottenere location vicine a delle certe coordinate possiamo andare ad utilizzare gli **operatori spaziali**:

```

1 db.locations.find({latlong:{$near: [40,70]}})

```

È comunque importante menzionare il fatto che per quanto sia possibile andare a memorizzare informazioni spaziali, MongoDB non è sicuramente il sistema più consono a questo scopo. Esistono infatti soluzioni più efficienti e studiate proprio per questo caso d'uso.

Ipotizziamo ora di voler aggiungere la possibilità per gli utenti di aggiungere delle *note* e dei *commenti* su ogni location.

Esempio: Locations v3 - Aggiunta la possibilità di lasciare commenti

```
1  location = {
2    name: "10gen East Coast",
3    address: "134 5th Avenue 3rd Floor",
4    city: "New York",
5    zip: "10011",
6
7    tags: ["business", "offices"],
8    latlong: [40.0, 72.0],
9    tips: [ // lista di oggetti complessi
10      {
11        user: "nosh", date: "6/26/2010",
12        tip: "stop by for office hours on Thursdays",
13      },
14      {...},
15    ]
16  }
```

Ipotizzando che la v3 sia la versione completa che ci serve per la collection **locations**, andiamo a vedere quali sono gli indici che ci sarà necessario definire per avere più efficienza:

```
1  db.locations.ensureIndex({tags:1})
2  db.locations.ensureIndex({name:1})
3  db.locations.ensureIndex({latlong:"2d"})
```

Quando andiamo a creare un indice su una lista, questo verrà creato su ogni elemento della lista. Assieme alle possibilità già viste per effettuare query è anche disponibile la funzionalità delle regular expression:

```
1  db.locations.find({name: /^typeaheadstring/})
```

Andiamo ora a vedere come sfruttare le operazioni CRUD viste in precedenza applicate a questo contesto. Per andare ad inserire gli elementi nella collection utilizziamo il comando `insert`:

```
1  db.locations.insert(location) // per definizione di location
   di vedano gli esempi (v3 in particolare)
```

Per andare a modificare una specifica location andiamo ad utilizzare il comando `update`, specificando una query e come andremo a modificare tale documento; in questo caso andremo ad aggiungere un tip, ipotizzando che questo non fosse già presente:

```

1  db.locations.update(
2    {name: "10gen HQ"}, // query
3    // push è usato per aggiungere elementi ad una lista (tips)
4    {$push: {tips:
5      {
6        user: "nosh", date: "2/26/2010",
7        tip: "stop by for office hours on Thursdays",
8      }
9    }}
10  }
11 )

```

Rappresentazione dei check-in

Per andare a rappresentare i vari check-in degli utenti abbiamo a disposizione varie scelte:

- Possiamo scegliere come con i vari tips, di associarli alla collection delle locations, memorizzando per ogni location una lista di check-in
- Possiamo andare a creare una collection `user` all'interno della quale memorizzeremo per ogni utente i check-in da questo effettuati
- Possiamo utilizzare una nuova collection specifica per i check-in che verrà gestita allo stesso modo di come trattiamo una relazione **many-to-many**

La scelta dell'approccio da utilizzare dipende più che altro dall'utilizzo che andremo a fare dei dati: in particolare, in questo caso, dal tipo di statistiche che vogliamo estrarre (o che vogliamo estrarre più frequentemente rispetto alle altre):

- Ci potrebbe interessare capire per ogni utente quale luogo è stato più frequentato, in questo caso forse è meglio salvare i check-in nella collection degli utenti
- Ci potrebbe interessare capire quale è location più frequentata tra tutte, e in questo caso sarebbe utile avere i check-in come attributo delle locations
- Nel caso in cui abbiamo bisogno di entrambe le statistiche, probabilmente sarebbe il caso di utilizzare una collection separata per i soli check-in

Utenti con check-in

Andiamo a mostrare come sia possibile mostrare i check-in come proprietà di un utente. La modalità non dovrebbe sorprendere dal momento che il meccanismo è analoga quello per i *tips* nella collection delle *location*.

```

1  user = {
2    name: "nosh",
3    email: "nosh@10gen.com",
4    ... // altre proprietà dell'utente
5    checkins: [
6      {
7        location: "10gen HQ",
8        timestamp: "9/20/2010, 10:12:00",

```

```

9      ... // altre proprietà
10    },
11    ... // altri check-in dell'utente
12  ]
13 }

```

Per andare ad estrarre delle statistiche è possibile utilizzare le seguenti query:

```

1  // estrazione di tutti gli utenti che hanno effettuato un
   check-in in una location
2  db.users.find({"checkins.location": "10gen HQ"})
3
4  // estrae i 10 utenti che hanno effettuato più check-in in una location
5  db.users.find({"checkins.location": "10gen HQ"}).sort({ts:-1}).limit(10)
6
7  // estrae quanti utenti hanno effettuato un check-in in una location dopo
   un certo timestamp
8  db.users.find({
9    "checkins.location": "10gen HQ",
10    timestamp: {$gt: ...$}}
11 ).count()

```

Evidentemente è ancora possibile calcolare statistiche riguardo alle specifiche location, ma è necessario in questo caso andare a scorrere tutti i record della collection `users`, risultando potenzialmente inefficiente nel caso in cui ogni utente abbia effettuato molti check-in.

Rappresentazione separata dei check-in

Possiamo scegliere di gestire separatamente la collection dei vari check-in, per fare ciò andremo a salvare un campo `checkins` all'interno dei record `user` che sarà costituito da una serie di references ai record della collection `checkins`

```

1  user = {
2    name: "nosh",
3    email: "nosh@10gen.com",
4    ... // altre proprietà
5    checkins = [e4af242f, cfeb950a, a542e63e]
6  }

```

L'utilizzo di `ObjectID` ci consente di avere accesso in lettura molto efficiente, tuttavia nel caso in cui avessimo bisogno di proprietà di utenti e locazioni che non sono presenti all'interno dei record della collection `checkins` in quel caso tali attributi dovrebbero essere replicati al loro interno, in modo da evitare di dover eseguire operazioni di aggregazione. Questo potrebbe risultare problematico, specialmente nel caso in cui sia necessario eliminare dei dati, infatti duplicando le informazioni, sarebbe complicato capire cosa andare ad eliminare e dove andarlo a fare.

2.2.4. Operazioni di Aggregazione in MongoDB

All'interno di MongoDB le operazioni di aggregazione funzionano in maniera molto diversa da come funzionano in un database relazionale.

Pipeline di Aggregazione

All'interno di MongoDB abbiamo a disposizione una **pipeline di aggregazione**.

Esempio: Semplice pipeline di aggregazione in MongoDB

```
1 db.orders.aggregate([  
2   // filtraggio  
3   {$match: {status: "A"}},  
4   // raggruppamento  
5   {$group: { id: "$cust_id", total: {$sum: "$amount"}}},  
6 ])
```

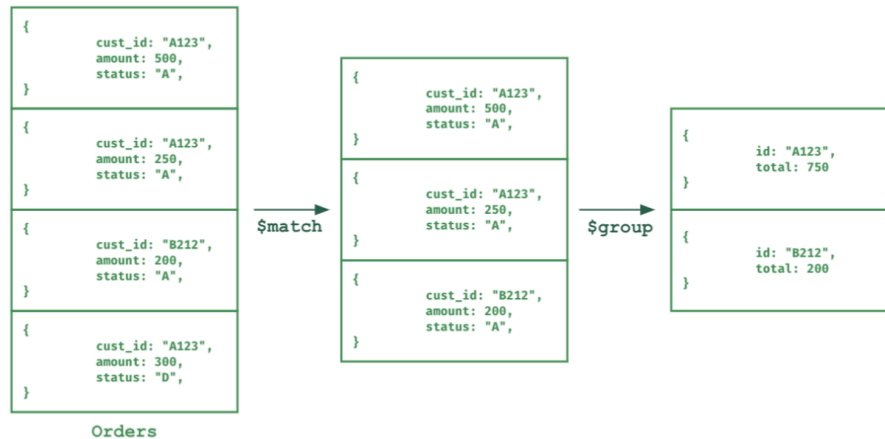


Figura 2.5: Rappresentazione grafica della pipeline di aggregazione specificata nel codice sopra

Nell'esempio di Figura 2.5 si nota come i passaggi principali di cui questa è costituita sono due: **matching** e **grouping**. Tuttavia queste non sono le uniche operazioni che è possibile effettuare in una pipeline di aggregazione. Andiamo di seguito a mostrare varie operazioni assieme ad una breve descrizione:

- **match** : filtra i documenti in ingresso in base a una certa condizione
- **group** : raggruppa i documenti in sulla base di attributi comuni e calcola valori aggregati per ogni gruppo
- **project** : consente di modificare la struttura dei documenti in ingresso in vari modi, ad esempio modificando i nomi degli attributi, creando nuovi attributi o eliminando attributi esistenti
- **sort** : ordina i documenti in ingresso sulla base dei criteri che vengono specificati
- **limit** : limita il numero di documenti in uscita a un certo numero
- **skip** : salta un certo numero di documenti in ingresso
- **unwind** : consente di "esplodere" il contenuto di una lista, ottenendo un documento per ogni elemento della lista. Si usa tipicamente per andare a filtrare o raggruppare in

base a valori che si trovano all'interno di liste che non sarebbero altrimenti accessibili in maniera diretta

- **geonear** : consente di effettuare un'operazione molto simile all'operatore **\$near** , ma all'interno di una pipeline di aggregazione e con maggiore flessibilità (più parametri possono essere specificati)

Come visto nell'esempio di sopra, all'interno dell'operazione di **grouping** è possibile utilizzare varie funzioni di aggregazione per calcolare valori aggregati sui gruppi creati. Le funzioni più comuni sono:

- **\$sum** : somma i valori di un certo attributo
- **\$avg** : calcola la media dei valori di un certo attributo
- **\$min** : calcola il valore minimo di un certo attributo
- **\$max** : calcola il valore massimo di un certo attributo
- **\$push** : crea una lista con tutti i valori di un certo attributo

Esempio: Pipeline di aggregazione su una collection 'sales'

```
1 [
2   { item: "apple", qty: 10, store: "A", region: "north"},
3   { item: "apple", qty: 5, store: "B", region: "south"},
4   { item: "banana", qty: 7, store: "A", region: "north"},
5   { item: "banana", qty: 8, store: "B", region: "south"},
6   { item: "banana", qty: 3, store: "C", region: "north"}
7 ]
```

Possiamo applicare la seguente pipeline di aggregazione:

```
1 db.sales.aggregate([
2   // filtri i vari documenti
3   { $match: { region: "north" }, // filtra per regione 'north'
4     qty: { $gte: 5 },           // e quantità maggiore o uguale a 5
5   },
6
7   {
8     $group: {
9       _id: "$item", // raggruppa per item
10      totalQty: { $sum: "$qty" }, // somma le quantità per ogni item
11      stores: { $push: "$store" }, // crea una lista di store per
12        ogni item
13      }
14    })
```

Il risultato di questa pipeline sarà il seguente:

```

1  [
2    { "_id": "apple", "totalQty": 10, "stores": ["A"] },
3    { "_id": "banana", "totalQty": 10, "stores": ["A", "C"] }
4  ]

```

JSON

È possibile andare ad applicare un'operazione nominata **aggregazione “by window”**, della quale vediamo un esempio.

Esempio: Aggregazione 'by window'

```

1  db.cakeSales.aggregate([
2    $setWindowFields: { // inizio dell'operazione di windowing
3      partitionBy: "$state", // partiziona per stato
4      sortBy: {orderDate: 1}, // ordina per data
5      output: { // definizione del campo di output
6        cumulativeQuantityForState: {
7          $sum: "$quantity",
8          window: {
9            documents: ["unbounded", "current"] // considera documenti
10             dal primo a quello corrente (somma cumulativa)
11          }
12        }
13      }
14  ])

```

JS JavaScript

Dall'esempio di sopra possiamo comprendere alcuni aspetti legati al **windowing**:

- `$setWindowFields` serve a permettere di calcolare funzioni *windowed* su ogni documento; permettendo di aggiungere o sostituire campi basati sui valori in una determinata finestra
- `$partitionBy` divide i documenti in gruppi separati su cui calcolare la funzione di finestra; in questo caso i calcoli saranno effettuati separatamente per ogni stato
- `$sum` calcola la somma dei valori del campo `quantity` all'interno della finestra specificata
- `window: {documents: ["unbounded", "current"]}` specifica che la finestra deve includere tutti i documenti dal primo fino a quello corrente, permettendo così di calcolare una **somma cumulativa**

Oltre all'aggregazione “by window” è possibile effettuare un'altra tipologia di aggregazione, detta **“bucket aggregation”**. Di seguito ne vediamo un esempio.

Esempio: Bucket Aggregation

```
1  {
2    $bucket: {
3      groupBy: <expression>, // espressione su cui basare il
        raggruppamento
4      boundaries: [ <lowerbound1>, <lowerbound2>, ... ], // definizione
        dei confini dei bucket
5      default: <literal>, // bucket di default per valori fuori dai
        confini
6      output: { // definizione dei campi di output per ogni bucket
7        <field1>: { <accumulator1> : <expression1> },
8        ...
9        <fieldN>: { <accumulatorN> : <expressionN> }
10   }
11 }
```

Per quanto le varie aggregazioni presentate possano risultare concettualmente simili è importante capire quali siano le differenze tra di esse. Andremo perciò a mostrarle nella seguente tabella.

- **group** : raggruppa secondo un valore discreto, calcolando un documento per gruppo
- **bucket** : raggruppa per intervalli di valori, calcolando un documento per intervallo (bucket) specificato
- **window** : calcola valori cumulativi o su finestre mobili, calcolando un documento per ogni documento in ingresso; la peculiarità in questo caso è che si rende necessario specificare un ordine sui documenti in ingresso

Vediamo di seguito alcuni ulteriori esempi di operazioni di aggregazione in MongoDB.

Esempio: Aggregazione 1

```
1  db.things.aggregate([
2    { $group: { _id: $parity$, sum: {$sum: $value} }}
3  ])
4
5  > { "result" : [
6    { "_id": "even", "sum": 102 },
7    { "_id": "odd", "sum": 97 }
8  ]
9  }
```

Esempio: Aggregazione 2

```
1 db.zipcodes.aggregate([  
2   { $group: {  
3     _id: "$state",  
4     totalPop: { $sum: "$pop" },  
5   }},  
6   {  
7     $match: { totalPop: { $gte: 10^6 } }  
8   }  
9 ])
```

JS JavaScript

Esempio: Aggregazione 3

La seguente query consente di trovare per ogni stato la città più grande e la più piccola in termini di popolazione:

```
1 db.zipcodes.aggregate([  
2   { $group: { _id: {"$state", city: "$city"}, pop: {$sum: "$pop"} }},  
3   { $sort: { pop: 1 } },  
4   { $group: {  
5     _id: "$_id.state",  
6     biggestCity: {$last: "$_id.city"},  
7     biggestPop: {$last: "$pop"},  
8     smallestCity: {$first: "$_id.city"},  
9     smallestPop: {$first: "$pop"}  
10  }},  
11   { $project: {  
12     _id: 0, // non mostrare il campo _id  
13     state: "$_id",  
14     biggestCity: {name: "$biggestCity", population: "$biggestPop"},  
15     smallestCity: {name: "$smallestCity", population: "$smallestPop"},  
16   } }  
17 ])
```

JS JavaScript

Esempio: Aggregazione 4

La query seguente mostra il funzionamento dell'operatore `$geoNear` all'interno di una pipeline di aggregazione:

```
1 db.places.aggregate([  
2   {  
3     $geoNear: {  
4       near: {type: "Point", coordinates: [ -73.9667, 40.78 ]},
```

JS JavaScript

```

5     distanceField: "dist.calculated",
6     maxDistance: 2,
7     query: { type: "public" }, // filtra per tipo 'public'
8     includeLocs: "dist.location",
9     spherical: true
10  }
11 }, // ... altri step della catena
12 ])
13
14 > {
15   "_id": 8,
16   "name": "Sara D. Roosevelt Park",
17   "type": "public",
18   "location": {
19     "type": "Point", "coordinates": [ -73.9935, 40.7186 ]
20   },
21   "dist": {
22     "calculated": 1.8259649934237,
23     "location": {
24       "type": "Point", "coordinates": [ -73.9935, 40.7186 ]
25     }
26   }
27 }

```

Map Reduce in MongoDB

MongoDB supporta nativamente l'utilizzo di Map-Reduce per effettuare operazioni di aggregazione sui dati memorizzati. Di seguito viene mostrata la sintassi per effettuare questa operazione.

```

1 db.collection.mapReduce(
2   <mapFunction>, // funzione di mapping
3   <reduceFunction>, // funzione di riduzione
4   {
5     out: <collection>, // nome della collection di output
6   }
7 )

```

Vediamo di seguito un esempio di utilizzo di questo comando per replicare i risultati della pipeline di aggregazione in Figura 2.5.

Esempio: Algoritmo MapReduce in MongoDB

```

1 db.orders.mapReduce(

```

```

2  function() {emit(this.cust_id, this.amount);}, // map function
3  function(key, values) {return Array.sum(values);}, // reduce function
4  {
5    query: {status: "A"}, // filtro per status 'A'
6    out: "order_totals" // output nella collection 'order_totals'
7  }
8 )

```

In generale l'algoritmo di MapReduce è estremamente flessibile e potente, tuttavia presenta alcuni svantaggi:

- Le performance sono generalmente inferiori rispetto all'utilizzo della pipeline di aggregazione, specialmente per operazioni semplici
- La scrittura delle funzioni di mapping e riduzione richiede l'utilizzo di JavaScript, il che può risultare scomodo per chi non ha familiarità con questo linguaggio
- La manutenzione del codice può risultare più complessa rispetto all'utilizzo della pipeline di aggregazione, specialmente per operazioni complesse

Alcune evoluzioni di MongoDB

A partire dalla versione 3.2 di MongoDB sono state introdotte alcune funzionalità che vanno a migliorare le capacità di aggregazione del sistema. In particolare è stata introdotta la possibilità di utilizzare un **left-outer join**. L'utilizzo di questo operatore è possibile all'interno delle pipeline di aggregazione combinato a tutti gli altri operatori già visti e viene utilizzato tramite il comando **lookup**. Il comportamento che ci attendiamo da questo operatore è visibile in Figura 2.6.

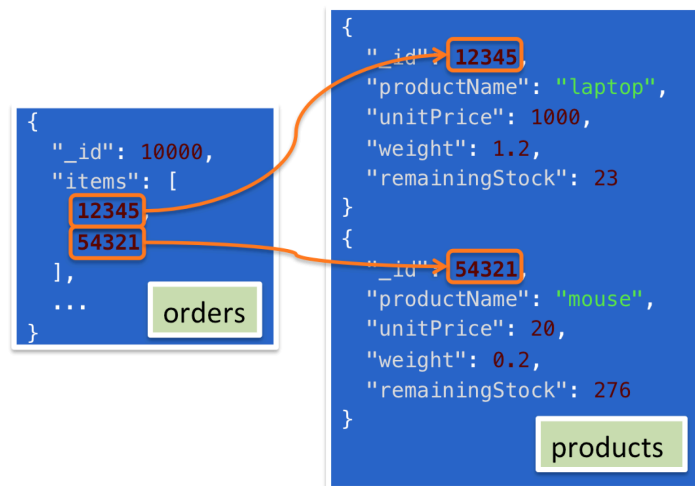


Figura 2.6: Esempio di utilizzo dell'operatore `$lookup` per effettuare un left-outer join

Di seguito andiamo a mostrare la sintassi del comando

```
1 {
2   $lookup: {
3     from: <collection to join>,
4     localField: <field from the input documents>,
5     foreignField: <field from the documents of the "from"
6     collection>,
7     as: <output array field>
8   }
}
```

A partire da MongoDB 3.4 e 3.6 sono state introdotte ulteriori funzionalità:

- **graphLookup** : consente di eseguire ricerche ricorsive all'interno di una gerarchia di documenti, permettendo di esplorare relazioni complesse tra i dati (**aggregazione ricorsiva**)
- **lookup** con condizioni di join multiple
- **views**: permettono di creare viste virtuali basate su query di aggregazione, consentendo di presentare i dati in modi diversi senza duplicarli fisicamente

Altre importanti funzionalità introdotte nelle versioni successive includono:

- **Transazioni multi-documento**
- **Transazioni distribuite**: consentono di eseguire transazioni che coinvolgono più shard in un cluster distribuito
- **Views materializzate**: viste che memorizzano fisicamente i risultati di una query di aggregazione per migliorare le performance
- **Unione di Pipeline**: consente di combinare i risultati di più pipeline di aggregazione in un'unica pipeline
- **Funzioni di aggregazione personalizzate**: permette di definire funzioni di aggregazione personalizzate per operazioni specifiche non coperte dalle funzioni predefinite
- **Supporto alle time series**: ottimizzazioni specifiche per la gestione di dati temporali, come serie storiche di misurazioni o eventi

2.2.5. Todo Maybe: forse torneremo qui per parlare di sharded deployments

3. Column Stores

Fino a questo momento abbiamo di varie tipologie di basi, ovvero modelli relazionali, key-value stores e document stores. Tra queste tipologie il modello NoSQL si rendono necessari nel momento in cui non serve più garantire le proprietà ACID, ma si preferisce garantire efficienza e velocità di accesso ai dati.

In tutti i precedenti capitoli è sempre stata discussa la **modalità di interazione** con il **data model** per ognuna delle categorie viste ma non è mai stato mostrato **come** i dati vengano effettivamente memorizzati all'interno dei database.

Prima di introdurre i **column stores** è necessario fare una breve digressione dove andremo a vedere come i dati vengono memorizzati all'interno dei database relazionali, useremo i database relazionali dal momento che sono quelli con la quale la maggior parte ha familiarità. Tutto questo verrà ulteriormente approfondito in uno dei successivi capitoli.

3.1. Architettura di un R-DBMS

Quello che vediamo in Figura 3.1 è una rappresentazione dell'organizzazione interna delle componenti di un DBMS relazionale.

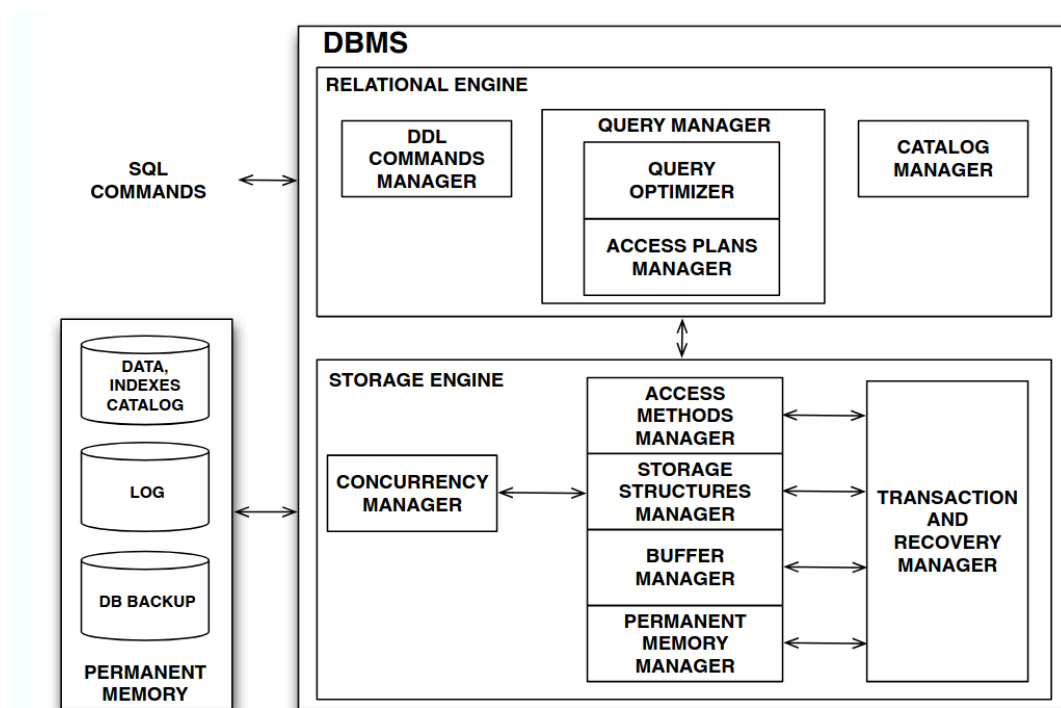


Figura 3.1: Organizzazione interna delle componenti di un R-DBMS

Ciò che è importante notare è che possiamo distinguere due macro aree:

- La parte superiore dell'immagine rappresenta la parte logica del database, anche detta **relational engine**, la quale si occupa di gestire le query, l'ottimizzazione delle stesse e la gestione delle transazioni e accede alla parte inferiore tramite delle API
- La parte inferiore dell'immagine rappresenta la parte fisica del database, anche detta **storage engine**, la quale si occupa di gestire l'effettiva memorizzazione dei dati

Possiamo notare come la gestione delle transazioni sia demandata alla parte dello storage engine, il quale è l'unico componente che ha accesso diretto al file system. Ciò che è importante per questo capitolo è comprendere il funzionamento delle seguenti componenti:

- **Storage structures manager**
- **Buffer manager**
- **Persistent memory manager**

Andremo ad analizzare il funzionamento di queste componenti nelle sezioni successive.

Persistent Memory Manager

La componente del **persistent memory manager** si occupa di **astrarre** i dispositivi fisici di memorizzazione fornendone una **vista logica**. Tipicamente la memoria che viene gestita si può vedere costituita di due livelli:

- **Memoria principale** (RAM), che è molto veloce (10-100ns), piccola (nell'ordine dei GigaByte), volatile e molto costosa
- **Memoria secondaria** (Dischi), che è al contrario permanente, di più grande capacità (svariati TeraByte) ed economica ma molto più lenta (5-10ms).

Il principale motivo della latenza nell'accesso ai dati su un disco fisico (hard disk) è dovuto al fatto che il disco è un dispositivo meccanico, dunque per accedere fisicamente ai file è necessario accedere alla posizione dei dati richiesti muovendo la testina sulla porzione di disco corretta. Dal momento che questo movimento è estremamente lento rispetto al resto del sistema, è ottimale cercare di minimizzare il numero di accessi, cercando di trasferire la maggior quantità di dati utili possibile.

Gestore del Buffer

Il compito del **buffer manager** è quello di trasferire pagine di dati tra la memoria principale e quella persistente, fornendo un'astrazione della memoria persistente come un insieme di pagine utilizzabili in memoria principale, nascondendo di fatto il meccanismo di trasferimento dei dati tra memoria persistente e *buffer pool*.

L'obiettivo principale di questo componente è quello di evitare il più possibile che vengano effettuate letture ripetute di una stessa pagina, cercando di mantenere una sorta di **cache**, e di minimizzare il numero di scritture su disco.

L'utilizzo di questo componente è fondamentale per poter sfruttare al meglio il principio della località spaziale e temporale, ovvero il fatto che se un dato viene richiesto, è probabile che vengano richiesti anche dati "vicini" (spaziale) o che lo stesso dato venga richiesto più volte in un breve lasso di tempo (temporale).

Strutture di Memorizzazione

In questa sezione andiamo a vedere come vengono memorizzati i record all'interno di una base di dati. In generale possiamo dire che i dati vengono salvati su **file** ovvero degli oggetti che siano **linearmente indirizzabili** (tramite degli indirizzi di un certo numero di byte).

Sappiamo che all'interno di una base di dati relazionale, i dati all'interno di una tabella sono organizzati in **record**, dove ogni record è costituito da un insieme di **fields** (o attributi), ognuno di questi può essere strutturato in maniera diversa:

- Lunghezza fissa: il campo occupa sempre lo stesso numero di byte (es. `integer`, `float`, `date`)
- Lunghezza variabile: il campo può occupare un numero variabile di byte (es. `char(n)`, `varchar(n)`, `text`)

I record possono inoltre essere separati tra loro tramite un **delimitatore** (es. nei file `csv`) o ancora possono avere un prefisso speciale che ne indica la lunghezza: `record = (prefix, fields)`, dove il prefisso può contenere diversi tipi di informazioni:

- La lunghezza in byte della parte del valore
- Il numero di componenti
- L'offset di inizio del record successivo
- Altre informazioni di controllo

Normalmente i record vengono memorizzati all'interno di una **pagina di memoria** e per ognuno viene memorizzato un **physical record identifier** che ne indica la posizione tramite le seguenti informazioni:

- Page number: il numero della pagina di memoria
- Offset: la posizione del record all'interno della pagina

Il modo più semplice in cui i record possono essere memorizzati all'interno di una pagina è quello di utilizzare un approccio **seriale**, ovvero memorizzare i record uno di seguito all'altro. Questo approccio però non consente di accedere ai dati in maniera efficiente.

Supponiamo per esempio che per una relazione siano presenti dei campi che sono più importanti di altri, per esempio l'età di una persona (ad esempio in un contesto in cui si voglia fare analisi demografica). Se riuscissimo a memorizzare i record in maniera ordinata rispetto a questo campo, potremmo velocizzare di molto le operazioni di lettura che coinvolgono questo campo (ad esempio tutte le persone con età maggiore di 30 anni). Questo approccio prende il nome di **sequenziale**. Il problema di questo approccio è che le operazioni di **inserimento**, cancellazione e aggiornamento diventano molto più complesse e costose dal momento che è necessario mantenere l'ordinamento. Tutto ciò che viene dopo il record inserito deve essere spostato in avanti.

3.2. Column Stores

I **column stores** (o **columnar databases**) sono una tipologia di basi di dati NoSQL che memorizzano i dati organizzandoli per colonne invece che per righe come avviene nei database relazionali tradizionali (row stores).

In un database relazionale tradizionale, nel momento in cui siamo interessati a leggere uno specifico valore di un record, siamo costretti a leggere l'intero record e scartare tutto ciò che non serve allo scopo della query in esame; questo è particolarmente inefficiente nel momento in cui i record sono composti da molti campi. Consideriamo per esempio la seguente relazione `BookLending`:

BOOKLENDING	BOOKID	READERID	RETURNDATE
	123	225	25-10-2016
	234	347	31-10-2016

All'interno di un **row store** i dati verrebbero memorizzati in questo modo: 123, 225, 25-10-2016, 234, 347, 31-10-2016,

Utilizzando un column store, questa operazione diventa molto più efficiente, dal momento che ci permettono di leggere solo i campi di interesse. Questo approccio è particolarmente vantaggioso in scenari di **analisi dei dati** e **data warehousing**, dove spesso si eseguono query che coinvolgono solo un sottoinsieme di colonne su grandi volumi di dati. La stessa tabella di sopra, all'interno di un **column store** verrebbe memorizzata nel modo seguente: 123, 234, 225, 347, 25-10-2016, 31-10-2016,

Come anticipato nella sezione precedente è possibile andare ad utilizzare questo approccio indipendentemente dal modello dei dati utilizzato, dunque è possibile trovare column stores che implementano modelli relazionali, key-value o documentali.

3.2.1. Vantaggi e Svantaggi dei Column Stores

Andiamo di seguito a specificare tutti i pro e contro dell'adozione di un column store:

- Soltanto i dati che sono effettivamente necessari vengono letti dal disco nella memoria principale, dal momento che le pagine non contengono più record, bensì colonne, riducendo così il carico di I/O ✓
- I valori di una colonna hanno tutti lo stesso dominio e possono essere **meglio compressi** grazie alla località dei dati ✓
- **Iterazione** e **aggregazione** di valori (es. calcolo di somma, media, massimi e minimi valori di una colonna) possono essere effettuate in maniera veloce, dal momento che i valori sono stati memorizzati in maniera contigua ✓
- Aggiungere **nuovi campi** ad una relazione (o documento) è molto semplice ✓
- **Combinare** i valori da diverse colonne è particolarmente costoso, dal momento che è necessario capire quali valori nelle colonne corrispondono ad una stessa tupla ✗
- Aggiungere **nuovi record** è particolarmente costoso, dal momento che è necessario andare ad aggiornare tutte le colonne ✗

3.2.2. Compressione delle colonne

Un algoritmo di compressione serve a prendere dei dati in input e a trasformarli in una rappresentazione più compatta in modo da ridurre lo spazio di memorizzazione necessario. Esistono diversi algoritmi di compressione, in particolare li possiamo dividere in algoritmi **lossless** (senza perdita di informazione) e **lossy** (con perdita di informazione). Nel contesto delle basi di dati, è di fondamentale importanza utilizzare algoritmi **lossless**, dal momento che non è accettabile perdere informazioni.

Esistono diversi modi di applicare compressione sui dati memorizzati in un column store, tutti questi sfruttano il fatto che i dati all'interno di una colonna sono spesso **ripetuti** o **simili** tra loro. Nelle prossime sezioni andremo a vedere gli approcci più comuni.

Run-Length Encoding (RLE)

Si tratta di un algoritmo di compressione alla base del quale c'è il principio di contare quante volte consecutive un certo valore viene ripetuto. Dopo la compressione, ogni valore verrà rappresentato dalla tupla che segue: (value, start row, run-length) . Figura 3.2 mostra un esempio del funzionamento di questo algoritmo.

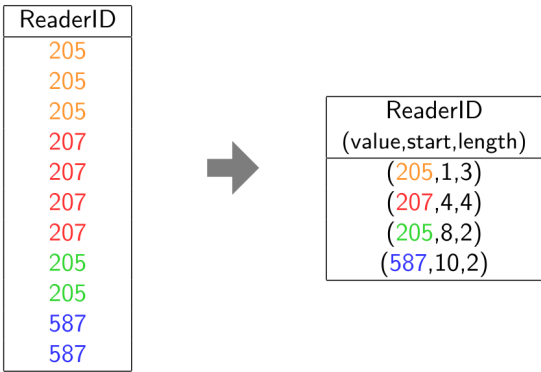


Figura 3.2: Esempio di Run-Length Encoding

 Osservazione

Apparentemente mantenere memorizzata anche l'informazione legata al valore di start di un record sembra superfluo, tuttavia questa informazione si rivela fondamentale nel momento in cui vogliamo andare a ricostruire dati parziali.

Uno dei primi strumenti che hanno utilizzato questo tipo di encoding è stato quello per comprimere immagini bianco e nero in formato PBM (Portable Bitmap). Il grande vantaggio dell'utilizzo di questo approccio consiste nel forte tasso di compressione.

Bit Vector Encoding

In questo algoritmo di compressione, per ogni valore tra quelli presenti nella colonna viene creato un vettore con un bit per ogni riga. Se il valore è presente allora il bit viene settato a 1, altrimenti a 0. Figura 3.3 mostra un esempio del funzionamento di questo algoritmo.

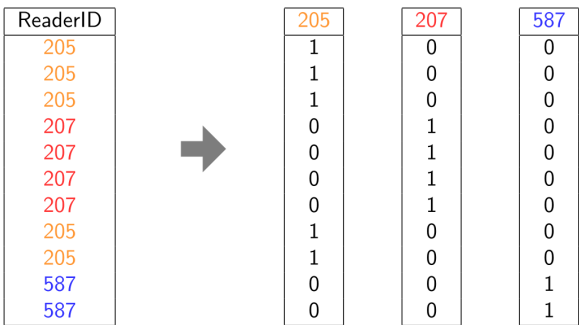


Figura 3.3: Esempio di Bit-vector encoding

Chiaramente, l'utilizzo di questo approccio risulta più oneroso in termini di spazio utilizzato rispetto a RLE, ma consente di effettuare facilmente operazioni di **AND**, **OR** e **NOT** tra vettori, il che lo rende particolarmente adatto per operazioni di filtro e selezione.

Dictionary Encoding

In questo algoritmo di compressione, ogni valore viene rimpiazzato ogni valore con valori che siano rappresentabili in maniera più efficiente. Per esempio, effettuando le seguenti associazioni: 1 → 205, 2 → 207 e 3 → 587 otteniamo la rappresentazione di Figura 3.4.

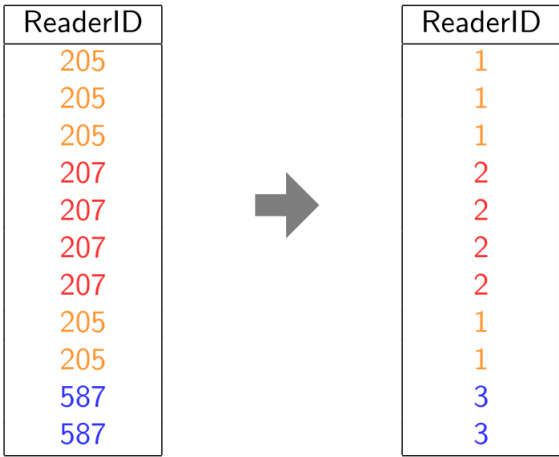


Figura 3.4: Esempio di dictionary-encoding

Questo approccio risulta particolarmente vantaggioso nel momento in cui i valori presenti (e ripetuti) all'interno di una colonna sono particolarmente lunghi o complessi, come stringhe di testo o strutture dati complesse. Chiaramente nel momento in cui si voglia operare sulla colonna sarà necessario andare a ricostruire i valori originali utilizzando il dizionario.

È anche possibile andare ad utilizzare questo approccio per **sequenze** di valori, andando a mappare intere sequenze a valori più compatti; questo approccio risulta di complicata implementazione ma può portare a tassi di compressione molto elevati. Per esempio effettuando il mapping 1 → 1002, 1010, 1004, 2 → 1008, 1006 otteniamo la rappresentazione di Figura 3.5

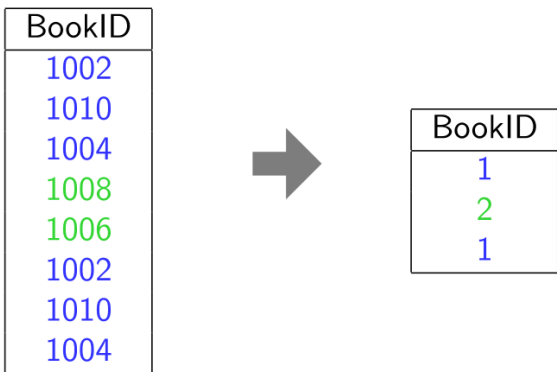


Figura 3.5: Esempio di dictionary-encoding per sequenze

Frame of Reference Encoding (FoR)

In questo algoritmo di compressione, viene scelto un **valore di riferimento** (es. minimo, mediano, massimo, ...) e ogni valore della colonna viene rappresentato come un **offset** (positivo o negativo) rispetto al riferimento.

È possibile scegliere che ci sia un valore massimo di offset dopo il quale i valori non vengano più compressi e marcati con un simbolo speciale. Questo potrebbe essere particolarmente utile per identificare e gestire i **valori anomali** (outliers). Figura 3.6 mostra un esempio del funzionamento di questo algoritmo.

ReturnDate		ReturnDate Reference: 25-10-2016
22-10-2016		-3
27-10-2016		+2
25-10-2016		0
22-10-2016		-3
28-10-2016		+3
14-10-2016		# 14-10-2016
26-10-2016		+1
21-10-2016		-4
25-10-2016		0

Figura 3.6: Esempio di Frame-of-Reference encoding con valori oltre l'offset massimo

Una variante di questa tecnica di encoding è data dal **differential encoding**, dove invece di memorizzare l'offset rispetto ad un valore predefinito, ogni valore viene memorizzato come differenza rispetto al valore precedente. Questo approccio potrebbe aiutare specialmente nella riduzione del numero di valori che sono fuori dall'offset massimo. Figura 3.7 mostra un esempio del funzionamento di questo algoritmo.

ReturnDate		ReturnDate
22-10-2016		22-10-2016
27-10-2016		+5
25-10-2016		-2
22-10-2016		-3
28-10-2016		+6
14-10-2016		# 14-10-2016
26-10-2016		-2
21-10-2016		-5
25-10-2016		+4

Figura 3.7: Esempio di encoding differenziale con valori oltre l'offset massimo

Osservazione

Si noti come in Figura 3.7 il valore successivo al valore fuori dall'offset sia preso a partire dal primo valore “valido”. Questo potrebbe tornare particolarmente utile per trovare outlier.

Altre tecniche di compressione

Ci possiamo trovare davanti a situazioni in cui abbiamo colonne con molti **valori nulli**, se vogliamo effettivamente essere in grado di ricostruire colonne con questi valori è comunque necessario memorizzarli. Esistono diverse tecniche per andare a comprimere questi valori:

- **Bitmap encoding**: viene creato un vettore di bit dove ogni bit indica se il valore in quella riga è nullo o meno. I valori nulli non vengono memorizzati
- **Sparse encoding**: vengono memorizzate solo le righe che contengono valori non nulli, insieme ai loro identificatori di riga
- **Run-length encoding per nulli**: viene applicato un algoritmo di run-length encoding specifico per i valori nulli, memorizzando le sequenze di valori nulli in modo compatto
- **Dictionary encoding per nulli**: viene creato un dizionario che include un'entrata speciale per i valori nulli, permettendo di rappresentarli in modo compatto

È possibile andare a rappresentare un **documento complesso** (es. JSON) utilizzando un approccio colonnare. Possiamo vedere il documento come un albero e andare a memorizzare *una colonna per ogni path root-leaf* dell'albero. In questo modo è possibile andare a memorizzare in maniera efficiente anche documenti con strutture molto complesse. Questa tecnica è nota con il nome di **column striping**.

Query su Dati Compressi

In questa sezione andiamo a concentrarci su quale approccio sia necessario per effettuare query su dati compressi. In alcune situazioni è possibile effettuare le operazioni di cui abbiamo bisogno direttamente sui dati compressi. Ne vediamo di seguito un esempio.

Esempio: Query su dati compressi

Supponiamo di avere effettuato una compressione *run-length encoding* sulla colonna `ReaderID` di una tabella `BookLending`, ottenendo la seguente compressione:

$((205, 0, 3), (207, 4, 2), (206, 6, 1))$

Supponiamo di avere la seguente query in linguaggio naturale: “*Quanti libri ha ciascun lettore?*”, che si può tradurre in SQL tramite il seguente codice:

```
1 SELECT ReaderID, COUNT(*)
2 FROM BookLending
3 GROUP BY ReaderID
```

Utilizzando l'encoding run-length è possibile semplicemente ritornare la somma dei run-length per ogni valore di `ReaderID`, ottenendo così il risultato della query senza dover de-comprimere i dati:

$\{(205, 4), (207, 2)\}$

Osservazione

Si noti come nell'esempio in precedenza, se non avessimo avuto a disposizione i run-length, avremmo dovuto de-comprimere l'intera colonna e poi effettuare il conteggio per ogni ReaderID, il che sarebbe stato molto più costoso in termini di tempo di calcolo. Questo ci suggerisce come sia importante scegliere algoritmi di compressione adeguati al tipo di query che ci aspettiamo di dover eseguire sui dati.

3.2.3. Alcuni Prodotti Commerciali

Di seguito sono elencati alcuni dei più noti prodotti commerciali che implementano uno storage engine colonnare:

- **Monet DB:** si tratta del primo database basato su storage engine colonnare, supporta il modello relazionale.
- **Maria DB ColumnStore:** si tratta di un'estensione di MariaDB che implementa uno storage engine colonnare, supporta il modello relazionale.
- **Apache Parquet:** si tratta di un formato di file che implementa uno storage engine colonnare, utilizzato principalmente in ambito big data.

4. Extensible Record Stores

Nello scorso capitolo abbiamo visto come a volte un cambio di paradigma dello **storage engine** possa portare a dei grandi benefici in termini di prestazioni sfruttando la *località dei dati*. Vogliamo provare a spingere questo concetto ancora oltre, andando a considerare un particolare tipo di database NoSQL chiamato **extensible record store** (ERS).

Ipotizziamo di dover gestire una base di dati che memorizzi informazioni riguardanti delle persone, ogni persona avrà delle informazioni che sono particolarmente importanti, come ad esempio la data e il luogo di nascita, la città in cui questa ha residenza e così via. Probabilmente tante di queste informazioni saranno accedute “assieme” (es. difficilmente avremo bisogno di sapere la data di nascita senza sapere anche il luogo di nascita).

4.1. Modello Logico dei Dati

In questa sezione andremo a vedere come è possibile modellare i dati in un sistema di questo tipo. È importante andare a vedere le seguenti regole, che si pongono alla base del funzionamento degli extensible record store:

- Le colonne sono raggruppate in insiemi detti **column families**, le quali hanno lo scopo di raggruppare le colonne che sono spesso accedute assieme
- Ogni column family deve essere creata prima che possa essere utilizzata (similmente a quanto avviene con una tabella SQL)
- All'interno di una column family è possibile aggiungere nuove colonne in maniera arbitraria, senza dover modificare uno schema predefinito
- Ogni riga di una column family può avere un insieme di colonne diverso dalle altre righe della stessa column family

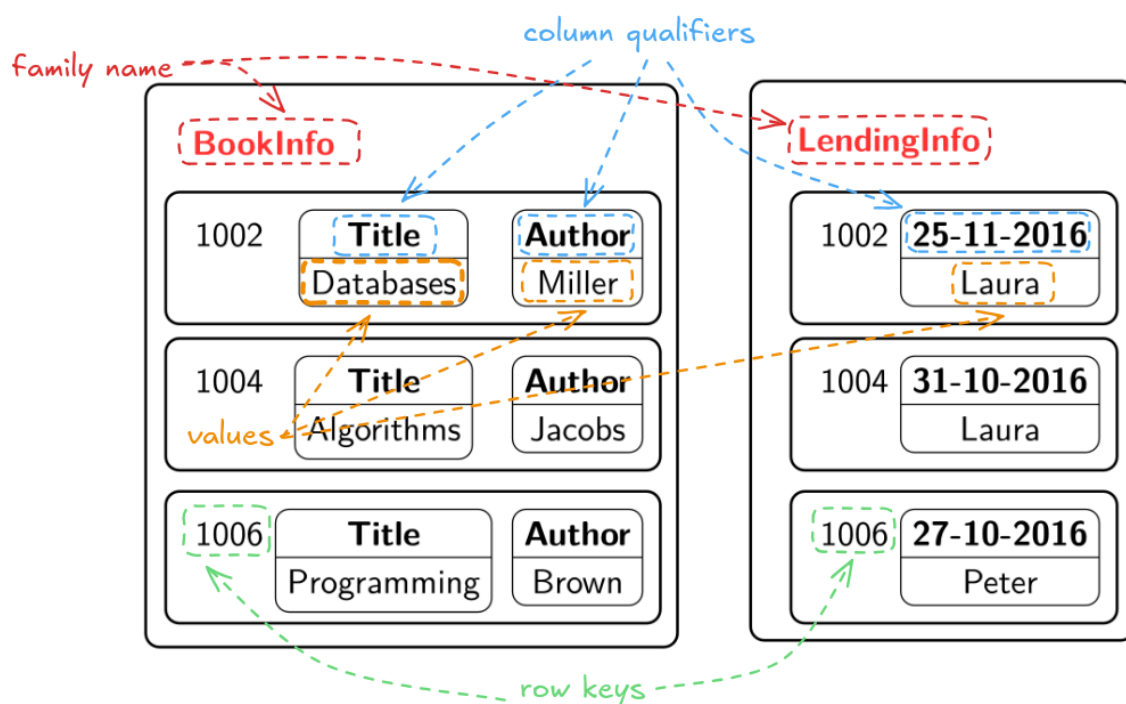


Figura 4.1: Terminologia degli Extensible Record Stores

Figura 4.1 mostra con chiarezza i vari elementi che troviamo all'interno un extensible record store: andiamo a definirli brevemente:

- Una **column family** è un insieme di colonne che sono spesso accedute assieme
- Una **row key** è l'identificativo univoco di una riga all'interno di una column family
- Un **column qualifier** è il nome di una colonna all'interno di una column family

Osservazione

Tipicamente i *column qualifiers* sono dei metadati e sono **costanti**, tutta via in Figura 4.1 è stato scelto di utilizzare una data come qualificatore per una colonna. Questo è possibile dal momento le colonne sono arbitrarie e possono avere valori diversi per ogni riga (quindi nulli nel caso di dati non pertinenti). Il motivo per cui questo è stato fatto è che se possiamo accedere ai dati tramite nome di una colonna, è possibile accedere a tutti i prestiti di un giorno in maniera immediata.

Per quanto riguarda la memorizzazione, fino a questo momento ci basta sapere che data una column family, ogni row viene memorizzata in maniera consecutiva a quella precedente, questo consente di ottenere **località spaziale**.

Accesso ai Valori di una Colonna

L'accesso a un valore di una colonna è possibile tramite l'utilizzo della sua **full key**:

```
<row key>:<column family>:<column qualifier>
```

Per esempio, con riferimento all'immagine in Figura 4.1, la full key `1006.BookInfo.Author` identifica in maniera univoca il valore `Brown`.

È inoltre possibile andare ad aggiungere una dimensione ulteriore, quella **temporale**, aggiungendo alla full key un *timestamp*, questo garantisce che sia possibile avere diverse versioni dello stesso dato, ad esempio per tenere traccia delle modifiche avvenute nel tempo.

4.2. Storage Fisico dei Dati

In questa sezione andremo a vedere più nel dettaglio, dato il modello logico precedentemente descritto, come i dati vengano effettivamente memorizzati sul disco. Il principio fondamentale di questa tipologia di basi di dati è quello di consentire alta velocità in scrittura, anche al costo di avere delle letture un po' più lente.

4.2.1. Flusso delle operazioni di scrittura

Tipicamente i dati più recenti vengono scritti all'interno di una struttura chiamata **memtable** che risiede in memoria principale. Tipicamente viene mantenuta una memtable per ogni column family. Ogni record nella memtable viene identificato tramite la sua full key (`row + column family + qualifier + timestamp[ms]`).

Una volta che una memtable è completa, avviene un'operazione di **flushing**, che consiste nel salvare i dati in memoria su disco. Potremmo tranquillamente scrivere i dati in un unico file, mettendo i dati nuovi in coda a quelli inseriti per ultimi, ma questo porterebbe a dover scorrere tutto il file per trovare un dato specifico, risultando in un costo estremamente elevato in

lettura. Questo costo potrebbe essere notevolmente abbassato memorizzando i dati in maniera ordinata, ma questo, utilizzando un singolo file, comporterebbe il dover ogni volta cercare la posizione di ogni dato da scrivere e il dover spostare i dati in avanti per fare spazio a quelli nuovi, risultando in *scritture lente*.

Per mitigare il costo in lettura, viene scelto di effettuare un'operazione di **ordinamento** su ogni memtable e di memorizzare la tabella ordinata in un singolo file in memoria. Questo è il motivo per cui sostanzialmente ha senso avere una memtable per ogni column family: il fatto che l'ordinamento possa essere effettuato sulla singola column family. In Figura 4.2 è mostrato il flusso delle operazioni di scrittura in un extensible record store.

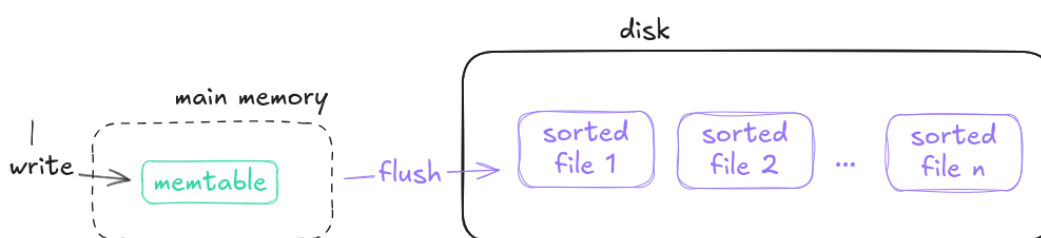


Figura 4.2: Flusso delle operazioni di scrittura in un Extensible Record Store

Osservazione

Si noti come una struttura di questo genere possa essere estesa in maniera estremamente efficace ad un'architettura distribuita e ad un approccio parallelizzabile, rendendo di fatto la ricerca ancora più efficiente, lasciando ogni server a lavorare sul suo o i suoi file ordinati.

4.2.2. Modifica e Cancellazione dei Dati

Per garantire che le scritture siano il più veloci possibili, è chiaro che anche le operazioni di modifica e/o cancellazione debbano essere diverse. Sarebbe impensabile andare a cercare il record da modificare in memoria principale e cambiarlo, dal momento che questo potrebbe essere scritto in qualsiasi posto del disco.

Per questo motivo viene imposto il vincolo che i dati memorizzati siano **immutabili** e che le modifiche di un record possano soltanto essere “simulate” andando a scrivere un nuovo record con la stessa full key e un timestamp più recente. In questo modo, quando si andrà a leggere un record, si prenderà sempre quello con il timestamp più recente, che sarà quello valido.

Osservazione

Questa modalità di effettuare l'aggiornamento dei dati, permette di avere senza costi aggiuntivi l'implementazione di una sorta di **versioning** dei record.

Se per andare a **modificare** dei dati basta semplicemente scrivere un nuovo record con un nuovo timestamp, per effettuare la **cancellazione** è possibile utilizzare un meccanismo simile, andando ad aggiungere un nuovo record con la stessa full key e un timestamp più recente, ma con un particolare valore chiamato **tombstone** che indica che quel record è stato cancellato. Di seguito andiamo ad elencare le varie caratteristiche di questi valori:

- I tombstone vanno a *mascherare* tutti i record precedenti con la stessa full key e timestamp precedente a quello del tombstone
- Esistono diversi tipi di tombstone, a seconda del tipo di dato che si vuole cancellare:
 - si può cancellare una singola **versione** di una colonna, andando a scrivere il **timestamp** relativo alla versione che si vuole cancellare
 - è possibile cancellare l'intera **colonna** con tutte le versioni
 - è possibile andare a cancellare un'intera **column family** andando di fatto a rimuovere tutte le versioni di tutte le colonne
- Nuove versioni per una stessa chiave possono essere scritte anche dopo la cancellazione, in questo caso il nuovo record non sarà mascherato dal tombstone

4.2.3. Flusso di Lettura dei Dati

Come già anticipato, le operazioni di lettura in un sistema di questo genere sono più lente e complesse rispetto a quelle di scrittura. Questo perché i dati sono memorizzati in maniera *sparsa* su disco, e per trovare un dato specifico potrebbe essere necessario andare a leggere più file.

In particolare, per leggere un dato specifico si rende necessario **combinare** i dati che provengono sia dalla memtable (dati più recenti) sia dai file memorizzati su disco. La versione corretta dei dati da recuperare è quella più recente, informazione che sarà accessibile tramite timestamp. Figura 4.3 mostra chiaramente questo processo.

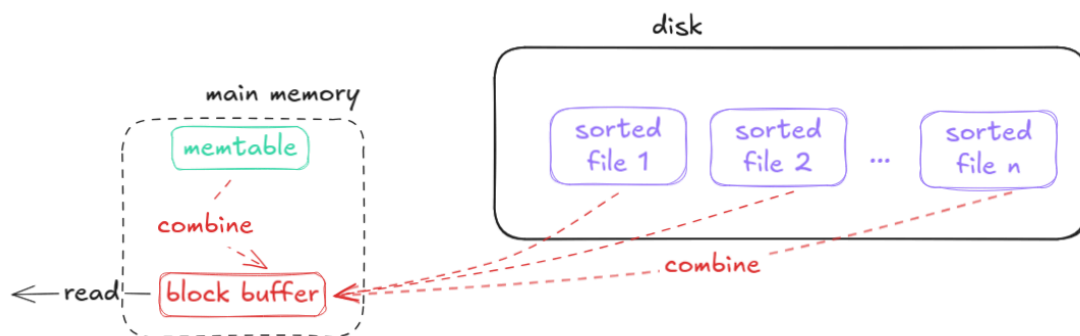


Figura 4.3: Flusso delle operazioni di lettura in un Extensible Record Store

Formato dei File

Andiamo ora a concentrare l'attenzione su come i dati vengono rappresentati all'interno dei file ordinati in memoria secondaria. Iniziamo a vedere come è strutturato un **file** (struttura ordinata proveniente dal flush della memtable):

- Ogni file si può vedere come composto da molti **blocchi** (data blocks)
- Dal momento che all'interno di un file sono presenti molti blocchi viene tenuto un **indice** che consente di trovare rapidamente il blocco che contiene il dato cercato
- Insieme all'indice viene mantenuto un **trailer** che consente di memorizzare informazioni per la gestione del file, come ad esempio la posizione dell'indice stesso

Osservazione

Il motivo per cui l'indice viene inserito in fondo al file, è legato al fatto che questa scelta consente una scrittura su disco più veloce: possiamo costruire l'indice man mano che scriviamo i record uno dopo l'altro, invece di dover prima leggere tutti i record da scrivere per poter inserire l'indice in testa.

Ogni **data block** è sostanzialmente composto da una lista di **key-value** pairs. Ogni key è la full key del record (row key + column family + column qualifier + timestamp) e il value è il valore associato a quella specifica chiave. Andando più nel dettaglio, queste key-value pairs sono memorizzate assieme ad un campo **type** che ci serve a distinguere se il record è un record normale *inserimento*, una *modifica* o una *cancellazione* (tombstone). Figura 4.4 mostra la gerarchia appena descritta.

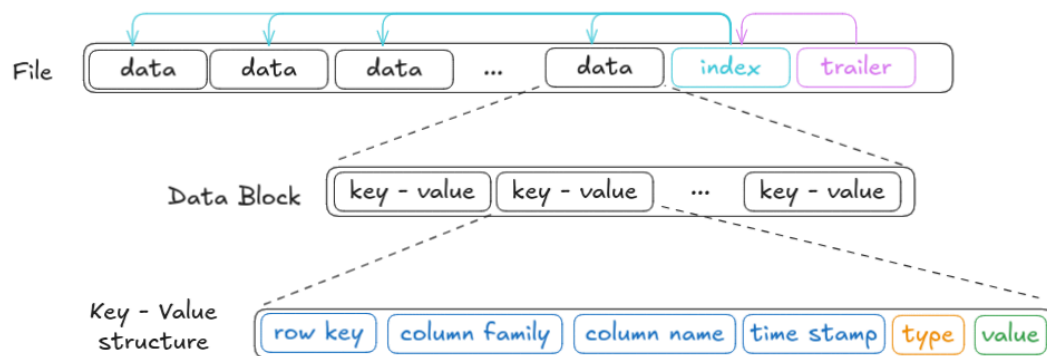


Figura 4.4: Formato di un file in un Extensible Record Store

4.2.4. Ulteriori Accorgimenti

Write-Ahead Logging

Il sistema progettato fino a questo momento è estremamente vulnerabile nel caso di **crash improvvisi**: tutto ciò che è presente in memtable non sarebbe infatti recuperabile. Per garantire che i dati non vadano persi in questi casi, viene utilizzata una tecnica chiamata **write-ahead logging**. Sostanzialmente ogni operazione di scrittura viene effettuata due volte: una volta su un **file di log** memorizzato su disco, e una seconda volta nella **memtable**.

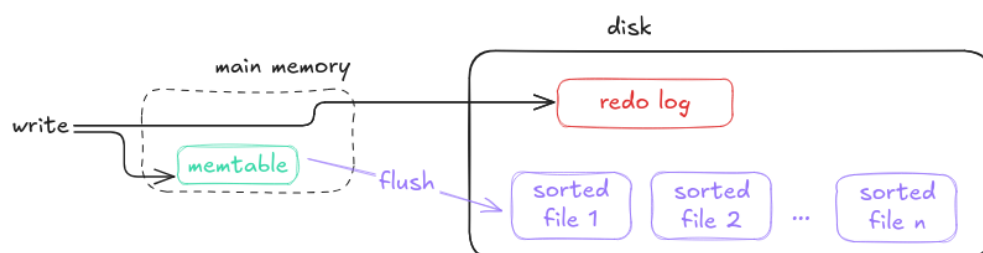


Figura 4.5: Write-Ahead Logging in un Extensible Record Store

Utilizzando questo approccio, in caso di system crash, è possibile recuperare tutte le operazioni di scrittura effettuate leggendo il file di log e reinserendole nella memtable. Figura 4.5

mostra questo processo. In pratica, il **redo log** viene svuotato ogni qualvolta che avviene un'operazione di flushing della memtable su disco. Se al verificarsi di un crash di sistema il log contiene dei record, significa che è necessario ripristinare lo stato.

Compattazione dei File

Dopo che si sono verificate **molte operazioni di flushing**, si verranno a creare **molti file** ordinati su disco. Questo comporta che le *operazioni di lettura* diventino *sempre più lente*. Questo perché per leggere un dato specifico, potrebbe essere necessario andare a leggere molti file diversi, andando così a vanificare il vantaggio di avere i dati ordinati all'interno di ogni singolo file.

Per capire l'idea alla base di questo approccio, è necessario considerare i seguenti punti:

- Nel momento in cui cerchiamo una chiave, dobbiamo cercarla all'interno di ogni file presente all'interno del disco
- Il costo di ricerca della chiave all'interno di un file consiste nella ricerca del trailer (1 accesso a pagina), della ricerca dell'indice (1 accesso a pagina) e nel caso in cui la chiave sia presente nel file, il costo di ricerca all'interno del blocco dati (1 accesso a pagina), per un totale di massimo 3 pagine per file
- Possiamo concludere che il costo della ricerca di una chiave sia dominato dal numero di file presenti su disco $O(n)$, dove n è il numero di file (o di operazioni di flushing effettuate fino ad un certo punto)

L'idea potrebbe essere quella di andare a **ridurre il numero di file**, per fare ciò, senza perdita di informazione, la soluzione consiste nel **combinare** più file all'interno di un unico file più grande. La creazione di file più grandi nativi non è supportata, dal momento che la dimensione di questi dipende dalla dimensione della *memtable*. Dal momento che effettuare ordinamento su disco è particolarmente costoso, questa operazione viene effettuata soltanto l'effort necessario è compensato dall'aumento di velocità in lettura.

Durante l'operazione di **compaction** è possibile anche decidere di eliminare i record che sono stati marcati come cancellati tramite tombstone, così da liberare spazio su disco.

Osservazione

Possiamo associare questa operazione di compactazione a quella di **deframmentazione di un disco** o a quella di **ricostruzione dell'indice** di una base di dati relazionale.

Osservazione

Dal momento che i file che stiamo riunendo in un unico file ordinato sono già rispettivamente ordinati, possiamo applicare un comunissimo algoritmo di ordinamento: il **merging**, che corrisponde alla seconda fase dell'algoritmo *merge-sort*.

Figura 4.6 mostra il flusso di operazioni che avvengono durante una compactazione di file in un extensible record store.

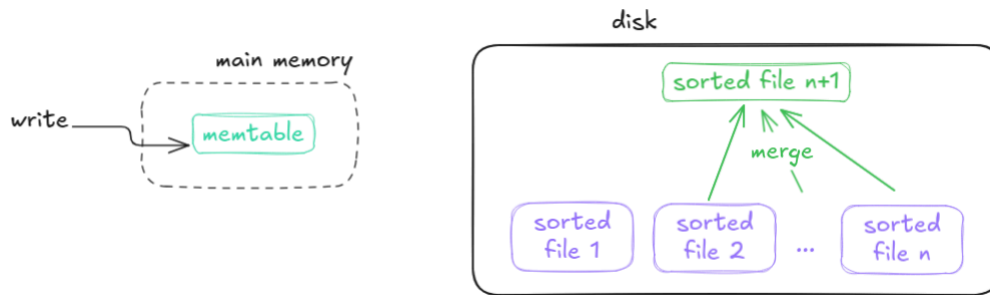


Figura 4.6: Compattazione dei file in un Extensible Record Store

Bloom Filters

Per migliorare ulteriormente le prestazioni in lettura, è possibile applicare una nuova tecnica chiamata **bloom filter**. Si tratta di un *meccanismo probabilistico* utilizzato per determinare l'appartenenza ad un insieme. In questo specifico caso viene utilizzato per determinare se una **chiave** è presente o meno all'interno di un **file specifico** o direttamente in un **data block**.

Questo filtro viene posizionato tra il trailer e l'indice all'interno del file contenente i vari data block e viene memorizzata la sua posizione all'interno del trailer in modo che sia di facile accesso. In Figura 4.7 è mostrata la nuova struttura.



Figura 4.7: Bloom Filter all'interno dei file in un Extensible Record Store

Il workflow di lettura di un dato specifico con l'utilizzo del bloom filter è il seguente:

- Viene *letto il trailer* per recuperare la posizione dell'indice e del bloom filter
- L'indice indica quali siano le chiavi minima e massima di ogni data block
- Se esiste un range che può contenere la chiave cercata, si legge il data block, cercando la chiave al suo interno
- Il bloom filter viene utilizzato per determinare se la chiave è presente o meno all'interno del blocco che contiene chiavi nel range prestabilito

Ipotizziamo di avere un **oracolo** che ci dica se una chiave è presente o meno all'interno di un data block, possiamo avere i seguenti casi:

- L'oracolo indica che la chiave esiste, ma non è vero, abbiamo un **true positive**, in questo caso andremmo a leggere il blocco dati e non troveremo la chiave cercata, andando a "sprecare" computazione, ma è una situazione accettabile
- L'oracolo indica che la chiave esiste, ed in effetti esiste, siamo nel caso di un **true positive**. In questo caso andremo a leggere il blocco dati e troveremo la chiave cercata
- Il filtro indica che la chiave non esiste, ma in realtà esiste, siamo nel caso di un **false negative**. Se ci fidassimo dell'oracolo, non andremmo a leggere il blocco dati e perderemmo l'informazione, questa situazione è inaccettabile
- Il filtro indica che la chiave non esiste, ed effettivamente non esiste, siamo nel caso di un **true negative**. In questo caso potremmo evitare di leggere il blocco dati, risparmiando tempo di computazione

Alla luce delle considerazioni appena fatte, i bloom filters sono progettati in modo tale da *poter restituire dei falsi positivi*, che sono appunto tollerabili, ma in modo tale da *non restituire mai dei falsi negativi*, che sono non accettabili.

L'idea dietro ad un bloom filter è quella di usare una piccola quantità di memoria per andare a tracciare appartenenza o non appartenenza ad un insieme. Quando andiamo a memorizzare un record, questo ha delle determinate caratteristiche, alcune di queste possono essere condivise da altri record. Alla luce di questo concetto, sappiamo che nel momento in cui cerchiamo un record, con delle date caratteristiche, se un certo insieme ha elementi con queste caratteristiche, potrebbe (**falsi positivi accettabili**) contenere record di interesse, al contrario sicuramente non conterrà il record cercato (**falsi negativi inaccettabili**).

Esempio: Applicazione di Bloom Filtering ad un semplice contesto

Supponiamo di essere in una classe, e voler cercare di stimare se un generico studente è presente al suo interno. Possiamo considerare un array di 26 elementi (uno per lettera dell'alfabeto), quando una persona entra in classe andiamo a memorizzare che uno studente con una certa iniziale è entrato in aula.

Supponiamo che entrino in aula gli studenti "Bob", e "Alice", andremo a settare gli elementi corrispondenti alle lettere "A" e "B" nell'array. Ora, se vogliamo sapere se "Charlie" è presente in aula, andremo a controllare l'elemento corrispondente alla lettera "C". Dal momento che questo elemento non è settato, possiamo concludere che "Charlie" non è presente in aula (true negative).

Osservazione

Possiamo notare come la scelta delle caratteristiche da utilizzare nel bloom filter sia fondamentale per garantire che il filtro non produca troppi falsi positivi. Nel caso di sopra, stiamo inoltre introducendo un **bias**, dal momento che ci sono iniziali che sono molto più comuni di altre.

Alla luce dell'osservazione precedente, desidereremmo che la distribuzione delle caratteristiche fosse il più **uniforme** possibile. Per fare ciò i bloom filter utilizzano **funzioni di hashing** per andare a mappare i valori delle caratteristiche in indici di un array di bit. In questo modo, anche se le caratteristiche non sono uniformemente distribuite, gli indici generati dalle funzioni di hashing lo saranno. Il filtro che progetteremo avrà lo scopo di decidere se un certo valore **non è presente** in un insieme.

Di seguito andiamo ad elencare i passi per la ricerca di un valore all'interno di un bloom filter:

- Viene richiesto al bloom filter se un certo valore c è presente nell'insieme S
- Vengono utilizzate k funzioni di hashing $h_1, \dots, h_k$
- Ogni funzione di hashing h_i mappa il valore c in maniera casuale in un indice dell'array di bit di dimensione m : $h_i : c \rightarrow [0, m - 1]$, ossia $h_{i(c)} \in \{1, \dots, m\}$

Nel caso in cui due diversi valori c, c' vengano mappati nello stesso valore: ($h_{i(c)} = h_{i(c')}$), si parla di **collisione**. Chiaramente le collisioni sono la causa principale dei falsi positivi.

Di andiamo ad approssimare il numero di falsi positivi che possiamo aspettarci da un bloom filter. Supponiamo di avere a disposizione k funzioni di hashing, un array di bit di dimensione m e di voler memorizzare n elementi nel filtro. Il numero di falsi positivi si può approssimare tramite la formula in Equazione (4.1).

$$\Pr[\text{false positive}] = \left(1 - \left(1 - \frac{1}{m}\right)^{kn}\right)^k \approx \left(1 - e^{-\frac{kn}{m}}\right)^k \quad (4.1)$$

Figura 4.8, mostra come si comporta un bloom filter nel caso in cui sia necessario leggere dei dati, sia nel caso di chiave presente sia nel caso di chiave assente.

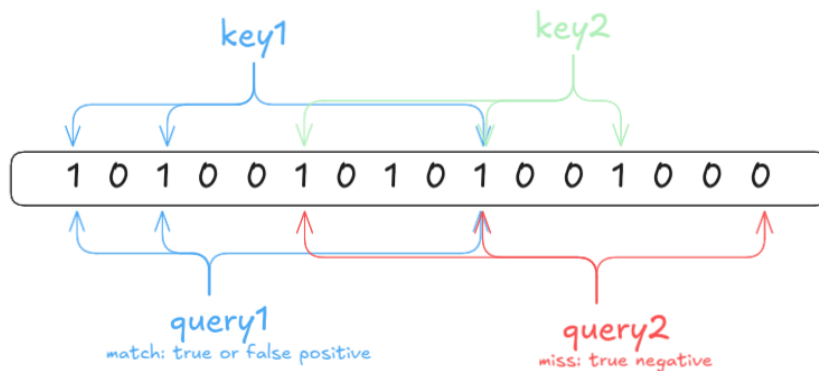


Figura 4.8: Due esempi di utilizzo di un Bloom Filter durante la lettura in un Extensible Record Store, `query1` mostra il caso in cui la chiave cercata è presente, `query2` mostra il caso in cui la chiave cercata non è presente

4.3. Alcune Implementazioni

Tra le implementazioni più famose di extensible record store troviamo:

- Apache Cassandra, che utilizza CQL, un linguaggio **SQL-like**, nel quale un inserimento avviene nel modo seguente:

```
1 INSERT INTO bookinfo (book_id, title, author)
2 VALUES (1234, 'Foo', 'Bar', ' ')
```

CQL

- HBase, che è costruito sopra a Hadoop HDFS e utilizza il framework MapReduce per l'elaborazione distribuita dei dati

5. Graph Databases

Nel corso di questo capitolo andremo ad affrontare basi di dati organizzate come **grafo**. Nella prima parte andremo a vedere i concetti fondamentali, per poi passare a vedere una delle implementazioni più diffuse: **Neo4j**.

5.1. Teoria dei Grafi

In questa sezione andiamo a porre le basi teoriche per comprendere il concetto al fondamento di questa categoria di base di dati: i **grafi**, di cui andiamo a dare una definizione.

Definizione 5.1 (Grafo)

Un grafo G è una struttura costituita da un insieme di **vertici** e di **archi** $G = (V, E)$, dove:

- ogni *vertice* $v \in V$ rappresenta un'entità o un oggetto;
- ogni *arco* $e \in E$ rappresenta una relazione o connessione tra due vertici.
- Gli archi possono essere **diretti** o indiretti, a seconda che la relazione abbia una direzione specifica o meno.

Un grafo è detto **diretto** se tutti i suoi archi hanno una direzione associata, al contrario, se nessun arco ha direzione, il grafo si dice **indiretto**. Figura 5.1 mostra un esempio di grafo diretto e indiretto.

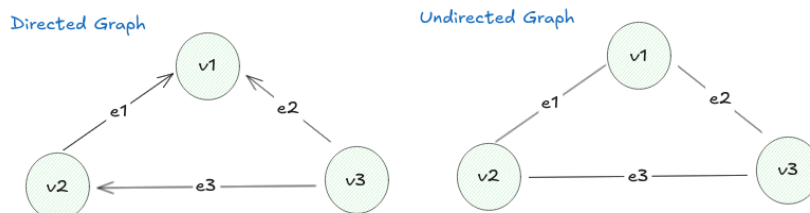


Figura 5.1: Esempio di grafo diretto (a sinistra) e indiretto (a destra).

Un grafo si dice un **multigrafo** se ha una coppia di nodi che è connessa tramite più di un arco. Questa caratteristica è utile per rappresentare relazioni complesse tra entità, come ad esempio in una rete di trasporti dove più linee possono collegare due stazioni. Figura 5.2 mostra un esempio di multigrafo.

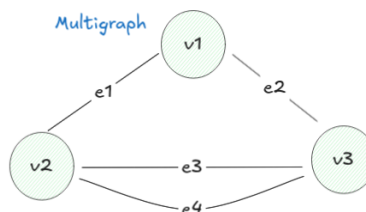


Figura 5.2: Esempio di multigrafo con più archi tra alcuni nodi. e_3, e_4 sono archi multipli tra i nodi v_2 e v_3 .

Possiamo avere anche degli **ipergrafi**, in cui un arco può connettere più di due vertici. Tuttavia, per i nostri scopi, ci concentreremo su grafi più semplici.

Definizione 5.2 (Adiacenza tra Nodi)

Due vertici v_1, v_2 sono detti **adiacenti**, se sono “vicini”, ovvero se esiste un arco che li connette in maniera diretta.

Definizione 5.3 (Incidenza)

Un arco e è detto **incidente su un vertice** se è connesso a quel vertice. Nel caso di grafi diretti distinguiamo due casi:

- **positivamente** incidente se l'arco parte dal vertice,
- **negativamente** incidente se l'arco arriva al vertice.

Vedremo in seguito come i concetti introdotti da Definizione 5.2^o e Definizione 5.3^o siano fondamentali per comprendere come andare a memorizzare in maniera efficiente un grafo all'interno della memoria.

5.1.1. Navigazione in un Grafo

La navigazione di un grafo è nota anche con il nome di **graph traversal** e sostanzialmente è alla base di tutte le operazioni che possiamo effettuare su un grafo.

Un **path** (o percorso) in un grafo è una sequenza di vertici e archi che collega un vertice di partenza a un vertice di arrivo.

Esempio: Path in un Grafo

Consideriamo il grafo in Figura 5.3. Possiamo notare che esiste un path tra il nodo “Alice” e il nodo “Marcus” che passa per “Bob”. Il fatto che possiamo giungere da Alice a Marcus, passando per Bob, implica che Alice e Marcus sono “conoscenti di conoscenti”.

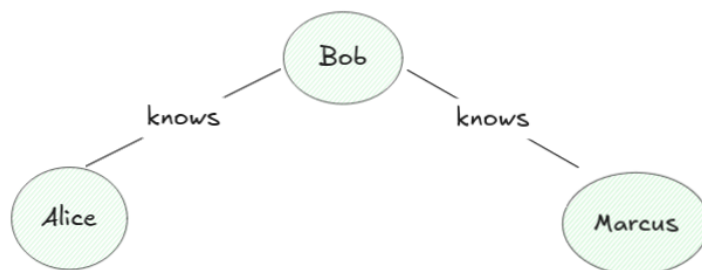


Figura 5.3: Rappresentazione delle relazione 'conoscenti di conoscenti' tramite un grafo

In riferimento all'esempio precedente, possiamo notare come alla definizione di grafo introdotta da Definizione 5.1^o manchi un concetto fondamentale nel contesto delle basi di dati: il dare un nome agli archi, che rappresentano relazioni tra i nodi.

Quando andiamo a visitare un grafo, possiamo porci una domanda fondamentale: «**In che ordine dovremmo visitare i nodi?**». Esistono diverse alternative per rispondere a questa domanda, le più comuni sono:

- **Depth-First Search (DFS)**: questa strategia esplora il più lontano possibile lungo ogni ramo prima di tornare indietro. In pratica, si visita un nodo, poi si visita uno dei suoi vicini, e così via, fino a quando non si raggiunge un nodo senza vicini non visitati. A quel punto, si torna indietro e si esplorano gli altri vicini.
- **Breadth-First Search (BFS)**: questa strategia esplora tutti i vicini di un nodo prima di passare ai nodi di livello successivo. In pratica, si visita un nodo, poi si visitano tutti i suoi vicini, poi i vicini dei vicini, e così via.
- **Altri algoritmi specializzati**: esistono molti altri algoritmi di navigazione dei grafi, come Dijkstra per il calcolo del percorso più breve, A^* , Dijkstra, Bellman-Ford, e molti altri, ciascuno con le proprie caratteristiche e casi d'uso specifici.

5.1.2. Problemi sui Grafi

I grafi sono utilizzati per modellare una vasta gamma di problemi nel mondo reale. Sono stati dunque sviluppati numerosi problemi classici sui grafi, tra cui:

- **Graph Traversal**: anche se già menzionato in precedenza è importante sottolineare che la navigazione di un grafo è un problema fondamentale, con molte varianti e applicazioni. Consiste nella visita di *tutti i nodi* al suo interno
- **Eulerian Path**: trovare un percorso che attraversa tutti i nodi passando per ogni arco esattamente una volta.
- **Eulerian Cycle**: iniziando da un path euleriano, vorremmo iniziare e terminare nello stesso nodo.
- **Hamiltonian Path**: trovare un percorso che visita ogni nodo esattamente una volta.
- **Hamiltonian Cycle**: in maniera simile a prima, vorremmo un path hamiltoniano che inizi e termini nello stesso nodo.
- **Minimum Spanning Tree**: trovare un albero che oltre a collegare tutti i nodi, minimizza il costo totale degli archi utilizzati.

5.2. Basi di Dati a Grafo

L'idea alla base delle basi di dati a grafo è quella che sia estremamente importante rappresentare i **collegamenti** tra dati memorizzati. In una base di dati relazionale, le relazioni tra entità sono rappresentate tramite chiavi esterne, che per quanto funzionali, non sono così intuitive, l'utilizzo di un grafo è quanto di più naturale per rappresentare queste relazioni. Di seguito Qualche esempio in cui basi di dati di questo tipo sono particolarmente utili:

- **Social Networks**: le relazioni tra utenti, amici, follower, ecc. sono naturalmente rappresentate come grafi.
- **Recommendation Systems**: le connessioni tra utenti e prodotti possono essere modellate come graf
- **Web Semantico**: le relazioni tra pagine web, link, e contenuti possono essere rappresentate come grafi.

- **Sistemi Informativi Geografici:** le reti stradali, i percorsi di trasporto, e le connessioni geografiche sono spesso modellate come grafi.
- **Bioinformatica:** le reti di interazioni tra proteine, geni, e altre entità biologiche sono spesso rappresentate come grafi.

Figura 5.4 mostra un esempio di come una base di dati a grafo possa essere utilizzata per rappresentare le relazioni tra utenti in un social network e in un sistema geospaziale.

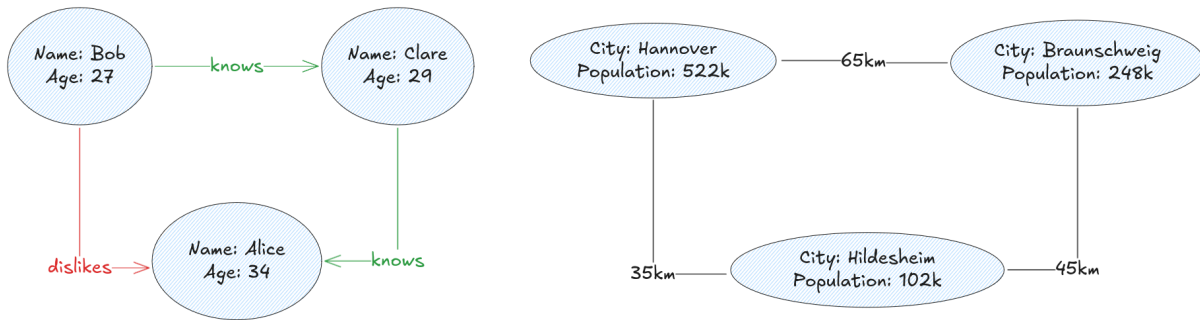


Figura 5.4: Esempi di basi di dati a grafo in un social network (a sinistra) e in un sistema geospaziale (a destra).

5.2.1. Modello dei Dati

Le basi di dati a grafo utilizzano un modello di dati chiamato **Property Graph Model**, che andiamo di seguito a definire.

Definizione 5.4 (Property Graph Model)

Un **property graph** è un **multigrafo diretto** che memorizza informazioni (proprietà) sia sui nodi che sugli archi. Una **proprietà** è una coppia chiave-valore (es. Name: Alice). A volte è possibile avere proprietà **multi-valore**: una chiave e un insieme di elementi associati (es. Hobbies: {Reading, Hiking, Coding}).

Per ogni nodo e arco viene definita una proprietà di default chiamata **Id** che identifica univocamente l'elemento all'interno del grafo.

In Figura 5.5 è mostrato un esempio di property graph che rappresenta una piccola rete sociale.

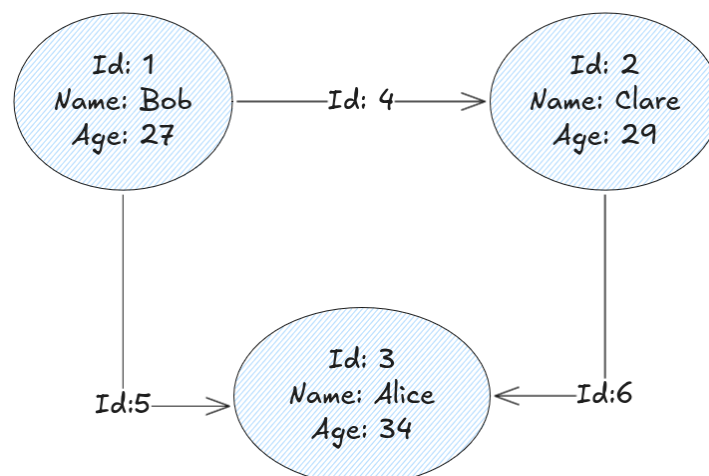


Figura 5.5: Esempio di Property Graph che rappresenta una rete sociale.

In generale non viene imposto alcun vincolo sul tipo di dati, che possono essere coppie chiave-valore di **tipo arbitrario**. Ad ogni modo è buona pratica utilizzare tipaggio per dare valore semantico ai valori memorizzati:

- Per i *vertici* viene utilizzata una proprietà di *default*, chiamata `Type`
- Per gli *archi* viene utilizzata una proprietà di *default*, chiamata `Label`, opzionalmente è anche possibile andare a definire quali edge labels possono connettere certi nodi in base al tipo che viene specificato per questi

Possiamo notare che l'utilizzo delle proprietà `Type` e `Label` corrisponde circa a dire che un certo nodo o una certa relazione tra nodi fanno parte di una certa **classe**. In questo modo, l'esempio in Figura 5.5 può essere riformulato reintroducendo il valore semantico delle relazioni nel grafo come viene fatto in Figura 5.6.

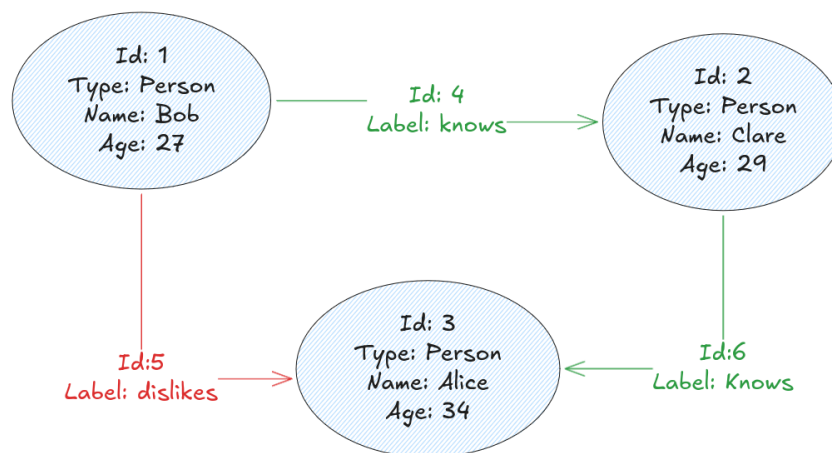


Figura 5.6: Esempio di Property Graph con tipi e label che rappresenta una rete sociale.

Non è obbligatorio, ma in generale possiamo aspettarci che nodi e archi che fanno parte di una stessa “classe” condividano le stesse proprietà. Introduciamo un ultimo particolare vincolo, che riguarda la presenza di **multi-edge**, questi non possono connettere gli stessi nodi con la stessa «Label» di relazione. Vogliamo evitare questa situazione per evitare di avere **ambiguità** nell'interpretazione del grafo. Figura 5.7 mostra un esempio di multi-edge non ammesso.

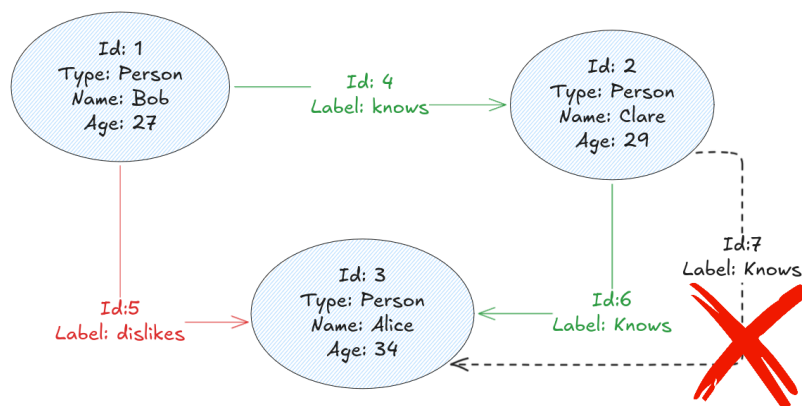


Figura 5.7: Esempio di multi-edge non ammesso dal momento che ha la stessa label di un altro arco già presente per collegare gli stessi nodi.

5.2.2. Memorizzazione di un Grafo

Questa sezione è dedicata a illustrare come viene salvata in memoria la struttura del grafo. Se per quanto riguarda i vari nodi, potrebbe risultare intuitivo come andare a memorizzarli, la questione sia complica quando andiamo a cercare di rappresentare gli **archi**.

Abbiamo infatti bisogno di raggiungere un buon **tradeoff** tra **memorizzazione rapida** e **possibilità di utilizzare di dati in modo efficiente** (es. graph traversal).

Strategia Naive

Una possibile strategia molto semplice è quella di memorizzare l'insieme dei nodi V e l'insieme degli archi E come liste separate. Ogni arco conterrà un riferimento ai nodi che connette. Di seguito andiamo ad elencare i pro e contro di questa strategia:

- L'**inserimento** è molto rapido, dal momento che basta aggiungere un nuovo nodo o un nuovo arco alla rispettiva lista, l'unica cosa di cui abbiamo bisogno è un puntatore alla prossima posizione in cui andiamo a scrivere ✓
- La determinazione delle **adiacenze** è particolarmente inefficiente, dal momento che dobbiamo scorrere l'intera lista degli archi per trovar quelli di interesse ✗
- La gestione delle operazioni di **cancellazione** è particolarmente inefficiente, dal momento che dobbiamo scorrere l'intera lista di interesse, cancellare il dato e ricompattare il tutto ✗

Matrice di Adiacenza

Una possibile strategia possibilmente più efficace è quella di utilizzare una **matrice di adiacenza**. In questo caso andremo a creare una matrice quadrata di dimensione $|V| \times |V|$. Ogni riga e colonna andrà a rappresentare i vertici $v_1, \dots, v_n$.

Nel caso di grafi senza archi, la matrice di adiacenza conterrà unicamente zeri. Nel caso in cui siano presenti degli archi abbiamo due possibilità in base al tipo di grafo:

- Per **grafi non diretti a-ciclici**: se esiste un arco tra il nodo v_i e v_j , allora andremo a scrivere uno nella cella (i, j) e nella cella (j, i) della matrice di adiacenza.
- Per **grafi non diretti con cicli**: nel caso di un loop tra il v_i e sé stesso, andremo a scrivere 2 nella cella (i, i) della matrice di adiacenza.
- Per **grafi diretti a-ciclici**: se esiste un arco dal nodo v_i al nodo v_j , allora andremo a scrivere uno nella cella (i, j) della matrice di adiacenza.
- Per **grafi diretti con cicli**: nel caso di un loop tra il v_i e sé stesso, andremo a scrivere 1 nella cella (i, i) della matrice di adiacenza.

Osservazione

Nel caso di grafi non diretti la matrice di adiacenza sarà sempre simmetrica, mentre non lo sarà necessariamente nel caso di grafi diretti.

Osservazione

Il motivo per cui in un grafo non diretto con ciclo andiamo a scrivere due nella cella (i, i) sta nel fatto che in grafi non diretti ogni arco è contato in maniera simmetrica, dunque un loop viene contato due volte. Ciò non avviene nel caso di grafi diretti.

Andiamo ora a mostrare cosa succede nel caso di **multigrafi non diretti**:

- Nel caso di multigrafo diretto **a-ciclico**: se esistono k archi tra il nodo v_i e v_j , allora andremo a scrivere k nella cella (i, j) e nella cella (j, i) della matrice di adiacenza.
- Nel caso di multigrafo diretto **con cicli**: nel caso in cui si verifichino k loops del tipo v_i, v_i , andremo a scrivere $2 \cdot k$ nella cella (i, i) della matrice di adiacenza.

Nel caso in cui di **multigrafi diretti** avremo la seguente situazione:

- Nel caso di multigrafo diretto **a-ciclico**: se esistono k archi dal nodo v_i al nodo v_j , allora andremo a scrivere k nella cella (i, j) della matrice di adiacenza.
- Nel caso di multigrafo diretto **con cicli**: nel caso in cui si verifichino k loops del tipo v_i, v_i , andremo a scrivere k nella cella (i, i) della matrice di adiacenza

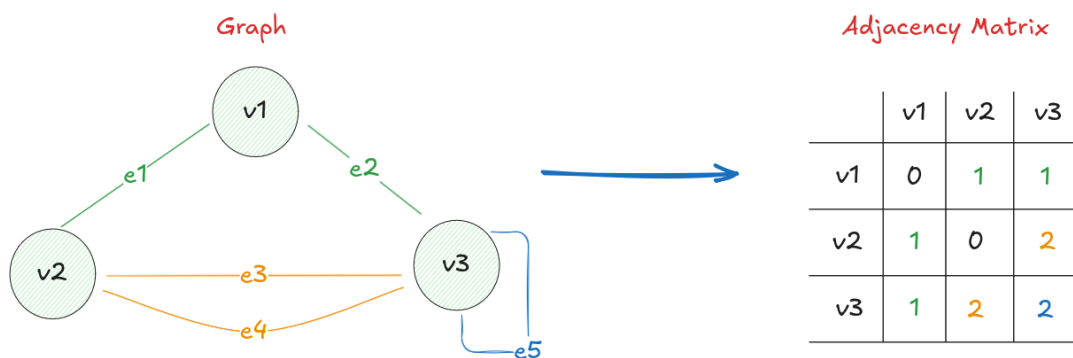


Figura 5.8: Esempio di multigrafo non diretto con cicli (a sinistra) e la sua matrice di adiacenza (a destra).

Di seguito andiamo ad elencare vantaggi e svantaggi di questo approccio:

- La determinazione delle **adiacenze** è molto rapida, dal momento che basta accedere alla cella di interesse della matrice ✓
- È estremamente facile **inserire** un nuovo arco, dal momento che basta incrementare il valore nella cella di interesse ✓
- È particolarmente **costoso** dal punto di vista della memoria, dal momento che dobbiamo memorizzare una matrice di dimensione $|V| \times |V|$, specialmente quando i nodi sono molti la dimensione della matrice potrebbe diventare di difficile gestione ✗
- Spesso e volentieri i grafi sono **sparsi**, ovvero hanno pochi archi rispetto al numero di nodi, in questo caso la matrice di adiacenza conterrà molti zeri, andando così a sprecare memoria ✗
- Le operazioni di **inserimento di nuovi nodi** sono particolarmente costose, dal momento che dobbiamo aumentare la dimensionalità della matrice ✗

- Non è possibile memorizzare iper-grafi, dal momento che la matrice di adiacenza può rappresentare solamente archi che connettono due nodi ✗
- Determinare **tutte le adiacenze** di un nodo è particolarmente inefficiente, dal momento che dobbiamo scorrere l'intera riga o colonna della matrice di interesse, che nel caso di grafi con molti nodi potrebbe essere un'operazione lenta ✗

Matrice di Incidenza

Un possibile alternativa alla matrice di adiacenza è la **matrice di incidenza**. In questo caso andremo a creare una matrice di dimensione $|V| \times |E|$. Ogni riga andrà a rappresentare i vertici $v_1, \dots, v_n$, mentre ogni colonna andrà a rappresentare gli archi $e_1, \dots, e_m$.

I valori nelle celle della matrice di incidenza saranno assegnati in maniera simile a quanto visto per la matrice di adiacenza, con la differenza che dovremo tenere conto della positività e della negatività delle incidenze.

Figura 5.9 e Figura 5.10 mostrano le differenze nella matrice di adiacenza nel caso di grafi diretti e indiretti.

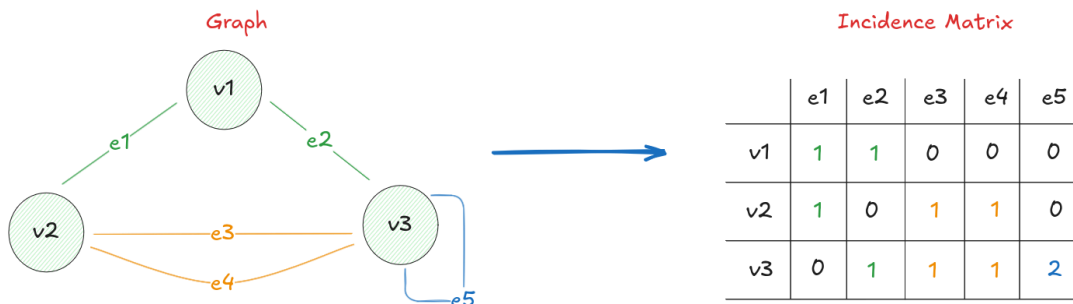


Figura 5.9: Esempio di multigrafo diretto con cicli (a sinistra) e la sua matrice di incidenza (a destra).

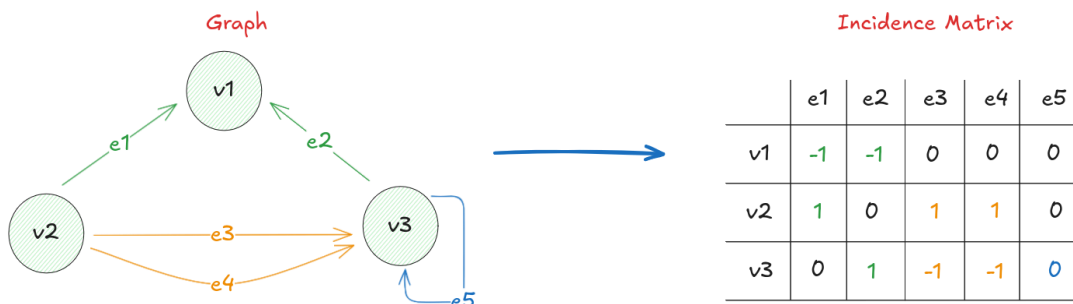


Figura 5.10: Esempio di multigrafo diretto con cicli (a sinistra) e la sua matrice di incidenza (a destra).

Di seguito andiamo a riportare vantaggi e svantaggi di questo approccio:

- Non avremo colonne di soli zeri, dal momento che queste rappresentano gli archi e usiamo colonne solo per gli archi esistenti ✓
- È possibile memorizzare iper-archi ✓
- Dal punto di vista dello **storage** si tratta di un approccio particolarmente **intensivo** ($n \times m$) ✗

- Nel caso di grafi con molti nodi, le **colonne** avranno comunque **molti zeri**, dal momento che una colonna rappresenta un arco
- Determinare le **adiacenze** per un vertice richiede il lookup di tutta la riga corrispondente ✗
- L'**inserimento** di un nuovo nodo richiede l'aggiunta di una nuova riga alla matrice, risultando **costoso** ✗

Invece di utilizzare **matrici** per rappresentare i grafi, un'alternativa più efficiente è quella di utilizzare delle **liste di adiacenza**, questo permette di risparmiare memoria fondamentale e di velocizzare l'esecuzione di operazioni comuni sui grafi.

Liste di Adiacenza

L'idea alla base di una lista di adiacenza è quella di memorizzare per ogni nodo, una lista di nodi adiacenti. In questo modo, possiamo rappresentare un grafo come un array di liste. Figura 5.11 mostra come questo approccio possa essere implementato nel caso di un grafo indiretto a sinistra e di uno diretto a destra.

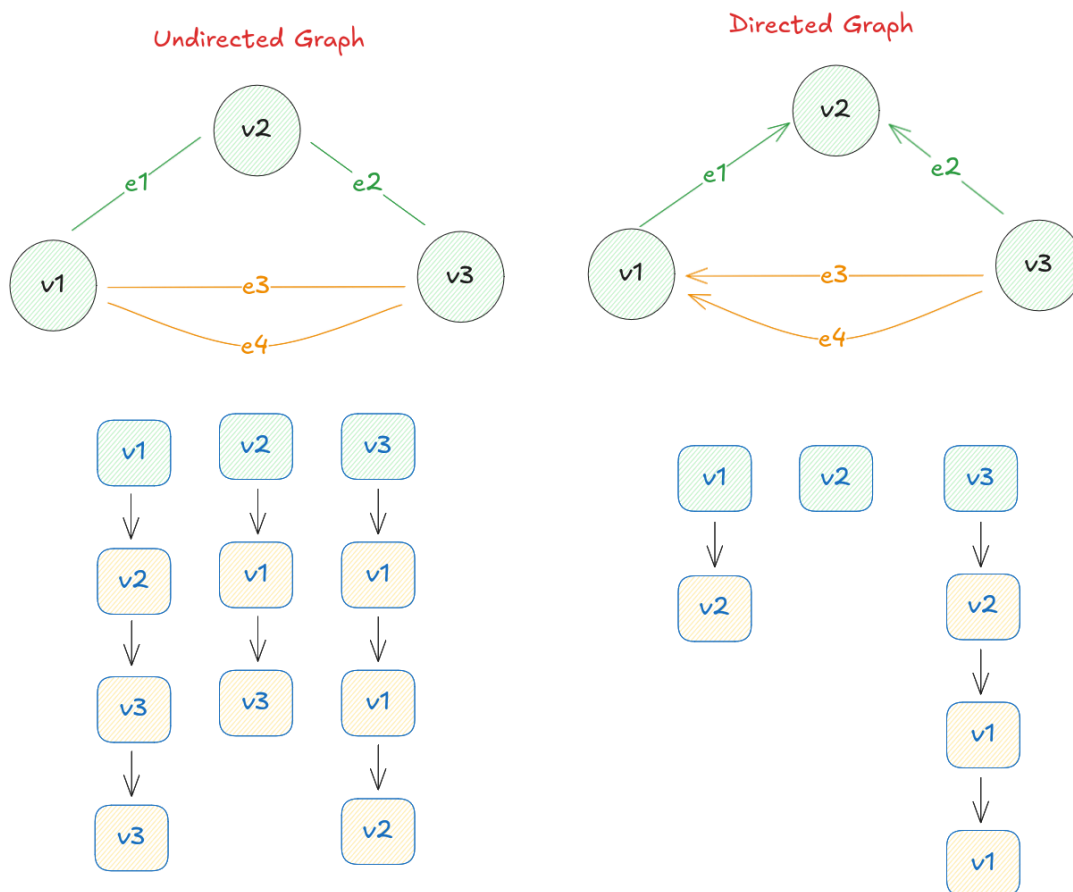


Figura 5.11: Esempio di rappresentazione di un grafo indiretto (a sinistra) e diretto (a destra) tramite liste di adiacenza.

Di seguito andiamo ad elencarne vantaggi e svantaggi:

- Effettuare un **lookup** di tutti i vertici adiacenti a un nodo è molto **efficiente**, dal momento che basta accedere alla lista corrispondente ✓
- Vengono memorizzati soltanto le informazioni rilevanti, non abbiamo **overhead** di memoria ✓

- L'**inserimento** di un nuovo nodo è **efficiente**, dal momento che basta aggiungere una nuova lista all'array ✓
- L'**inserimento** di un nuovo arco è **efficiente**, dal momento che basta aggiungere il nodo di destinazione alla lista del nodo/i di partenza/arrivo ✓
- È possibile memorizzare iper-archi ✓
- Determinare l'esistenza di un **arco specifico** può essere costoso, dal momento che potrebbe essere necessario scorrere l'intera lista di adiacenza ✗

Osservazione

Questa struttura per quanto efficiente non consente ancora di andare a memorizzare le proprietà associate a nodi ed archi, ma non si tratta di una operazione complicata, basta infatti memorizzare le **proprietà di un nodo** possiamo memorizzarle nel punto di **partenza della lista**, mentre le **proprietà di un arco** possono essere memorizzate nel **nodo di destinazione** all'interno della lista di adiacenza.

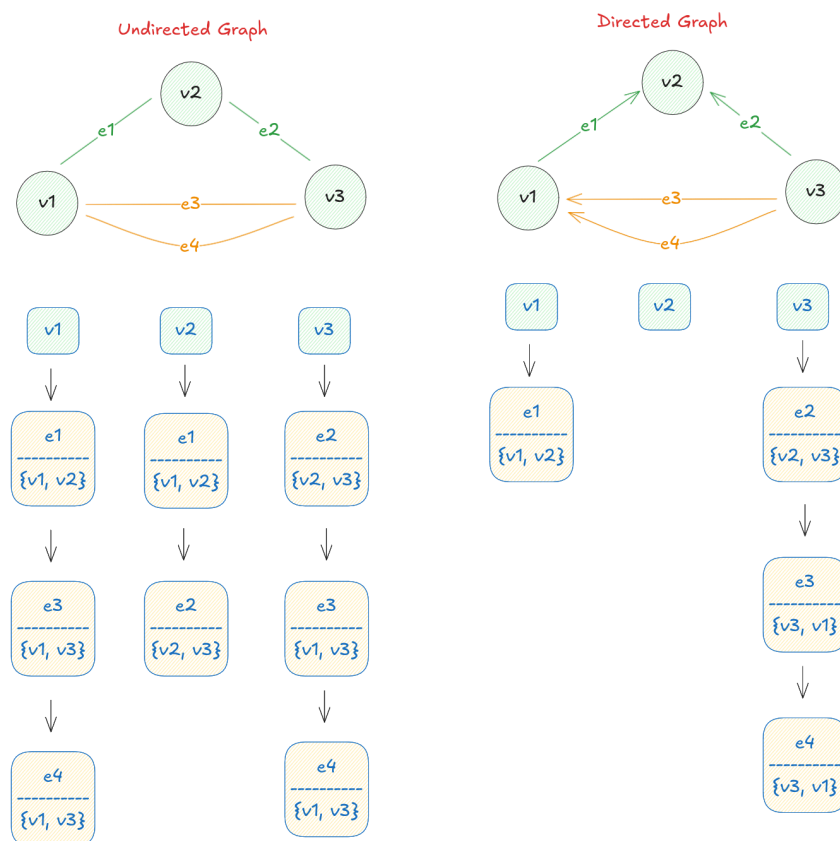


Figura 5.12: Esempio di rappresentazione di un grafo indiretto (a sinistra) e diretto (a destra) tramite liste di incidenza.

Liste di Incidenza

Un'ulteriore possibile strategia per memorizzare un grafo è quella di utilizzare delle **liste di incidenza**. In questo caso, per ogni nodo andremo a memorizzare una **lista di archi** che sono incidenti su quel nodo. Figura 5.12 mostra come questo approccio possa essere implementato nel caso di un grafo indiretto a sinistra e di uno diretto a destra.

Osservazione

Nel caso in cui volessimo andare a memorizzare degli **iper-grafi** questo sarebbe possibile, basterebbe andare ad aggiungere alla lista di incidenza relativa all'iper-arco tutti i nodi che questo coinvolge.

5.2.3. Graph Database Management Systems & API

Il focus di questa sezione è quello di andare ad illustrare come possiamo **accedere** ai dati memorizzati in un graph database. In realtà abbiamo diversi sistemi per farlo. Di seguito andiamo a mostrarne alcuni ognuno con le sue caratteristiche.

TinkerPop

Si tratta di un framework open-source per la gestione di grafi che fornisce un API standardizzata per interagire con un graph db.

```
1 Graph g = TinkerGraph.open();
2 Vertex alice = g.addVertex("name", "Alice");
3 alice.property("age", 34);
4 Vertex bob = g.addVertex("name", "Bob");
5 alice.addEdge("knows", bob, "since", 2020);
```

Gremlin

Di seguito andiamo a vedere un semplice codice per effettuare traversal del grafo creato tramite *method chaining*:

```
1 g.traversal().V() // prendiamo la lista dei vertici
2 .has("name", "Alice") // filtriemo per il vertice con nome "Alice"
3 .out("knows") // andiamo agli archi in uscita con label "knows"
4 .values("name"); // prendiamo i nomi dei vertici raggiunti
```

Gremlin

Cypher

Un'alternativa all'approccio di TinkerPop è quello di utilizzare un linguaggio di query tramite il quale andare a **descrivere pattern** riguardo **nodi** o **paths**. Il modo in cui possiamo definire questi pattern avviene tramite l'utilizzo del linguaggio **Cypher**. Di seguito andiamo a vederne alcuni elementi sintattici:

- **START** : viene utilizzato per specificare i *nodi* di partenza per una query
- **MATCH** : viene utilizzato per la specifica dei *pattern* da ricercare
- **WHERE** : viene utilizzato per specificare delle *condizioni* sui *nodi* o sugli *archi*
- **RETURN** : viene utilizzato per specificare quali dati vogliamo vengano restituiti dalla query

In riferimento al grafo giocattolo costruito nell'esempio di prima possiamo costruire una query come quella descritta di seguito per trovare tutti i nodi adiacenti ad "Alice":

```
1 START alice = (people_idx, name, "Alice") // selezioniamo il
   nodo di partenza
```

Cypher

```
2 MATCH (alice)-[:knows]->(aperson) // definiamo il pattern da cercare
3 return (aperson)
```

Quello che viene fatto nella query di sopra è:

- Selezionare il nodo di partenza “Alice” tramite l’uso dell’indice `people_idx`
- Definire il pattern da cercare, ovvero tutti i nodi connessi ad “Alice” tramite un arco con label “knows”
- Restituire i nodi trovati.

Osservazione

Si noti come, dal momento che non sono stati specificati vincoli sui nodi o sugli archi, la query restituirà tutti i nodi adiacenti ad “Alice” tramite un arco con label “knows”, non necessariamente nodi con il suo stesso tipo (es. `Person`).

5.3. Neo4j

Al contrario di moltissime altre tecnologie per la gestione dei dati, Neo4j è un **Graph Database nativo**, ovvero è stato progettato fin dall'inizio per memorizzare e gestire dati in forma di grafo.

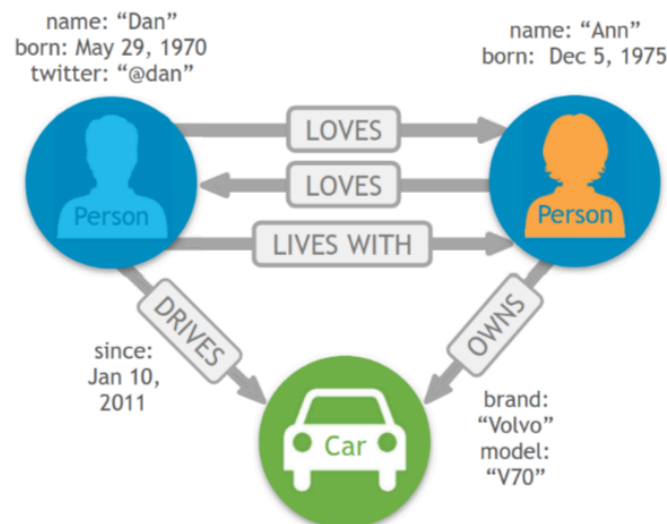
Anche un database relazionale può essere visto come una struttura a grafo, ma in questo caso i grafi sono **derivati** dalle tabelle e dalle relazioni tra esse. Questo comporta che le operazioni su database relazionali possano diventare estremamente complesse e inefficienti quando si tratta di navigare attraverso molteplici relazioni.

5.3.1. Property Graph Model

Questa sezione va velocemente a riassumere i concetti fondamentali alla base di un qualsiasi database basato su un grafo, ovvero il *property graph model*. Di seguito andiamo a rielencare i concetti principali:

- **Nodi**: rappresentano gli oggetti (o entità) del grafo, ogni nodo può avere una **label** (o più di una) che ne definisce il tipo (es. `Person`, `Movie`, ecc.)
- **Relazioni**: rappresentano le connessioni (archi) tra i nodi, ogni relazione ha una **direzione** e una **label** che ne definisce il tipo (es. `KNOWS`, `ACTED_IN`, ecc.)
- **Proprietà**: sia i nodi che le relazioni possono avere proprietà, che sono coppie chiave-valore che memorizzano informazioni aggiuntive (es. `name: "Alice"`, `since: 2020`, ecc.)

L'immagine che segue mostra un esempio di un property graph model che contiene molteplici relazioni tra nodi, molte proprietà e tipi di nodi e relazioni diversi:



L'idea alla base di Neo4j è quella di fornire uno strumento il più intuitivo e vicino possibile a ciò che un utente si aspetta quando pensa a un grafo.

5.3.2. Graph Querying: Qualche Esempio

Dato un grafo abbiamo bisogno di un modo per poter interrogare i dati in esso contenuti. Neo4j fornisce un linguaggio di query chiamato **Cypher**, che è stato progettato specificamente per lavorare con grafi. Si tratta di un linguaggio **dichiarativo** che pone alla sua base il concetto di

pattern matching sui grafi, in maniera simile a quello che facciamo con le *espressioni regolari* sui testi. Figura 5.13 mostra un esempio di come possiamo rappresentare dei pattern in Cypher.

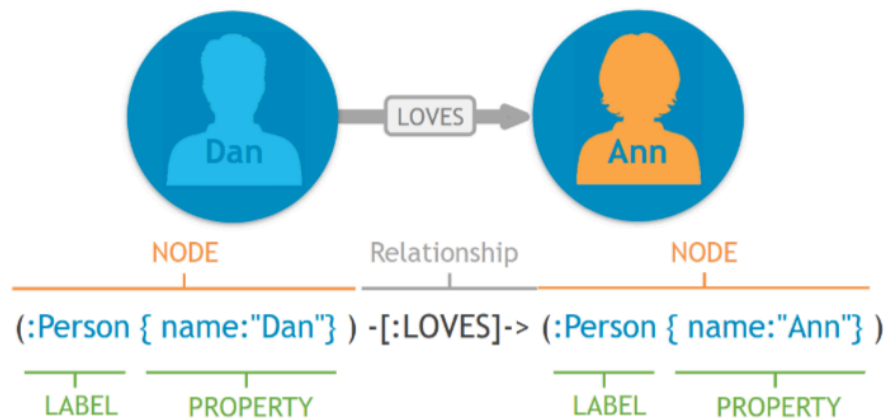


Figura 5.13: Esempio di pattern matching in Cypher per trovare nodi e relazioni specifiche all'interno di un grafo.

Dato il pattern rappresentato in Figura 5.13 possiamo andare a **creare** o a **ricercare** nodi e relazioni all'interno del grafo.

Esempio: Creazione della relazione 'Loves' tra 'Dan' e 'Ann'

```
1 CREATE
2   (:Person {name: "Dan"})
3   -[:LOVES]->
4   (:Person {name: "Ann"})
```

Cypher

Esempio: Ricerca delle persone amate da 'Dan'

```
1 MATCH
2   (:Person{name: "Dan"})
3   -[:LOVES]-> (whom)
4 RETURN whom
```

Cypher

Nell'esempio di sopra non è stato specificato che l'entità amata da "Dan" dovesse essere di tipo `Person`, dunque la query restituirà qualsiasi entità connessa a "Dan" tramite una relazione di tipo `LOVES`. È tuttavia possibile aggiungere ulteriori vincoli per restringere i risultati ottenuti.

Vediamo ora un esempio leggermente più complesso. Consideriamo il grafo in Figura 5.14, che rappresenta una rete sociale in cui gli utenti sono connessi tramite relazioni di amicizia e hanno ristoranti preferiti.

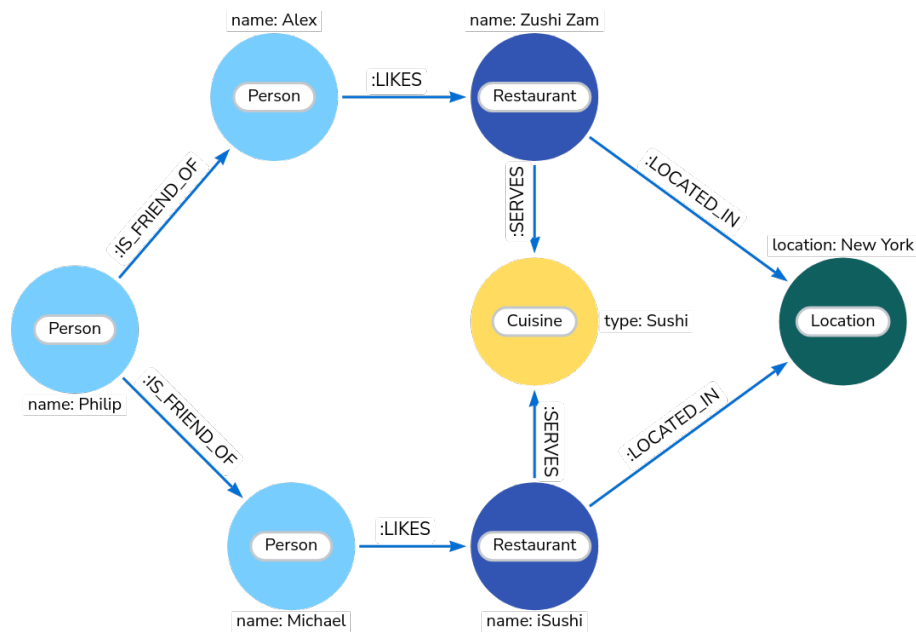


Figura 5.14: Grafo che rappresenta una rete sociale con relazioni di amicizia e ristoranti preferiti.

Vorremo andare a trovare i ristoranti che sono graditi dagli amici di “Philip” che servono “sushi” locati a “New York”. La query in Cypher per ottenere queste informazioni è la seguente:

```

1 MATCH (person:Person) -[:IS_FRIEND_OF]-> (friend),
2     (friend) -[:LIKES]-> (restaurant),
3     (restaurant) -[:LOCATED_IN]-> (loc:Location),
4     (restaurant) -[:SERVES]-> (type:Cuisine),
5 WHERE person.name = 'Philip'
6 AND loc.location = 'New York'
7 AND type.cuisine = 'Sushi'
8 RETURN restaurant.name

```

Cypher

5.3.3. Introduzione a Cypher

Dopo aver visto qualche esempio di query eseguita in Cypher andiamo a vedere più nel dettaglio la **sintassi** e gli **elementi fondamentali** per costruire query con questo linguaggio.

Sintassi

- un **nodo** viene rappresentato tramite delle parentesi tonde `()`, al cui interno possiamo specificare una **label** e delle **proprietà**. Esempio: `(n:Person {name: "Alice", age: 30})`
- una **relazione** viene rappresentata tramite una freccia `-->` o `<--`, al cui interno possiamo specificare una **label** e delle **proprietà** tra parentesi quadre `[]`. Esempio: `-[:KNOWS {since: 2020}]->`
- un **pattern** viene di solito costruito tramite una combinazione di nodi e relazioni: `()-[]-()`, `()-[]->()`, `()<-[]-()`.

Componenti di una Query

I due componenti principali di una query in Cypher sono sostanzialmente due:

- **MATCH** : viene utilizzato per specificare i pattern da cercare all'interno del grafo.
- **RETURN** : viene utilizzato per specificare quali dati che hanno registrato una corrispondenza nella clausola **MATCH** vogliamo vengano restituiti dalla query

Esempio: Basica Query in Cypher 2

```
1 MATCH (m:Movie) // considera tutti i nodi m di tipo Movie
2 RETURN m // restituisce tutti i nodi m trovati
```

Cypher

Esempio: Basica Query in Cypher 2

```
1 MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)
2 RETURN p, r, m
```

Cypher

In questo caso **MATCH** e **RETURN** sono due parole chiave del linguaggio; **p**, **r** e **m** sono delle **variabili** che rappresentano rispettivamente i nodi di tipo **Person**, le relazioni di tipo **ACTED_IN** e i nodi di tipo **Movie**.

È anche possibile utilizzare interi pattern come variabili, per esempio nel caso in cui vogliamo vedere dei **path** completi. Lo vediamo nell'esempio che segue.

Esempio: Query di Path in Cypher

```
1 MATCH p = (p:Person)-[r:ACTED_IN]->(m:Movie)
2 RETURN p
```

Cypher

In questo caso la **variabile** **p** rappresenta l'intero path che va dal nodo di tipo **Person**, tramite la relazione di tipo **ACTED_IN**, fino al nodo di tipo **Movie**.

È anche possibile andare ad accedere in maniera selettiva alle **proprietà** dei nodi e delle relazioni. Per fare ciò la sintassi è **{variable}.{property_key}**. Lo vediamo nell'esempio che segue.

Esempio: Query con Proprietà in Cypher

```
1 MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)
2 RETURN p.name, m.title
```

Cypher

In questo caso stiamo restituendo solamente le proprietà **name** del nodo di tipo **Person** e **title** del nodo di tipo **Movie**.

Osservazione

Si noti che questa operazione è molto simile a quella di eseguire una **proiezione** in algebra relazionale o ad applicare costrutti equivalenti in SQL.

Un'altra questione che seppur di minore importanza è necessario menzionare è il funzionamento e la gestione del **casing** in questo linguaggio:

- le *label* di nodi, i *tipi* delle relazione, le *property key* sono sempre **case sensitive**
- tutte le *keyword* di *Cypher* sono **case insensitive**

5.3.4. Indici e vincoli

Un costrutto molto importante è quello dei **vincoli** (constraints), che permettono di imporre delle regole sui dati memorizzati all'interno del grafo, in modo da garantire l'integrità, coerenza dei dati e altre interessanti proprietà.

All'interno di Neo4j e in Cypher è inoltre possibile andare a specificare delle maniere per **ottimizzare** le performance di alcune operazioni, lo strumento utilizzato, similmente a quanto avviene con altre tecnologia è quello degli **indici**.

Vincoli (Constraints) Unique

Andiamo a vedere ora il principale vincolo che possiamo andare a specificare in Neo4j, ovvero il vincolo di **unicità**. Questo assolve a due funzioni fondamentali:

- Garantisce che non esistano due nodi con la stessa proprietà chiave-valore per una certa label
- Permette un **accesso molto veloce** a nodi che corrispondono certe coppie «label-property»

Per la creazione di questo vincolo andiamo a utilizzare la seguente sintassi:

```
1 CREATE CONSTRAINT ON (label:Label)
2 ASSERT label.property_key IS UNIQUE
```

Cypher

Esistono in verità **tre tipologie** di vincoli di unicità che andiamo di seguito ad illustrare:

- **Unique Node Property:** garantisce che non esistano due nodi con la stessa proprietà chiave-valore per una certa label.

```
1 CREATE CONSTRAINT ON (label:Label)
2 ASSERT label.property_key IS UNIQUE
```

Cypher

- **Node Property Existence:** impedisce la creazione di nodi che per una certa label non abbiano valori

```
1 CREATE CONSTRAINT ON (label:Label)
2 ASSERT exists(label.name)
```

Cypher

- **Relationship Property Existence:** impone che per un certo tipo di relazione esista sempre una certa proprietà

```
1 CREATE CONSTRAINT ON ()-[rel:REL_TYPE]->()
2 ASSERT exists(rel.name)
```

Cypher

Indici

Come già visto in altri sistemi e linguaggi, l'utilizzo di **indici** serve a garantire un **lookup** veloce di nodi che soddisfano una certa condizione di tipo «label-property». In Neo4j possiamo creare un indice tramite la sintassi che segue:

```
1 CREATE INDEX ON :Label(property_key)
```

Cypher

Il lookup efficiente e rapido è garantito quando andiamo ad utilizzare i seguenti **predicati** che utilizzano gli indici by design:

- *Uguaglianza*: =, <>
- STARTS WITH
- CONTAINS
- ENDS WITH
- Ricerche su **range** di valori
- Ricerche su **valori null**

È importante fare una distinzione tra l'utilizzo di indici in questo contesto e in altri sistemi come quello relazionale: se nel mondo relazionale, l'uso dell'indice è fatto per **cercare righe** di certe tabelle, in un graph db l'indice serve a **cercare nodi** di partenza per una certa query.

5.3.5. Scrittura di Query in Cypher: Utilizzi Avanzati

Dopo aver visto le basi del linguaggio Cypher andiamo ora a vedere qualche costrutto più avanzato che ci consentirà di scrivere query più complesse e potenti.

Clausola CREATE

La clausola **CREATE** viene utilizzata per andare a creare nuovi nodi e relazioni all'interno del grafo. La sintassi è molto simile a quella utilizzata per specificare i pattern nella clausola **MATCH**:

```
1 CREATE (m:Movie(title: 'Mystic River', released:2003))  
2 RETURN m
```

Cypher

Nell'esempio di sopra stiamo andando a creare un'entità **Movie** con le proprietà **title** e **released** e ritorniamo all'utente l'entità appena creata.

È possibile andare a utilizzare la clausola **CREATE** anche per creare relazioni tra nodi esistenti:

```
1 MATCH (m:Movie {title: 'Mystic River'})  
2 MATCH (p:Person {name: 'Kevin Bacon'})  
3 CREATE (p)-[r:ACTED_IN {roles: ['Sean']}]>-(m)  
4 RETURN p, r, m
```

Cypher

In questo caso vediamo come sia stato prima effettuato il **lookup** dei nodi di interesse tramite la clausola **MATCH**, per poi andare a creare la relazione **ACTED_IN** tra i due nodi, con una proprietà **roles** che è una lista che conterrà i ruoli interpretati dall'attore nel film.

Osservazione

Una volta che andiamo ad utilizzare la clausola `CREATE` questa provvederà a creare tutto ciò che viene specificato al suo interno. Nel caso in cui vogliamo creare una relazione tra due nodi abbiamo due possibilità diverse:

- *entrambi i nodi sono già esistenti*: in tal caso andremo soltanto a creare la relazione
- *solo uno dei nodi o nessuno esiste*: ipotizzando che la persona con nome `Kevin Bacon` non fosse già esistente, la clausola `CREATE` la avrebbe creata per fare in modo che il path desiderato fosse coerente

Clausola `SET`

Oltre a creare entità, è possibile anche andare a modificare le proprietà delle entità, per questo scopo utilizziamo la clausola `SET`:

```
1 MATCH (m:Movie {title: 'Mystic River'})
2 SET m.released = 2004
3 SET m.tagline = 'A movie about life and loss'
4 RETURN m
```

Cypher

La query presentata sopra va sia a modificare una proprietà già esistente (`released`), sia ad aggiungerne una nuova (`tagline`).

Clausola `MERGE`

La clausola `MERGE` viene utilizzata per creare nodi o relazioni in modalità “**upsert**”, combinando le funzionalità di `MATCH` e `CREATE`. In pratica, `MERGE` cerca di trovare un nodo o una relazione che corrisponde al pattern specificato; se non lo trova, lo crea.

```
1 MERGE (p:Person {name: 'Tom Hanks'})
2 RETURN p
```

Cypher

Avvertenza

È importante considerare che l'utilizzo di questa clausola potrebbe portare a risultati inattesi se utilizzata senza la dovuta attenzione. Supponiamo di voler andare a rivisitare la l'operazione di upsert presentata sopra, ma questa volta aggiungendo il la proprietà `oscar` alla persona `Tom Hanks`. A una prima occhiata potrebbe venirci in mente di utilizzare la seguente query:

```
1 MERGE (p:Person {name: 'Tom Hanks', oscar: true})
2 RETURN p
```

Cypher

Tuttavia, questa query non funzionerà come ci aspettiamo. Se esiste già un nodo `Person` con il nome “Tom Hanks” ma senza la proprietà `oscar`, la clausola `MERGE` non lo troverà come corrispondente e creerà un nuovo nodo con la proprietà `oscar` impostata a `true`.

Di conseguenza, ci ritroveremo con due nodi distinti per “Tom Hanks”, uno con la proprietà `oscar` e uno senza. La query giusta da utilizzare in questo caso è la seguente:

```
1 MERGE (p:Person {name: 'Tom Hanks'})  
2 SET p.oscar = true  
3 RETURN p
```

Cypher

È possibile utilizzare la clausola `MERGE` anche per creare relazioni in modalità upsert. Oltre a tutte queste funzionalità, `MERGE` supporta la possibilità di specificare delle **azioni** da eseguire in base al fatto che l'entità sia stata trovata o creata, tramite l'uso delle clausole `ON MATCH` e `ON CREATE`.

```
1 MERGE (p:Person {name: 'Your Name'})  
2 ON CREATE SET p.created = timestamp(), p.updated = 0  
3 ON MERGE SET p.updated = p.updated + 1  
4 RETURN p.created, p.updated
```

Cypher

5.3.6. Data Ingestion in Neo4j

Dopo aver visto come interrogare la nostra base di dati, andiamo a vedere come effettuare una delle operazioni più importanti nel mondo del data management, ovvero l'**ingestione** dei dati all'interno del database.

Cypher è dotato di una clausola specifica per questo scopo, che permette di partire da un file `.csv`, si chiama appunto `LOAD_CSV`. Di seguito ne mostriamo alcune caratteristiche:

- Permette di caricare dati in formato `.csv` da un file locale o da un URL
- Preso in input un file `.csv` fornisce uno **stream** di record che possono essere processati tramite le classiche clausole messe a disposizione da Cypher (`MATCH`, `CREATE`, `SET`, ecc.)
- Supporta l'esecuzione di **operazioni transazionali** sul grafo esistente
- È in grado di convertire i valori letti dal file `.csv` in tipi di dati nativi di Neo4j (es. stringhe, numeri, booleani, ecc.)

Ipotizziamo che il nostro property graph model sia descritto in Figura 5.15.

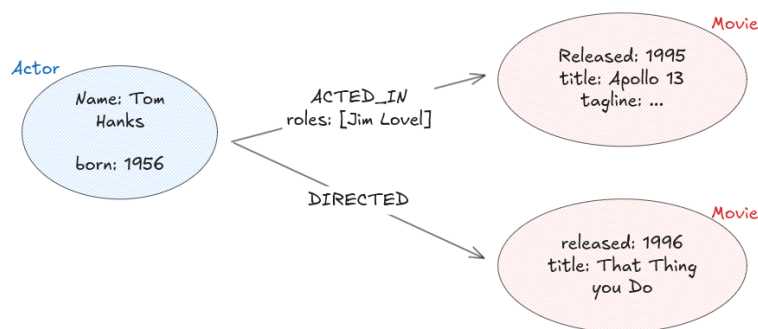


Figura 5.15: Esempio di property graph model che rappresenta persone e film con relazioni

Per ogni record del file `.csv` da utilizzare per l'ingestion vogliamo distinguere i casi seguenti:

- Creiamo un nodo di tipo `Person` o `Movie` se questo non esiste già:

```
1 CREATE (:Person {  
2     name:row.name,  
3     born:toInt(row.born)  
4 });
```

Cypher

- Cerchiamo nodo di inizio e nodo di fine e creiamo tr loro una relazione:

```
1 MATCH (m:Movie {title:row.movie}),  
2     (p:Person {name: row.person}),  
3 CREATE (p)-[:ACTED_IN {roles:split(r.roles)}]->(m);
```

Cypher

Una volta definiti i classici statements da utilizzare per fare ingestion, è necessario andare a mostrare in quale maniera dire a Neo4j di effettuare ingestion:

```
1 [USING PERIODIC COMMIT] // attiva le transazioni per  
  batchCypher  
2  
3 LOAD CSV // statement per iniziare a effettuare ingestion  
4  
5 WITH HEADERS // opzionalmente usare la prima riga del file come  
  chiave per gli elementi letti  
6  
7 FROM "url" // path o url da cui leggere il file csv  
8  
9 AS row // ritorna ogni riga come lista di stringe o mappa  
10  
11 FILEDTERMINATOR ";" // specifica qual è il separatore dei  
  valori  
12  
13  
14 ... gli altri statement che specificano come fare ingestion ...
```

6. Gestione di Dati Distribuiti

Dopo aver introdotto varie famiglie di basi di dati, e aver visto come in moltissimi contesti, la loro nascita è dovuta al fatto di permettere una scalabilità orizzontale, andremo ora a vedere come effettivamente queste basi di dati permettono di gestire **dati distribuiti** su diversi nodi.

6.1. Concetti di Base

Come già menzionato, il principio alla base delle basi di dati distribuite è quello della **scalabilità**. Nel particolare, esistono due tipologie di scalabilità di cui si può parlare:

- **Scale up** (o *verticale*): consiste nell'aggiunta di più potenza computazionale ad un singolo server che ospita la base di dati. Questo consente tipicamente di migliorare le prestazioni, ma ha dei limiti fisici e di costo.
- **Scale out** (o *orizzontale*): consiste nell'aggiunta di più nodi (server) al sistema di database. Questo approccio consente di distribuire il carico di lavoro e i dati.

Quando andiamo a parlare di basi di dati distribuite, andremo a riferirci alle seguenti componenti fondamentali:

- **Database distribuito**: si tratta di un insieme di dati che sono *logicamente interconnessi*
- **DBMS distribuito**: si tratta di un sistema per la gestione di *database distribuiti*, che ha la fondamentale capacità di rendere la **distribuzione trasparente**

Trasparenza

Proprio questo concetto di **trasparenza** è fondamentale da approfondire. In generale per un utente non dovrebbe essere rilevante come internamente il DBMS gestisce lo storage dei dati e l'esecuzione di query. Esistono diverse tipologie di trasparenza:

- **Access Transparency**: l'accesso ai dati è indipendente dalla struttura della rete o dall'organizzazione fisica dei dati.
- **Location Transparency**: gli utenti non devono conoscere come i dati sono distribuiti.
- **Replication Transparency**: gli utenti non devono conoscere se stanno accedendo a dati replicati o meno.
- **Fragmentation Transparency**: la frammentazione e lo sharding sono tipicamente gestiti internamente. Un utente dovrebbe poter effettuare query alla base di dati come se fosse un'unica entità, senza preoccuparsi di come i dati sono stati frammentati.
- **Migration Transparency**: i dati possono essere spostati tra nodi senza che gli utenti ne siano consapevoli.
- **Concurrency Transparency**: il sistema deve gestire l'accesso concorrente ai dati in modo che gli utenti non debbano preoccuparsi di conflitti o problemi di coerenza.
- **Failure Transparency**: il sistema deve essere in grado di gestire guasti dei nodi o della rete senza che gli utenti ne siano consapevoli.

6.2. Guasti nei Sistemi Distribuiti

È importante andare a considerare come in un sistema distribuito, i guasti sono inevitabili. Esistono diverse tipologie di guasti che possono verificarsi. Lo scopo di questa sezione è quello di andare a presentarli brevemente assieme alle tecniche più comuni per mitigarli.

Il primo caso, più semplice è quello in cui a guastarsi sia un singolo nodo (server del sistema). In questo caso andremo a parlare di **server failure**. Figura 6.1 mostra un'illustrazione di questo tipo di guasto.

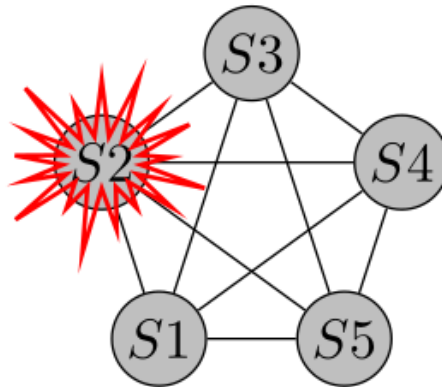


Figura 6.1: Illustrazione di un guasto su un nodo singolo in un sistema distribuito.

Un'altra tipologia di guasto è quella di **network failure**, in cui la comunicazione tra nodi viene interrotta. Questo può essere causato da problemi di rete, congestione o guasti hardware. Figura 6.2 mostra un'illustrazione di questo tipo di guasto.

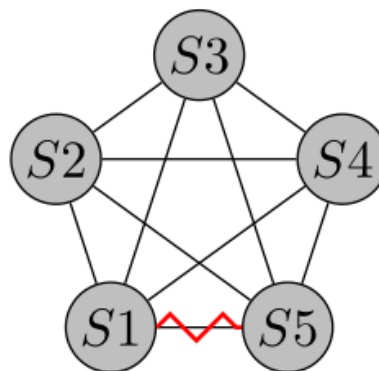


Figura 6.2: Illustrazione di un guasto di rete in un sistema distribuito.

Nell'eventualità in cui si verificano **guasti multipli**, potremmo trovarci in una condizione chiamata **network partitioning**, in cui la rete di comunicazione tra i server risulta divisa in più segmenti che non possono comunicare tra di loro. Figura 6.3 mostra un'illustrazione di questo tipo di guasto.

Osservazione

È importante considerare che un segmento di partizionamento potrebbe essere composto anche da un singolo nodo, nel caso in cui questo risulti isolato dal resto della rete.

Si noti come spesso e volentieri le situazioni di **network partitioning** siano estremamente difficili da distinguere rispetto a quelle di **server failure**, in quanto da un nodo potrebbe sembrare che un altro nodo sia semplicemente non raggiungibile.

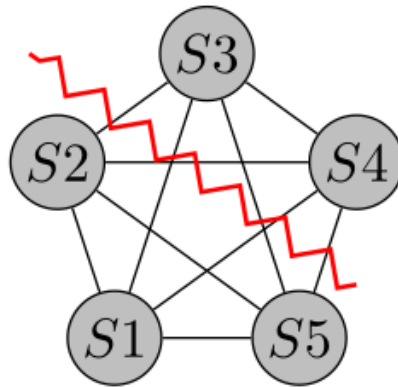


Figura 6.3: Illustrazione di un partizionamento di rete in un sistema distribuito.

6.3. Protocolli Epidemici

6.3.1. Background

Uno dei punti cruciali all'interno di un sistema distribuito è la **propagazione** delle **informazioni**. In un sistema distribuito, si tratta di un compito complesso, per due principali motivi:

- *Perdita di messaggi*: in una rete distribuita, i messaggi possono essere persi a causa di guasti di rete, congestione o altri problemi, e maggiore il numero di nodi e connessioni, maggiore è la probabilità che questo accada.
- *Forte agitazione dei nodi*: in un sistema distribuito, i nodi possono entrare e uscire dalla rete in qualsiasi momento, rendendo difficile mantenere una visione coerente dello stato del sistema.

Una delle tecniche più interessanti per mitigare questi problemi è l'utilizzo di **protocolli epidemici**, *protocolli peer to peer* ispirati al modo in cui le infezioni o il “gossip” si diffondono in una popolazione. Per capire il motivo dietro al quale utilizziamo una modalità peer to peer, consideriamo il caso in cui usassimo un modello **point to point**: per utilizzare questa strategia ogni nodo dovrebbe quantomeno essere a conoscenza dell'esistenza di tutti gli altri nodi. Per garantire questo avremmo due possibilità:

- Ogni nodo mantiene una lista di tutti gli altri nodi: questo approccio non scala bene, in quanto ogni volta che un nodo entra o esce dalla rete, tutti gli altri nodi devono essere aggiornati.

- Utilizzare un nodo centrale per mantenere la lista dei nodi: questo approccio introduce un singolo punto di fallimento e può diventare un collo di bottiglia.

Il ruolo dei protocolli epidemici è dunque quello di permettere propagare le informazioni ricevute da un nodo a tutti gli altri, nel modo più efficiente possibile, e con la massima affidabilità possibile. All'interno di un protocollo epidemico, ogni nodo potrebbe trovarsi in uno dei seguenti stati:

- **Infetto**: il nodo ha ricevuto l'informazione e la sta *propagando ad altri nodi*
- **Suscettibile**: il nodo non ha ancora ricevuto l'informazione, ma è in grado di riceverla.
- **Rimosso**: il nodo ha ricevuto l'informazione, ma non la propaga più ad altri nodi.+

Il motivo per cui un i nodi possono essere infetti o suscettibili appare abbastanza chiaro: senza nodi suscettibili, l'informazione non si potrebbe propagare, e in modo analogo senza nodi infetti, non ci sarebbe nessuno a propagarla. Per quanto riguarda lo stato di **rimosso** invece, questo viene introdotto per evitare che l'informazione venga propagata all'infinito. Per fare questo i nodi infetti possono decidere di diventare rimossi dopo un certo periodo di tempo, o dopo aver propagato l'informazione ad un certo numero di nodi.

6.3.2. Varianti di Protocolli Epidemici

Esistono diverse varianti di protocolli epidemici, ognuna con le proprie caratteristiche e vantaggi. Di seguito andiamo a presentarne alcune.

Variante Anti-Entropica

Un approccio di tipo **anti-entropico** è tra i più semplici che si possono applicare. Ogni nodo in questo protocollo può essere soltanto **infetto** o **suscettibile**. I nodi infetti *periodicamente* propagano l'informazione ai loro nodi vicini suscettibili. Questo processo continua fino a quando tutti i nodi sono stati infettati.

Il problema maggiore di questo approccio consiste nell'**eccessivo utilizzo di banda** e che ad ogni round, ogni nodo controlla che i suoi vicini siano suscettibili a nuove informazioni.

Approccio Rumor Spreading

La propagazione delle informazione viene avviata solamente nel momento in cui un nodo riceve **nuova informazione** o può essere attivata in maniera **periodica**. La differenza rispetto a prima è che in questo caso le dinamiche di propagazione sono differenti:

- La quantità di nodi infetti viene fatta decrescere nel tempo mano a mano che il numero di nodi **rimossi** aumenta
- Ogni server può passare da **infetto** a **rimosso** sulla base di alcune euristiche, per esempio con una certa probabilità ad ogni round, o dopo aver inviato l'informazione ad un certo numero di nodi.

6.3.3. Hash Trees

Come già anticipato, è necessario garantire che ogni coppia di nodi possa stabilire quali informazioni sono rispettivamente note ad ognuno. Per questa operazione si potrebbe pensare di applicare una semplice operazione di **differenza insiemistica**. Tuttavia un'operazione di questo tipo potrebbe risultare inefficiente, richiedendo un numero di operazioni pari ad $O(n)$,

dove n è il numero di elementi da confrontare. Per ovviare a questo problema, si può utilizzare una struttura dati chiamata **hash tree** (o **merkle tree**).

Un **hash tree** può essere visto come un **indice** gerarchico di hash. Ogni nodo foglia dell'albero rappresenta un blocco di dati, e ogni nodo interno rappresenta l'hash dei suoi nodi figli. In questo modo, ogni nodo può rappresentare un insieme di dati tramite un singolo hash. Questo permette di confrontare rapidamente grandi insiemi di dati. Figura 6.4 ne mostra un esempio.

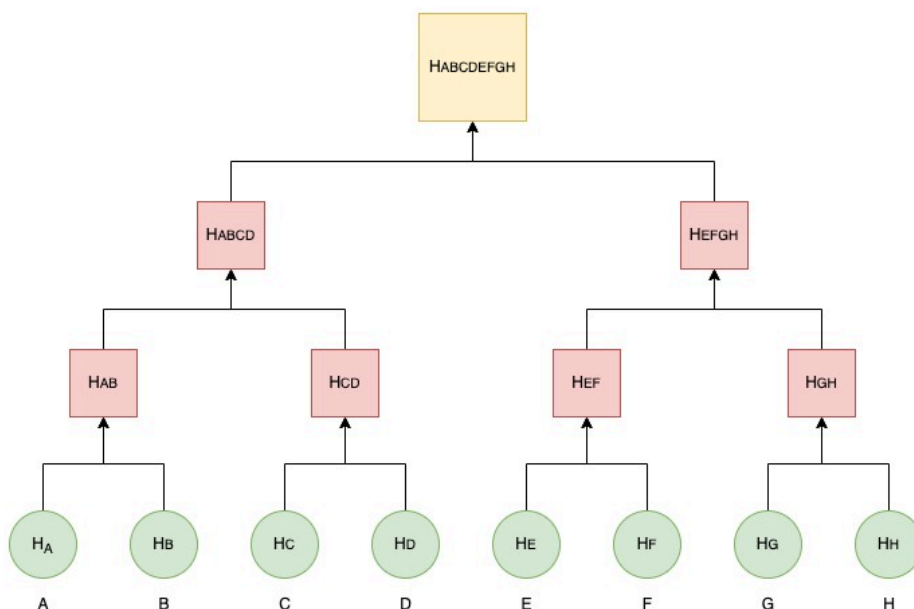


Figura 6.4: Esempio di Hash Tree con 8 nodi foglia data una lista di messaggi A,B,C,D,E,F,G,H.

Nel caso in cui tutti i messaggi ricevuti da due nodi siano gli stessi, anche gli hash dei nodi radice saranno gli stessi. In caso contrario, i nodi possono scendere lungo l'albero confrontando gli hash dei nodi figli per identificare quali blocchi dati differiscono.

Osservazione

Chiaramente l'utilizzo di una struttura come gli Hash Tree implica che tutti i nodi della rete devono concordare sull'ordinamento dei messaggi che pervengono e su una funzione di hashing univoca.

6.4. Frammentazione («Sharding»)

Una delle tecniche più comuni per la gestione di dati grandi e massivi è quella di ricorrere alla **frammentazione** degli stessi. L'idea è quella di suddividere appunto grandi quantità di dati in **frammenti più piccoli** che possano essere gestiti in maniera più agile. Di seguito presentiamo vantaggi e svantaggi di questa tecnica.

- **Località dei dati:** possiamo delegare ad un nodo locale la gestione e la computazione di operazioni su un frammento di dati comunicando il risultato finale agli altri nodi ✓
- **Riduzione dei costi di comunicazione:** svolgendo le operazioni a livello di singolo frammento in locale, è possibile ridurre la quantità di dati che deve essere trasferita nella rete ✓

- **Performance migliorate:** operazioni su frammenti più piccoli di dati tendono ad essere più veloci rispetto a operazioni su grandi insiemi di dati ✓
- **Indici più efficienti:** gli indici possono essere creati e mantenuti più facilmente su frammenti più piccoli di dati ✓
- Possibilità di applicare **load balancing**: distribuendo i frammenti di dati tra diversi nodi, è possibile bilanciare il carico di lavoro e migliorare le prestazioni complessive del sistema ✓
- **Query più complesse:** le query che coinvolgono più frammenti di dati possono diventare più complesse da gestire e ottimizzare ✗
- **Gestione più complessa:** operazioni di backup e recovery possono diventare più complesse in un sistema frammentato ✗

6.4.1. Allocazione

Ogni volta che tentiamo di memorizzare delle nuove informazioni queste vengono suddivise in **unità di allocazione** e serve decidere quanti e quali nodi dovranno essere impiegati per memorizzare i frammenti. Un'altra tematica importante è quella del **load balancing**: se tutti i frammenti andassero a finire su un singolo nodo, questo diventerebbe un collo di bottiglia per l'intero sistema. Abbiamo diverse possibilità per gestire l'allocazione:

- **Range-based allocation:** supponiamo che i nodi abbiano un identificativo numerico. In questo caso possiamo decidere di allocare i frammenti in base a degli intervalli di identificativi. Per esempio, il nodo 1 potrebbe essere responsabile per i frammenti con ID da 0 a 99, il nodo 2 per quelli da 100 a 199, e così via. Questo approccio è semplice da implementare, ma può portare a squilibri se la distribuzione dei dati non è uniforme.

Esempio: Distribuzione uniforme vs. non-uniforme

se memorizziamo date di nascita con solo giorno e mese, la distribuzione sarà pressoché uniforme, mentre andando a memorizzare anche gli anni questo sarà diverso dal momento che alcuni anni sono più rappresentati di altri

- **Cost-based allocation:** il nodo sul quale verranno allocati i frammenti viene scelto in base al costo stimato di accesso ai dati. Questo approccio può essere più complesso da implementare, ma può portare a una migliore distribuzione del carico di lavoro.
- **Hash-based allocation:** utilizza una funzione di hash per mappare i frammenti ai nodi. Si tratta di una tecnica molto simile a quella "range-based" con il vantaggio di distribuire i frammenti più uniformemente tramite le funzioni di hash.

Per quanto la **hashed based allocation** sembri essere una tecnica molto valida, questa presenta due importanti criticità:

- necessità di una conoscenza a priori del numero di nodi nel sistema
- nel momento in cui un nodo subisse dei guasti, molti hashcodes verrebbero invalidati, richiedendo una riallocazione massiva dei frammenti

Consistent Hashing

Una tecnica molto interessante per ovviare ai problemi della **hash-based allocation** è quella del **consistent hashing**. In questo approccio la funzione di hash viene calcolata sia sui **nodi** che sui **frammenti di dati**.

Tutti i valori di hash vengono mappati su un **hash ring**, un intervallo circolare di valori di hash. Ogni frammento di dati viene allocato al nodo il cui valore di hash è più vicino al valore di hash del frammento, procedendo in senso orario lungo l'anello. Figura 6.5 ne mostra un esempio.

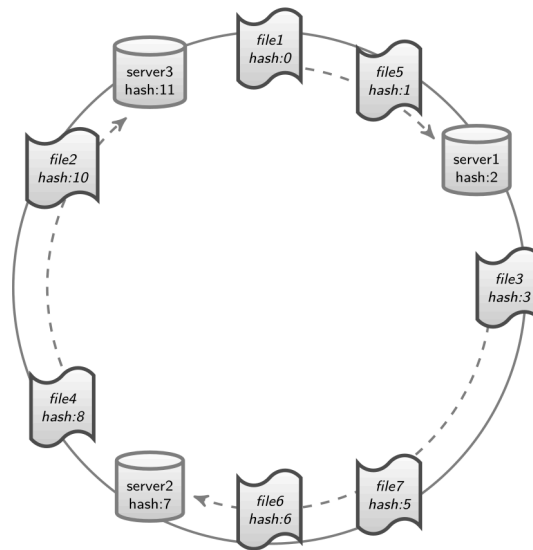


Figura 6.5: Esempio di Consistent Hashing con 3 nodi e 6 frammenti di dati.

Nel caso di **rimozione di un nodo**, tutti i frammenti del nodo vengono copiati nel nodo successivo nell'anello. Questa situazione è chiaramente illustrata in Figura 6.6.

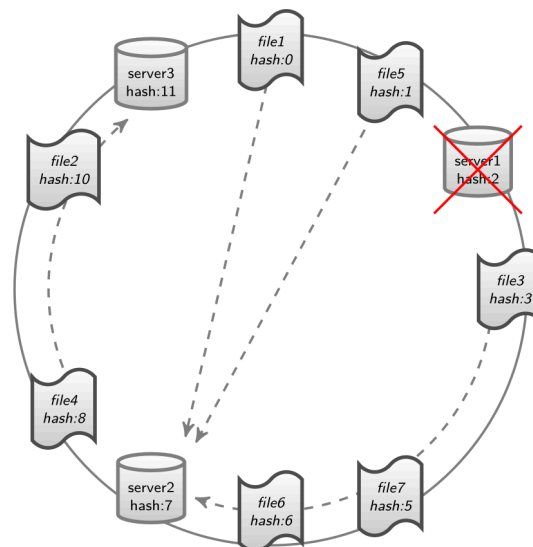


Figura 6.6: Esempio di Consistent Hashing con rimozione di un nodo.

In maniera abbastanza simile, nel caso in cui si proceda con l'**aggiunta di un nodo**, andremo a considerare il nodo precedente nell'anello e riallocheremo tutti i frammenti che ora ricadono sotto la responsabilità del nuovo nodo. Figura 6.7 illustra questa situazione.

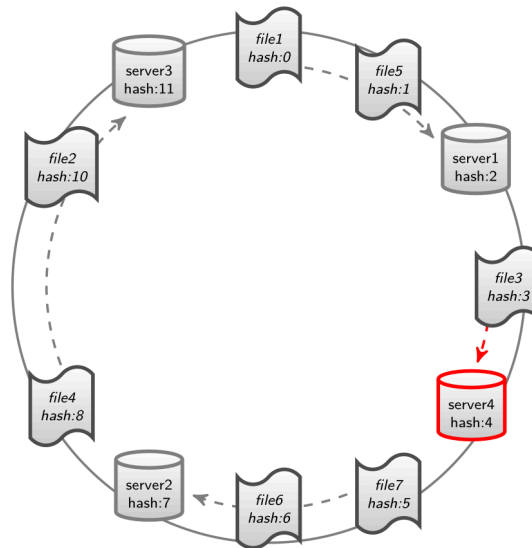


Figura 6.7: Esempio di Consistent Hashing con aggiunta di un nodo.

Ciò che manca ora è capire come andare a gestire lo stato di questo anello. Supponiamo di avere un client che prova a connettersi in scrittura al nostro sistema distribuito. Per capire in quale nodo scrivere i dati dovremo:

- Calcolare l'hash di un frammento di dati, contattare un nodo casuale in scrittura. A questo punto abbiamo due possibilità:
- Il nodo ha conoscenza dell'intero anello: in questo caso può calcolare direttamente il nodo responsabile per il frammento e inoltrare la richiesta.
- Il nodo può provare ad inoltrare la richiesta al nodo successivo nell'anello, che a sua volta può fare lo stesso fino a quando la richiesta non raggiunge il nodo responsabile.

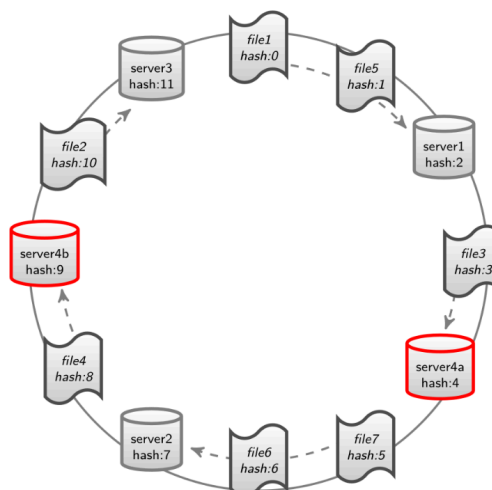


Figura 6.8: Illustrazione dell'utilizzo di nodi virtuali nel Consistent Hashing.

Evidentemente la seconda opzione è quella che potrebbe richiedere meno sforzo di mantenimento dello stato dell'anello, ma potrebbe richiedere più tempo per raggiungere il nodo responsabile. Per mantenere lo stato dell'anello, l'unica informazione necessaria in ogni nodo è l'identificativo del nodo successivo nell'anello.

Spesso e volentieri, non è verosimile che all'interno di un sistema distribuito tutti i nodi abbiano la stessa capacità computazionale e di storage. In una situazione tale, potremmo non voler distribuire i frammenti in maniera uniforme, andando a prediligere nodi con maggior capacità. Per risolvere questo requisito, possiamo *simulare la presenza di nodi aggiuntivi* gestiti da uno stesso server, chiamati **nodi virtuali**. Utilizzando questo approccio possiamo assegnare più nodi virtuali a server con maggior capacità, e meno nodi virtuali a server con minore capacità. In questo modo, i frammenti di dati verranno distribuiti in maniera proporzionale alla capacità di ogni server. Figura 6.8 illustra questo concetto.

6.5. Replicazione dei Dati

Oltre a frammentare i dati per rendere le computazioni più efficienti e sostenibili, un'altra tecnica molto comune per la gestione di dati distribuiti è quella della **replicazione**. In questo contesto ci sono due termini che sono importanti da conoscere:

- Le copie dei dati che vengono effettuate sono chiamate **repliche**
- Il numero di nodi su cui le repliche sono memorizzate è chiamato **replication factor**

L'impiego di questa tecnica serve sostanzialmente a due scopi:

- **Affidabilità e disponibilità:** avendo più copie dei dati distribuiti su diversi nodi, il sistema può continuare a funzionare anche in caso di guasti di uno o più nodi.
- **Minor Latenza:** le repliche possono essere utilizzate per effettuare load balancing, e parallelizzazione delle letture, riducendo la latenza di accesso ai dati.

Ovviamente la replicazione dei dati introduce anche delle sfide, tra cui:

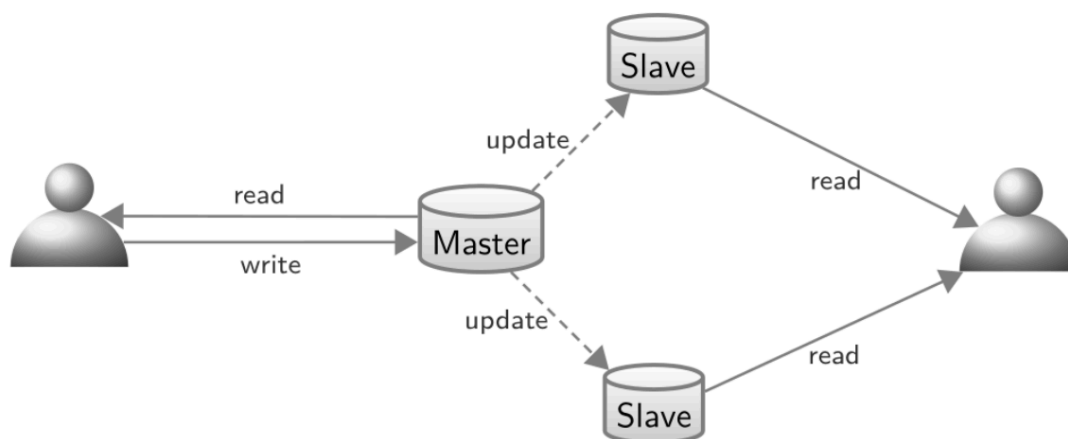
- **Coerenza dei dati:** mantenere tutte le repliche aggiornate può essere complesso
- **Concorrenza:** gestire accessi concorrenti ai dati replicati può portare a conflitti

6.5.1. Replicazione Master-Slave

Si tratta di una delle architetture di replicazione più semplici. In questo modello, un nodo viene designato come **master** mentre gli altri nodi vengono scelti come **slave**. Di seguito andiamo ad elencarne le peculiarità:

- Tutte le operazioni di **scrittura e aggiornamento** sono svolte sul nodo **master**
- Dopo una certa quantità di tempo, il master propaga le modifiche agli **slave**
- Le operazioni di **lettura** possono essere svolte su qualsiasi nodo, sia master che slave

Chiaramente, questo modello presenta un grosso svantaggio: il nodo master rappresenta un **single point of failure**. In caso di guasto del master, il sistema non può più accettare operazioni di scrittura fino a quando un nuovo master non viene eletto. L'immagine presentata di seguito da una rappresentazione schematica del funzionamento di questa architettura.



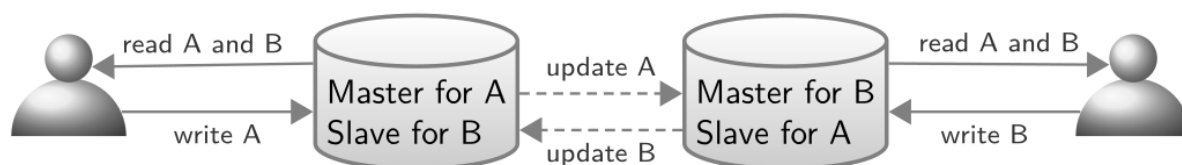
6.5.2. Replicazione Multi-record

Questo approccio si ispira al modello master-slave introducendo alcune modifiche. Sostanzialmente ogni nodo nel sistema può agire sia come **master** per certe informazioni, sia come **slave** per altre.

Ogni nodo può accettare operazioni in **scrittura** per dati di sua competenza, e propagare le modifiche agli altri nodi. Le operazioni di **lettura** possono essere svolte su qualsiasi nodo. La figura mostrata di seguito illustra questo concetto.

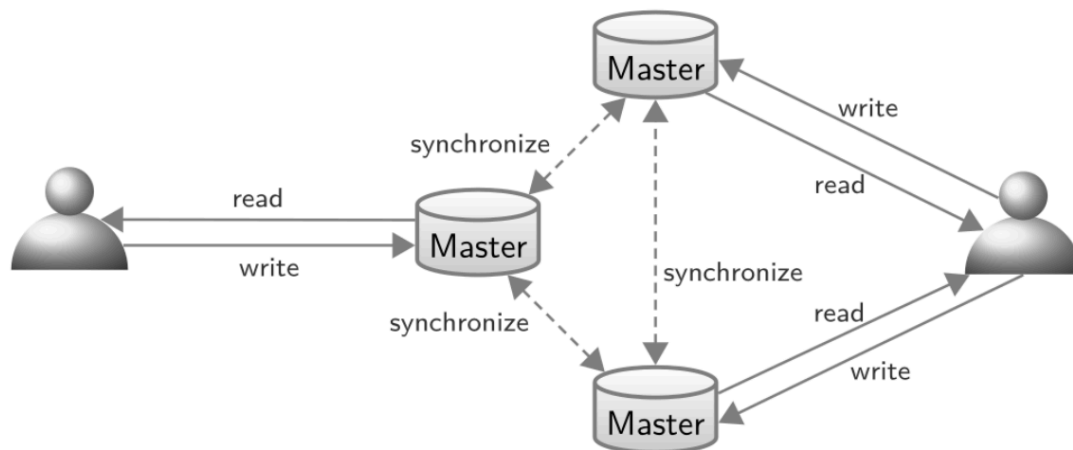
Per andare a sincronizzare le modifiche tra i vari nodi, abbiamo a disposizione due possibili strategie:

- **Replicazione immediata:** ogni volta che un nodo master viene modificato, queste modifiche sono immediatamente propagate agli altri nodi. Questo approccio garantisce una maggiore coerenza, ma può introdurre latenza nelle operazioni di scrittura.
- **Replicazione differita:** le modifiche vengono propagate agli altri nodi dopo un certo intervallo di tempo o quando viene raggiunta una certa soglia di modifiche. Questo approccio può migliorare le prestazioni, ma può portare a situazioni di incoerenza temporanea tra i nodi.



6.5.3. Replicazione Multi-Master

Questo approccio, detto anche «update-anywhere», permette a qualsiasi nodo di accettare operazioni di scrittura e aggiornamento. Si tratta di un approccio che consente **maggiore disponibilità** e **tolleranza ai guasti**. Permette di avere **tempi di risposta migliorati**. Tuttavia, questo modello introduce delle importanti sfide legate alla **coerenza dei dati**: spesso si rende infatti necessario implementare meccanismi di risoluzione dei conflitti per gestire situazioni in cui più nodi tentano di aggiornare gli stessi dati contemporaneamente.



L'immagine presentata sopra illustra il funzionamento di questa architettura. Ogni nodo può accettare operazioni di scrittura e aggiornamento, e le modifiche vengono propagate agli altri nodi. In caso di conflitti, il sistema deve essere in grado di risolverli in modo coerente.

6.5.4. Guasti e Recovery

Avendo discusso la replicazione dei dati, il prossimo passaggio è quello di andare a comprendere come i guasti vengano gestiti in un'architettura distribuita. Ipotizziamo di avere un **replication factor** di 2. Nel caso in cui uno dei due server subisse un guasto, o fosse momentaneamente non raggiungibile, l'altro server dovrebbe essere in grado di continuare a servire le richieste di lettura e scrittura. Nel momento poi in cui il server guasto riprendesse a funzionare, sarà necessaria una **sincronizzazione** con l'altro server per portare il sistema in uno stato coerente. Questa situazione è illustrata in Figura 6.9.

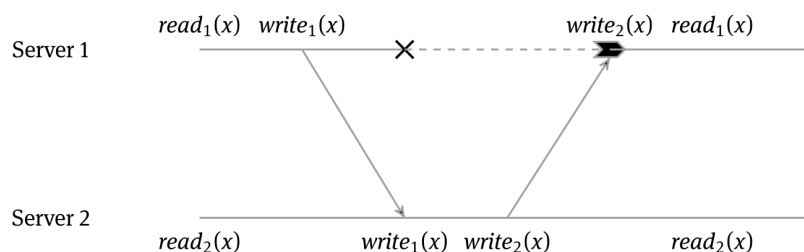


Figura 6.9: Illustrazione di un guasto di singolo server con replication factor = 2.

Nel caso però in cui, sempre con un **replication factor** di 2, se il secondo server fallisse prima che il primo possa tornare disponibile, tutte le scritture accettate dal secondo server non saranno visibili dal primo. Questo scenario è illustrato in Figura 6.10.

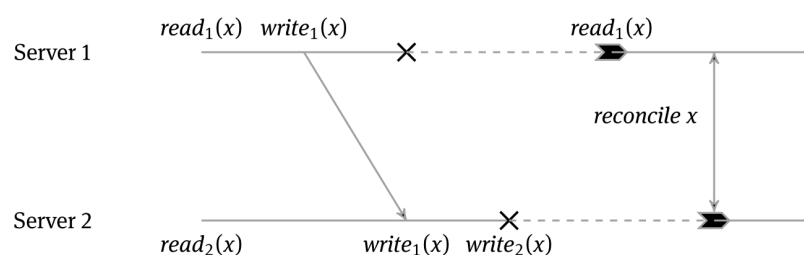


Figura 6.10: Illustrazione di un guasto di entrambi i server con replication factor = 2.

In questo caso, nel momento in cui entrambi i server torneranno funzionanti, tutte le richieste di scrittura accettate in maniera indipendente dai server andranno **riconciliate**. Questa situazione ci mette davanti ad un quesito fondamentale, legato a *quale sia il corretto **replication factor*** da utilizzare. Tenzialmente una convenzione accettata abbastanza in generale è quella di utilizzare un replication factor pari a 3.

Di seguito andiamo a presentare due tecniche molto comuni per la gestione dei guasti e della successiva riconciliazione dei dati.

Hinted Handoff

Questa tecnica viene utilizzata per gestire i guasti temporanei dei nodi in un sistema distribuito. Nel seguente elenco puntato andiamo a mostrarne il flusso operativo:

- nel momento in cui una replica non è disponibile, le richieste in scrittura vengono **delegate** ad un altro nodo disponibile
- il nodo che riceve le richieste di scrittura riceve un **hint** che indica quale nodo era il destinatario originale della scrittura, in modo tale da poter inoltrare la scrittura non appena il nodo originale torni disponibile

Read Repair

Nell'utilizzo di questa tecnica un *coordinatore* manda una serie di richieste di lettura alle repliche da lui conosciute. Nel momento in cui i nodi rispondono al coordinatore, questo effettua un **majority vote** per capire cosa rispondere al client.

Oltre a fornire risposte il più coerenti possibili al client, il coordinatore si occupa di inviare istruzioni alle repliche che hanno risposto con dati incoerenti in modo da **sincronizzare lo stato** del sistema.

Osservazione

Si noti come, nel caso in cui utilizzassimo un singolo nodo coordinatore, andremmo ad introdurre un **single point of failure**. Per ovviare a questo problema non viene mai scelto un nodo coordinatore fisso, ma questo viene eletto secondo un protocollo di **leader election** ogni volta che un client invia una richiesta.

6.6. Gestione della Concorrenza

Come già menzionato in precedenza, uno dei principali problemi all'interno di un sistema distribuito è quello della gestione della **concorrenza**. Potremmo infatti trovarci nella situazione in cui più componenti tentino di lavorare sugli stessi dati in contemporanea. In questi casi, è fondamentale garantire che tutti i nodi rimangano sincronizzati e l'operazione venga conclusa in maniera coerente.

Commit a 2 Fasi

Il **Two-Phase Commit (2PC)** è un protocollo utilizzato per garantire che le **transazioni distribuite** vengano completate in modo atomico, anche in presenza di guasti. L'idea è che tutti gli agenti devono essere d'accordo sulla finalizzazione di una transazione. Il protocollo si

divide in una fase di **voting** e in una fase di **decisione**. Anche in questo caso avremo bisogno di un nodo che funga da **coordinatore**. Di seguito andiamo ad elencare alcune altre specificità:

- Il fallimento di un singolo “agente” implica che l’intera transazione venga abortita
- Lo stato che si interpone tra le due fasi è detto “**doubt-phase**”, dato che gli agenti non sanno se la decisione del coordinatore sarà di committare o abortire la transazione
- In caso di guasto del coordinatore, gli agenti rimangono bloccati senza poter committare o abortire

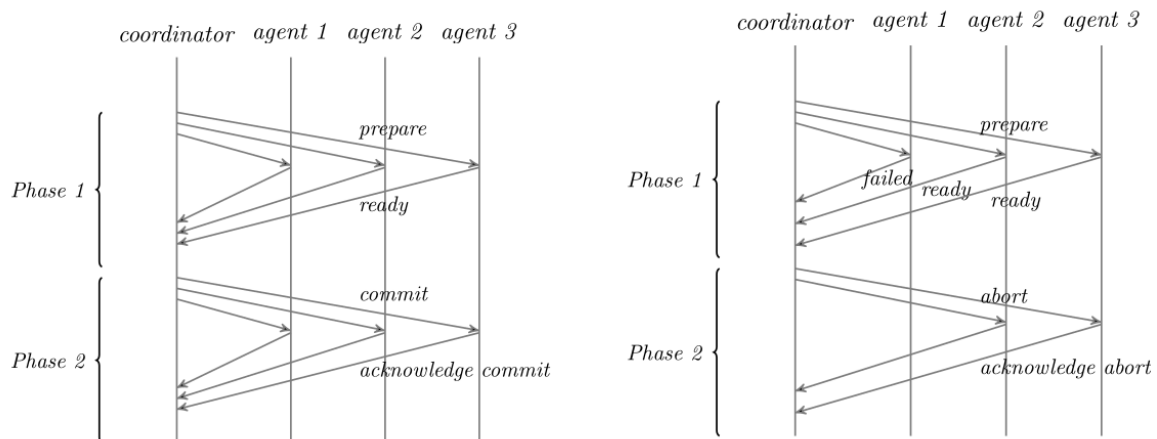


Figura 6.11: Illustrazione del Two-Phase Commit: a sinistra un commit riuscito, a destra un commit abortito.

6.7. Proprietà dei Database Distribuiti

Per concludere questa panoramica sulle basi di dati distribuite, andiamo a presentare le **proprietà** che queste devono / possono garantire.

Consistenza

Quando un nodo modifica un valore in una replica, il valore dovrebbe essere modificato in tutte le repliche; o se non immediatamente, almeno prima che ci sia la necessità di leggere un certo dato. Si tratta di un processo particolarmente **costoso** nel caso in cui venga utilizzato un **replication factor grande**. Un modo per implementarlo potrebbe consistere nel mantenere bloccate in lettura tutte le repliche fino a quando tutte non siano state aggiornate.

Disponibilità

In un sistema distribuito, è fondamentale che ogni richiesta di lettura o scrittura riceva una risposta anche in presenza di guasti. In particolare vengono tenute in considerazione due *metriche*: **efficienza** in risposta alle query e **bassi tempi di risposta**.

Tolleranza alle Partizioni

Un sistema distribuito deve essere in grado di continuare a funzionare anche in presenza di *partizioni di rete*. Questo significa che anche se alcuni nodi non possono comunicare tra di loro, il sistema deve essere in grado di accettare e processare le richieste.

6.7.1. Congettura di Brewer

Dopo aver definito le proprietà, in questa sezione andiamo a presentare la **congettura di Brewer** che stabilisce una sorta di relazione tra queste proprietà.

Teorema 6.1 (Congettura CAP di Brewer)

Date le proprietà sopra elencate (*consistency, availability, partition tolerance*), in un sistema distribuito è possibile garantire **al massimo due** di queste proprietà contemporaneamente.

In sostanza, in base ai requisiti del sistema che abbiamo la necessità di implementare, andremo a scegliere due delle proprietà che vogliamo andare a garantire. Per esempio, in un sistema in cui è richiesta *alta disponibilità*, è sarà necessario sacrificare la consistenza a favore di una maggiore tolleranza alle partizioni.

Sempre in merito alla consistenza, possiamo distinguere tra due principali modelli:

- **Consistenza forte:** dopo un aggiornamento, ogni lettura successiva restituirà il valore aggiornato e corretto
- **Consistenza debole:** il sistema non garantisce che dopo un aggiornamento le letture seguenti restituiscano il valore aggiornato immediatamente.

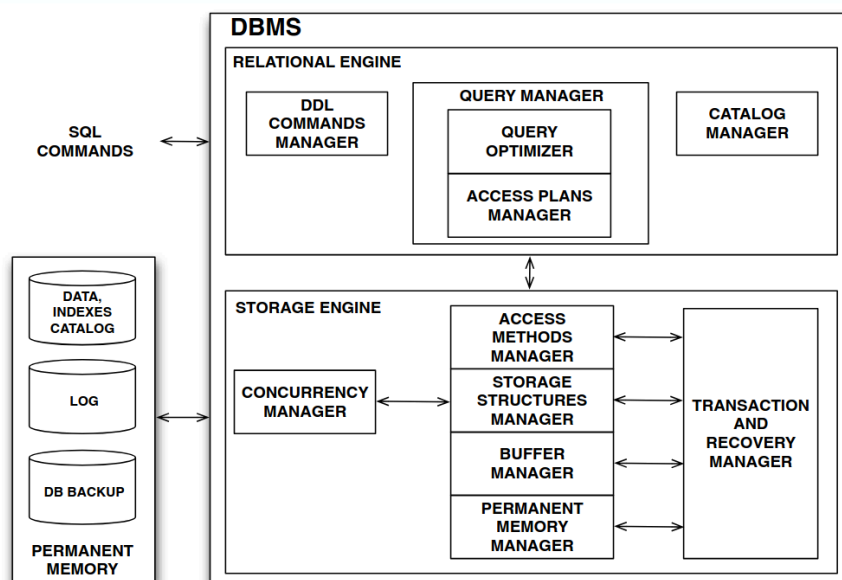
Proprio in merito a questa forma *rilassata* di consistenza, possiamo introdurre due concetti. Il primo è quello di **finestra di inconsistenza**, che rappresenta un intervallo di tempo tra un aggiornamento il momento in cui tutte le repliche non sono ancora state aggiornate. Durante questa finestra, le letture potrebbero restituire valori obsoleti.

Dato il concetto di finestra di inconsistenza, possiamo introdurre la nozione di **eventual consistency** che garantisce che, in *assenza di aggiornamenti*, tutte le repliche convergeranno verso lo stesso valore dopo un certo periodo di tempo, tale periodo è appunto la finestra di inconsistenza.

Uno dei metodi utilizzato per cercare di ottenere consistenza in un sistema distribuito è quello di utilizzare il concetto di **quorum**. In questo approccio, per ogni operazione di lettura o scrittura viene richiesto un numero minimo di repliche per completare l'operazione. Questo concetto potrà essere meglio approfondito all'interno del corso di sistemi distribuiti.

7. Architetture per Basi di Dati

Questo capitolo dà inizio alla seconda parte del corso. Tutto ciò che tratteremo da questo momento in poi supporrà che il punto di partenza sia quello di un **java relational system**. La struttura di base di tale elemento è quella che è già stata presentata in Figura 3.1, che per completezza viene di seguito riportata.



La scelta di considerare questa architettura viene fatta per semplicità, e per la maggiore familiarità che si ha tipicamente con una base di dati relazionale. Tuttavia, è importante considerare che la maggioranza delle famiglie di database già presentate condivide la gran parte delle componenti con questa architettura.

Come già menzionato l'architettura presentata può essere rappresentata in **due blocchi**

- un blocco che presenta delle componenti specifiche alla famiglia di database che abbiamo scelto di utilizzare, che in questo caso è il **relational engine**
- un blocco che è deputato alla gestione della memorizzazione e il recupero dei dati in memoria principale, a cui ci si riferisce tipicamente con il nome di **storage engine**

Questo schema a blocchi non è sempre presente, esistono casi in cui lo storage engine viene fuso alla componente che normalmente dovrebbe essere più specifica al tipo di dati che vogliamo rappresentare nella nostra base di dati.

In linea di massima però relational (o document database, ...) e storage engine sono tra loro *indipendenti*. Questo maggiore livello di astrazione ci consente di trattare in maniera separata il modo in cui vengono rappresentati a livello logico i dati dal modo in cui questi vengono effettivamente trattati all'interno della memoria secondaria.

Nel corso del capitolo procederemo a presentare le varie componenti seguendo un approccio **bottom-up**, partendo dunque dallo storage engine. Dal momento che non si tratta di componenti fondamentali per il funzionamento di una base di dati, eviteremo di trattare, per ora, il gestore delle transazioni, del recovery e della concorrenza. Andremo dunque a concentrarci sulla colonna centrale dello storage engine.

Vedremo inoltre come questo stack di componenti sia molto simile nel concetto allo stack ISO/OSI, dove ogni componente fornisce servizi per il componente di livello superiore e ne utilizza dal livello inferiore, fino ad arrivare al livello più basso possibile.

7.1. Persistent Memory Manager

Come già menzionato in qualche capitolo precedente il principale ruolo del gestore della memoria persistente è quello di fornire un'**astrazione linearmente indirizzabile** della memoria, nascondendo ai livelli superiori l'effettiva struttura della memoria.

È importante andare a vedere come la memoria secondaria sia strutturata, questa informazione sarà estremamente utile per comprendere anche le sezioni successive. Possiamo dire in generale che la memoria permanente di una DBMS è organizzata a livello logico come un insieme di “database”, ognuno di questi si può vedere come un **insieme di file**, che a loro volta sono organizzati in **pagine** linearmente indirizzabili

Tipicamente vengono utilizzati più **livelli di memoria** in maniera molto simile a quanto accade all'interno di un normale calcolatore. A taglie di memoria più grande, corrisponderà maggior latenza, mentre man mano che si diminuisce la capacità si va a migliorare il tempo di accesso ai dati. Tipicamente i livelli che troviamo in ambienti production scale:

- *glacier storage*: utilizzati per storage di dati a lunghissimo termine
- *data lakes*: utilizzati per memorizzare oggetti usati più frequentemente
- *cloud storage / network drives*: basati su file system distribuiti remotamente
- *drive locali*: dispositivi locali tipicamente accessibili all'interno di una stessa rete
- *gerarchia di memoria del calcolatore*

Ciò che differenzia i vari tipi di memoria è che ad un certo punto della gerarchia esiste una sorta di “barriera” dalla quale in poi tutto quanto ciò che memorizziamo è volatile. Tipicamente questa soglia è costituita dalla **memoria principale**.

7.1.1. Geometria di un Hard Disk

Per quanto possa sembrare poco attuale parlare della geometria di un disco magnetico in questo periodo storico, è importante sottolineare che questo tipo di supporto ha una caratteristica molto conveniente rispetto ad un SSD. Oltre al costo che è indubbiamente minore, ciò che è più importante è il fatto che questo tipo di supporti **non degrada** nel tempo.

È noto infatti che tecnologie come gli SSD sono altamente soggette ad **isteresi**, quel fenomeno per cui lo stato di una porzione della memoria non è più dipendente dai dati memorizzati in un certo momento, ma anche dalla storia di tutti i dati memorizzati in quella porzione di memoria. Questo rende, dopo una certa quantità di tempo, un disco a stato solido inutilizzabile. Sebbene all'interno di un personal computer questo fenomeno possa essere trascurabile, in una base di dati, dove si rende necessario scrivere e leggere dati in maniera intensiva, questo processo potrebbe notevolmente amplificarsi.

All'interno di un supporto magnetico possiamo identificare le seguenti componenti:

- **Tracce**: percorsi circolari su un disco
- Ogni traccia è divisa in **settori** che vanno da 512 a 4KB

- Ogni traccia è composta da un numero variabile di settori, tra i 500 e i 1000
- Un **cilindro** è composto da un insieme di tracce che si trova nella stessa posizione relativamente alla testina.

Su tutta la superficie del disco abbiamo circa 100k tracce. Per quanto tutte le tracce abbiano una dimensione fisica differente, queste hanno tutte la **stessa capacità di memorizzazione**. Tracce di dimensione minore conterranno informazione in maniera più densa di altre tracce a dimensionalità maggiore.

Ogni traccia è divisa **logicamente** in **blocchi** di dimensione fissa, che rappresentano i blocchi di trasferimento tra i vari livelli di memoria.

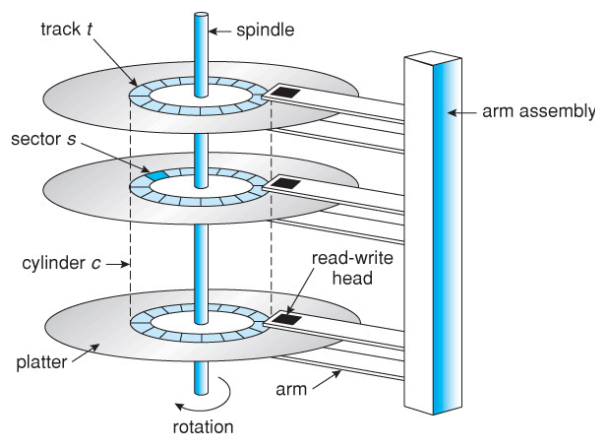


Figura 7.1: Rappresentazione schematica di un disco magnetico

7.1.2. Lettura da Disco Magnetico

Nel momento in cui andiamo a leggere dei dati da un disco magnetico dobbiamo tenere in considerazione i seguenti aspetti:

- è necessario trasformare l'indirizzo lineare in un **indirizzo fisico**
- una volta trovato l'indirizzo fisico *muoviamo la testina* sul **cilindro** di competenza
- una volta trovato il cilindro, andiamo a muovere la testina per farla finire sul **settore** corretto della traccia di interesse

Tutte queste operazioni hanno un costo che possiamo rappresentare tramite la seguente formula:

$$\text{Block Access Time} = t_s + t_r + t_b \quad (7.1)$$

dove:

- t_s sta per il **seek time**, ovvero il tempo necessario per muovere la testina sul cilindro corretto (3.5/4 ms in media)
- t_r sta per il **rotational latency**, ovvero il tempo necessario per far arrivare il settore corretto sotto la testina (2 ms in media)
- t_b sta per il **transfer time**, ovvero il tempo necessario per trasferire i dati dal disco alla memoria principale (0.1 ms per 4KB circa)

Evidentemente il tempo di trasferimento di un blocco dati è dominato da latenza e seek time. Per questo motivo, invece che andare a leggere un singolo blocco alla volta, è preferibile

leggere più blocchi in sequenza (**pagine**), in modo da minimizzare il numero di seek e rotazioni. Questo fenomeno rende molto importante il concetto di **località spaziale**:

- per trasferire un blocco il tempo necessario è $t_s + t_r + t_b$
- per trasferire n blocchi in sequenza il tempo necessario è $t_s + t_r + n \cdot t_b$

7.2. Buffer Manager

Come già anticipato, l'unità di misura per il trasferimento dei dati all'interno dei supporti di memorizzazione è quella delle **pagine**. Il compito del gestore del **buffer** è proprio quello di spostare pagine *tra la memoria permanente e quella principale*.

Nel fare questo fornisce una **vista logica** della memoria persistente sotto forma di pagine che possono essere utilizzate in memoria principale. I suoi obiettivi sono i seguenti:

- **limitare** il più possibile **letture ripetute** di una stessa pagina
- **minimizzare** il numero di **accessi in scrittura**

La porzione di memoria gestita dal buffer manager viene chiamata **buffer pool**. Tipicamente questi buffer possono essere molto grandi e risiedono in memoria principale. Il meccanismo che governa il funzionamento del buffer pool è estremamente simile a quello in uso nei sistemi operativi per la gestione della memoria virtuale o di una cache.

7.2.1. Bozza di Interfaccia del Buffer Manager

Di seguito viene presentato un insieme di operazioni che un buffer manager dovrebbe essere in grado di fornire come interfaccia sulla quale costruire il resto del sistema.

GB_getAndPinPage(pageID) --> Page

Si tratta di un'operazione fondamentale per l'effettiva lettura di una pagina: recupera la pagina con identificativo `pageID` dalla memoria persistente spostando la pagina dalla memoria permanente al buffer. L'operazione di **pinning** consente nel rappresentare l'informazione che la pagina è in uso, non si tratta di un flag, quanto più di un contatore che tiene traccia di quante entità stanno utilizzando la pagina.

GB_setDirty(pageID)

Questa operazione è fondamentale nel momento in cui andiamo a modificare dati all'interno di una pagina gestita nel buffer pool. Segnala che la pagina con identificativo `pageID` è stata modificata in memoria principale e deve essere scritta nuovamente su memoria permanente prima di essere rimossa dal buffer.

GB_unpinPage(pageID)

Quando abbiamo terminato di utilizzare una pagina, possiamo fare in modo che questa, a discrezione del del buffer, possa essere rimossa dalla porzione di memoria di sua competenza. Segnala che la pagina con identificativo `pageID` non è più in uso per una certa risorsa diminuendone il contatore di pinning. Se il contatore di pinning arriva a zero, la pagina può essere rimossa dal buffer pool.

GB_flushPage(pageID)

Questa operazione serve nel momento in cui vogliamo forzare la scrittura di una pagina su memoria permanente. Scrive la pagina con identificativo `pageID` su memoria permanente se questa è stata segnalata come **dirty**.

GB_getNewPage(Field, Type) --> Page

Alloca una nuova pagina all'interno della memoria permanente e la porta in memoria principale. La pagina viene inizializzata in base al tipo di dati che deve contenere.

7.2.2. Buffer Pool

Come già anticipato, il principale componente di competenza del gestore del buffer è il **buffer pool**. Figura 7.2 presenta una rappresentazione schematica di un buffer pool.

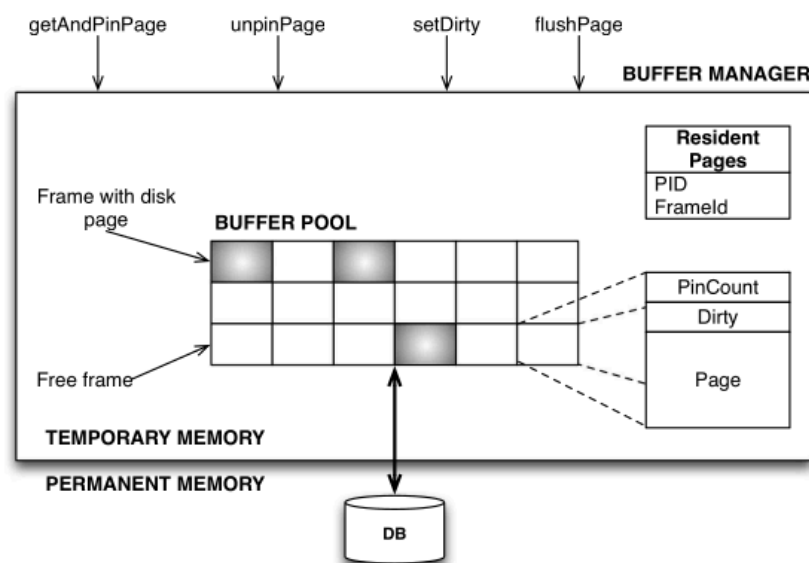


Figura 7.2: Rappresentazione schematica del buffer pool

Possiamo notare come le funzioni di gestione del buffer siano rese disponibili all'esterno del buffer manager tramite un'interfaccia. Possiamo notare come il buffer pool sia composto da un insieme di **frame** di dimensione fissa, tipicamente della stessa dimensione di una pagina. Ogni frame può contenere una pagina che è stata portata in memoria principale dalla memoria permanente. Ogni frame contiene inoltre delle informazioni di **metadati** che servono al buffer manager per tenere traccia dello stato della pagina contenuta al suo interno:

- un **contatore di pinning** che tiene traccia di quante entità stanno utilizzando la pagina
- un flag **dirty** che segnala se la pagina è stata modificata in memoria principale
- un campo **pageID** che identifica in maniera univoca la pagina contenuta all'interno del frame

Scrittura di Pagine su Memoria Permanente

Oltre che tramite l'API di flushing, è possibile che una pagina venga scritta in memoria permanente nel momento in cui, avendo il contatore di pinning a zero, il buffer manager decide di rimuoverla dal pool per fare spazio ad una nuova richiesta. In questo caso, se la pagina è

segnalata come dirty, questa viene scritta in memoria permanente prima di essere rimossa dal buffer pool.

Out of Memory Error

Nel caso in cui il buffer pool sia pieno e venga richiesta una nuova pagina, il buffer manager deve decidere quale pagina rimuovere per fare spazio alla nuova richiesta. Tipicamente ciò che viene fatto è cercare una pagina con contatore di pinning a zero, e tra queste sceglierne una. Nel momento in cui non fosse presente nessuna pagina con contatore di pinning a zero, il buffer manager non sarebbe in grado di soddisfare la richiesta e verrebbe segnalato un errore del tipo **out of memory**.

7.2.3. Tabella delle Pagine Residenti

Una componente fondamentale per il funzionamento del buffer manager è la **tabella delle pagine residenti** (resident page table). Si tratta di una struttura dati che consente di tenere traccia di quali pagine sono attualmente presenti all'interno del buffer pool.

Mentre sul disco le pagine sono identificate da un indirizzo fisico, il **pageID**, all'interno del buffer pool le pagine potrebbero essere assegnate in maniera *dinamica* a diversi frame. La tabella delle pagine residenti consente di tenere traccia di questa associazione tra **pageID** e frame del buffer pool. Figura 7.3 presenta una rappresentazione schematica della tabella delle pagine residenti.

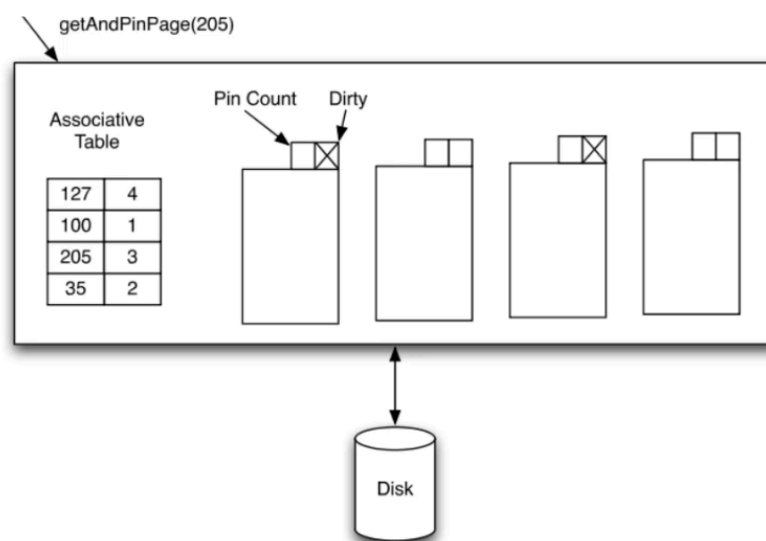


Figura 7.3: Rappresentazione schematica della tabella delle pagine residenti

Gestione di una richiesta di una pagina

Di seguito andiamo a mostrare il flusso operativo che avviene all'interno del buffer manager nel momento in cui si rende necessario gestire la richiesta di una pagina:

- Se la pagina è già presente nel buffer pool, andiamo semplicemente ad incrementare il pin count ritornandone il riferimento al richiedente
- Se la pagina non è presente procediamo come di seguito:

- Scegliamo una frame libero, o una frame dove il pin count sia zero tramite l'utilizzo di una **politica di sostituzione**. Se tale pin count è zero, e la pagina è dirty, procediamo a scriverla su memoria permanente
- Carichiamo la pagina richiesta dalla memoria permanente alla frame scelta, impostando il pin count a 1 e il flag dirty a false
- Aggiorniamo la tabella delle pagine residenti ritornando al chiamante il riferimento alla pagina appena caricata

Politica di Sostituzione

Nel momento in cui si rende necessario scegliere una pagina da rimuovere dal buffer pool, il buffer manager deve adottare una **politica di sostituzione** per scegliere quale pagina rimuovere.

La possibilità più semplice è quella di utilizzare un approccio **LRU**. Questo approccio è particolarmente adatto alla situazione in cui dobbiamo effettuare un **JOIN**, tuttavia non è detto che sia sempre la scelta ottimale. In alcuni caso **MRU** potrebbe essere una scelta migliore.

Nel caso in cui avessimo degli indici, allora potremmo aver bisogno di applicare politiche di sostituzione più sofisticate, che meglio si adattano al tipo di accesso che stiamo effettuando sui dati. In questo caso avremo però bisogno di utilizzare un **buffer pool separato** da quello normale per gestire le pagine degli indici, in modo da poter applicare politiche di sostituzione differenti.

7.2.4. Struttura di una Pagina di Memoria

Abbiamo già visto come l'unità di trasferimento dei dati tra memoria principale e secondaria sia la **pagina**. Fino a questo momento abbiamo soltanto visto come una pagina sia caratterizzata da un identificativo univoco.

Da un punto di visto **fisico** (all'interno di un file, che ricordiamo essere un insieme di pagine) una pagina è una struttura dati di **dimensione fissa**. Dal punto di vista **logico** (all'interno di un buffer) una pagine è una struttura che contiene le seguenti informazioni:

- **informazioni di servizio**: metadati necessari per la gestione della pagina all'interno del buffer pool
- **record**: i dati veri e propri memorizzati all'interno della pagina

Ora che abbiamo intuito come lo scopo di una pagina è quello di contenere diversi record, è necessario capire come effettuare accesso ai vari record. Abbiamo già citato in capitoli precedenti come ogni record sia fatto da **campi** (fields). Ognuno di questi campi può essere memorizzato in modo diverso:

- **dimensione fissa**: per esempio potremmo allocare un certo numero di byte per memorizzare certi valori. In questo caso l'accesso ai campi è molto semplice, in quanto basterà conoscere l'offset del campo all'interno del record per poterlo recuperare. Tuttavia questo approccio, nel caso di dati con dimensione variabile, potrebbe portare a sprechi di spazio, in quanto potremmo allocare più spazio del necessario
- **separatore speciale**: potremmo scegliere di utilizzare un carattere special per separare i valori all'interno di un record

- utilizzo di un **prefisso** che indica la **lunghezza del campo**

Tipicamente è presente una corrispondenza univoca tra record e pagine: ogni record è contenuto in una sola pagina. Tuttavia, ci sono dei casi, per esempio con dati in formato **BLOB**, in cui un singolo record potrebbe essere più grande della dimensione di una pagina. Per gestire questo caso avremo bisogno di una *sovrastuttura* che andremo a introdurre più avanti nel corso. L'idea è quella di memorizzare in record in un contenitore sufficientemente grande e memorizzare un riferimento a questo contenitore.

Esempio: Memorizzazione di record con campi a dimensione fissa

Si consideri la seguente tabella rappresentante un record con diversi campi ognuno con diverse dimensioni ma memorizzati con delle dimensionalità fisse.

ATTRIBUTO	POSIZIONE	TIPO DEL VALORE	VALORE
Name	1	char(10)	Rossi
StudentCode	2	char(6)	456
City	3	char(2)	MI
BirthYear	4	int(2)	68

In questo caso il numero totale di caratteri utilizzato effettivamente sarà 20. Di seguito mostriamo come sia possibile memorizzarlo secondo i vari approcci presentati sopra:

- *Dimensione fissa*: Rossi...456...MI68
- *Separatore speciale*: Rossi|456|MI|68|
- *Indice* che indica l'inizio di ogni campo: (1,6,9,11)Rossi456MI68

Per fare **riferimento** ad un **record** utilizziamo un identificativo univoco chiamato **record identifier**, che è composto da due campi:

- un **pageID** che identifica la pagina in cui il record è memorizzato
- un **offset** che indica la posizione del record all'interno della pagina

L'utilizzo di questi record identifier è particolarmente utile nel momento in cui andiamo ad utilizzare degli **indici** per velocizzare l'accesso ai dati.

Quello appena presentato, chiamato **riferimento diretto**, non è però il modo migliore per fare riferimento a record. Immaginiamo infatti di star indicizzando un insieme di record che si possono trovare su più pagine, e l'indice punta al riferimento diretto di ogni record. Se andiamo ad eliminare dei record in una pagina potremmo aver bisogno di *deframmentare* la pagina per recuperare spazio. In questo caso, i riferimenti diretti all'interno dell'indice andrebbero a puntare a posizioni errate. Ricalcolare l'indice sarebbe estremamente costoso.

Per questo motivo si preferisce mantenere un livello di **indirezione** tra l'indice e il record vero e proprio. In questo caso l'indice punterà ad un elemento dello **slot array** che sarà una struttura presente a livello di pagina. Sarà poi compito dello slot array tenere traccia del riferimento diretto (che in termini pratici, consiste solamente nell'*offset*) di ogni record. In questo modo, nel momento in cui andiamo a deframmentare la pagina, sarà sufficiente aggiornare lo slot array senza dover toccare l'indice. Figura 7.4 mostra schematicamente la differenza tra l'approccio indiretto e diretto.

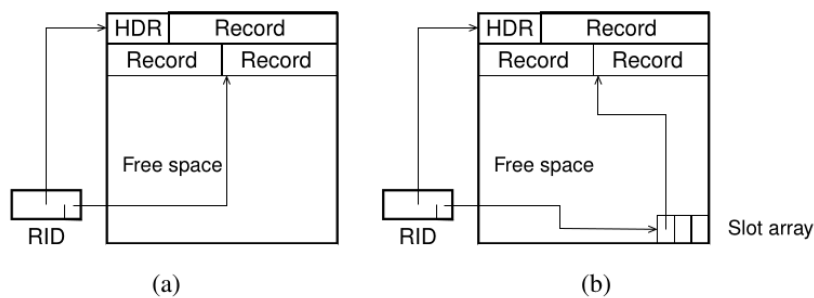


Figura 7.4: a) Rappresentazione di utilizzo di riferimenti diretti per i record all'interno di una pagina. b) Rappresentazione di utilizzo di uno slot array per i record all'interno di una pagina.

Osservazione

Lo slot array viene posizionato tipicamente alla fine della pagina, in modo da poter crescere verso l'alto. Se andassimo infatti a posizionare questo elemento all'inizio della pagina, ogni elemento aggiunto potrebbe farlo crescere di dimensionalità, comportando una nuova modifica di tutti gli offset.

7.3. Strutture per File Management

Come già noto dalla sezione del permanent memory manager, sappiamo che un **file** si può vedere dal punto di vista logico come un **insieme di pagine**.

Fino a questo momento ci siamo sempre concentrati sulla gestione dei record all'interno di una pagina. Abbiamo imparato come leggere dei record e come andarli a memorizzare all'interno di una pagina, dando per assunto di conoscere la pagina in cui i record sono memorizzati. Nella prossima sezione andremo a concentrarci su come venga gestito lo spazio all'interno di un file, che ricordiamo essere un insieme di pagine:

- come viene **scelto** dove memorizzare un nuovo record
- come **modificare** il contenuto di un **file** per riciclare memoria
- come **compattare** lo spazio all'interno di un **file**

In generale il tema di maggiore importanza è capire come **scegliere** una pagina sufficientemente capiente da poter memorizzare un nuovo record; sempre che questa pagina esista, in caso contrario sarà necessario allocare una nuova pagina all'interno del file.

In questa sezione andremo a vedere diversi approcci per la gestione dell'allocazione dello spazio all'interno di un file: *seriale*, *sequenziale*, *associativo* (per attributi unici o non unici).

Modello di Costo

Prima di andare a vedere i vari modelli di allocazione dello spazio all'interno di un file, è importante definire un **modello di costo** che ci consenta di valutare l'efficacia dei vari approcci. Andremo a tenere in considerazione i seguenti aspetti:

- **spazio utilizzato**: quantità di spazio effettivamente utilizzato per memorizzare i record, specialmente nel caso in cui venga utilizzata memoria ausiliaria
- **performance**: numero di operazioni read/write necessario per eseguire determinate operazioni come inserimento, aggiornamento, cancellazione e ricerca

All'interno di questo modello di costo, considereremo un file R con N_{rec} record di lunghezza L_r e un numero di pagine N_{page} di dimensione D_{page} .

Osservazione

Si noti come tutti i parametri definiti nell'ultimo paragrafo siano **necessari** per poter valutare l'efficacia del nostro modello di allocazione.

In base alle informazioni fornite dal nostro modello di costo, avremo la possibilità di comprendere quale sia il modello di allocazione più adatto alle nostre esigenze. Tipicamente l'**unità di misura** tramite cui esprimiamo il costo è il numero di **accessi a pagina** necessari per eseguire una certa operazione (sia in lettura, sia in scrittura).

7.3.1. Modello Seriale e Sequenziale

È stato scelto di raggruppare questi due approcci per l'organizzazione dei file in quanto condividono molte caratteristiche. La loro peculiarità consiste nel fatto che entrambi gli approcci **non utilizzano strutture dati ausiliarie**, la memoria viene utilizzata solamente per tenere traccia delle pagine che compongono il file.

La differenza sostanziale tra i due approcci consiste nel fatto che il modello **seriale** (*heap file*) non suppone alcun ordinamento dei valori all'interno del file, mentre il modello **sequenziale** suppone che i record siano memorizzati in ordine in base al valore di uno o più attributi.

Organizzazione Seriale

Il modello *heap file* è una tecnica molto semplice ed efficiente in termini della memoria utilizzata. Il problema è che nel caso di file molto grandi potrebbe risultare in performance scadenti. È ottimale nel caso invece in cui i file gestiti siano di piccole dimensioni. Si tratta dell'organizzazione per file che viene utilizzata di default nella maggior parte dei DBMS. Di seguito andiamo a vedere come vengono gestite le varie operazioni sui file:

- **ricerca** di un singolo valore per un attributo: è necessario effettuare una scansione sequenziale fino a quando non viene trovato il valore cercato, o fino a quando la fine della lista non viene raggiunta
- **ricerca** di un **intervallo** di valori: similmente al caso precedente è richiesta una scansione completa. La differenza rispetto a prima è che abbiamo un "stop criteria" differente, dal momento che dobbiamo continuare a leggere fino a quando non superiamo il valore massimo dell'intervallo

- l'**inserimento** di un nuovo record avviene semplicemente aggiungendo il record alla fine dell'ultimo file
- la **cancellazione** e l'**aggiornamento** di un record vengono effettuate tramite un'operazione di ricerca del record e di *risrittura*.

Costi del Modello Seriale

Di seguito andiamo ad illustrare i vari costi associati all'utilizzo di una architettura di questo genere. In generale, in termini di **spazio** avremo bisogno di una quantità espressa dall'equazione di seguito:

$$N_{\text{page}}(R) = N_{\text{rec}}(R) \cdot \frac{L_r}{D_{\text{page}}} \quad (7.2)$$

Osservazione

Il valore di L_r è tipicamente esatto, nel caso in cui abbiamo a che fare con valori provenienti da un dominio "statico", nel caso in cui avessimo valori a lunghezza variabile, il suo sarà una media delle varie dimensioni memorizzate fino ad un certo punto.

Possiamo notare come in Equazione (7.2) il costo in termini spazio sia unicamente dipendente dall'informazione memorizzata all'interno di un file. Andiamo ora ad osservare il costo in termini di numero accessi alle pagine per le varie operazioni:

- Per quanto riguarda **cancellazione** e **aggiornamento** avremo bisogno di un numero di accessi alle pagine pari a quello necessario per una ricerca più un accesso in scrittura per riscrivere il record: $C_d = C_u = C_s + 1$

Il motivo per cui il costo di una cancellazione di valore è uguale a quello di un aggiornamento è dovuto al fatto che, per evitare di dover spostare tutti i record cancellandone uno, si preferisce sovrascrivere il record o impostare un flag di cancellazione. In questo modo il costo risulta essere identico.

- per l'**inserimento** di un record avremo già conoscenza riguardo a quale sia l'ultima pagina del file, e potremo quindi effettuare una lettura della pagina di interesse e un conseguente inserimento: $C_i = 2$
- per la **ricerca** di un **singolo valore** per un attributo: $C_s \approx \lceil N_{\text{page}} \frac{R}{2} \rceil$ mediamente nel caso in cui il valore sia presente, in caso contrario avremo $C_s = N_{\text{page}}(R)$
- per la **ricerca** di un **intervallo** di valori: $C_r = N_{\text{page}}(R)$, non essendo infatti i valori ordinati, si rende necessario scorrere l'intero file per essere sicuri di aver trovato tutti i valori nell'intervallo.

Organizzazione Sequenziale

L'idea alla base dell'organizzazione sequenziale è quella di mantenere i record ordinati rispetto ad una chiave K e memorizzarli in memoria continua. Questo principio consente di abbassare notevolmente i costi di ricerca, sia per un singolo valore, sia per un intervallo di valori.

Lo svantaggio principale di questo approccio è legato al **mantenimento dell'ordinamento**. Questo comporta infatti che ogni inserimento e aggiornamento comporti una riorganizzazione del file, che potrebbe essere estremamente costosa. Nello specifico, le operazioni di aggiornamento sono ancora più complicate, dal momento che possono o non possono cambiare l'organizzazione del file a seconda del valore della chiave K .

Per tutte queste ragioni, l'utilizzo di questo modello di organizzazione, sebbene interessante dal punto di vista teorico, non viene praticamente mai utilizzato nei DBMS moderni. L'unico caso in cui risulti utile è quello in cui il file venga utilizzato in sola lettura, come per esempio nel caso di **data warehousing**.

Ricerca di un singolo valore

Nel momento in cui dobbiamo andare a cercare una determinata chiave k all'interno di un file sequenziale, possiamo sfruttare l'ordinamento di questo. A livello teorico possiamo utilizzare tutti i possibili algoritmi già noti per la ricerca in liste ordinate, come per esempio la **ricerca binaria**.

Osservazione

Si rende necessario tenere in considerazione che i dati vengono trasferiti **una pagina alla volta** (o a blocchi di pagine). Questo porta ci porta ad adattare i nostri algoritmi di ricerca.

Pensiamo ad esempio alla ricerca binaria in cui andiamo normalmente a scegliere un “*pivot*” e sulla base del valore di questo andiamo a decidere se cercare nella metà sinistra o destra della lista di valori. In questo caso non abbiamo più soltanto un “*pivot*”, ma ci servirà utilizzare “**pagine pivot**”:

- Scegliamo una “*pagina pivot*” (tipicamente al centro del file) P_i , con $1 \leq i \leq N_{\text{page}}(R)$
- Prendiamo m_i, M_i come rispettivamente il valore minimo e massimo dell'attributo cercato nella pagina P_i
- Se $m_i \leq k \leq M_i$ allora abbiamo trovato la pagina di nostro interesse e possiamo cercare k al suo interno, in caso contrario se $k < m_i$ allora andiamo a cercare nella metà sinistra del file, altrimenti andiamo a cercare nella metà destra del file.
- Se l'intervallo di pagine da cercare è vuoto, allora il valore non è presente nel file

Osservazione

Si noti come il costo di ricerca di un valore all'interno di una pagina una volta che questa è stata trovata si può considerare **trascurabile**, dal momento che il costo totale (in termini di tempo e latenza) sarà dominato dal numero di accessi alle pagine.

7.3.2. Algoritmi di Ricerca e Costi

Di seguito andiamo a presentare varie possibilità per implementare la ricerca di un singolo valore all'interno di un file sequenziale andandone a valutare i costi associati.

Ricerca Binaria

Come noto da precedenti corsi di algoritmi, la ricerca binaria, che va a scegliere il “**pivot**” al **centro** dell'**intervallo di ricerca** in cui cercare produce un costo medio dato da:

$$C_s = \log_2(N_{\text{page}}(R))$$

Sebbene in un normale algoritmo di ricerca questo costo sia accettabile se non addirittura buono, in un contesto come il nostro dobbiamo fare di meglio. Infatti nel caso in cui avessimo 400k pagine, il costo medio sarebbe di circa 19 accessi a pagina. Questo costo è ancora troppo elevato per essere accettabile in un DBMS.

Ricerca Interpolata

Un miglioramento rispetto alla ricerca binaria può essere ottenuto tramite l'utilizzo della **ricerca interpolata**. L'idea è quella di scegliere il **pivot** in maniera più intelligente, andando a stimare la posizione del valore cercato all'interno del valore di ricerca.

L'idea di base è che le chiavi siano distribuite in maniera all'incirca **uniforme** nell'intervallo di ricerca. In questo modo possiamo stimare la *probabilità che una chiave sia minore di un certo k* tramite la seguente formula:

$$p_k = \frac{k - k_{\min}}{k_{\max} - k_{\min}}$$

Dato il valore di p_k possiamo stimare la posizione della pagina pivot tramite la seguente formula:

$$P_i = \left\lceil P_1 + p_k \cdot (P_{N_{\text{page}}(R)} - P_1) \right\rceil$$

In sostanza p_k , supponendo che le chiavi siano distribuite uniformemente, ci consente di scalare il valore di k all'interno dell'intervallo di ricerca. Utilizzando questa tecnica i costi saranno i seguenti:

- $C_s = O(\log_2 \log_2 N_{\text{page}}(R))$ nel caso “medio”
- $C_s = O(N_{\text{page}}(R))$ nel caso peggiore, tuttavia questo caso è molto raro

Utilizzando ricerca interpolata nel caso di 400k pagine avremo un costo medio di circa 4 accessi a pagina, che è decisamente migliore rispetto alla ricerca binaria.

Altri costi per la ricerca interpolata

Ipotizzando di andare ad utilizzare la ricerca interpolata, andiamo a vedere quali sono gli altri costi associati alle altre operazioni sui file:

- Nel caso di **ricerca** di un **intervallo** ($k_1 \leq k \leq k_2$), ipotizzando di avere chiave uniformemente distribuite nell'intervallo $[k_{\min}, k_{\max}]$ avremo quanto segue:

$$f_s = \frac{k_2 - k_1}{k_{\max} - k_{\min}}$$

che corrisponde alla frazione totale delle pagine coperte dall'intervallo di ricerca rispetto al totale, questo elemento prende il nome di **fattore di selettività**. Dato questo fattore di selettività, il costo totale si può esprimere come:

$$\lceil \log_2 N_{\text{page}}(R) \rceil + \lceil f_s \cdot N_{\text{page}}(R) \rceil - 1$$

dove il primo membro della somma corrisponde al **costo per la ricerca della prima pagina** dell'intervallo, mentre il secondo membro corrisponde al **costo per la lettura di tutte le pagine successive** nell'intervallo. Chiaramente andiamo a sottrarre 1 in quanto la **prima pagina è già stata letta**.

- Nel caso di **inserimento**, dobbiamo innanzitutto individuare la pagina corretta in cui inserire il nuovo record. Questo comporta la scansione di tutte le pagine precedenti rispetto a quella di destinazione. Nel caso peggiore, quando tutte le pagine sono completamente occupate, è necessario spostare i record verso le pagine successive. Tuttavia, siccome il costo che ci interessa misurare è quello in termini di **accessi a pagina**, è sufficiente considerare lo spostamento di **un solo record per pagina** verso la pagina successiva, per ciascuna delle pagine che seguono quella di inserimento. Il costo totale dell'operazione di inserimento è quindi dato da:

$$C_i = C_s + N_{\text{page}}(R) + 1$$

dove, il primo termine rappresenta il costo per la **ricerca della pagina di inserimento**, il secondo termine rappresenta il costo per la **scansione delle pagine successive per lo spostamento dei record**, e l'ultimo termine rappresenta il costo per la **scrittura del nuovo record nella pagina di destinazione**.

- Nel caso di **cancellazione**, andiamo a considerare soltanto il caso di cancellazione **logica**, possiamo dire che questo ha costo come nel caso seriale: $C_d = C_s + 1$.

Confronto tra Modello Seriale e Sequenziale

Di seguito andiamo a presentare una tabella riassuntiva che confronta i due modelli di organizzazione per file appena presentati in termini dei costi a loro associati.

Type	Memory	Single value search	Interval search	Insertion	Deletion
Serial	$N_{\text{pag}}(R)$	$\lceil N_{\text{pag}}(R)/2 \rceil$	$N_{\text{pag}}(R)$	2	$C_s + 1$
Sequential	$N_{\text{pag}}(R)$	$\lceil \log_2 N_{\text{pag}}(R) \rceil$	$C_s + 1 + \lceil f_s \times N_{\text{pag}}(R) \rceil$	$C_s + 1 + N_{\text{pag}}(R)$	$C_s + 1$

Figura 7.5: Confronto tra modello seriale e sequenziale

In generale non abbiamo un vincitore assoluto tra i due approcci, ognuno ha i suoi pro e contro:

- il modello **seriale** è ideale nel caso in cui abbiamo bisogno di effettuare tanti **inserimenti**, ma è pessimo nel caso in cui dobbiamo effettuare ricerche molto piccole
- il modello **sequenziale** è ideale nel caso in cui abbiamo bisogno di effettuare tante **ricerche**, ma è pessimo nel caso in cui dobbiamo effettuare tanti inserimenti

7.4. Problemi di Ordinamento

Come già noto, l'ordinamento di dati è uno dei problemi più classici ed importanti nell'ambito dell'informatica. Anche nelle basi di dati si tratta di un'operazione molto comune: spesso infatti capita che sia necessario ordinare i dati in base a certi attributi per poter rispondere a query specifiche.

Altri casi molto comuni in cui è necessario ordinare i dati sono i seguenti:

- unione di dati provenienti da più tabelle
- eliminazione di duplicati
- raggruppamenti rispetto a determinate chiavi
- operazioni insiemistiche (unioni, intersezioni, differenze)

Seppur siano già state viste numerose tecniche di ordinamento in corsi precedenti, all'interno di un DBMS l'algoritmo di ordinamento maggiormente utilizzato è **external merge-sort**. Questo algoritmo funziona in due fasi:

- **ordinamento**: crea piccoli ordinamenti detti **run**
- **merge**: fonde più run ordinati in un unico file ordinato

Di seguito andiamo a vedere cosa succede ad ogni passo dell'algoritmo.

Fase di ordinamento

All'interno del **buffer** vengono ordinate B (dimensione del buffer) pagine per volta tramite un algoritmo di **sort interno** e scrive il risultato su disco come un **run** ordinato. Questo processo viene ripetuto fino a quando tutte le pagine del file sono state lette e scritte su disco come run ordinate. Al termine del sorting otteniamo n run dove:

$$n = \left\lceil \frac{N_{\text{page}}(R)}{B} \right\rceil$$

Fase di merge

In questa avviene l'ordinamento vero e proprio. Sappiamo che abbiamo un buffer di B pagine a disposizione, che è uno spazio limitato rispetto a quello effettivamente necessario. Per superare questo limite andremo a considerare $Z = B - 1$ run. Per ognuna di queste run andremo a tenere all'interno del buffer una pagina, riservando l'ultima pagina per scrivere il risultato.

In questo modo, ad ogni passo andremo a scegliere il record più piccolo tra quelli nelle Z pagine, scrivendolo nella pagina di output. Man mano che la pagina di output si riempie questa viene scritta su disco e andrà a dar forma al nuovo file ordinato, nel frattempo le Z pagine vengono riempite progressivamente.

Questo processo verrà ripetuto per $\frac{Z}{n}$ fasi di merge, tutti i risultati prodotti saranno **intermedi** e verranno fusi nuovamente fino a quando non rimarrà un unico file ordinato.

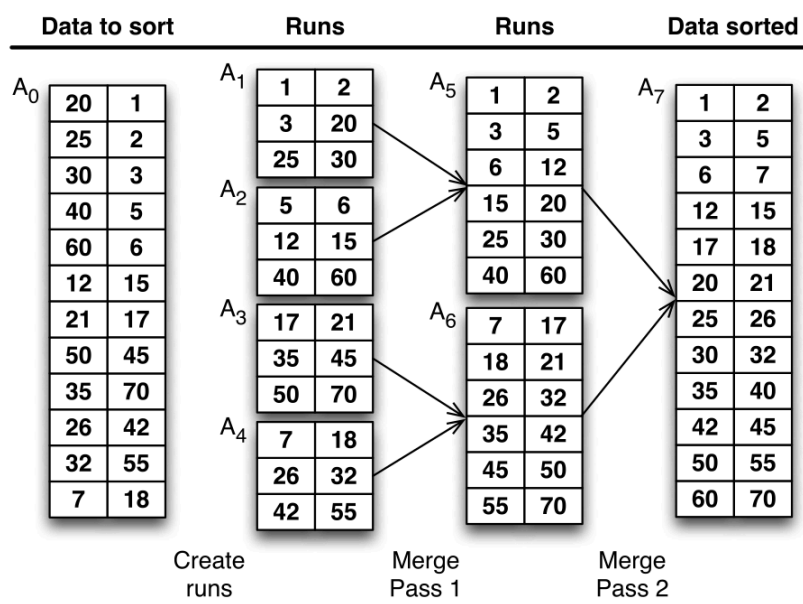


Figura 7.6: Rappresentazione schematica dell'algoritmo di external merge-sort con dimensione del buffer di 3 pagine, e pagine di dimensione 2

Figura 7.6 mostra schematicamente il funzionamento di external merge sort. Si noti come dal momento che il buffer può contenere 3 pagine di dimensione 2, allora $Z = 2$. Non a caso il risultato della fase di sort consiste in 4 run ordinate di 3 pagine ciascuna.

Costo di External Merge-Sort

Andiamo ora a vedere quali sono i costi associati all'algoritmo di external merge-sort. In generale questo costo è estremamente dipendente dal numero di volte che i dati vengono spostati tra il buffer e la memoria permanente.

Per quanto riguarda l'algoritmo in sé, sappiamo che questo può essere visto come diviso in un singolo passo di ordinamento e un certo numero k di passi di merge. Indipendentemente dal tipo di operazione effettuata, ogni passo comporta la lettura e la scrittura di tutte le pagine del file. Definiamo quindi il costo di singolo passo $C_{\text{step}} = 2 \cdot N_{\text{page}}(R)$.

Il costo totale dell'algoritmo sarà dunque dato da:

$$C_{\text{tot}} = (1 + k) \cdot C_{\text{step}} = (1 + k) \cdot 2 \cdot N_{\text{page}}(R)$$

A questo punto non resta che andare a investigare il numero di passi di merge k . Sappiamo che il numero di run iniziali S è strettamente legato alla dimensione del buffer B : $S = \lceil N_{\text{page}} \frac{R}{B} \rceil$. In ogni passo di merge, possiamo fondere $Z = B - 1$ run, riducendo dunque il numero di run di un fattore Z : $k = \lceil \log_Z S \rceil = \lceil \log_{B-1} \lceil N_{\text{page}} \frac{R}{B} \rceil \rceil$. Possiamo dunque riscrivere il costo totale come:

$$2 \cdot N_{\text{page}}(R) \cdot (1 + \lceil \log_Z S \rceil) \quad (7.3)$$

Nello specifico, nel caso in cui avessimo $N_{\text{page}}(R) < B^2$ allora avremmo bisogno di una sola fusione; questo dal momento che $S = N_{\text{page}} \frac{R}{B} < B$. In questo caso il costo totale si ridurrebbe a $C_{\text{tot}} = 4 \cdot N_{\text{page}}(R)$.

Di seguito vediamo alcuni benchmark che mostrano il costo di external merge-sort in funzione del numero di pagine avendo fissati $B = 3$, $Z = 2$ e la differenza rispetto ad avere $B = 257$ e conseguentemente un valore di $Z = 256$.

$N_{\text{pag}}(R)$	B = 3, Z = 2			B = 257, Z = 256		
	Merge Passes	Cost	Time (m)	Merge Passes	Cost	Time (m)
100	6	1400	0.23	0	200	0.03
1000	9	20 000	3.33	1	4000	0.67
10 000	12	260 000	43.33	1	40 000	6.67
100 000	16	3 400 000	566.67	2	600 000	100.00
1 000 000	19	40 000 000	6666.67	2	6 000 000	1000.00

Figura 7.7: Benchmark di external merge-sort in funzione del numero di pagine per due diverse dimensioni del buffer: a sinistra $B=3$, a destra $B=257$

8. Organizzazioni per la Ricerca di Chiavi

Nell'ultima sezione del capitolo precedente abbiamo illustrato come l'organizzazione la gestione dei file in cui sono memorizzate le pagine vada ad influire enormemente sulle prestazioni di una base di dati. Oltre all'organizzazione seriale e sequenziale abbiamo a disposizione strutture molto più sofisticate che vedremo all'interno di questo capitolo.

Come già citato lo scopo di queste organizzazioni è quello di **localizzare un record** nella maniera più **veloce** possibile. Possiamo distinguere due metodi per organizzare i file:

- **organizzazioni primarie:** determinano come vengono memorizzati *fisicamente* i dati all'interno del file. Nel momento in cui una chiave in questa organizzazione viene modificata, è anche necessario modificare la posizione fisica del record.
- **organizzazioni secondarie:** strutture dati che permettono di localizzare i record a partire da strutture dati secondarie, tipicamente si tratta di strutture basate su indici

Possiamo identificare diverse famiglie di organizzazione per i file. Per quanto riguarda le strutture primarie abbiamo quelle **hash based** o **tree based**. Per quanto riguarda le strutture secondarie abbiamo invece gli **indici**. Tutte queste famiglie possono essere inoltre classificate in **statiche** e **dinamiche**, a seconda che la struttura dati possa adattarsi a modifiche dettate da inserimenti e cancellazioni. Nel caso in cui la struttura sia *statica* sarà necessario prevedere un meccanismo di *riorganizzazione*, mentre per quanto riguarda strutture *dinamiche* queste saranno in grado di adattarsi automaticamente alle modifiche.

8.1. Hashing Statico

Il principio alla base di queste tecniche è quello di utilizzare una **funzione di hash** per trasformare chiavi che potenzialmente non sono distribuite in maniera uniforme in valori che lo sono, permettendo così di distribuire i record in maniera uniforme all'interno dei vari file.

L'idea dietro a queste tecniche è abbastanza simile a quella dell'*accesso diretto*: supponiamo che dato un valore di chiave K esista un unico posto in cui il record k possa essere memorizzato. Questo approccio è teoricamente molto efficiente, perché sapremmo per ogni chiave, dove ricercarla in maniera puntuale. Il problema è che questo richiede uno spazio di memorizzazione grande quando il dominio dei valori. L'utilizzo di una funzione di **hash** ci permette di **restringere il dominio** dei possibili valori.

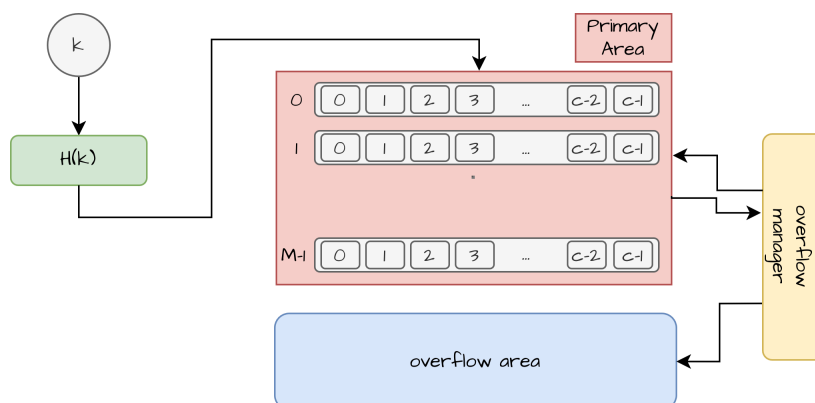


Figura 8.1: Workflow di un'organizzazione hash-based

Come vediamo in Figura 8.1 ogni record viene sostanzialmente memorizzato in una pagina, a seconda di quello che è il valore di hash. Ovviamente se il dominio delle chiavi è molto grande, è da tenere in considerazione la possibilità di avere delle **collisioni**: per due valori diversi la funzione di hash potrebbe restituire lo stesso indirizzo. Quando lo spazio di una pagina viene esaurito, è necessario prevedere un meccanismo di **overflow** per gestire i record in eccesso.

Il motivo per cui questo approccio è detto **statico** è che una volta che un record viene memorizzato in una pagina, questo rimane lì fino a quando non viene cancellato. Come vediamo in Figura 8.1, in questo approccio è necessario prevedere di gestire i seguenti aspetti:

- una **funzione di hash** $h(K)$ che data una chiave K restituisce un indirizzo di pagina
- un modulo per la **gestione dell'overflow** che si occupa di gestire i record in eccesso quando una pagina è piena
- tenere in conto del **loading factor**: quanto spazio vuoto per ogni pagina andiamo a lasciare per gestire gli overflow
- tenere in considerazione la **capacità di una pagina**

Avvertenza

È importante tenere in considerazione che questo approccio non è progettato per ricercare valori all'interno di un **intervallo**; non esiste infatti un meccanismo di ordinamento dei valori per cui non è possibile effettuare in maniera efficiente ricerche di questo tipo.

8.1.1. Funzione di Hashing

La funzione di hashing H è il cuore di questa organizzazione. Si dice che la funzione di hash è **uniforme** nel momento in cui tutti gli indirizzi sono prodotti in modo uniforme nell'intervallo $[0, M - 1]$. Si dice che due chiavi k_1 e k_2 sono **sinonimi** quando $H(k_1) = H(k_2)$, ossia, producono una *collisione*.

Se il numero di collisioni su una pagina è maggiore della capacità di una pagina, ci troveremo in una situazione di **overflow**. Il numero di overflow in generale va ad aumentare il costo delle operazioni di ricerca, dal momento che per uno stesso hash si renderà necessario l'accesso ad un numero elevato di pagine.

Quando una funzione di hashing è ben progettata (80% occupazione delle pagine), possiamo assumere di non avere un numero overflow, dunque ogni operazione di ricerca avrà costo *unitario*. Una tipica funzione di hashing è quella che effettua il **modulo** del valore della chiave rispetto al numero di pagine M :

$$H(k) = k \bmod M_p$$

dove M_p è il numero di pagine del file. Chiaramente il valore “80% di page occupancy” è un valore ideale, che non è così immediato ottenere. Questo verrà ulteriormente discusso nella sezione dedicata alla gestione del loading factor.

8.1.2. Gestione dell'overflow

Come già citato parlando delle funzioni di hashing, la gestione dell'overflow è di fondamentale importanza per garantire efficienza in lettura. Possiamo gestire l'overflow in diversi modi.

Open Overflow (Open Addressing)

In questo approccio i record che non trovano spazio nella pagina designata da $H(k)$ vengono memorizzati in altre pagine libere della zona **primaria**. Si tratta di un approccio estremamente semplice e di facile gestione, il problema sorge nel momento in cui la probabilità di overflow aumenta: diventerà sempre più necessario andare a cercare nelle pagine libere, aumentando così il costo delle operazioni di ricerca.

Un altro importante problema di questo approccio è che, per quanto stiamo risolvendo il problema relativo all'overflow per una dato hash-code, stiamo andando a *rubare* spazio per record che verranno mappati in altre pagine.

Chiaramente, per quanto questo approccio sia semplice, ha la caratteristica di **degradare velocemente** le prestazioni all'aumentare del numero di record memorizzati.

Chained Overflow (Closed Addressing)

Questo approccio prevede di utilizzare una zona di overflow separata dalla zona primaria. L'idea è quella di utilizzare l'ultima parte di ogni pagina per memorizzare l'overflow. Possiamo vedere ogni pagina come divisa in due porzioni. La prima porzione sarà quella utilizzata per memorizzare i record che mappano in quella pagina, mentre la seconda porzione sarà destinata record in eccedenza, l'ultima parte di questa porzione viene a sua volta utilizzata per memorizzare un puntatore alla pagina di overflow successiva. Questo meccanismo viene mostrato in Figura 8.2.

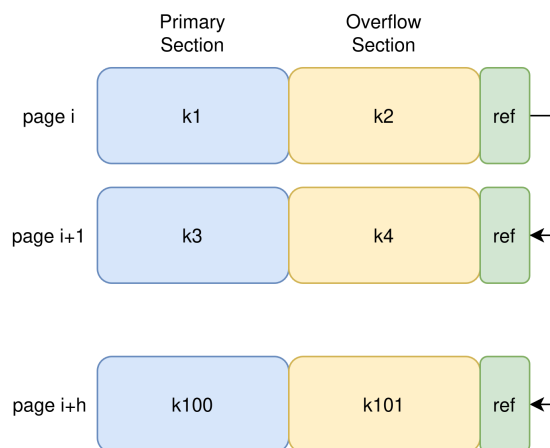


Figura 8.2: Gestione dell'overflow con chained overflow

Questo approccio consente di andare, pur riducendo lo spazio associato ad ogni pagina primaria, di riservare una porzione di pagina ai record che dovrebbero essere effettivamente memorizzati in quella pagina.

Gestione completamente separata

In questo approccio, che è esattamente quello illustrato in Figura 8.1, viene prevista una zona di overflow completamente separata dalla zona primaria. In questo caso ogni pagina di overflow contiene record che non hanno trovato spazio nella zona primaria. Tuttavia è comunque necessario definire in quale maniera i record in overflow vengono memorizzati, il che risulta in un problema simile a quello dell'organizzazione primaria.

8.1.3. Loading Factor

Dato il numero di pagine M e la capacità di una pagina c , possiamo definire il **loading factor** $d = \frac{N}{M \cdot c}$, dove N è il numero di record memorizzati. Sostanzialmente si tratta di una metrica che rappresenta la percentuale media di elementi memorizzati in ogni pagina.

Come si può immaginare, un loading factor elevato implica una maggior probabilità di overflow e un conseguente aumento del costo delle operazioni di ricerca.

Il grafico in Figura 8.3 mostra come il costo medio delle operazioni di ricerca vada ad aumentare all'aumentare del loading factor, per i diversi approcci di gestione dell'overflow.

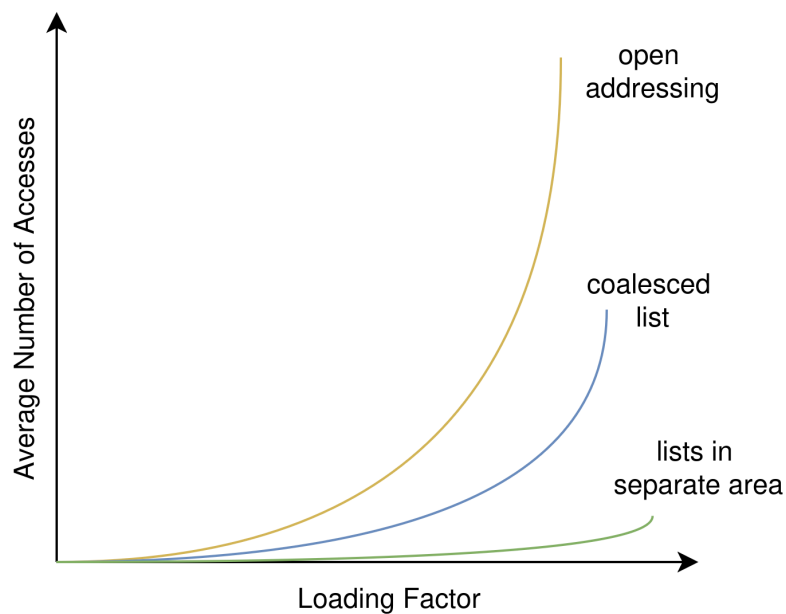


Figura 8.3: Costo medio delle operazioni di ricerca in funzione del loading factor

La voce *coalesced list* sta a rappresentare un approccio in cui vengono messi insieme elementi appartenenti a diverse "sorgenti di overflow".

8.1.4. Costi dell'approccio

Nel caso di ricerca di una **singola chiave** una scelta ragionevole dei parametri del nostro sistema (funzione di hash, gestione dell'overflow, loading factor) ci permette di ottenere un costo medio per operazione di ricerca **prossimo a 1 page access**. Per ottenere tali risultati alcuni valori tipicamente utilizzati sono un loading factor $d < 0.7$, $c \gg 10$.

Come già anticipato non è possibile andare a ricercare valori in **intervalli**, dal momento che non esiste un ordinamento tra i valori memorizzati.

Per quanto riguarda gli **inserimenti** e le **cancellazioni**, queste operazioni hanno il costo di 1 operazione in lettura e 1 operazione in scrittura, dunque hanno costo costante.

8.1.5. Riorganizzazione della struttura

Prima di spiegare come avviene questo delicato processo, è importante sottolineare che questa viene effettuata in situazioni abbastanza particolari:

- la quantità di **overflow** è particolarmente elevata (nel caso in cui l'area primaria è utilizzata per gli overflow, altrimenti non serve)
- quando un numero significativo di record sono stati **cancellati**, e dunque è necessaria una **compattazione** dei record

Per effettuare la riorganizzazione si utilizza un file *temporaneo* su cui verranno copiati tutti i record. Dopo aver copiato tutti i record verrà effettuata un'operazione di **bulk load**:

- il file temporaneo viene ordinato in base all'hash code $H(k)$
- caricamento dei record che non causano overflow in ciascun hash file
- caricamento dei record in overflow nelle pagine di overflow

8.2. Hashing Dinamico

Sebbene l'**hashing statico** sia un approccio semplice che può funzionare in maniera estremamente efficiente a *determinate condizioni*, il problema è che nel momento in cui la situazione inizia a degradare, questa arriva ad essere catastrofica molto rapidamente, richiedendo una riorganizzazione completa della struttura.

L'idea è che, dal momento che la riorganizzazione **non è evitabile** in alcun modo o, per meglio dire, è probabile che nel corso del tempo sarà un processo necessario, vorremmo quantomeno evitare di dover fermare l'intero sistema per effettuare questa operazione.

Ci piacerebbe dunque avere un sistema che ci permetta di eseguire in maniera preventiva alcune delle operazioni necessarie per la riorganizzazione, in modo tale che, rendendo leggermente più lenta l'amministrazione ordinaria, possiamo garantire un sistema più leggero nei confronti della riorganizzazione. Questa è proprio l'idea che sta alla base degli **hash file dinamici**. Abbiamo a disposizione due principali metodologie:

- hashing dinamico che utilizza strutture **ausiliarie**, come ad esempio il *virtual hashing*, *extensible hash*, *dynamic hash*.
- hashing dinamico **senza** strutture **ausiliarie**, come ad esempio il *linear hashing* o l'*hashing a spirale*.

In questa sezione andremo ad affrontare unicamente hashing lineare e virtuale.

8.2.1. Hashing virtuale

Come sappiamo, il problema della riorganizzazione consiste nel dover riorganizzare un **file intero alla volta**. Immaginiamo di non ammettere overflow, ma di permettere che, nel momento in cui ci troveremmo di fronte a uno sfioramento, possiamo creare più spazio per i record. Invece però di andare a creare più spazio per tutti i record, andremo a creare più spazio solo per i record che mapperebbero in una pagina con overflow.

Di seguito mostriamo il workflow di questo approccio:

1. Il file in memoria contiene inizialmente un numero M di pagine contigue con capacità c . Ogni pagina è identificata da un indirizzo nell'intervallo $[0, M - 1]$.
2. Un bit vector $\mathcal{B}$ viene utilizzato per tenere traccia delle pagine che contengono almeno un record al loro interno
3. Inizialmente la funzione di hash H_0 è utilizzata per mappare ogni chiave k in un indirizzo $H_0(k) \in [0, M - 1]$. Se dovesse venir generato un overflow:
 - il numero di pagine continue viene raddoppiato, creando M nuove pagine
 - viene creata una nuova funzione di hash H_1 tale che $m \in [0, 2M - 1]$
 - la funzione H_1 viene applicata alla chiave k e a tutti i record nella pagina m che ha causato overflow in modo da distribuire i record tra m e m'

In caso di overflow tutti i record nelle pagine diverse dalla pagina m non verranno in alcun modo toccati. In questo modo l'operazione di riorganizzazione viene limitata a un sottoinsieme di pagine, riducendo notevolmente il costo dell'operazione. Chiaramente questo approccio richiede di utilizzare una serie di funzioni $H_0, \dots, H_r$, dove in generale H_r produce pagine appartenenti all'intervallo $[0, 2^r M - 1]$. Questo meccanismo viene spiegato di seguito.

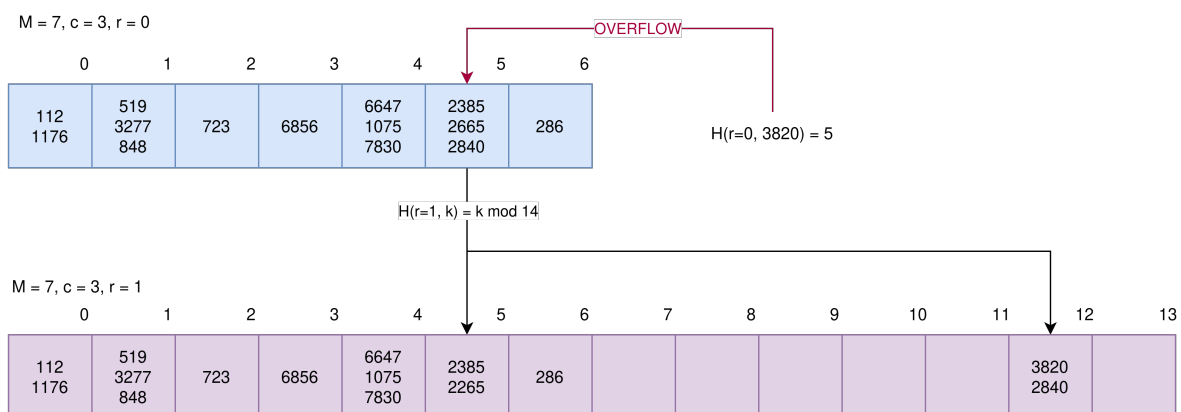


Figura 8.4: Esempio di gestione di overflow in una situazione di virtual hashing.

Nota Bene: il bit-vector viene mostrato ma è da considerare presente.

Per andare a gestire la ricerca di un record data una chiave k possiamo far riferimento al seguente algoritmo:

```

1  def PageSearch(r, k: int) -> int:
2      if r < 0
3          # l'algoritmo ha provato tutte le funzioni di hash senza successo
4          raise Exception("Inexistent key")
5      elif B(H(r,k)) == 1:
6          return H(r,k)
7      else:
8          return PageSearch(r-1, k) # ricerca tramite hash precedente
9
10 # esempio di utilizzo
11 page = PageSearch(r_max, key) # r_max è il massimo hash level in uso

```

Osservazione

Per quanto possa sembrare superflua, il mantenimento del bit vector $\mathcal{B}$ rende possibile evitare di accedere a pagine vuote, riducendo notevolmente il costo delle operazioni di ricerca.

In riferimento a Figura 8.4, immaginiamo di voler ora inserire la chiave 3343. Andremo in primo luogo a calcolare $H_1(3343) = 11$. Dal momento che la pagina 11 ha indicatore a 0, andremo a calcolare $H_0(3343) = 4$. Se la pagina 4 fosse vuota, potremmo semplicemente andare ad aggiungere il nuovo valore a questa, ma essendo piena, andremo a ricalcolare H_1 su tutti gli elementi della pagina, andando a distribuire i record tra la pagina 4 e la pagina 11, segnalando inoltre che la pagina 11 ora contiene dei record nel bit-vector $\mathcal{B}$.

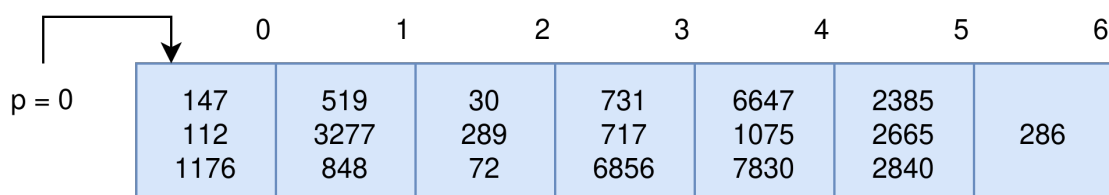
Osservazione

Possiamo notare che in realtà, andando a tenere traccia dell'occupazione della pagina, possiamo andare a **recuperare un record in un solo accesso** a pagina, questo perché, utilizzando le hash function da quella di ordine maggiore, possiamo essere sicuri che se una pagina è occupata, il record che stiamo cercando si trova sicuramente in quella pagina.

8.2.2. Hashing Lineare

Al contrario dell'hashing virtuale, nel quale l'obiettivo è quello di evitare l'avvenimento di overflow, nell'hashing lineare l'obiettivo è quello di andare a regolamentare la politica con cui questo viene gestito. L'idea è quella di andare a aumentare il numero di pagine non appena abbiamo un overflow. Di seguito mostriamo il flusso operativo di questo approccio:

- Inizialmente vengono allocate M pagine continue con capacità c
- La funzione di hash iniziale H_0 è tale che $H_0(k) = k \bmod M$
- Viene inizializzato un puntatore $p = 0$ che punta alla prossima pagina da **splittare**
- Quando si verifica un overflow nella pagina $m = H_{i(k)}$:
 - ▶ se $m = p$, viene effettuato uno **split** della pagina p , riorganizzando i record con la funzione $H_{\{i+1\}}$
 - ▶ se $m > p$, viene creata mantenuta una catena di overflow per la pagina m e viene aggiunta una nuova pagina vuota alla fine del file che rappresenta lo split della pagina p , nuovamente i dati verranno riorganizzati
 - ▶ se $m < p$, non serve effettuare alcun tipo di split dal momento che la pagina è già stata "splittata" precedentemente



	0	1	2	3	4	5	6
$p = 0$	147 112 1176	519 3277 848	30 289 72	731 717 6856	6647 1075 7830	2385 2665 2840	286

Figura 8.5: Posizione di partenza per linear hashing

Figura 8.5 mostra una possibile situazione di partenza per il nostro scenario di linear hashing. Ipotizziamo che siano stati inseriti dei record, rispettivamente con valore 569 e 563, i cui hash sono $H_0(569) = 2$, $H_0(563) = 3$. Chiaramente guardando alla situazione illustrata si verificheranno due overflow con due conseguenti split e riorganizzazioni delle pagine 0 e 1. Questo viene rappresentato in Figura 8.6.

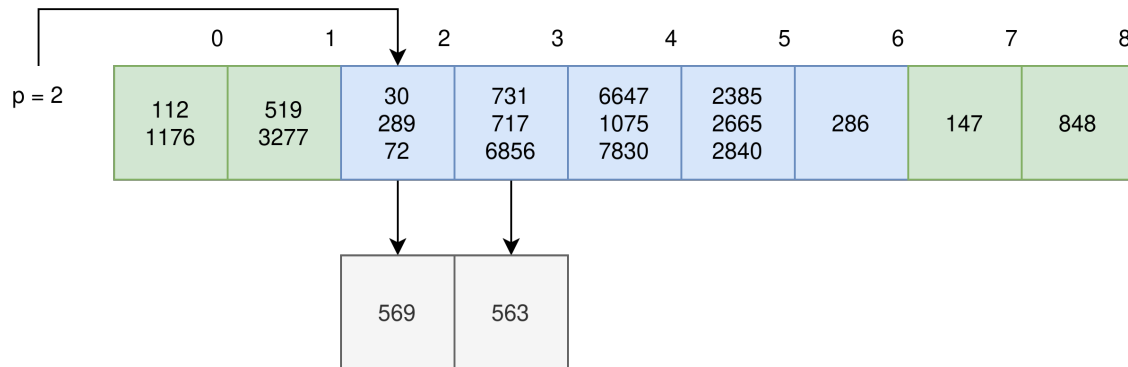


Figura 8.6: Situazione dopo due split in linear hashing

Ipotizziamo di voler andare ora ad aggiungere la chiave 3820, il cui $H_0(3820) = 5$. Andremo a creare una **catena di overflow** sulla pagina 5 e a effettuare uno split della pagina due, riorganizzandone i valori andando a eliminare l'overflow per quella pagina. Questo viene mostrato in Figura 8.7.

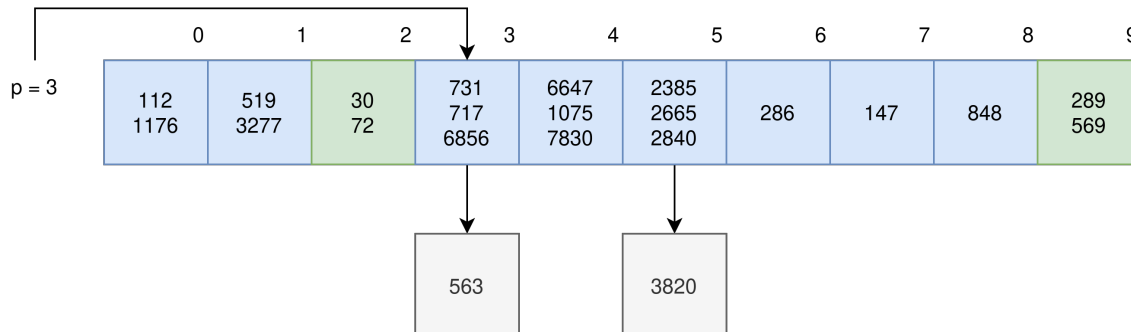


Figura 8.7: Situazione dopo l'inserimento di 3820 in linear hashing

Di seguito mostriamo l'algoritmo per la ricerca di una chiave k tramite linear hashing:

```

1 def SearchPage(p, k: int) -> int:
2     if H_0(k) < p:
3         return H(1, k)
4     else:
5         return H(0, k)

```

Python

Dopo che si saranno verificati M overflow e conseguenti split, il puntatore p tornerà ad essere 0 e le funzioni di hash saranno sovra-scritte, ossia $H_0 \leftarrow H_1$, $H_2 = k \bmod 2^2 M$, e così via.

Nonostante questo approccio goda di ottimi costi medi, e di una buona gestione dell'occupazione dello spazio è ancora impossibile effettuare ricerche su intervalli di valori, è inoltre difficile valutare le performance nel caso peggiore.

8.3. Strutture basate su B+ Tree

Introduciamo in questa sezione quella che è la soluzione più utilizzata in generale, dal momento che è in grado di bilanciare in maniera ottimale l'utilizzo dello spazio garantendo anche **ricerche su intervalli**. La soluzione in questione è fornita dalla struttura **B+ Tree**.

Si tratta di un albero **multi-way**, ossia un albero in cui ogni nodo può avere più di due figli. Si rende infatti necessario aumentare il branching factor rispetto all'albero binario che non è facilmente paginabile.

All'interno di questa struttura i record vengono memorizzati in **pagine foglia**, mentre tutti i nodi interni contengono valori chiave per **direzionare la ricerca** verso le pagine foglia corrette. Se l'**ordine** (o branching factor di ogni nodo) è m , ogni nodo conterrà al suo interno al massimo $m - 1$ elementi. Il minimo numero di elementi in un nodo è invece $\lceil \frac{m}{2} \rceil - 1$.

Un'importante proprietà di questo tipo di alberi è che *tutte le pagine foglia* si trovano allo *stesso livello* di profondità. Questo garantisce che tutte le ricerche abbiano lo stesso (equo) costo.

Osservazione

Il numero minimo di elementi in un nodo è necessario per garantire la proprietà di **bilanciamento**. Questo si rivela di fondamentale importanza per garantire che le stime dei costi delle operazioni siano veritiere e affidabili anche nei casi pessimi.

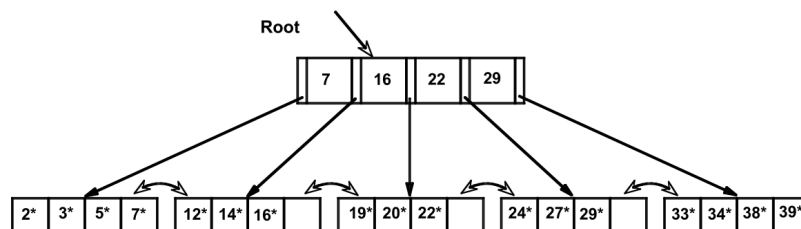


Figura 8.8: Esempio di B+ Tree di ordine 4

Il fatto che distingue fondamentalmente un B+ Tree da un B-Tree è che in un B+ Tree tutti i record sono memorizzati in nodi foglia, mentre i nodi interni contengono informazioni per il “direzionamento” della ricerca. Questo comporta che nel momento in cui vogliamo andare a scorrere tutti i nodi foglia, possiamo farlo in maniera estremamente efficiente, dal momento che questi nodi sono collegati tra di loro tramite puntatori. Questo rende i B+ Tree estremamente efficienti per ricerche su intervalli di valori, come mostrato in Figura 8.8.

8.3.1. Ricerca di una chiave

Nel caso di un B-Tree se avessimo voluto cercare tutti i valori inferiori a 12 in Figura 8.8, avremmo dovuto seguire il primo puntatore del nodo radice, un poi, una volta terminato, tornare alla radice per seguirne il secondo valore. Utilizzato un B+ Tree abbiamo invece la possibilità di scendere il primo puntatore utile e, dal momento che le foglie sono ordinate secondo un attributo (**sequenzialità**), scorrere i collegamenti tra le pagine foglia evitando di dover risalire l'albero ogni volta.

Questa struttura spesso il nome di **index-organized table** dal momento che l'intera tabella viene memorizzata all'interno della struttura ad albero.

È molto facile vedere come la ricerca di una **range di valori** $[k_1, k_2]$ possa essere effettuata in maniera molto efficiente, andando a cercare il primo valore k_1 e scorrendo le pagine foglia fino a raggiungere il valore k_2 .

Osservazione

Possiamo in un certo senso vedere la struttura dei nodi interni del B+ Tree come una sorta di **indice multi-livello** che ci permette di localizzare in maniera efficiente le pagine foglia.

8.3.2. Inserimento di una chiave

Nel momento in cui volessimo andare ad inserire una nuova chiave k all'interno del B+ Tree, distinguiamo due casi:

- la pagina foglia in cui dovrebbe essere inserita la chiave k ha spazio sufficiente per inserirla: in questo caso andremo semplicemente ad inserire la chiave nella pagina foglia, mantenendo l'ordinamento dei valori al suo interno.
- la pagina foglia in cui dovrebbe essere inserita la chiave k non ha spazio a sufficienza: andiamo di seguito a esplorare come avviene questa delicata operazione.

Possiamo avere due tipologie di overflow su un nodo dell'albero da gestire:

- **overflow** su una **pagina foglia**: in questo caso il nodo foglia viene diviso, il primo nodo conterrà $\lceil \frac{M}{2} \rceil - 1$ chiavi, mentre il secondo conterrà le chiavi rimanenti. Il nodo padre di questo nodo foglia si vedrà aggiunto il primo elemento della foglia di destra
- **overflow** su un **nodo intermedio**: in questo caso, analogamente a quanto avviene per le pagine foglia, il nodo intermedio viene diviso in due nodi, con il primo che conterrà $\lceil \frac{M}{2} \rceil - 1$ chiavi. Di nuovo la più piccola delle chiavi del nodo di destra verrà promossa al nodo padre

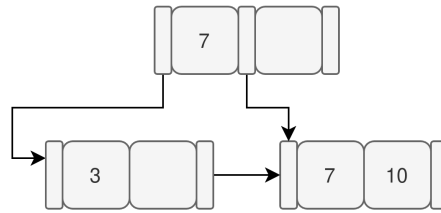
Supponiamo di avere un B+ Tree di ordine $m = 3$, questo significa che ogni nodo può contenere al massimo 2 chiavi e 3 puntatori. Ipotizziamo che questo albero sia inizialmente vuoto e che vogliamo andare ad inserire le chiavi 3, 10, 7, 15, 20, 13 in questo ordine.

Dopo l'inserimento delle chiavi 3 e 10 la situazione sarà quella mostrata di seguito, con entrambe le chiavi memorizzate nella stessa pagina foglia e nessun nodo intermedio.

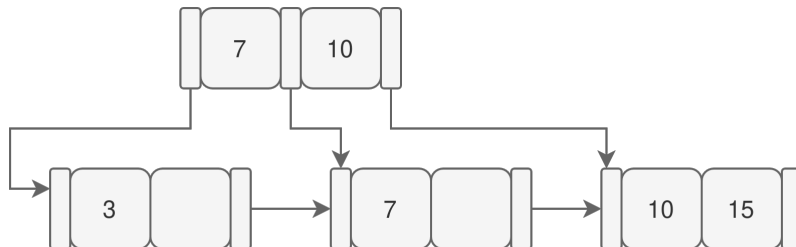


Ipotizziamo ora di dover inserire la chiave 7. Questa dovrebbe essere inserita tra 3 e 10, ma la pagina foglia non ha sufficiente spazio. È quindi necessario effettuare uno **split** della pagina foglia. Per capire come dividere i valori nelle foglie di solito andiamo a mettere $\lceil \frac{M}{2} \rceil - 1$ nodi a sinistra e i nodi rimanenti nel nodo di destra. Dal momento che non abbiamo nodi intermedi per gestire il flusso di ricerca, andremo ad inserire un nodo intermedio che conterrà il valore

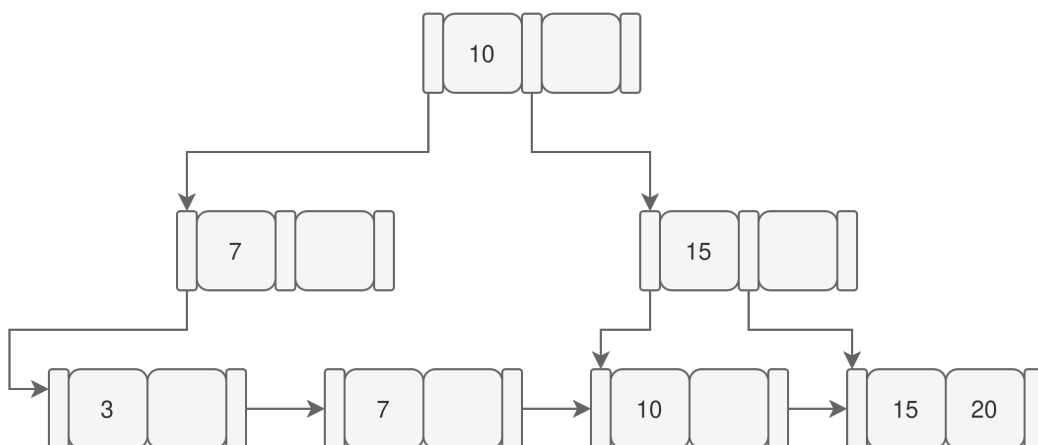
di chiave più piccolo del nodo di destra, in questo caso 7. La situazione dopo l'inserimento della chiave 7 sarà dunque la seguente:



Volendo ora inserire il 15, chiaramente questo andrà inserito nella pagina foglia di destra. Nuovamente sarà necessario operare uno split, dal momento che la pagina foglia non ha sufficiente spazio. Dopo aver effettuato lo split ci troveremo con due nuovi nodi foglia, uno contenente unicamente il valore 7 e uno contenente i valori 10 e 15. Dopo questo passaggio è inoltre necessario aggiungere al nodo intermedio precedente (che è anche la radice) un nuovo valore di chiave, che sarà il 10. Questo viene mostrato nell'immagine che segue:



È ora il momento di aggiungere la chiave 20. Questa andrà inserita nell'ultima pagina foglia ma, non essendo disponibile spazio sufficiente, sarà necessario effettuare un nuovo split. Dopo aver effettuato lo split, andremo ad aggiungere il valore 15 al nodo intermedio, il quale non avendo a sua volta sufficiente spazio, dovrà essere anch'esso diviso. L'inserimento della chiave 13, andrà in seguito a riempire una pagina con spazio sufficiente, dunque non sarà necessario effettuare alcuno split. Di seguito mostriamo la situazione dopo l'inserimento di tutti le chiavi.



8.3.3. Cancellazione di una chiave

Come possiamo immaginare, la cancellazione viene tipicamente operata solamente sulle pagine foglia. È tuttavia necessario che un nodo non vada in “**underflow**”, ossia che non contenga meno della soglia minima di elementi, che ricordiamo essere $\lceil \frac{M}{2} \rceil - 1$.

In questo caso sarà necessaria una riorganizzazione della struttura. Per fare questo abbiamo tipicamente a disposizione due operazioni:

- **rotazione**: viene tipicamente utilizzata nel momento in cui un nodo fratello ha più del numero minimo di elementi. In questo caso andremo a prendere in prestito un elemento dal nodo fratello, aggiornando di conseguenza il nodo padre
- **merging**: viene utilizzata quando il fratello immediatamente successivo ha esattamente il minimo degli elementi. In questo caso andremo a fondere i due nodi, eliminando il puntatore al nodo fratello dal nodo padre. Questa operazione può causare underflow anche nel padre, il quale andrà gestito in maniera ricorsiva.

8.3.4. Profondità di un B+ Tree

Come possiamo immaginare, e come abbiamo anche visto studiando le strutture ad albero in generale, la **profondità** di questo albero va a determinare in maniera significativa il costo delle operazioni di ricerca. Possiamo andare a stimare la profondità dell'albero andando a stabilire un lower bound (caso *migliore*) e un upper bound (caso *peggiore*):

$$\log_{m(N+1)} \leq h \leq \log_{\lceil \frac{m}{2} \rceil} \left(\frac{N+1}{2} \right) \quad (8.1)$$

Ovviamente il caso peggiore va a verificarsi quando ogni nodo è occupato dal numero minimo di elementi, mentre il caso migliore si verifica quando ogni nodo è completamente pieno.

Applicazioni Pratiche dei B+ Tree

Andiamo a vedere alcuni valori tipici dei parametri per la costruzione di un B+ Tree in applicazioni pratiche. Normalmente l'ordine m di un albero è un valore $\in [100, 200]$, questo permette di avere alberi con un fill-factor che si aggira attorno al 67% circa.

Questi valori consentono di avere alberi strutturati in modo tale che la parte dell'indice (**nodi intermedi**) sia ospitabile all'interno del buffer pool, consentendo così di ridurre notevolmente il numero di accessi a disco necessari per le operazioni di ricerca.

8.4. Indici

Dopo aver introdotto alcune tipologie di organizzazioni primarie per i file, andiamo a vedere da questo momento in poi le organizzazioni **secondarie**, ossia strutture dati basate su **indici**.

Definizione 8.1 (Indice)

Un **indice** è una collezione di record con campi $[k_i, r(k_i)]$, dove k_i è una chiave e $r(k_i)$ è un **record identifier** (RID) per il record con chiave k .



Utilizzare una struttura secondaria come un indice, permette di non dover modificare il riferimento fisico dei record nel file principale. Questo ci consente di poter utilizzare organizzazioni primarie più semplici ed economiche, come quella seriale (heap file), dal momento che la gestione dell'ordinamento e della localizzazione dei record viene demandata all'indice secondario.

Dato un file con molteplici record, è possibile stabilire diversi tipi di indici, ognuno dei quali con una chiave di ricerca differente. Il fatto di poter utilizzare più indici su uno stesso file consente di poter effettuare ricerche efficienti su condizioni differenti, senza dover modificare la struttura primaria del file. Questo sarebbe impossibile nel caso di organizzazioni primarie basate su alberi o heap file sequenziale, dal momento che in questi casi l'ordinamento fisico dei record è vincolato alla chiave primaria.

Indici Clustered e Unclustered

Sebbene si tratti di una struttura secondaria, potrebbe capitare che in maniera casuale, l'indice vada più o meno a ricalcare la struttura ordinata di un file primario. In questi casi si parla di **indici clustered**, in caso contrario si parla di **indici unclustered**.

TABLE				Index(CODE)		Index(BY)	
RID	Code	City	BY	Code	RID	BY	RID
1	100	MI	1972	100	1	1972	2
2	101	PI	1970	101	2	1970	4
3	102	PI	1971	102	3	1971	5
4	104	FI	1970	104	4	1970	3
5	106	MI	1970	106	5	1970	1
6	107	PI	1972	107	6	1972	6

Figura 8.9: Esempio di indice clustered (sinistra) e unclustered (destra) data una tabella

Indici Densi e Sparsi

Un'altra importante distinzione che è possibile fare è quella tra indici **densi** e **sparsi**. Un indice è detto *denso* quando il numero di entry è uguale al numero di record che sono memorizzati in un file. Al contrario un indice *sparso* contiene un sottoinsieme delle entry del file primario. Un esempio di indice *denso* si può trovare in entrambi i casi di Figura 8.9, in cui ogni record della tabella (file) ha una entry nell'indice.

Un esempio di indice sparso è un indice in cui ogni elemento di questo ha un puntatore ad un numero di pagina, non ad un RID. Sebbene possa sembrare un approccio meno utile rispetto ad un indice denso, è importante ricordare che il collo di bottiglia è sempre l'accesso alla pagina, non tanto alla ricerca di uno specifico record all'interno di questa. Concretamente un esempio di indice sparso si può vedere in un B+ Tree: i nodi interni contengono informazioni riguardo ad intervalli di valori, e ci permettono di navigare verso la pagina che contiene un certo record.

8.4.1. Modalità di Costruzione di un Indice

Ipotizziamo di trovarci in una situazione in cui vorremmo andare a tenere in considerazione più attributi come chiavi di ricerca su uno stesso file. In questo caso, è possibile definire diversi indici, ognuno dei quali basato su una chiave differente.

La gestione degli indici in presenza di più chiavi dipende fortemente dal tipo di organizzazione adottata per il file.

Organizzazione Secondaria o Primaria Basata su Heap File

Quando i dati sono organizzati come heap file, non esiste un ordinamento fisico basato su una particolare chiave. In questo scenario sarà necessario creare un indice per ogni chiave su cui desideriamo effettuare ricerche efficienti.

Tutti gli indici creati in questo scenario seguiranno lo schema $K \rightarrow \text{RID}$, dove K è la chiave su cui l'indice è basato e RID è il record identifier del record associato a quella chiave. Ipotizzando dunque di avere una chiave primaria (a livello logico) K_p e un'altra chiave K , sarà sostanzialmente indifferente creare un indice su K_p o su K , entrambi seguiranno lo schema $K \rightarrow \text{RID}$.

Organizzazione Primaria Basata su Chiave K_p

Quando i dati sono già organizzati fisicamente secondo una chiave primaria K_p (ad esempio tramite hash o B+ Tree), la situazione cambia: non è infatti necessario creare un indice per K_p poiché è l'organizzazione primaria stessa a fornire accesso diretto ai record tramite questa chiave logica.

Per quanto riguarda la chiave K , l'indice che andremo a costruire non avrà più uno schema di tipo $K \rightarrow \text{RID}$, ma piuttosto uno schema di tipo $K \rightarrow K - p$, dal momento che l'organizzazione primaria fornisce già un meccanismo efficiente per localizzare i record con chiave di ricerca K_p .

Sebbene da un punto di vista teorico sia possibile creare un indice secondario su K che mappi a RID, questo non è generalmente pratico o efficiente: nel caso di strutture statiche come l'hash statico, potrebbe essere fattibile, nel momento in cui adottassimo un'organizzazione dinamica, l'indice creato verrebbe invalidato ogni volta che un record si sposta fisicamente.

8.4.2. Costi degli Indici

In questa sezione andiamo ad analizzare il costo in termini di numero di accessi a pagina per effettuare diversi tipi di operazioni a seconda dell'indice che si intende utilizzare. Supporremo che gli indici presi in considerazione siano **densi**.

Il costo a livello generale si può suddividere in due componenti principali:

- costo di **accesso all'indice**
- costo di **accesso ai dati**

In ogni caso, il costo di accesso all'indice potrà sempre essere considerato come un singolo accesso a pagina, nel caso peggiore, nel momento in cui l'indice non sia presente in memoria. Per la **ricerca** di un **singolo valore** (equality search), il costo è sempre 1 accesso a pagina, dal momento che l'indice fornisce un puntatore diretto al record.

Ciò in cui possiamo trovare differenze è il costo per la ricerca di un intervallo di valori. Ricordiamo in primo luogo l'importanza del **fattore di selettività** $f(p)$ che serve a dare una stima del numero di record che hanno valori appartenenti ad un certo intervallo. La formula per stimarlo è la seguente:

$$f(p) = \frac{v_2 - v_1}{V_{\max} - V_{\min}} \quad (8.2)$$

La differenza tra la ricerca di **intervalli** è mostrata anche in maniera visuale in Figura 8.10.

Indici Unclustered

Nel caso di indici unclustered, nei quali i record non sono memorizzati in modo ordinato rispetto alla chiave di ricerca. In questo caso il costo di accesso all'indice potrebbe essere sia 0 che 1, a seconda che l'indice sia presente in memoria o meno.

Sappiamo che tutte le foglie di un indice, sono sempre ordinate rispetto alla chiave di ricerca. Tuttavia la struttura su cui sono memorizzati i record potrebbe non esserlo, si veda Figura 8.9 per maggiore chiarezza.

Il costo di accesso all'indice sarà dato dal fattore di selettività moltiplicato per il numero di foglie presenti nell'indice. Una volta ottenuti i puntatori a record, non potremo fare altro che accedere a ciascun record singolarmente, dal momento che non esiste un ordinamento fisico che ci permetta di accedere a più record in un singolo accesso a pagina. Dunque il costo totale per la ricerca di un intervallo di valori in un indice unclustered sarà:

$$C_{\text{unclustered}} = f(p) \cdot N_{\text{leaves}} + f(p) \cdot N_{\text{records}} \quad (8.3)$$

Indici Clustered

Nel caso in cui lavorassimo con un indice di tipo clustered, dove l'ordinamento delle chiavi nell'indice rispecchia l'ordinamento fisico dei dati, per una ricerca un un range di valori avremo la possibilità di accedere a più record in un singolo accesso a pagina sfruttando la località spaziale dei dati.

Il costo di accesso all'indice sarà sempre dato dal fattore di selettività moltiplicato per il numero di foglie presenti nell'indice. Tuttavia, una volta ottenuti i puntatori ai record, dovremo accedere a un numero di pagine proporzionale al fattore di selettività. Dunque il costo totale per la ricerca di un intervallo di valori in un indice clustered sarà:

$$C_{\text{clustered}} = f(p) \cdot N_{\text{leaves}} + f(p) \cdot N_{\text{pages}} \quad (8.4)$$

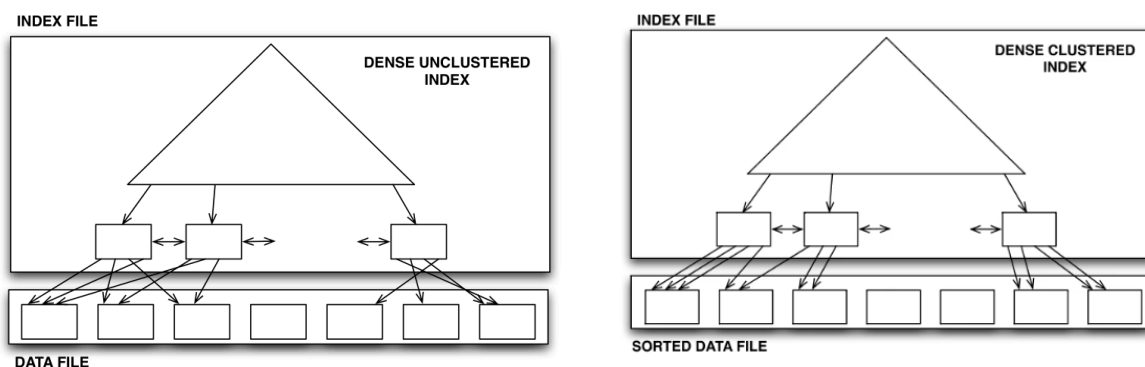


Figura 8.10: Ricerca di un intervallo di valori in un indice unclustered e clustered

8.5. Indici su Attributi Non Chiave

Fino a questo momento abbiamo sempre utilizzato indici che lavorano con attributi **chiave**, ossia attributi il cui valore è univoco per ogni record. Tuttavia, in molti casi potremmo aver bisogno di effettuare ricerche su attributi diversi, non chiave, i cui valori potrebbero **non esistere** o essere **ripetuti** in più record.

Ipotizzando di costruire un B+ Tree su un attributo totale, andremmo a “rinunciare” a una proprietà sostanziale dei B+ Tree, ossia il fatto che esso induca un **ordinamento totale** sui record. Nel caso di attributi non univoci potremmo avere delle uguaglianze tra chiavi. Chiaramente questo approccio potrebbe essere utilizzato ma possiamo immaginare che non sia il più efficiente. Basti semplicemente pensare al fatto che staremmo “sprecando” delle foglie per memorizzare dei record che potrebbero essere memorizzati insieme.

8.5.1. Indici a Lista Invertita

Per capire il senso di questo approccio, proviamo a guardare alla situazione seguente:

Index on Quantity		Sales					
Quantity	RID	RID	Date	Product	City	Quantity	
1	5	1	20090102	P1	Lucca	2	
2	1	2	20090102	P2	Carrara	8	
2	8	3	20090103	P3	Firenze	5	
2	9	4	20090103	P1	Arezzo	10	
5	3	5	20090103	P1	Pisa	1	
5	7	6	20090103	P4	Pisa	8	
5	10	7	20090103	P2	Massa	5	
8	2	8	20090104	P2	Massa	2	
8	6	9	20090105	P4	Massa	2	
10	4	10	20090103	P4	Livorno	5	

Figura 8.11: Esempio di tabella con attributo non chiave 'Quantity'

Possiamo notare che l'indice sulla tabella mostrata, contiene un sacco di entry duplicate, dal momento che l'attributo “Quantity” non è una chiave univoca. In questo caso potremmo andare a creare un indice di tipo **lista invertita**.

Inverted Index on Quantity			Sales					
Quantity	n	RID list	RID	Date	Product	City	Quantity	
1	1	5	1	20090102	P1	Lucca	2	
2	3	1, 8, 9	2	20090102	P2	Carrara	8	
5	3	3, 7, 10	3	20090103	P3	Firenze	5	
8	2	2, 6	4	20090103	P1	Arezzo	10	
10	1	4	5	20090103	P1	Pisa	1	
			6	20090103	P4	Pisa	8	
			7	20090103	P2	Massa	5	
			8	20090104	P2	Massa	2	
			9	20090105	P4	Massa	2	
			10	20090103	P4	Livorno	5	

Figura 8.12: Esempio di indice a lista invertita per l'attributo 'Quantity'

Possiamo notare come l'utilizzo di questo indice invertito ci permetta di memorizzare in maniera più efficiente i record, raggruppando assieme i record che condividono lo stesso valore per l'attributo "Quantity". Un altro aspetto importante che possiamo notare rispetto agli indici invertiti è il fatto che, come vediamo in Figura 8.12, la lista dei RID per ciascun valor è memorizzata in **ordine**.

Vantaggi degli Indici invertiti

Di seguito andiamo ad elencare i vantaggi dell'impiego di questo tipo di struttura per la gestione di indici su attributi non chiave:

- il risultato di operazioni di **conteggio** su attributi non chiave può essere ottenuto in maniera estremamente efficiente, andando unicamente a controllare l'indice
- l'organizzazione fisica del file è totalmente indipendente dall'organizzazione dell'indice, permettendo così di utilizzare organizzazioni primarie più semplici

Requisiti di memoria

Dato un insieme di dati R , possiamo identificare i seguenti parametri:

- $N_{\text{rec}}(R)$: il numero di record in R
- $N_{\text{pag}}(R)$: il numero di pagine utilizzato per memorizzare i record in R
- L_R : il numero di byte necessari per memorizzare numero di record $N_{\text{rec}}(R)$
- $N_{I(R)}$: il numero di indici creati su R
- L_I : il numero medio di byte necessario a rappresentare un valore per la chiave sulla quale è costruito l'indice I
- $N_{\text{key}}(I)$: il numero di valori distinti per la chiave dell'indice I
- $N_{\text{leaf}}(I)$: il numero di foglie nell'indice I
- L_{RID} : il numero di byte necessari per memorizzare un RID

Dati i parametri sopra descritti possiamo descrivere il costo di memorizzazione degli indici definiti su R come segue:

$$\begin{aligned} \mathcal{M} &= \sum_{i=1}^{N_I(R)} N_{\text{key}}(I_i) (L_{I_i} + L_R) + N_I(R) N_{\text{rec}}(R) L_{\text{RID}} \\ &\approx N_I(R) N_{\text{rec}}(R) L_{\text{RID}} \end{aligned} \quad (8.5)$$

Dove, la **prima parte** rappresenta il costo di memorizzazione della chiave per ogni record nell'indice, mentre la **seconda parte** rappresenta il costo di memorizzazione delle liste di RID per ogni lista invertita di ogni indice. Il motivo per cui la prima parte viene omessa è dato dal fatto che, tipicamente, in un indice a lista invertita, la terza colonna, ovvero la lista dei RID per ciascun valore è ciò che domina il costo di memorizzazione dell'indice. Se questo non fosse il caso, potremmo semplicemente utilizzare un indice B+ Tree standard.

Costo Ricerca di una Chiave

Per quanto riguarda il costo legato alla ricerca di una chiave sappiamo che questo si può scomporre in due elementi principali. Il costo legato all'accesso all'indice e il costo legato all'accesso ai dati.

Dal momento che stiamo lavorando su un indice definito su valori non univoci, possiamo immaginarci che il processo di ricerca sia simile a quello che avviene con la ricerca su un intervallo. Andiamo dunque a definire il **fattore di selettività**:

$$f_s(\psi) = f_s(A = v) = \frac{1}{N_{\text{key}}(I)}$$

Possiamo vedere questo fattore di selettività come la probabilità che un record abbia un certo valore v per l'attributo A . Non sempre possiamo dare un valore preciso a questo numero, ma dal momento che tipicamente le chiavi sono distribuite in maniera uniforme possiamo assumere che la questo valore sia stimabile con la formula mostrata in precedenza. Il costo totale di accesso all'indice sarà dunque:

$$C_I = \lceil f_s(\psi) N_{\text{leaf}}(I) \rceil = \left\lceil \frac{N_{\text{leaf}}(I)}{N_{\text{key}}(I)} \right\rceil$$

Per quanto riguarda il costo di accesso ai dati abbiamo, in maniera analoga al caso in cui stiamo ricercando un intervallo di valori univoci, che il costo varia tra indici clustered e unclustered. Iniziamo con il definire E_{rec} che rappresenta il numero di record che rappresenta il numero di record che soddisfano la condizione ψ :

$$E_{\text{rec}} = N_{\text{rec}}(R) f_s(\psi)$$

A questo punto possiamo andare a vedere come variano i costi di accesso ai dati. Nel caso di un **indice clustered** il costo sarà dato da:

$$C_D = \lceil N_{\text{pag}}(R) f_s(\psi) \rceil \quad (8.6)$$

in maniera analoga a quanto visto per la range search. Nel caso invece in cui avessimo a che fare con un **indice unclustered** con liste ordinate di RID il costo sarà dato da:

$$C_D = \Phi(E_{\text{rec}}, N_{\text{pag}}(R)) \quad (8.7)$$

dove la funzione $\Phi(x, y)$ è anche detta formula di **Cardenas**: $\Phi(k, n) = n \left(1 - \left(1 - \frac{1}{n} \right)^k \right)$, ed è una funzione limitata superiormente dal minimo tra k e n . Possiamo vedere il grafico di questa funzione in Figura 8.13.

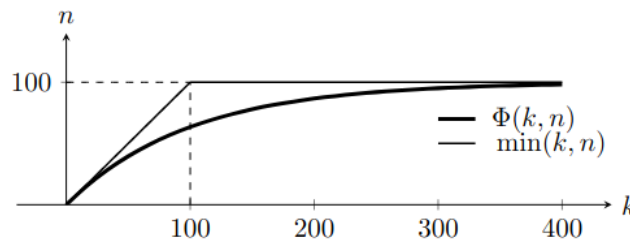


Figura 8.13: Grafico della funzione di Cardenas

Costo di Ricerca di un Range di Valori

Nel caso in cui vogliamo ricercare un range di valori $[v_1, v_2]$ per un attributo non chiave, possiamo andare a definire il fattore di selettività come:

$$f_s(\psi) = f_s(A \in [v_1, v_2]) = \frac{v_2 - v_1}{\max(A) - \min(A)}$$

A questo punto possiamo vedere come il costo di accesso all'indice sia dato, come usuale, dal numero di foglie moltiplicato per il fattore di selettività:

$$C_I = \lceil N_{\text{leaf}}(A_i) \cdot f_s(\psi) \rceil$$

Questo dal momento che avremo bisogno di accedere ad un numero di foglie proporzionale a quante foglie andrà a coprire l'intervallo di valori. Per quanto riguarda il costo di accesso ai dati, questo sarà in maniera molto generale dato dal prodotto tra il numero liste e il numero di pagine per ogni lista. Possiamo in primo luogo stimare il numero di liste selezionate dall'intervallo utilizzando il fattore di selettività:

$$N_{\text{lists}} = \lceil N_{\text{key}}(A) \cdot f_s(\psi) \rceil$$

Una volta ottenuto il numero di liste non ci resta che andare a studiare il numero di pagine. Come prevedibile, questo sarà diverso in base a se abbiamo a che fare con un indice clustered o unclustered. Nel caso di indici **clustered** avremo:

$$N_{\text{page}} = \left\lceil N_{\text{pag}(R)} \cdot \frac{1}{N_{\text{key}}(A)} \right\rceil$$

Questo dal momento che si suppone che i record siano distribuiti in maniera uniforme su tutte le chiavi. Nel caso di indici **unclustered** il numero di pagine sarà dato da:

$$N_{\text{page}} = \left\lceil \Phi \left(\frac{N_{\text{rec}}(R)}{N_{\text{key}}}(A), N_{\text{pag}}(R) \right) \right\rceil$$

8.5.2. Indici Bitmap

Un'altra interessante struttura dati che può essere utilizzata per la gestione di indici su attributi non chiave è quella dell'**indice bitmap**. Una bitmap è un modo alternativo per rappresentare le liste invertite viste in precedenza.

Sales			Bitmap index				
RID	...	Quantity	1	2	5	8	10
1	...	2	0	1	0	0	0
2	...	8	0	0	0	1	0
3	...	5	0	0	1	0	0
4	...	10	0	0	0	0	1
5	...	1	1	0	0	0	0
6	...	8	0	0	0	1	0
7	...	5	0	0	1	0	0
8	...	2	0	1	0	0	0
9	...	2	0	1	0	0	0
10	...	5	0	0	1	0	0

Figura 8.14: Esempio di indice bitmap per l'attributo 'Quantity'

Possiamo vedere l'indice bitmap come una tabella, in cui ogni colonna è un valore distinto per l'attributo su cui è costruito l'indice, mentre ogni riga rappresenta un RID. Figura 8.14 mostra come l'indice invertito di Figura 8.12 possa essere rappresentato come un indice bitmap.

Dimensione di un Indice Bitmap

È interessante andare a confrontare la memoria occupata da un indice invertito implementato tramite B+ Tree e quella di un indice bitmap costruiti sullo stesso attributo. Intuitivamente è facile immaginare come, nel caso in cui abbiamo pochi valori distinti per l'attributo, l'indice bitmap risulti essere più compatto rispetto all'indice invertito. Viceversa, nel momento in cui l'indice va a partizionare i dati sulla base di molteplici valori distinti, l'indice bitmap rischia di esplodere in dimensionalità rispetto all'indice invertito.

Nel caso di molti valori distinti, ci troveremmo infatti con un indice bitmap formato da molte colonne, ognuna delle quale andrebbe a contenere un numero di bit pari al numero di record totali. Al contrario, nel caso in cui abbiamo pochi valori distinti, l'indice bitmap risulterebbe essere estremamente compatto, dal momento che il numero di colonne sarebbe molto basso.

Vantaggi degli Indici Bitmap

Di seguito andiamo ad elencare i vantaggi dell'impiego di una simile struttura dati:

- si tratta di un approccio **leggero** nel caso in cui l'attributo su cui è costruito l'indice abbia pochi valori distinti
- le operazioni di **AND**, **OR** e **NOT** tra bitmap sono estremamente efficienti, dal momento che possono essere effettuate a livello di bit
- si tratta di un approccio molto efficiente per query che utilizzano il **conteggio** dei record che soddisfano una certa condizione

9. Organizzazioni e Indici Multidimensionali

Nello scorso capitolo abbiamo visto come i file possono essere organizzati in maniera tale da rendere più efficienti le operazioni di accesso ai dati sulla base di alcuni attributi. Come abbiamo visto possiamo fornire delle organizzazioni primarie, che possono essere statiche o dinamiche, oppure degli **indici** che permettono di accedere ai dati in maniera efficiente anche quando l'organizzazione primaria non è strutturata per soddisfare in maniera efficiente le richieste delle query.

Tutto ciò che abbiamo visto nel capitolo precedente si basa su strutture dati **mono-dimensionali**, ovvero strutture che permettono di organizzare i dati sulla base di un singolo attributo. Tuttavia nella maggior parte dei contesti avremo in realtà a che fare con dati che sono per loro natura **multi-dimensionali**.

Un esempio molto comune di dati multi-dimensionali sono i dati *spaziali*, ovvero dati che rappresentano oggetti nello spazio, come ad esempio punti, linee, poligoni, ecc. Questi dati sono caratterizzati da più attributi che rappresentano le coordinate spaziali (ad esempio latitudine e longitudine) e spesso è necessario effettuare query che coinvolgono più di un attributo (ad esempio trovare tutti i punti all'interno di una certa area geografica).

È necessario prestare particolare importanza alla maniera in cui vengono indicizzati i dati su più attributi: immaginiamo per esempio di avere un indice di primo livello su un singolo attributo (es. età) e un indice di secondo livello su un altro attributo (es. reddito) che mantiene un ordinamento però sulla base dei valori del primo attributo. In questo caso se volessimo effettuare una query che coinvolge unicamente il secondo attributo (ad esempio trovare tutte le persone in un certo intervallo di reddito) l'indice non sarebbe di alcun aiuto, in quanto l'ordinamento è basato sul primo attributo. In altri casi potrebbe essere necessario effettuare delle query che coinvolgono più attributi senza però avere una chiara gerarchia tra di essi (ad esempio trovare tutti i punti in un'area rettangolare definita da due coordinate).

Requisiti per le Organizzazioni Multidimensionali

Prima di andare a vedere alcuni degli approcci più comuni utilizzati è necessario capire come vorremmo organizzare le nostre gestire le nostre organizzazioni in caso di gestione di dati multi-dimensionali.

- per quanto riguarda le organizzazioni primarie, vorremmo fare in modo da dividere i dati in partizioni che raggruppano insieme i dati che verranno acceduti assieme
- per quanto riguarda le organizzazioni secondarie vorremmo comprendere in che modo dividere le “righe” dell'indice tra i nodi foglia dell'indice

Per rispondere a questo tipo di domande è quanto mai utile andare ad analizzare il **tipo di query** che vogliamo andare a supportare nei nostri sistemi, in modo da poter scegliere l'organizzazione più adatta alle nostre esigenze. Le query più comuni che vogliamo andare a supportare sono:

- ricerca di **punti** o di **regioni** specifiche: si tratta dell'equivalente delle *query di uguaglianza* nel caso monodimensionali. In sostanza vogliamo andare a verificare che un

punto specifico (o una regione, ossia un certo insieme di dati specifico) sia presente nei dati.

- ricerca su **range spaziali**: si tratta dell'equivalente delle query di intervallo nel caso monodimensionale. Vogliamo in questo caso andare a cercare tutti i punti che ricadono in un (iper) rettangolo o in una (iper) palla
- ricerca dei **k-nearest neighbors**: in questo caso vogliamo andare a cercare i k punti più vicini ad un punto specifico. Si tratta di una query molto comune in ambito spaziale e di machine learning.

9.1. Tipologie di Organizzazioni Multidimensionali

Esistono diversi modi per organizzare i dati multi-dimensionali, di seguito elenchiamo alcune delle metodologie più comuni.

Linearizzazione

La prima tecnica che possiamo provare ad utilizzare è quella di **linearizzare** lo spazio multi-dimensionale convertendolo in un nuovo spazio uni-dimensionale. In questo caso, abbiamo la necessità di costruire un **ordinamento totale** sui dati. Si rende dunque necessario trasformare i dati in una singola dimensione, in modo tale da poter utilizzare le strutture dati monodimensionali viste nel capitolo precedente.

Ovviamente questo approccio presenta il problema di non garantire che dati diversi vengano mappati in punti dello spazio diversi, ovvero si possono avere delle **collisioni** tra punti diversi che vengono mappati nello stesso punto dello spazio uni-dimensionale.

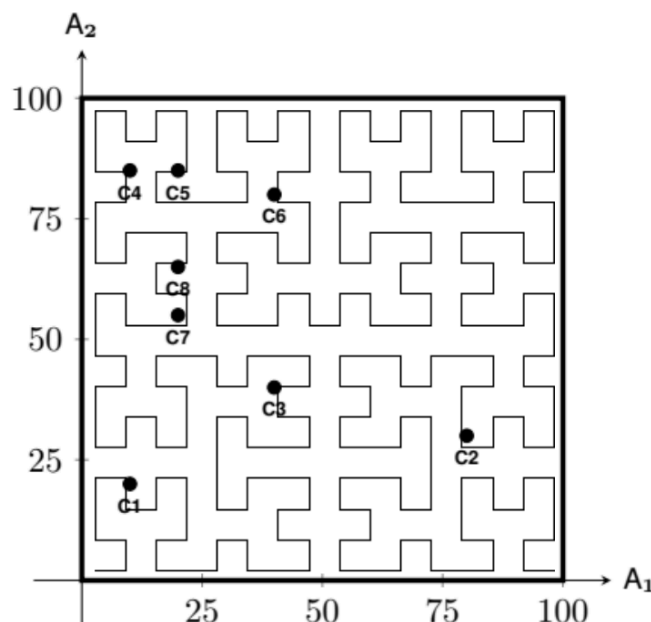


Figura 9.1: Esempio di linearizzazione dello spazio bidimensionale

Possiamo notare come nella precedente immagine ci si possa immaginare di avere una linea che va ad attraversare tutte le possibili posizioni nello spazio bidimensionale (ipotizzando che sia discreto) e che vada a mappare ogni punto in base all'ordine in cui questa linea va a attra-

versare i punti stessi. Un'altro modo per applicare questa tecnica è quello andare a stabilire una **gerarchia** tra gli attributi. Ad ogni modo, vorremmo fare in modo che i punti che sono vicini nello spazio multi-dimensionale rimangano vicini anche nello spazio uni-dimensionale, nel concreto, vorremmo che la curva che riempie lo spazio eviti di fare salti troppo ampi tra punti molto vicini. Questo è proprio ciò che otteniamo in Figura 9.1.

Partizionamento dello Spazio

Un altro approccio è quello di **partizionare** lo spazio in diverse regioni e andare a memorizzare assieme quegli elementi che sono appartenenti alla stessa partizione. Un esempio di come questo avviene si può osservare in Figura 9.2, dove lo spazio bidimensionale viene partizionato in regioni distinte.

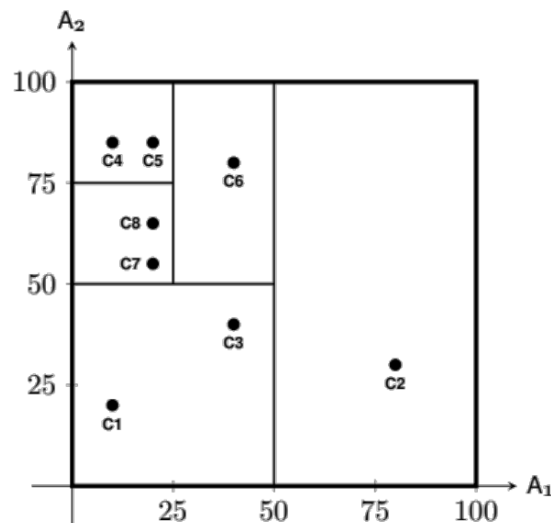


Figura 9.2: Esempio di partizionamento dello spazio bidimensionale

Albero di Ricerca Generalizzato

Questo approccio è molto simile al precedente, ma invece di partizionare tutto lo spazio di ricerca, si dividono i dati in **regioni** che ammettono **overlapping**. Possiamo vedere questo approccio illustrato in Figura 9.3.

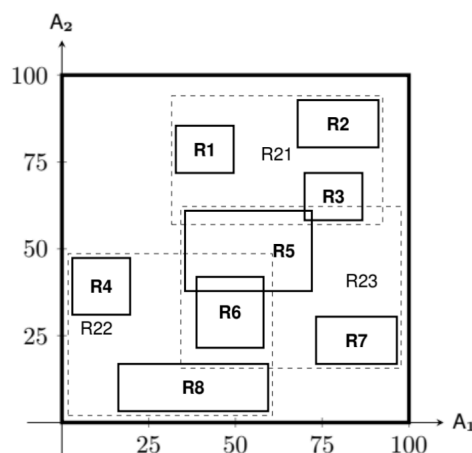


Figura 9.3: Esempio di albero di ricerca generalizzato

9.2. B+ Tree Multidimensionali

Una possibile soluzione per indicizzare dati multi-dimensionali è quella di provare a usare un B+ Tree mantenendo un **ordinamento lessicografico** sui valori degli attributi. Ad esempio, se abbiamo due attributi A e B, potremmo ordinare i dati prima per A e poi per B. In questo modo, quando effettuiamo una query che coinvolge entrambi gli attributi, possiamo navigare nell'albero seguendo l'ordinamento lessicografico.

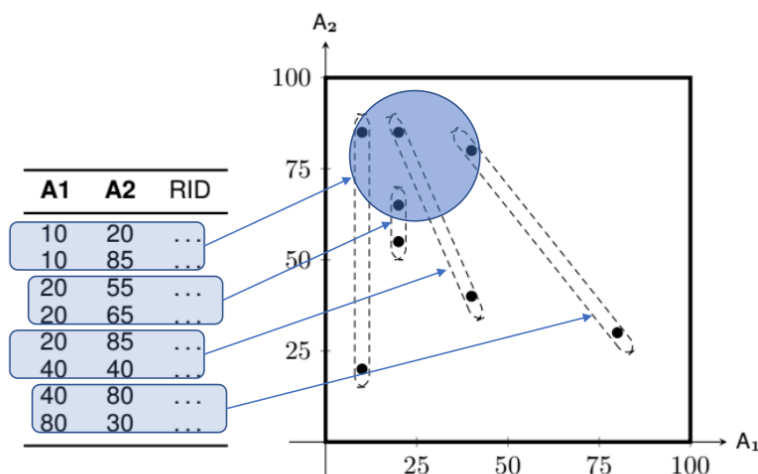


Figura 9.4: Esempio di B+ Tree con ordinamento lessicografico

Figura 9.4 mostra un esempio di come si possa utilizzare un B+ Tree per indicizzare dati sulla base di un attributo A e un attributo B utilizzando un ordinamento lessicografico tra i due attributi. Andiamo in sostanza a definire una **gerarchia** tra i due attributi, dove il secondo attributo serve a *risolvere la parità* sul primo attributo. Questo approccio va proprio ad implementare il concetto di **linearizzazione** mostrato in precedenza.

9.2.1. Vantaggi e Svantaggi

Questo approccio per quanto utile nel caso in cui si vogliano effettuare query **puntuali**. Funziona bene anche nel caso in cui vogliamo andare a effettuare ricerche su **range per il primo attributo**. Per il secondo attributo questo approccio risulta molto meno utile in quanto l'ordinamento è influenzato dal primo attributo.

Altro svantaggio di questo approccio è che non consente di effettuare query del tipo **k-nearest neighbors** in maniera efficiente, in quanto l'ordinamento lessicografico non riflette la distanza spaziale tra i punti.

9.2.2. Space Filling Curves

In questa sezione andiamo a mostrare come le curve di riempimento dello spazio variano a seconda della gerarchia che andiamo ad utilizzare per mappare lo spazio multi-dimensionale in uno spazio uni-dimensionale in un Bè tree.

Vedremo come la scelta dell'ordinamento tra gli attributi vada ad influenzare in maniera sostanziale la qualità della mappatura nello spazio mono-dimensionale.

Iniziamo ad analizzare l'effetto dell'**ordinamento lessicografico** che abbiamo ipotizzato, ossia dove l'attributo A_1 ha priorità sull'attributo A_2 . In questo caso la curva di riempimento dello spazio sarà come quella in Figura 9.5.

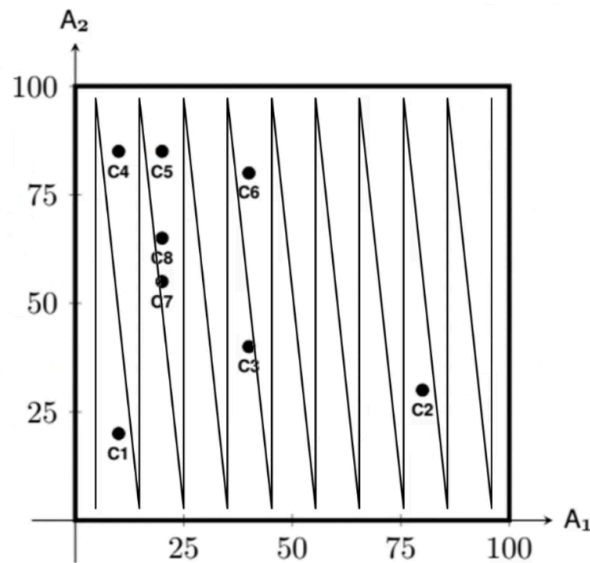


Figura 9.5: Curva di riempimento dello spazio con ordinamento lessicografico

Possiamo notare che in questo caso la curva tende a fare dei **salti molto ampi** in corrispondenza del cambio di valore dell'attributo A_1 . Il problema di questo tipo di mapping si mostra nel senso che a fronte di una leggerissima perturbazione dei valori di A_1 possiamo avere dei cambiamenti molto ampi nella posizione nello spazio uni-dimensionale.

In alternativa all'ordinamento lessicografico possiamo utilizzare un ordinamento che permette di ottenere dei salti più corti per alcuni valori, mentre per altri valori si avranno dei salti comunque grandi, addirittura più grandi rispetto all'ordinamento lessicografico. Questo viene chiamato **ordinamento diagonale** e viene mostrato in Figura 9.6.

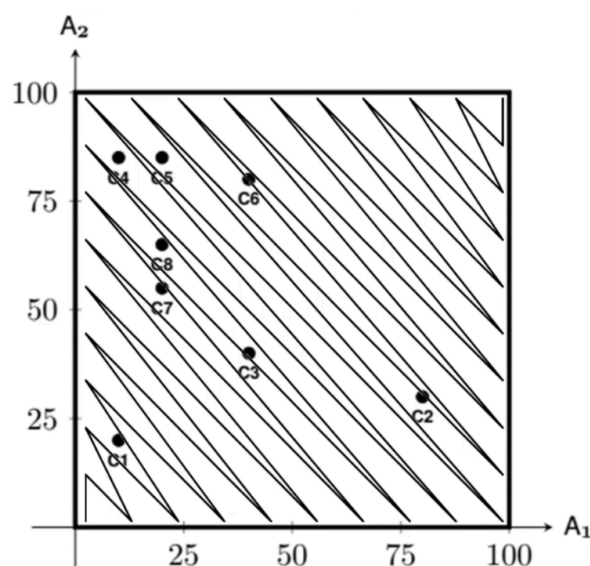


Figura 9.6: Curva di riempimento dello spazio con ordinamento diagonale

Per ottenere tale ordinamento si utilizza la **somma delle due coordinate**. Possiamo notare come in questo caso i salti siano più bilanciati, anche se comunque si hanno dei salti molto ampi in corrispondenza di alcune regioni dello spazio (specialmente sulla diagonale).

Un'alternativa a questo tipo di curva è la **Morton space filling curve** (o Z-order), che viene mostrata in Figura 9.7.

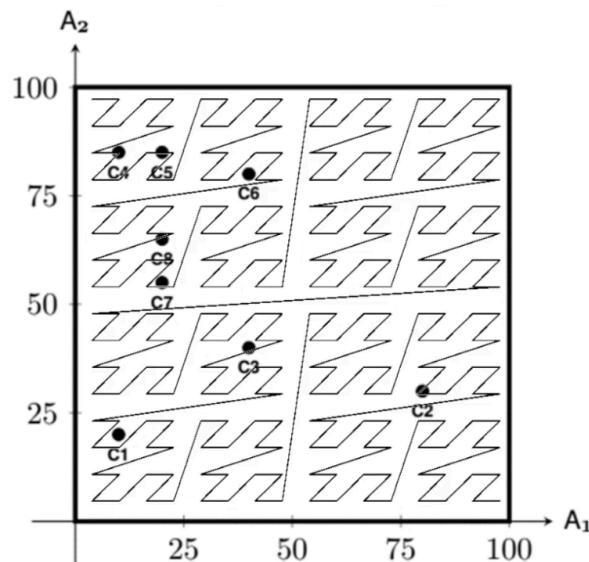


Figura 9.7: Curva di riempimento dello spazio con ordinamento Morton (Z-order)

Il motivo per cui questa curva è chiamata Z-order è che l'elemento di base è una "Z", e ci muoviamo nello spazio seguendo questa forma in maniera ricorsiva a seconda della risoluzione desiderata. Di seguito vediamo alcuni esempi:

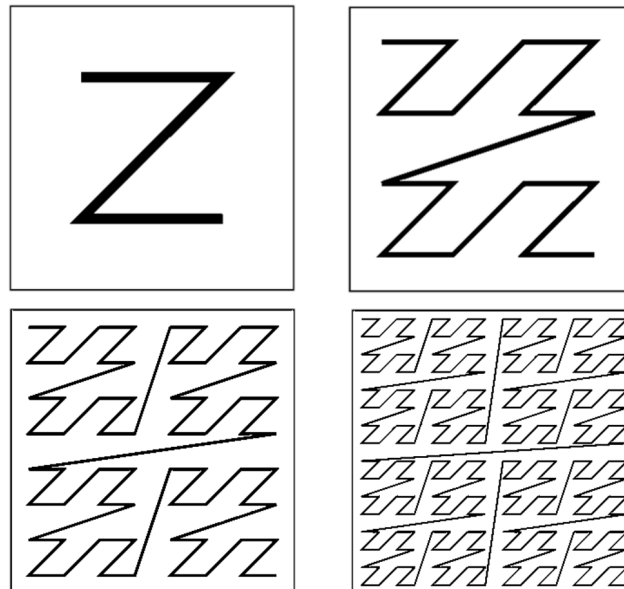


Figura 9.8: Varianti della curva di Morton a diverse risoluzioni

Ciò che accade aumentando il livello di ricorsività è che il numero di "Z" che la curva disegna aumenta esponenzialmente, andando a coprire lo spazio in maniera sempre più dettagliata. In particolare pensiamo alle coordinate come se fossero rappresentabili tramite numeri binari,

possiamo rappresentare ogni punto dello spazio coperto dalla curva tramite una sequenza alternata di bit presi da un attributo e dall'altro. Questo concetto è mostrato in Figura 9.9.

	x:	0	1	2	3	4	5	6	7
		000	001	010	011	100	101	110	111
y: 0	000	000000	000001	000100	000101	010000	010001	010100	010101
1	001	000010	000011	000110	000111	010010	010011	010110	010111
2	010	001000	001001	001100	001101	011000	011001	011100	011101
3	011	001010	001011	001110	001111	011010	011011	011110	011111
4	100	100000	100001	100100	100101	110000	110001	110100	110101
5	101	100010	100011	100110	100111	110010	110011	110110	110111
6	110	101000	101001	101100	101101	111000	111001	111100	111101
7	111	101010	101011	101110	101111	111010	111011	111110	111111

Figura 9.9: Rappresentazione binaria dei punti nello spazio secondo la curva di Morton

Questa proprietà rende il nostro sistema ancora più efficiente dal momento che rende il calcolo di operazioni bit-wise molto semplice ed efficiente.

Un'altra tecnica molto utilizzata è quella di data dalla **Peano space filling curve**. Il vantaggio principale di questa curva è quello che, con livelli di ricorsività molto elevanti la grandezza di “salto” tra punti particolarmente lontani è sempre limitata, rendendo dunque questo approccio più robusto e stabile. Vediamo questo approccio implementato in Figura 9.10.

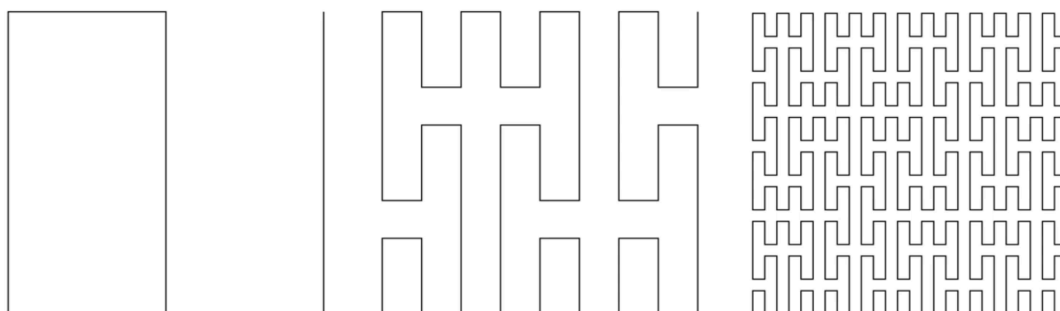


Figura 9.10: Curva di riempimento dello spazio con ordinamento Peano con diversi livelli di ricorsività

Un'altra curva estremamente utile e particolarmente efficiente da calcolare tramite un algoritmo ricorsivo è la **Hilbert space filling curve**. Il pattern di base di questa curva è una “U”. Vediamo come questa curva si comporta in Figura 9.11.

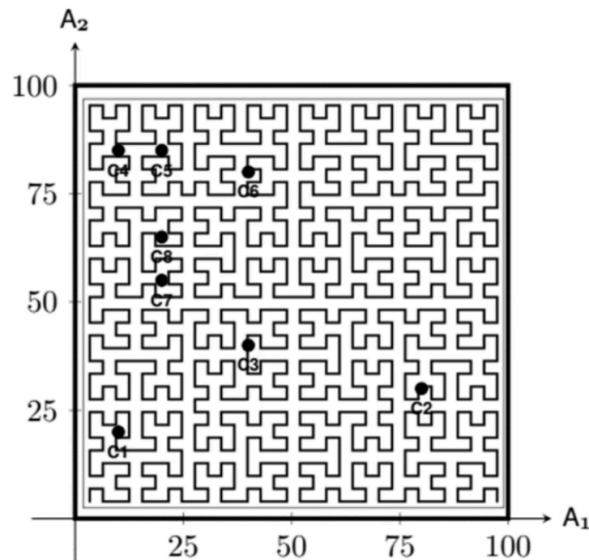


Figura 9.11: Curva di riempimento dello spazio con ordinamento Hilbert

9.3. Tecniche di Space Partitioning

In questa sezione andiamo a vedere quali tecniche sono utilizzate in contesti pratici per andare a gestire lo spazio multidimensionale tramite **partizionamento**. Abbiamo due quesiti principali da provare a risolvere in questo contesto:

- come partizionare lo spazio in regioni contigue senza sovrapposizioni in modo che contengano record che andrebbero memorizzati nella stessa pagina
- come trovare in maniera rapida ed efficiente la regione dello spazio partizionato che contiene i punti che soddisfano una determinata query

9.3.1. Point Region Quadtree

La prima alternativa che abbiamo a disposizione è il **point region quadtree**. Si tratta di un approccio molto simile a quello degli alberi binari. Sappiamo che in un albero binario ogni nodo divide lo spazio in due metà: in parole povere, possiamo immaginarci che ci sia una soglia, e che tutti i valori minori di questa soglia vadano a sinistra, mentre tutti i valori maggiori vadano a destra.

Nel caso di un quadtree (albero quaternario), ogni nodo divide lo spazio in base a entrambi gli attributi (nel caso in cui lo spaio sia bidimensionale). In questo caso ogni nodo avrà quattro figli, ognuno dei quali rappresenta una delle quattro regioni in cui lo spazio viene diviso.

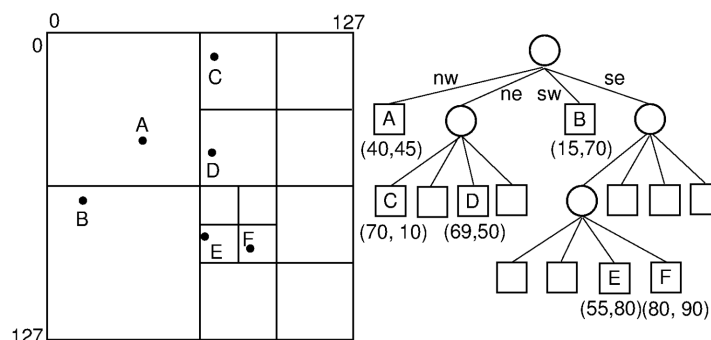


Figura 9.12: Esempio di Point Region Quadtree dato un dataset

Il principio alla base della costruzione di un albero di questo genere è molto semplice. Ipotizziamo di avere pagine di una data dimensione. Dopo aver partizionato lo spazio tramite un nodo. Possiamo andare a controllare quanti punti sono presenti in ogni partizione, se questa partizione contiene un numero minore o uguale al massimo numero di elementi in una pagina allora non è necessario procedere ricorsivamente, altrimenti si procede con una sottopartizione. Questo approccio costruttivo è illustrato in Figura 9.12.

Una naturale generalizzazione dell'albero quaternario precedentemente illustrato si può trovare in alberi che per ogni dimensione dello spazio, lo dividono in due metà. Supponendo infatti che lo spazio sia di dimensione $d = 3$, otterremo un **albero ottale** (octree), dove ogni nodo avrà 8 figli, ognuno dei quali rappresenta una delle 8 regioni in cui lo spazio tridimensionale viene diviso. Il problema consiste nel fatto che il numero di figli di un nodo cresce in maniera esponenziale con il numero delle dimensioni dello spazio. Mostriamo un esempio di octree in Figura 9.13.

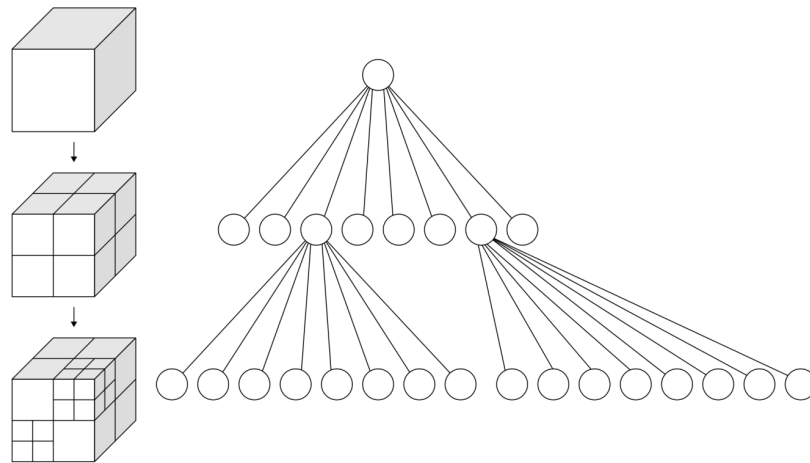


Figura 9.13: Esempio di costruzione di un point region octree

Il principale svantaggio legato a tecniche di questo genere è che spesso e volentieri ci troveremo ad avere **molto spazio inutilizzato**. Consideriamo l'esempio in Figura 9.12: possiamo notare che per separare in maniera efficiente i punti C e D si è reso necessario andare ad allocare due ulteriori pagine che però non contengono alcun punto. Questo problema viene ulteriormente amplificato nel caso di spazi con dimensioni maggiori di 2.

9.3.2. KDB - Tree

Per far fronte al problema della grande quantità di spazio inutilizzato andando ad applicare un partizionamento rigido come quello indotto da quadtree e octree, possiamo adottare una struttura più flessibile e bilanciata: il **KDB-Tree**. L'idea di base è la seguente:

- lo spazio viene suddiviso in maniera ricorsiva lungo assi alternati
- ogni **nodo interno** contiene un insieme di regioni rettangolari che suddividono lo spazio in partizioni non sovrapposte
- ogni **regione** è associata ad un puntatore che va o ad un altro nodo interno o ad una **point page** che contiene i record

Il numero di regioni in ogni nodo è limitato dalla **capacità della pagina**. Un esempio di utilizzo di questa struttura si può vedere in Figura 9.14.

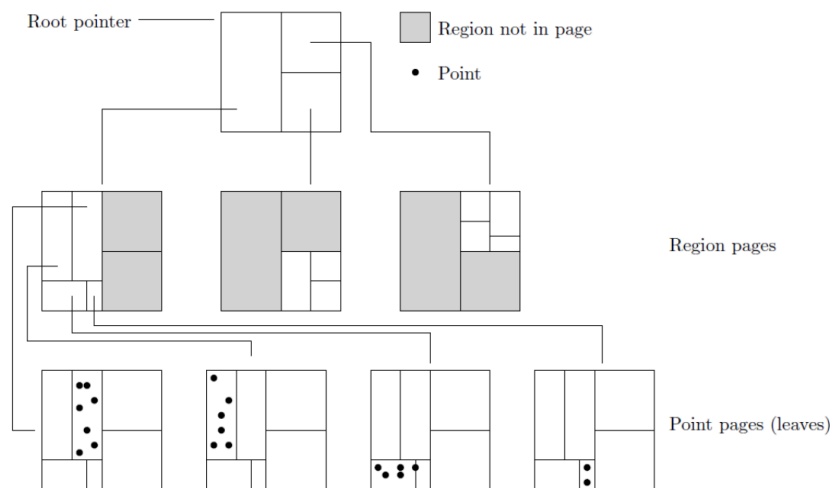


Figura 9.14: Esempio di KDB-Tree dato un dataset

Oltre alla possibilità di scegliere di partizionare lungo un solo asse alla volta, cosa che nei quadtree e octree non è possibile, si noti che è anche possibile scegliere una soglia di partizionamento che non sia necessariamente il punto medio dell'asse.

9.4. Alberi G - Partizionamento Non-Overlapping

Un'altra struttura dati molto utilizzata per la gestione di dati multi-dimensionali è data dai **G-Trees**. Si tratta di una struttura che combina il **partizionamento** dello spazio e dei **B+ Tree** in maniera originale: lo spazio è diviso in regioni non sovrapposte di dimensione variabile che vengono identificate tramite un codice. In seguito viene definito un ordinamento totale sui codici delle regioni, e vengono memorizzate in un B+ Tree.

9.4.1. Codifica delle Partizioni - Creazione del Partition Tree

Di seguito mostriamo come viene definito il codice di ogni regione nello spazio partizionato:

- la regione iniziale che contiene tutti i punti viene identificata tramite stringa vuota
- con il primo split lungo l'asse X , due partizioni sono prodotte ed identificate tramite i codici "0" e "1". Ipotizziamo che entrambi gli attributi prendano valori in $[0, 100]$. Tutti i punti che hanno $X < 50$ finiranno nella partizione sinistra (codice "0"), mentre tutti i punti con $X \geq 50$ finiranno nella partizione destra (codice "1").
- Quando una partizione contiene più punti di quanti ne possa contenere una pagina si procede con uno split sull'asse opposto (in questo caso Y) e nuovamente i punti vengono divisi in due partizioni, che vengono identificate tramite l'aggiunta di un ulteriore bit al codice della partizione padre.

In generale quando una partizione R con codice S viene divisa, le sotto-partizioni con valori minori della metà dell'intervallo considerato avranno codice $S0$, mentre le altre avranno codice $S1$. Un esempio di funzionamento di questo processo è illustrato in Figura 9.15.

L'albero costruito in Figura 9.15 è detto **partition tree**, e servirà ad andare a costruire la possibile partizione in cui un nodo potrebbe essere presente in fase di ricerca.

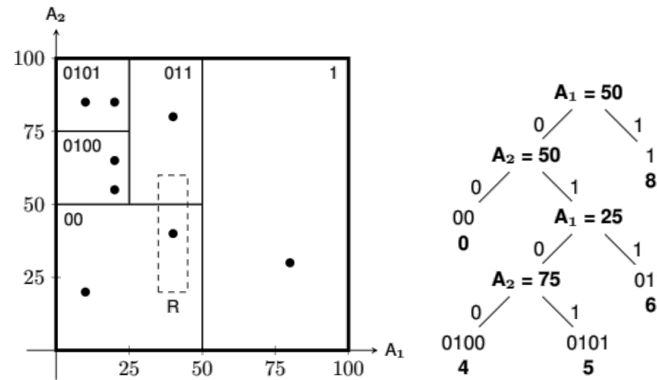


Figura 9.15: Esempio di G-Tree dato un dataset

Una volta stabiliti gli identificatori di ogni regione possiamo passare a costruire il B+ Tree che andrà a memorizzare i puntatori alle effettive regioni.

9.4.2. Creazione del G Tree

Una volta definito il **partition tree** possiamo procedere con la creazione del **G Tree**, che consiste semplicemente nel creare il B+ Tree associato alle regioni, basterà considerare gli encoding ottenuti con dello zero padding a destra e convertirli in decimale, nel caso dell'esempio in Figura 9.15 avremo i corrispettivi binari dei numeri 0, 4, 5, 6, 8. Andando ad effettuare un inserimento per ogni elemento (partendo da quello la cui rappresentazione binaria ha valore più alto in questo esempio) otterremmo l'albero binario rappresentato in Figura 9.16

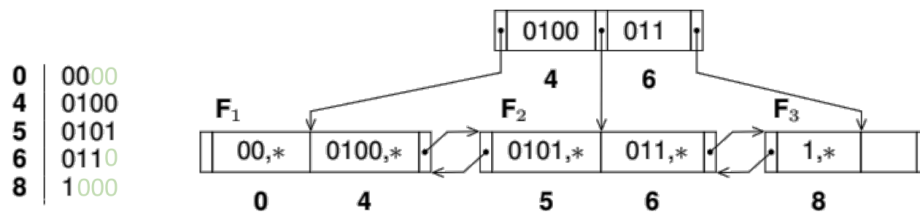


Figura 9.16: B+ Tree associato agli encoding delle partizioni identificate

Osservazione

Dal momento che le codifiche delle partizioni e il partition tree non sono parte integrante del G Tree, è importante che qualsiasi cambiamento nei dati al seguito di accessi in scrittura a dati nel G-Tree si riflettano anche su questi componenti.

9.4.3. Ricerca di un Punto

Supponiamo che M sia la massima lunghezza di un codice di partizione (in formato binario) all'interno del G-Tree. La ricerca di un punto di coordinate (x, y) procede nella maniera seguente:

- ricerca all'interno del **partition tree** per un codice S_P che contenga P se presente
- ricerca del **G-Tree** del codice di partizione S_P per verificare che P sia effettivamente nella partizione associata alle sue coordinate

Supponiamo per esempio di voler cercare $P = (30, 60)$: andando a scorrere il partition tree troveremmo che il codice di partizione ad esso associato è $S_P = 011$, andando a scorrere il G-Tree scopriamo che questo codice di partizione è quello relativo al valore 6, presente nel nodo foglia F_2 . Sarà a questo punto necessario controllare la pagina corrispondente per verificare la presenza del punto richiesto dalla query.

9.4.4. Ricerca di un Range Spaziale

Come sappiamo per la ricerca di punti in un range di valori, siamo interessati in tutti quei punti P_i di coordinate x_i, y_i tali che $x_1 \leq x_i \leq x_2$ e $y_1 \leq y_i \leq y_2$ che si trovano nella regione definita dalle query: $R = \{(x_1, y_1), (x_2, y_2)\}$. Per risolvere la query procediamo come segue:

- Cerchiamo la pagina F_h che contiene il punto identificato dalle coordinate (x_1, y_1)
- Cerchiamo la pagina F_k che contiene il punto identificato dalle coordinate (x_2, y_2)
- Per ogni nodo foglia tra F_h e F_k che sappiamo essere connessi tramite puntatori a livello delle foglie, vengono cercati gli elementi S tali che la regione a cui fa riferimento S sia sovrapposta o contenuta in R
- Se la regione è totalmente inclusa in R allora tutti i punti soddisfano la query, in caso contrario sarà necessario verificare punto per punto l'aderenza alla query

Osservazione

Si noti come, per permettere lo svolgimento di query di questo genere è necessario che venga prevista una funzione del tipo *RegionOf(S)* che dato in input il codice di una partizione, restituisca il vertice in basso a sinistra e quello in alto a destra della regione R_S .

Esempio: Ricerca di un range

Supponiamo di voler cercare tutti i punti nella regione

$$R = \{(35, 20), (45, 60)\}$$

facendo riferimento al grafico in Figura 9.15. Utilizzando la codifica delle partizioni tramite numeri interi e il partition tree, il vertice inferiore è nella partizione 0, mentre quello superiore nella partizione 6. Ciò significa che esaminando il G tree dovremo analizzare le partizioni 0, 4, 5, 6, ossia tutte quelle comprese tra la 0 e la 6.

Sappiamo che la partizione 0 corrisponde alla regione $R_0 = \{(0, 0), (50, 50)\}$. Questa partizione è sovrapposta ad R , dunque andrà recuperata la pagina associata e verificato che tutti i suoi punti ne facciano parte. Le partizioni 4 e 5 non sono sovrapposte, mentre la numero 6 lo è, portandoci dunque a dover esaminare ogni suo punto.

9.4.5. Inserimento di un Punto

Sia M la massima lunghezza di un partition code nel G-Tree. L'inserimento di un punto P con coordinate (x, y) si svolge nella maniera seguente:

- cerchiamo nel G-Tree la foglia F che dovrebbe contenere la partizione R_P che dovrebbe andare a contenere P . Sia S_P il codice di questa partizione.

- Se R_P non è completa, possiamo inserire direttamente il punto P , altrimenti R_P deve essere divisa in due sotto-partizioni R_{P_1}, R_{P_2} .
- I codici associati a queste partizioni sono rispettivamente $S_{P_1} = S_P0$ e $S_{P_2} = S_P1$. Nel caso in cui la lunghezza di questi codici sia maggiore di M allora $M = M + 1$ e le codifiche in intero di tutte le partizioni dovranno essere modificate di conseguenza.
- Tutti i punti di R_P vengono divisi nelle due sotto-partizioni. La partizione di destinazione è determinata tramite il primo passo dell'algoritmo di ricerca.
- L'elemento S_P con il relativo puntatore alla pagina di R_P viene sostituito con due nuovi elementi $(S_{P_1}, \text{ref}(R_{P_1}))$ e $(S_{P_2}, \text{ref}(R_{P_2}))$. Nel caso di overflow nel nodo foglia, si procede con la suddivisione del nodo come in un normale B+ Tree.

Facciamo sempre riferimento al solito esempio in Figura 9.15. Immaginiamo di volere aggiungere il punto $P_1 = (70, 65)$. In questo caso la sua regione di destinazione è quella identificata dal codice 8 con codice "1". Dal momento che la pagina ha ancora elementi liberi ($1 < 2$) possiamo procedere con l'inserimento diretto del punto.

Immaginiamo ora di volere inserire il punto $P_2 = (8, 65)$. La sua regione destinazione dovrebbe essere la numero 4 con codice "0100". La pagina associata però non ha sufficiente spazio. Viene dunque divisa in due sotto-partizioni causando una modifica delle codifiche e del B+ Tree ad esse associato. Figura 9.17 mostra il risultato finale dopo l'inserimento di entrambi i punti.

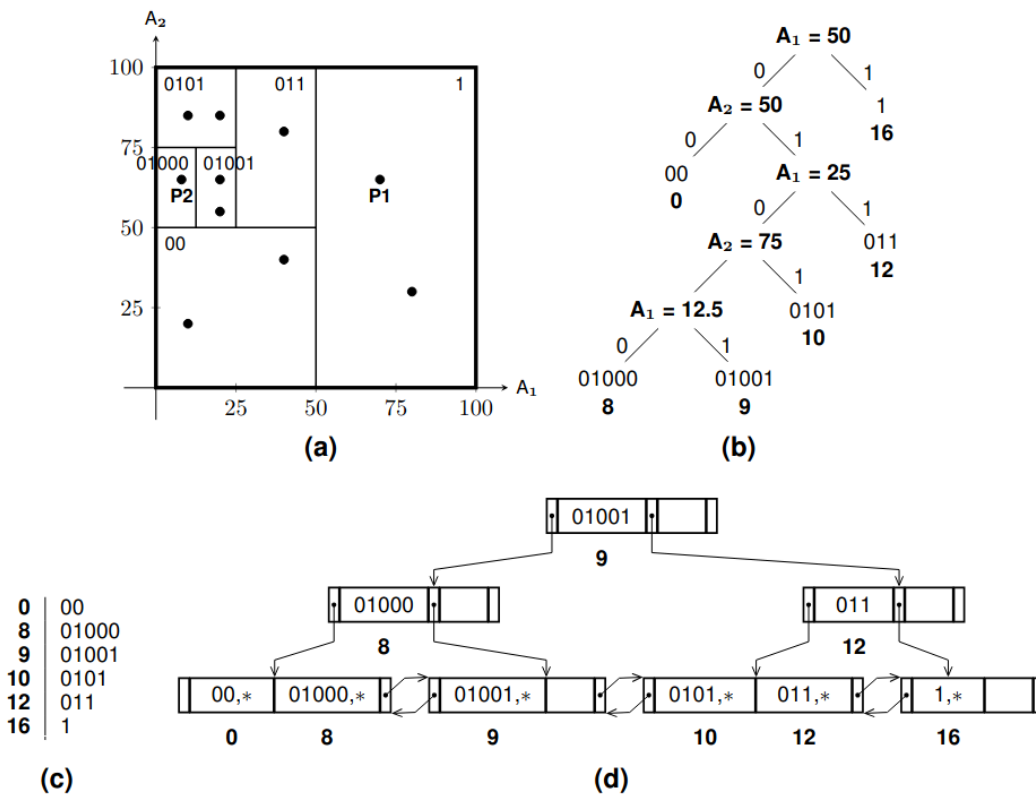


Figura 9.17: Esempio di G-Tree dopo l'inserimento di due punti P_1 e P_2

9.4.6. Cancellazione di un Punto

Di seguito mostriamo il procedimento di cancellazione di un punto $P = (x, y)$ dal G-Tree:

1. sia F la foglia associata alla partizione R_P che contiene il punto P e S_P il codice di questa partizione. Sia S' il codice della partizione R' ottenuta dal “padre” di R_P tramite uno split. Possiamo notare che R' ha codice esattamente uguale a S_P ad eccezione dell'ultimo bit.
2. eliminiamo il punto P dalla pagina associata ad R_P . Abbiamo ora due possibilità:
 - Se la “sorella” R' è già stata splittata, non è possibile applicare fusioni. Controlliamo che R_P non sia diventata vuota, in tal caso eliminiamo l'elemento S_P dalla foglia F . Se R_P è vuota andiamo a cancellare l'elemento S_P dalla foglia F .
 - Se la sorella R' non è stata divisa, allora dobbiamo controllare la somma del numero di elementi di R e R' . Se questa somma è maggiore della capacità di una pagina, allora l'operazione termina, altrimenti possiamo applicare una **fusione** delle due partizioni: S_P, S' vengono eliminate dall'albero e viene generata una nuova stringa S'' ottenuta cancellando l'ultimo bit da S_P .

Applicando un'operazione di cancellazione al G-Tree in Figura 9.17 per P_1, P_2 torniamo alla situazione mostrata in Figura 9.15 e Figura 9.16.

9.5. Alberi R^* - Partizionamento con Overlapping

Nell'ultima sezione di questo capitolo andiamo a illustrare una tecnica per gestire l'ultima opzione di partizionamento mostrata all'inizio del capitolo, ossia il **partizionamento che ammette overlapping** tra le regioni. La struttura dati utilizzata per questo scopo è quella degli **alberi R^*** (R^* -Trees).

9.5.1. Struttura degli Alberi R^*

Un albero R^* è una variante di un albero R . Si tratta di una struttura dinamica **perfettamente bilanciata** (similmente ai B+ Tree). Per semplicità consideriamo di nuovo il caso bidimensionale.

Dal momento che in questo tipo di organizzazione ha a che fare con regioni dello spazio, ogni elemento dell'albero andrà a contenere le informazioni relative ad un **minimum bounding rectangle**, ossia il rettangolo di dimensione minima che contiene i punti in una regione. Ogni rettangolo sarà identificabile tramite le coordinate del vertice in basso a sinistra e di quello in alto a destra.

Dal punto di vista pratico, i nodi più in alto dell'albero conterranno degli iper-rettangoli che al loro interno contengono regioni sempre più specifiche, fino ad arrivare ai nodi foglia che conterranno i minimum bounding rectangle dei singoli punti. Per rendere questa notazione più formale diremo che i **nodì foglia** saranno della forma R_i, O_i dove R_i rappresenta le coordinate che identificano il rettangolo, mentre O_i è un puntatore effettivo ai dati. I **nodì interni** invece saranno della forma (R_i, p_i) dove p_i è un riferimento alla radice di un sotto-albero, ed R_i contiene le informazioni per identificare il MBR che contiene tutti i rettangoli dei

nodi figli. Similmente ai B+ Tree, per mantenere il **bilanciamento** ogni nodo avrà un numero minimo e un numero massimo di elementi.

Per meglio comprendere come i vari minimum bounding bounding rectangle dei livelli superiori contengano quelli dei livelli inferiori possiamo fare riferimento a Figura 9.3, in riferimento alla quale mostriamo il corrispondente R*-Tree in Figura 9.18.

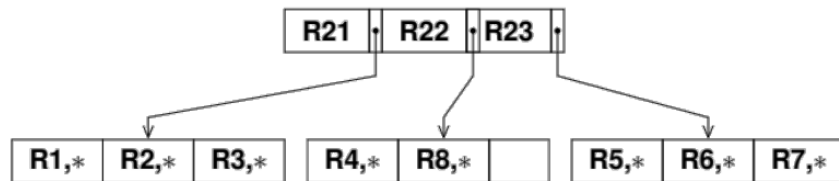


Figura 9.18: Esempio di R*-Tree dato un dataset

Dal momento che è necessario definire il numero minimo e massimo di elementi per ogni nodo, una pratica comune è che il minimo $m = 0.4 \cdot M$. Un R*-Tree soddisfa le proprietà seguenti:

- il nodo radice ha almeno due figli, a meno che non sia l'unico nodo dell'albero
- ogni nodo ha un numero di elementi compreso tra m e M a meno che non la radice
- tutti i nodi foglia sono allo stesso livello

Ci sono delle sostanziali differenze tra gli alberi R* e gli alberi B+, di seguito le illustriamo:

- gli elementi dei nodi di un albero R* **non sono ordinati** in alcun modo
- le regioni associate a diversi elementi di un nodo **possono sovrapporsi** in un B+ tree, questo invece è esattamente il requisito per un albero R*

9.5.2. Ricerca di Regioni sovrapposte

L'operazione principale che si può svolgere su un albero di questo tipo è la ricerca di tutte le regioni che sono sovrapposte ad una data regione specificata nella query R . La radice viene visitata in maniera da cercare elementi (R_i, p_i) tali che R_i sia sovrapposta a R . Per ogni elemento (R_i, p_i) che soddisfa questo requisito si procede ricorsivamente nel sotto-albero puntato da p_i . Quando raggiungiamo un nodo foglia, le regioni R_i nel risultato della query sono quelle che sono sovrapposte a R .

9.5.3. Inserimento nell'Albero

Sia S una nuova porzione di dati da voler inserire nell'albero. L'operazione è molto simile all'inserimento di una chiave all'interno di un B+ Tree. La differenza sostanziale consiste nel fatto che in un B+ Tree la posizione in cui aggiungere la nuova chiave è univocamente determinata dall'ordinamento.

In un albero R*, dal momento che regioni in diversi nodi interni allo stesso livello potrebbero sovrapporsi, la nuova regione S potrebbe sovrapporsi con più di questi nodi e potrebbe dunque essere inserita in più foglie con diversi possibili nodi interni come padri.

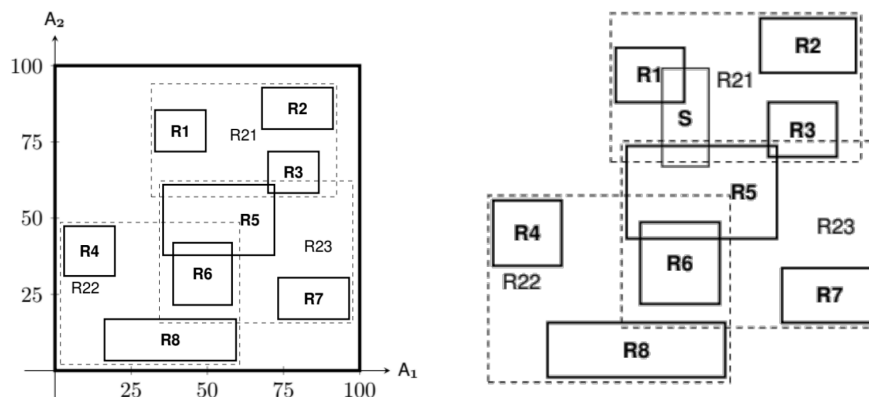
La scelta di quale regione considerare può essere effettuata andando a considerare il **grado di sovrapposizione** con S . Per esempio si potrebbe scegliere la regione che vedrebbe il minore cambiamento di area del bounding box per andare a contenere anche S . Dopo aver

scelto in quale nodo foglia inserire S , si procede con il suo inserimento, nel caso in cui non siano presenti overflow viene ricalcolata la regione e propagato il nuovo valore al nodo padre, altrimenti consideriamo due casi:

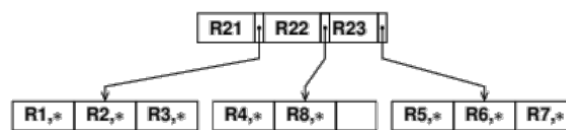
- **forced reinsert**: se abbiamo a che fare con il primo overflow da un nodo foglia, non viene diviso a metà, piuttosto p degli $M + 1$ ingressi vengono rimossi dal nodo e reinseriti nell'albero. A livello pratico un buon valore di p si aggira attorno al 30% del valore di M . Questo approccio potrebbe permettere di evitare di dover dividere il nodo, andando a ridurre la sovrapposizione tra le regioni.
- **split del nodo**: dopo il primo overflow, gli $M + 1$ elementi sono divisi tra due nodi e due nuovi elementi sono inseriti nel nodo padre, gli effetti vengono poi propagati in alto nell'albero. Nel momento in cui dovessimo avere più possibilità per dividere gli elementi del nodo scegliamo il criterio che minimizza il **perimetro totale** delle regioni ottenute.

Esempio: Inserimento di nuovi dati

Consideriamo il solito partizionamento di Figura 9.3 che riportiamo di seguito per comodità e assumiamo di voler aggiungere una nuova regione S rappresentata nell'immagine a fianco



Il processo di inserimento inizia con il nodo radice presentato nella figura seguente:



Le regioni candidate alla memorizzazione di S sono R_{23} e R_{21} . Dal momento che R_{21} richiede un aumento minore dell'area del bounding box per contenere S , scegliamo questa regione per l'inserimento. Seguendo il puntatore p_{21} nell'immagine precedente andremo a considerare il nodo foglia a sinistra; andando ad inserire il nuovo elemento otteniamo un **overflow**, dal momento che è il primo possiamo procedere con un **forced reinsert**. Supponiamo di provare a reinserire la data region R_1 . Ciò che avverrebbe in questo caso è che la regione R_1 verrebbe nuovamente inserita nella stessa sovra-regione R_{21} , andando a causare un **secondo overflow**.

A questo punto andrà a verificarsi una **suddivisione** della regione. Supponiamo di aver diviso la data region in $\{R_1, S\}$ e $\{R_2, R_3\}$. Siano rispettivamente R_{24} ed R_{25} i minimum bounding rectangle associati a queste due nuove regioni. Andremo ad aggiornare il nodo padre. A livello di nodo **radice** non ha alcun senso praticare reinserimento procediamo dunque con un nuovo **split** della radice ipotizzando che verrà divisa in due nuove regioni R_{26} e R_{27} che conterranno rispettivamente $\{R_{24}, R_{25}\}$ e $\{R_{22}, R_{23}\}$. Mostriamo il risultato finale dell'inserimento in Figura 9.19.

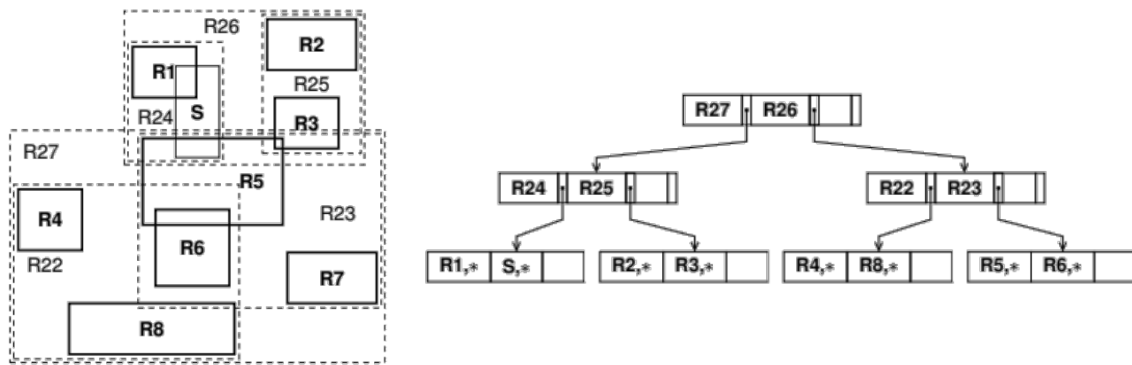
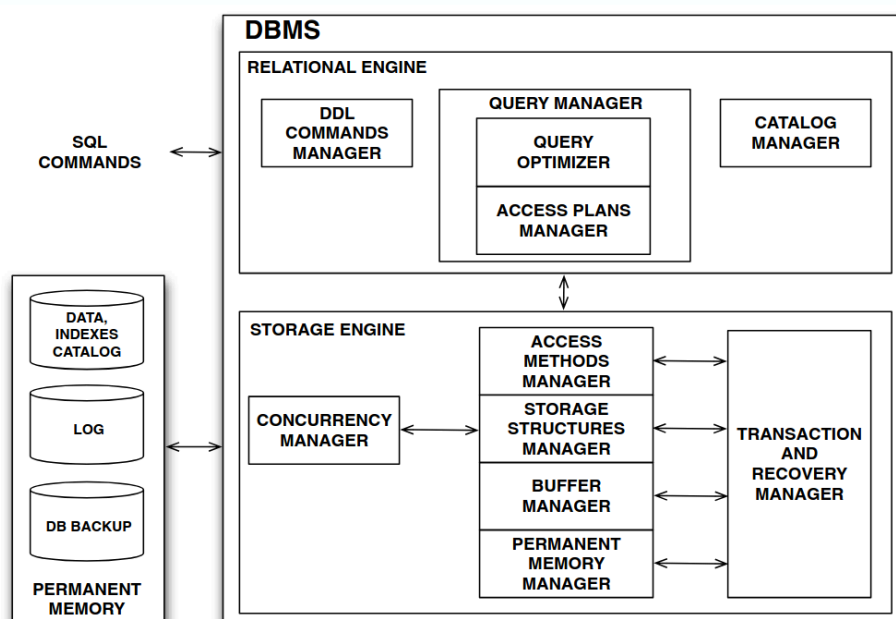


Figura 9.19: Esempio di inserimento di una nuova regione in un R*-Tree

10. Metodi di accesso e Operatori Fisici

Riprendiamo in considerazione il nostro classico schema che rappresenta un database management system (DBMS) già presentato in Figura 3.1.



Negli scorsi capitoli abbiamo visto le tre componenti che si pongono alla base dello **storage engine**, ossia il gestore della memoria secondaria, il gestore del buffer e il gestore delle strutture di memorizzazione. In questo capitolo ci concentriamo sui **metodi di accesso** ai dati, ossia l'interfaccia che lo storage engine offre al livello superiore del DBMS che si occupa della gestione **logica** dei dati e in quale modo gli strumenti logici che vengono utilizzati per manipolare i dati (ad esempio il linguaggio SQL) vengono tradotti in operazioni fisiche sullo storage.

10.1. Gestione dei Metodi di Accesso

Per meglio comprendere l'interfaccia che lo storage engine offre, andiamo a considerare il sistema relazionale JRS che memorizza le relazioni tramite **heap files** e utilizza **indici** basati su *B+ Tree*. Gli operatori esposti dallo storage engine sono procedurali e possono essere raggruppati nelle categorie che elenchiamo di seguito:

- operatori per la creazione e gestione della **base di dati**
- operatori per la creazione e gestione di **heap file** e **indici**
- operatori per la gestione delle **transazioni**

10.1.1. Gestione della Base di Dati

Di seguito andiamo a illustrare i vari operatori disponibili per la gestione della base di dati.

Creazione di un DB

Per la creazione di un database abbiamo a disposizione un operatore che dato un path, un nome e un identificativo di transazione con la quale creare la base di dati, si occupa di creare la struttura necessaria a contenere l'intera base di dati. La funzione preposta alla creazione è `createDB: path x dbName x TransactionId -> null`.

Creazione di un Heap File

Una volta creato il database, è possibile creare al suo interno gli **heap file** che andranno a contenere le relazioni della base di dati. La funzione prende un input il nome della base di dati e uno specifico path: `createHeap: path + dbName x heapName + TransID -> null`.

Creazione di un Indice

Data una relazione, che viene memorizzata per mezzo di un heap file, possiamo andare a creare **indici** per garantire ricerca più efficiente su degli attributi di questa relazione. La funzione è definita come segue:

```
createIndex: path x dbName x indexName x heapFile x attribute  
            x ordering x unique x _transactionId -> null
```

dove `unique` serve a stabilire se l'indice sta venendo creato su una chiave, e l'`ordering` stabilisce se l'indice deve essere creato in ordine crescente o decrescente.

Tutti gli operatori di creazione appena visti hanno un corrispettivo per la cancellazione delle strutture: `dropDB`, `dropHeapFile` e `dropIndex`.

10.1.2. Operazioni sugli Heap File

Come già anticipato ogni relazione di una base di dati viene memorizzata per mezzo di **heap file**. Il DBMS deve esporre una serie di operatori per permettere inserimenti, cancellazioni, ricerche, aggiornamenti e altre operazioni di gestione sugli heap file. Di seguito vediamo le funzionalità principali che il sistema JRS mette a disposizione:

- `HF_InsertRecord : Record → RID`, permette di inserire un nuovo record all'interno di uno specifico heap file e restituisce il "RID" (Record Identifier) del record inserito
- `HF_DeleteRecord : RID → null`, permette di cancellare un record specificato dal suo "RID"
- `HF_GetRecord : RID → Record`, permette di recuperare un record dato il suo "RID"
- `HF_UpdateRecord : RID x FieldNum x NewField → null`, permette di aggiornare un campo specifico di un record individuato dal suo "RID"

Potremmo avere bisogno di alcune operazioni per recuperare statistiche che riguardano il nostro heap file, come ad esempio il **numero di pagine** che esso utilizza e il **numero di record** memorizzati. Per questo motivo sono stati definiti gli operatori seguenti:

- `HF_GetNpage : null → integer`, restituisce il numero di pagine occupate
- `HF_GetNrec : null → integer`, restituisce il numero di record memorizzati

Il motivo per cui questi operatori sono necessari è che sono particolarmente utili per calcolare l'occupazione media di ogni pagina che ci consentirà di calcolare il valore atteso del costo di accesso ad un record.

10.1.3. Operazioni sugli Indici

Come sappiamo un indice consiste in un insieme di record di tipo **index entry**, organizzato (almeno in JRS) secondo una struttura B+ Tree. Un'index entry è costituita dagli attributi **value** e **RID**, dove il primo è la *chiave di ricerca* all'interno dell'indice e il secondo il riferimento al

record all'interno dell'heap file. Le operazioni ammesse sugli indici consentono di inserire e cancellare elementi, o di ottenere informazioni riguardo al B+ Tree stesso:

- $\text{Index_open} : \text{Path} \times \text{dbName} \times \text{IndexName} \times \text{TransId} \rightarrow \text{Index}$, apre un indice e restituisce un riferimento ad esso
- $\text{Index_close} : \text{Index} \rightarrow \text{null}$, chiude un indice precedentemente aperto
- $\text{I_isKey} : \text{Value} \rightarrow \text{boolean}$, consente di verificare se una specifica chiave è presente all'interno dell'indice
- $\text{I_InsertEntry} : \text{Value} \times \text{RID} \rightarrow \text{null}$, consente di inserire una nuova index entry all'interno dell'indice
- $\text{I_DeleteEntry} : \text{Value} \times \text{RID} \rightarrow \text{null}$, consente di cancellare una index entry dall'indice
- $\text{I_getNKey} : \text{null} \rightarrow \text{integer}$, restituisce il numero di chiavi presenti nell'indice
- $\text{I_getNleaf} : \text{null} \rightarrow \text{integer}$, restituisce il numero di foglie presenti nell'indice
- $\text{I_getMin} : \text{null} \rightarrow \text{Value}$, restituisce il valore minimo presente nell'indice
- $\text{I_getMax} : \text{null} \rightarrow \text{Value}$, restituisce il valore massimo presente nell'indice

10.1.4. Operatori dei Metodi di Accesso

Dopo aver osservato come è possibile interagire con i componenti di base di una base di dati, possiamo ora vedere come sia possibile utilizzarli in un contesto più ampio, ossia per rispondere a delle query.

Il gestore dei metodi di accesso fornisce gli operatori per trasferire dati tra la memoria permanente e la memoria principale in maniera tale da rispondere a delle query effettuate su una base di dati. Gli operatori che fornisce il gestore dei metodi di accesso sono utilizzati per implementare le gli operatori utilizzati a livello logico dai «query plan» che vengono generati dal **query optimizer**.

I record di un heap file o di un indice sono acceduti tramite operazioni di **scansione**. Per quanto riguarda un heap file scan, avremo un operatore che semplicemente legge un record dopo l'altro, mentre un index scan operator consiste di ottenere il RID di un record dato un valore chiave o un intervallo di valori. A livello pratico questi operatori per la scansione sono implementati sotto forma di **iteratori**, anche detti **cursori**, ossia oggetti con funzionalità che permettono di ottenere i risultati un record per volta. Questi iteratori vengono tipicamente creati per mezzo di una funzione *open* e mettono a disposizione le seguenti funzionalità:

- *isDone*: per capire se esistono ancora record da leggere per un certo risultato
- *getCurrent*: per ottenere il record attualmente puntato dall'iteratore
- *next*: per spostare l'iteratore al record successivo
- *close*: per chiudere l'iteratore e liberare le risorse ad esso associate

Una volta che un cursore è stato creato e aperto, possiamo avviare una **scansione** utilizzando uno schema simile allo pseudocodice che mostriamo in seguito.

```

1 while !cursor.isDone(){
2     value = cursor.getCurrent();
3
4     // operations with the value
5
6     cursor.next();
7 }

```

 Java

Heap File Scan

Sappiamo che nel momento in cui vogliamo utilizzare uno scan operator abbiamo bisogno sempre di utilizzare la sua funzionalità di *apertura*. Per lavorare su un heap file abbiamo due alternative a disposizione:

- HFS_Open : Heapfile $\rightarrow$ ScanHeapFile, apre uno scan su un heap file partendo dal suo primo record
- HFS_Open : Heapfile x RID $\rightarrow$ ScanHeapFile, apre uno scan su un heap file partendo dal record specificato

Dopo aver inizializzato il cursore tramite apertura, possiamo utilizzare varie funzionalità:

- HFS_IsDone : null $\rightarrow$ boolean, verifica se la scansione ha raggiunto la fine
- HFS_next : null $\rightarrow$ null, sposta la scansione al record successivo
- HFS_getCurrent : null $\rightarrow$ RID, restituisce il record attualmente puntato dal cursore
- HFS_Reset : null $\rightarrow$ null, riporta il cursore al primo record dell'heap file
- HFS_Close : null $\rightarrow$ null, chiude lo scan e libera le risorse ad esso associate

Index Scan

Nel caso di uno scan su indice, come al solito abbiamo necessità di “*aprire* lo scan operator” accedendo ad un iteratore. Per fare questo abbiamo a disposizione la seguente funzionalità:

- IS_Open : Index x FirstValue x LastValue $\rightarrow$ ScanIndex, apre uno scan su un indice per un intervallo di valori compreso tra FirstValue e LastValue.

Una volta aperto abbiamo a disposizione tutte le funzionalità già specificate anche per l'heap file scan, ossia IS_isDone, IS_next, IS_getCurrent, IS_Reset e IS_Close.

10.1.5. Esecuzione delle Query

Dopo aver visto tutti gli operatori messi a disposizione dal gestore dei metodi di accesso, andiamo a vedere alcuni esempi di programmi che utilizzano i metodi di accesso forniti dallo storage engine per eseguire delle semplici query SQL.

Mostreremo in maniera sommaria come una query SQL viene tradotta in una **programma procedurale** che utilizza gli operatori di accesso per ottenere i risultati desiderati. Per semplicità possiamo assumere che questo programma sia generale dal modulo **query optimizer**.

Supponiamo di voler considerare una relazione **Student** con gli attributi **Name**, **StudentNumber**, **Address** e **City**. Ipotizziamo di voler eseguire una query del tipo:

```
1 SELECT Name, FROM Students WHERE City = 'Pisa';
```

SQL

Assumendo che la relazione `Students` sia memorizzata in un heap file con lo stesso nome, la struttura di un possibile programma per eseguire questa query potrebbe essere:

```
1 Heapfile students = HF_Open(path, dbName, "students", transID);
2 ScanHeapFile iteratorHF = HFS_Open(students);
3
4 while (!iteratorHF.isDone()) {
5     RID rid = iteratorHF.getCurrent();
6     Record record = Students.HF_GetRecord(rid);
7     if (record.getField(4).("Pisa")) {
8         System.out.println(record.getField(1));
9     }
10    iteratorHF.HFS_next();
11 }
12
13 iteratorHF.Close();
14 students.Close();
```

Java

Supponiamo ora di avere a disposizione un **indice** `IdxCity` sull'attributo `City` della relazione. Possiamo ottenere un programma più efficiente per ottenere lo stesso risultato utilizzando uno scan su indice:

```
1 Heapfile students = HF_Open(path, dbName, "students", transID);
2 Index idxCity = I_open(path, dbName, "IdxCity", transID);
3
4 // apertura dello scan sull'indice per il valore 'Pisa'
5 ScanIndex iteratorIdx = IS_Open(idxCity, "Pisa", "Pisa");
6
7
8 while (!iteratorHF.isDone()) {
9     RID rid = iteratorIdx.IS_getCurrent().getRid();
10    Record record = Students.HF_GetRecord(rid);
11    System.out.println(record.getField(1));
12    iteratorIdx.IS_next();
13 }
14
15 iteratorIdx.IS_Close();
16 idxCity.I_Close()
17 students.HF_Close();
```

Java

10.1.6. Da un Operatore Logico a un Piano di Accesso Fisico

In questa sezione andiamo mostrare ad alto livello come, data una query, questa possa essere interpretata secondo un **albero logico**. Tuttavia questa rappresentazione non è così comoda per essere eseguita direttamente su una macchina. Per questo motivo mostreremo tutte le trasformazioni che portano da un albero logico ad un **piano di accesso fisico** ai dati.

Supponiamo di dover svolgere la seguente query SQL:

```
1 SELECT PkEmp, EName
2 FROM Employee JOIN Department
3 ON Employee.FKDept = Department.PkDept
4 WHERE Department.Location = 'Pisa' AND Employee.Salary > 2000;
```

Una prima traduzione della query in un albero logico si può vedere in Figura 10.1. Questa traduzione è piuttosto diretta e segue passo passo la struttura della query SQL, con il **join** tra le due relazioni che avvengono prima della selezione sui dati.

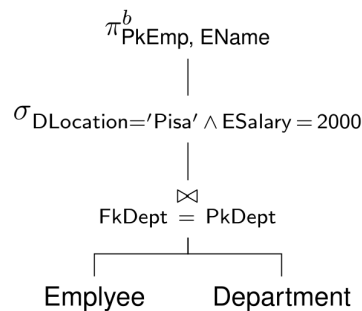


Figura 10.1: Albero logico della query in Algebra Relazionale

Volendo, possiamo applicare delle ottimizzazioni alla nostra query per fare in modo che il **join** avvenga dopo le selezioni che vengono applicate in maniera distinta su ognuna delle relazioni. In questo modo possiamo ridurre il numero di tuple che devono essere considerate durante il join stesso. Dopo aver ottenuto questa ottimizzazione è possibile andare ad ottenere un nuovo albero logico che sarà poi tradotto in un piano di accesso fisico utilizzando gli operatori messi a disposizione dallo storage engine.

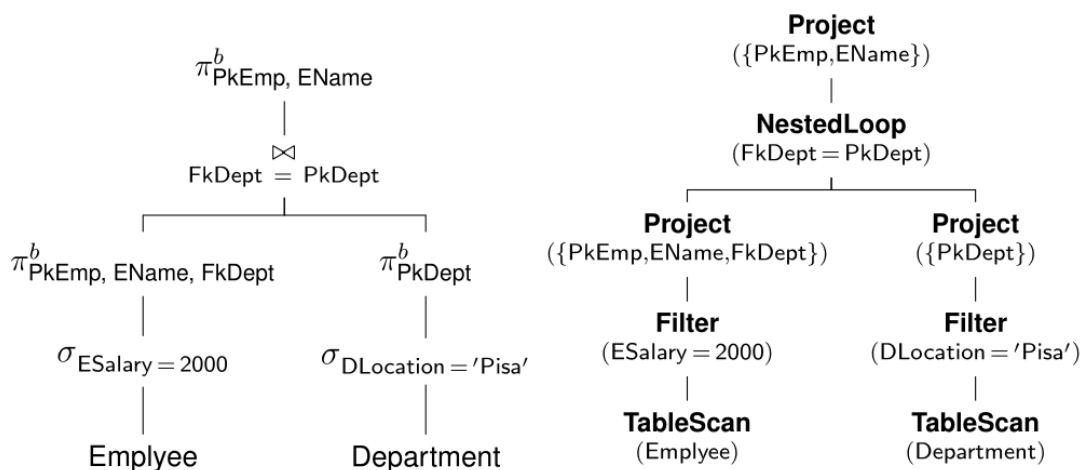


Figura 10.2: Piano di accesso fisico della query dopo ottimizzazione

10.2. Operatori Fisici

Nel momento in cui ci troviamo a dover svolgere una query, il gestore dei **piani di accesso** si occupa di creare un piano di accesso ai dati per lo **storage engine**, richiedendo che il piano di accesso venga eseguito dal gestore dei metodi di accesso.

In sostanza è necessario mappare gli operatori logici che possiamo vedere come la nostra algebra relazionale in operatori fisici che utilizzino le funzionalità messe a disposizione dallo storage engine. Per semplificare la trattazione in questo capitolo assumeremo di utilizzare un DBMS relazionale in cui le relazioni sono memorizzate in **heap file** o in **B+ Tree** utilizzati come organizzazioni primarie. Un'ulteriore assunzione che faremo è che avremo a disposizione vari **indici**: unici, non unici, clusterizzati e non clusterizzati.

10.2.1. Physical Query Plan

Un piano di accesso fisico è costituito da un insieme di operatori fisici che vengono collegati tra loro in una struttura ad albero. Ogni operatore è in realtà un **iteratore** che:

- ritorna una collezione di oggetti (come fanno gli operatori algebrici)
- hanno un certo tipo
- possono permettere accesso ai record di una collezione secondo un ordine

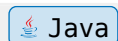
Per ognuno degli operatori fisici considerati andremo a studiare il **costo** di esecuzione C e la **cardinalità** dei **risultati** E_{rec} . Come già anticipato, possiamo andare a comporre un piano di accesso ai dati combinando vari operatori, in maniera tale che l'output di ogni operatore sia l'input per l'operatore successivo.

Una volta che lo storage engine riceve un piano di accesso, questo viene eseguito seguendo la logica definita dagli operatori fisici che lo compongono e restituendo i risultati al livello superiore del DBMS. Come abbiamo già visto nella sezione precedente, ogni iteratore può essere visto come un oggetto descritto da uno stato e che espone varie funzioni:

- *open*: per inizializzare l'iteratore
- *isDone*: per verificare se ci sono ancora record da leggere
- *next*: per spostarsi al record successivo
- *reset*: per riportare l'iteratore al primo record
- *close*: per chiudere l'iteratore e liberare le risorse ad esso associate

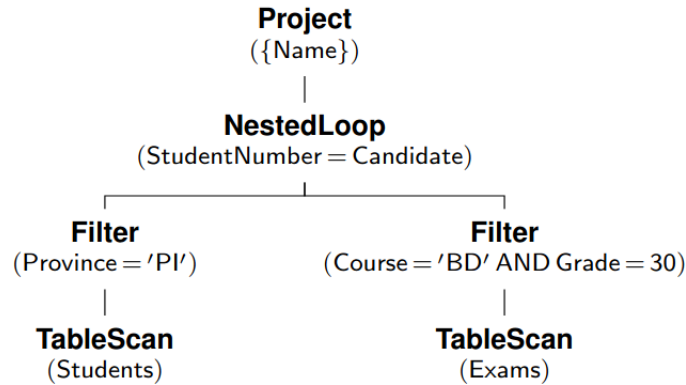
Data una query rappresentata tramite un piano di accesso fisico, lo storage engine segue il seguente schema per andare a eseguire il piano richiesto.

```
1 plan.open();
2 while (!plan.isDone()) {
3     System.out.println(plan.next());
4 }
5 plan.close();
```



Esempio: Svolgimento di un piano di esecuzione

Si consideri per esempio il piano di esecuzione che viene mostrato nell'immagine seguente:



Di seguito mostriamo i vari passaggi svolti dallo storage engine per svolgerlo:

1. L'operatore alla radice del piano è un nodo di **proiezione** che richiede un record al suo operando (il nodo figlio), ossia un **nested loop**
2. Il nested loop, che serve ad implementare la funzionalità di join, richiede due operandi:
 - l'operando a sinistra è un'operazione di **filtering** che a sua volta richiede un record all'operatore **tableScan** fintanto che questo non restituisce un operatore che soddisfi la condizione specificata nel filtro
 - l'operando a destra è un'altra operazione di filtraggio, che richiede record all'operazione **tableScan** sulla tabella **Exams** fintanto che non viene trovato un record che soddisfi la condizione del filtro

Possiamo notare il fatto che, utilizzando operatori di filtraggio a monte dell'operazione di join tra le due relazioni, riusciamo a ridurre significativamente il numero di record sui quali operare un confronto di uguaglianza tra gli attributi **StudentNumber** e **Candidate**.

10.2.2. Algebra Relazionale Estesa

Andiamo a definire all'interno di questa sezione un modello di algebra relazionale esteso che ci permetta di andare in seguito a definire gli operatori fisici che opereranno sui nostri dati. Il motivo per cui andiamo ad estendere l'algebra relazionale classica è che, quest'ultima suppone di lavorare con **insiemi**, che non ammettono elementi duplicati. Per questo motivo andremo ad *estendere* la nostra algebra in maniera tale che possa lavorare su **multiset** o **bag** di dati, in maniera tale da poter replicare il modello SQL. Di seguito illustriamo gli operatori:

- $\pi_X^b(O)$ **Proiezione con duplicati**: dati un multiset di record O e un insieme di attributi X restituisce un multiset di record contenente solo gli attributi in X .
- $\delta(O)$ **Eliminazione dei duplicati**: dato un multiset di record O restituisce un insieme di record eliminando i duplicati.
- $\tau_X(O)$ **Ordinamento dei duplicati**: dato un multiset O e un insieme di attributi X restituisce una lista di record ordinata secondo gli attributi in X . Corrisponde all'operatore SQL **ORDER BY**.

- **Unione, Intersezione e Differenza** di multiset di cui mostriamo in seguito il funzionamento in questo caso:

- $\{1, 1, 2, 3\} \cup^b \{2, 2, 3, 4\} = \{1, 1, 2, 2, 3, 2, 2, 3, 4\}$
- $\{1, 1, 2, 3\} \cap^b \{2, 2, 3, 4\} = \{2, 3\}$
- $\{1, 1, 2, 3\} -^b \{1, 2, 3, 4\} = \{1\}$

Una volta definite le componenti principali del nostro modello di algebra relazionale possiamo andare a mostrare come questi operatori vengono implementati e i vari costi legati alla loro esecuzione. Ricordiamo sempre che il nostro modello di costo è dato dal numero di pagine accedute in memoria secondaria durante l'esecuzione di un operatore. Nel caso in cui utilizziamo un indice, avremo anche un costo legato al numero di accessi alle pagine dell'indice stesso.

10.2.3. Operatori Fisici per le relazioni

È possibile leggere i record di una relazione tramite diverse funzionalità, ognuna di questa si può vedere come una funziona che ha come unico argomento il nome della relazione R da leggere. Questi nodi sono tipicamente le **foglie** di un piano di esecuzione.

In generale la cardinalità del risultato è un valore che non dipende da altro se non dalla numero di record della relazione stessa, ossia $E_{\text{rec}}(R) = N_{\text{rec}}(R)$.

Table Scan

L'operatore **TableScan**(R) ritorna tutti i record della relazione R nell'ordine in cui questi sono memorizzati all'interno dell'heap file. Il costo di questa operazione è dato dal numero di pagine occupate dalla relazione R :

$$C_{\text{TableScan}(R)} = N_{\text{pag}}(R)$$

SortScan

L'operatore **SortScan**($R, \{A_i\}$) ritorna tutti i record della relazione R secondo un ordine crescente stabilito sui valori dell'attributo A_i . L'ordinamento avviene tramite l'algoritmo di **external merge sort** già descritto in precedenza. In generale il costo di questo operatore dipende dal numero di pagine occupate dalla relazione, dal numero di pagine B nel buffer e dall'implementazione scelta per il merge sort.

Per quanto riguarda il costo di esecuzione, possiamo ipotizzare che $N_{\text{page}}(R) < B^2$ e possiamo ricordare che il costo di external merge sort è di $4 \cdot N_{\text{page}}(R)$, è però vero che la versione originale di external merge sort richiede di scrivere su disco il risultato dei merge incrementali, ciò non è richiesto qui dal momento che abbiamo a che fare con **iteratori**, dunque possiamo evitare di contare le scritture su file ordinato, la formula in Equazione (7.3) diventa dunque

$$2 \cdot N_{\text{page}}(R) \cdot (\lceil \log_Z S \rceil) \implies C_{\text{SortScan}(R, A_i)} = 3 \cdot N_{\text{page}}(R)$$

Index Scan

L'operatore **IndexScan**(A, Idx) ritorna i record della relazione R che sono memorizzati nell'indice Idx ordinati secondo l'ordine stabilito sull'attributo A_i . Il costo è legato al tipo di indice e al tipo di attributo su cui questo è definito. Lo vediamo in dettaglio di seguito:

$$C = \begin{cases} N_{\text{leaf}}(Idx) + N_{\text{pag}}(R) & \text{if } Idx \text{ is clustered} \\ N_{\text{leaf}(idx)} + N_{\text{rec}}(R) & \text{if } Idx \text{ is on a key, otherwise:} \\ N_{\text{leaf}(Idx)} + \left\lceil N_{\text{key}}(Idx) \cdot \Phi\left(\left\lceil \frac{N_{\text{rec}}(R)}{N_{\text{key}}(Idx)} \right\rceil, N_{\text{pag}}(R)\right) \right\rceil & \end{cases}$$

Applicando questo operatore, la prima eventualità è quella migliore, in cui l'indice riflette l'ordinamento della memoria. Nel secondo caso, invece l'indice non è più clusterizzato e non possiamo assumere che dopo aver acceduto ad una pagina per un RID, dovremo accedere ad un'altra pagina per il RID successivo. Il terzo caso è invece quello peggiore, l'unica cosa che possiamo fare è stabilire un limite superiore tramite la formula di Cardenas.

Index Sequential Scan

L'operatore **IndexSequentialScan**(A, Idx) ritorna tutti i record della relazione R che sono memorizzati con l'organizzazione primaria basata su B+ Tree ordinati in base all'attributi chiave della relazione. Il costo in questo caso è dato semplicemente da:

$$C_{\text{IndexSequentialScan}(A, Idx)} = N_{\text{leaf}(Idx)}$$

Notiamo che questa tecnica si differenzia rispetto alla precedente proprio per il fatto che usiamo un B+ Tree come organizzazione primaria della nostra relazione, le foglie del B+ Tree contengono esattamente i dati della relazione, mentre nel caso di un indice secondario è poi necessario andare a recuperare i record dall'heap file partendo dal RID memorizzato nell'indice.

10.2.4. Operatori di Proiezione e Deduplicazione

Andiamo ora ad analizzare i vari operatori fisici che ci permettono di effettuare operazioni di proiezione e deduplicazione sui record delle nostre relazioni.

Proiezione

L'operatore **Project**($O, \{A_i\}$) prende in input un multiset di record O e un insieme di attributi $\{A_i\}$ e restituisce un multiset di record contenente solo gli attributi in $\{A_i\}$. Il numero di record in output da questo operatore è lo stesso del numero di record in input, e analogamente, il costo in termini di numero di accessi a pagina è dato dal costo dell'origine O .

$$C = C(O) \quad E_{\text{rec}} = E_{\text{rec}}(O)$$

Index Only Scan

L'operatore **IndexOnlyScan**($R, Idx, \{A_i\}$) è in realtà un operatore per la scansione di una relazione, tuttavia ci consente di effettuare una **proiezione** in maniera intrinseca andando a ritornare record soltanto composti dagli attributi sui quali è costruito l'indice. Il costo di questo operatore è dato dal numero di foglie dell'indice.

Deduplicazione

L'operatore **Distinct**(O) prende in input un multiset di record O e restituisce un insieme di record eliminando i duplicati. Il costo è lo stesso del costo dell'origine O .

Chiaramente questo costo è così basso solamente nel caso in cui i dati iniziali siano già ordinati, in caso contrario sarà necessario effettuare un'operazione di ordinamento preliminare sui dati, il cui costo è quello di external merge sort.

Utilizzo di Hashing

Un'alternativa all'utilizzo dell'ordinamento per effettuare deduplicazione dei record è quello di utilizzare **hashing**. L'idea è quella che, utilizzando una qualsiasi funzione di hash, tutti i record duplicati avranno lo stesso hash code.

Possiamo dunque utilizzare questa funzione di hashing per dividere i record in **bucket** della dimensione della memoria o del buffer, in maniera da poter caricare in maniera atomica ogni bucket, procedendo poi ad eliminare i duplicati all'interno dei bucket già presenti in memoria.

Il costo di questo operatore sarà dunque dato da:

$$C = C(O) + 2 \cdot N_{\text{page}(O)}$$

Il motivo per cui moltiplichiamo il numero di pagine di O per due è che, una volta avremo bisogno di accedere alle pagine per creare i bucket, la seconda volta per caricare questi in memoria in modo tale da fare deduplicazione.

10.2.5. Operatori di Selezione

In questa sezione andiamo a mostrare alcune possibilità per effettuare operazioni di selezione sui record delle nostre relazioni.

Filter

La prima opzione è quella di applicare un banale operatore di **filtering**: **Filter**(O, ψ) che effettua selezione senza indici dei record provenienti dalla sorgente O . Il costo in questo caso è lo stesso della sorgente: $C = C(O)$.

Il grande vantaggio di questo strumento è che può essere sempre applicato, per qualsiasi condizione e su qualsiasi tipo di dato. Lo svantaggio principale è che non dà alcun vantaggio nel caso in cui abbiamo a disposizione strutture dati ausiliarie.

Index Filter

Vediamo ora come sia possibile migliorare sfruttare la presenza di un indice per la gestione delle selezioni. Questa operazione consiste nel trovare i record di R che soddisfano la condizione ψ utilizzando un indice I che sia definito sugli attributi coinvolti in ψ .

L'operatore **IndexFilter**(R, I, ψ) opera in due fasi: utilizza l'indice per trovare l'insieme ordinato dei RID che soddisfano ψ e poi li utilizza per accedere a questi record. L'operatore ha un costo che è dato dalla somma del costo di accesso all'indice e del costo di accesso ai record stessi: $C = C_I + C_D$.

Index Sequential Filter

L'operatore **IndexSequentialFilter**(R, I, ψ) ritorna i record di R che soddisfano ψ in maniera ordinata. In questo caso la condizione deve essere riferita unicamente ad attributi che sono parte della chiave di costruzione dell'indice. L'operatore ha costo $C = \lceil s_f(\psi) \cdot N_{\text{rec}}(R) \rceil$

È anche possibile utilizzare una variante di questo operatore per effettuare anche una **proiezione**, ossia **IndexOnlyFilter**($R, I, \{A\}, \psi$) che ritorna i record di R proiettati sugli attributi $\{A\}$ che soddisfano ψ . Anche in questo caso il costo è dato da $C = \lceil s_f(\psi) \cdot N_{\text{rec}}(R) \rceil$.

Filtri con Predicati Complessi

Spesso può capitare di dovere svolgere operazioni di selezione che coinvolgono più attributi. In questi casi, avremo bisogno di utilizzare più indici, uno per ogni attributo coinvolto nella selezione. Possiamo vedere questa selezione come divisa in due parti. La prima parte è quella della ricerca su indice, che sia nel caso in cui le condizioni sono in *and* o in *or* ha costo:

$$C_I = \left\lceil \sum_{k=1}^n C_I^k \right\rceil$$

dove k varia tra i n indici utilizzati per la selezione. La seconda parte è quella dell'accesso ai dati, che dipende dal tipo di condizione, possiamo in ogni caso stimare il numero di record selezionati dalla condizione ψ tramite $E_{\text{rec}} = \lceil s_f(\psi) \cdot N_{\text{rec}}(R) \rceil$ e a questo punto utilizzare la formula di Cardenas per stimare il costo di accesso ai dati:

$$C_D = \lceil \psi(E_{\text{rec}}) \cdot N_{\text{pag}}(R) \rceil$$

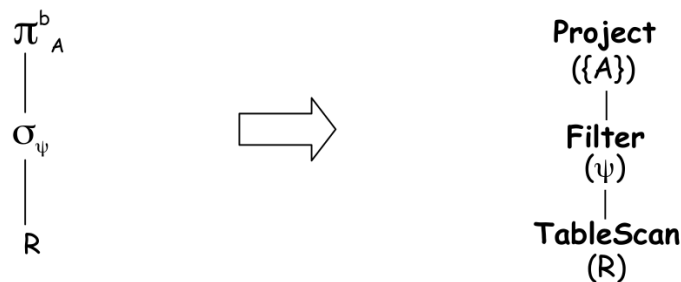
Esempio di Piano di Accesso con Filtraggio

Supponiamo di avere la seguente query SQL:

```
1 SELECT * A
2 FROM R
3 WHERE A BETWEEN 50 AND 100;
```

SQL

Possiamo vedere questa query risolta tramite l'albero logico che viene trasformato in un piano di accesso fisico come mostrato nell'immagine che segue:



In questo caso, il costo dell'intero accesso ai dati sarà dato semplicemente dal numero di pagine necessarie per effettuare la tableScan. Gli operatori successivi, essendo iteratori, non fanno altro che analizzare uno ad uno i record già forniti dall'operazione di scansione.

Supponiamo però di avere a disposizione un indice definito sull'attributo **A** e di modificare leggermente la nostra query per fare in modo da non dover proiettare sul solo attributo **A**. In questo caso il possiamo sfruttare l'operatore **IndexFilter** per ottenere un piano di accesso fisico più efficiente.

10.2.6. Operatori Fisici per il Raggruppamento

Andiamo ora a mostrare come è possibile tradurre operazioni di raggruppamento in operatori fisici che possano essere eseguiti dallo storage engine. In algebra relazionale questo operatore viene denotato tramite $\{A_i\} \gamma \{f_i\}$.

Operatore GroupBy

L'operatore **GroupBy**($O, \{A_i\}, \{f_i\}$) viene utilizzato per ordinare i record dell'origine O secondo gli attributi $\{A_i\}$ utilizzando le **funzioni di aggregazione** $\{f_i\}$. Più specificamente, queste funzioni di aggregazione servono ad implementare le funzionalità di aggregazione che è possibile utilizzare in SQL come **SUM**, **COUNT**, ... oppure a replicare il comportamento della clausola **HAVING**.

Similmente a quanto accade per l'operatore di deduplicazione, anche in questo caso è necessario che i dati provenienti dalla sorgente O siano **ordinati** sulla base dell'attributo o degli attributi $\{A_i\}$. Il costo di questa operazione è dunque dato dal costo della sorgente $C = C(O)$.

Hashing per il Raggruppamento

In alternativa a supporre che i dati siano ordinati, è possibile, come nel caso della deduplicazione, utilizzare tecniche basate su **hashing**. In questo caso abbiamo a disposizione l'operatore **HashGroupBy**($O, \{A_i\}, \{f_i\}$). Il funzionamento di questo operatore è il seguente:

- per ogni riga di input viene calcolato un hash code basato sugli attributi $\{A_i\}$ in modo da partizionare i record nei rispettivi gruppi. A questo punto vengono utilizzate le funzioni $\{f_i\}$ per ottenere i valori aggregati per ogni gruppo.

Chiaramente è ancora possibile avere delle collisioni e che gruppi diversi finiscano nello stesso bucket, ma dobbiamo sempre ricordare il fatto che, una volta che un bucket è caricato in memoria, possiamo trascurare i costi computazionali. Il costo di questa operazione è dunque dato da: $C = C(0) + 2 \cdot N_{\text{pag}}(O)$.

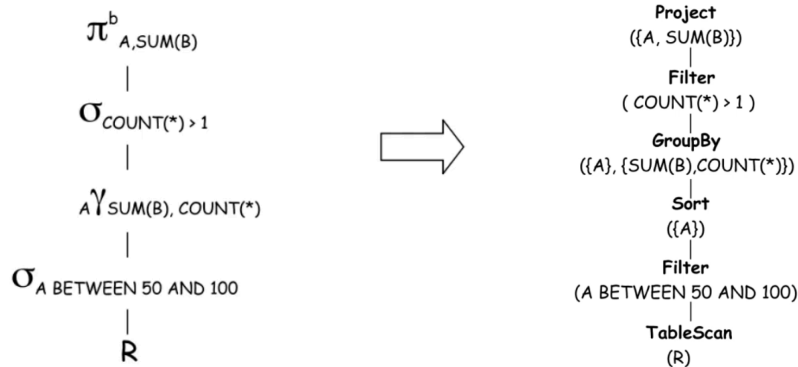
Esempio di Piano di Accesso con Raggruppamento

Andiamo di seguito a vedere come un piano di albero relazionale rappresentate una query che prevede di usare raggruppamenti possa essere trasformato in piano di accesso fisico. Supponiamo di dover lavorare con la seguente query:

```
1 SELECT A, SUM(B)
2 FROM R
3 WHERE A BETWEEN 50 AND 100
4 GROUP BY A
5 HAVING COUNT(*) > 1;
```

SQL

La seguente query può essere trasformata in albero logico e successivamente in piano di accesso fisico come mostrato nell'immagine che segue:



10.2.7. Operatori Fisici per il JOIN

Dopo aver visto tutti gli operatori fisici che è possibile applicare ad una singola relazione, andiamo ora a vedere come sia possibile implementare dal punto di vista fisico le operazioni di **join** tra due relazioni.

In questa trattazione andremo a considerare unicamente il caso di **equi-join** tra due origini O_E ed O_I ossia origine esterna e origine interna. Andremo a denotare il tutto nella maniera seguente: $\left(O_E \bowtie_{\psi_J} O_I \right)$ dove ψ_J è la **condizione di join**.

Nested Loop JOIN

La prima e molto comune tecnica che mostriamo per andare ad effettuare un join tra due sorgenti è quella di utilizzare un **nested loop join**. Si tratta di un'implementazione molto semplice che segue la seguente logica:

```

1  for (Record r: R)      // scorriamo la sorgente esterna
2    for (Record s: S)    // per ogni record esterno, scorriamo gli interni
3      if joinCondition(r, s) // controlliamo la condizione di join
4        output.add((r,s));

```

Il costo di questo algoritmo è dato dal costo di accesso alla sorgente esterna, più il costo di accesso alla sorgente interna moltiplicato per il numero atteso di record nella sorgente esterna:

$$C_{\text{NestedLoop}} = C(O_E) + E_{\text{rec}}(O_E) \cdot C(O_I)$$

Osservazione

È di fondamentale importanza notare che in questo caso l'ordine delle sorgenti è cruciale. Il costo del primo operando della somma è sostanzialmente trascurabile, ciò che è di grande impatto è il prodotto.

In generale sarà di fondamentale importanza scegliere come sorgente esterna quella che ha un numero significativamente minore di record. Nel caso in cui non si abbia grande differenza in termini di numero di record, sarà preferibile scegliere quella che occupa il numero minore di pagine.

Una proprietà molto interessante di questo algoritmo è che gli elementi di O_E vengono letti secondo l'ordine con cui sono memorizzati nell'heap file. Se dunque la sorgente esterna è “più importante” di quella interna, otterremo una sorta di **ordinamento lessicografico**.

Page Nested Loop JOIN

Nel caso precedente abbiamo un importante punto di inefficienza: la sorgente interna viene letta completamente ogni qual volta viene letto un record della sorgente esterna. Una soluzione a questo è utilizzare la scansione della sorgente interna soltanto una volta per ogni **pagina** della sorgente esterna.

```

1  for (Page p_r: R)      // scorriamo R per pagine
2    for (Page p_s : S)  // per ogni pagina interna scorriamo S
3      for (Record r : p_r) // per ogni record nella pagina esterna
4        for (Record s : p_s) // per ogni record nella pagina interna
5          if joinCondition(r, s)
6            output.add((r,s));

```

Il costo di questo algoritmo sarà ora dato dalla seguente equazione:

$$\begin{aligned}
 C_{\text{PageNestedLoop}} &= C(O_E) + N_{\text{pag}}(O_E) \cdot C(O_I) \\
 &= N_{\text{pag}}(O_E) + N_{\text{pag}}(O_E) \cdot N_{\text{pag}}(O_I)
 \end{aligned}$$

Vediamo come in questo caso il costo sarà dominato dal prodotto tra il numero di pagine delle due sorgenti, mentre prima era dominato dal prodotto tra il numero di record di una sorgente e il numero di pagine dell'altra, il che rappresenta un miglioramento significativo.

Per quanto sembri che dal punto di vista del numero di operazioni effettuate questo algoritmo sia quasi peggiore del precedente, in realtà il due cicli `for` interni non avranno particolare influenza, dal momento che siamo interessati solamente al numero di accessi a pagina.

Ulteriori miglioramenti a questo algoritmo possono essere ottenuti caricando in memoria **più pagine** dallo storage secondario. Questo permetterà di ridurre ulteriormente il numero di scansioni della sorgente interna. Se assumiamo di poter caricare tutta la sorgente esterna in memoria, il costo di questo algoritmo diventa:

$$C_{\text{PageNestedLoop}} = N_{\text{pag}}(O_E) + N_{\text{pag}}(O_I)$$

Dal momento che la sorgente esterna è tutta in memoria, non avremo più bisogno di effettuare scansioni multiple della sorgente interna. Per quanto questo algoritmo sia molto più efficiente, ci sono casi in cui vorremmo che il join restituisca risultati ordinati in maniera “lessicografica”, in questo caso questo algoritmo non può essere applicato.

Index Nested Loop JOIN

Un'ulteriore ottimizzazione che possiamo applicare al nested loop join è quella di sfruttare un indice per risolvere le **condizioni di uguaglianza** presenti nella condizioni di join. In questo caso l'algoritmo diventa:

```

1  for (Record r : O_E)

```

```

2   for (Record s : IndexFilter(0_I, Idx, r.A_j = s.A_j))
3       output.add((r,s));

```

La chiave di questo algoritmo è utilizzare l'operatore **Index Filter** utilizzando il valore della chiave del record esterno per cercare i record corrispondenti nella sorgente interna in maniera efficiente. Il costo di questo algoritmo sarà dato da:

$$C_{\text{IndexNestedLoop}} = C(O_E) + E_{\text{rec}}(O_E) \cdot (C_I + C_D)$$

Merge JOIN

Un'altra tecnica comune per effettuare il join tra due sorgenti è quella di utilizzare un **merge join**. Se le due sorgenti sono *ordinate* secondo gli attributi coinvolti nella condizione di join, possiamo utilizzare questo algoritmo:

```

1   Record r = 0_E.first(); // null if empty
2   Record s = 0_I.first(); // null if empty
3
4   while (r IS NOT NULL and s IS NOT NULL) {
5       if r[A_i] = s[A_j] { // controllo della condizione di join
6           output.add((r,s));
7           s = 0_I.next(); // i valori s soddisfano la condizione di join
8       } else {
9           r = 0_E.next(); // spostiamo il cursore della relazione esterna
10      }
11 }

```

Il costo di questo algoritmo è semplicemente dato dalla somma degli accessi alle due sorgenti:

$$C_{\text{MergeJoin}} = C(O_E) + C(O_I)$$

Ovviamente, per quanto sia un algoritmo molto efficiente, la condizione sull'ordinamento delle due sorgenti lo rende applicabili soltanto in casi particolari.

Hash JOIN

Senza dubbio l'algoritmo di merge join è uno dei più efficienti, tuttavia richiede una precondizione relativamente stringente. Per questo motivo viene in nostro aiuto l'algoritmo di **hash join** che come nei casi precedenti sfrutta tecniche di hashing per sorvolare il requisito di ordinamento dei dati. Questo algoritmo funziona in due fasi: **partizionamento** e **probing**.

La differenza rispetto agli algoritmi precedenti consiste nel fatto che in questo caso abbiamo a che fare con due relazioni, non più con una relazione soltanto.

Nella fase di *partizionamento* i record delle due sorgenti vengono divisi in **bucket** tramite una funzione di hashing basata sugli attributi coinvolti nella condizione di join. In questo modo i record che possono soddisfare la condizione di join finiranno tutti nello stesso bucket.

Nella fase di *probing* per ogni partizione B_i i record di O_E vengono letti e inseriti in una hash table con B pagine di memoria utilizzando la funzione di hashing h_2 . La stessa funzione

(basata sugli attributi di join) viene utilizzata una volta letti i record di O_I per cerchiamo i record corrispondenti nella hash table tramite h_2 . Una volta trovati i record, controlleremo la condizione di join e in caso positivo li aggiungeremo all'output. Notiamo che questa operazione può essere effettuata in memoria, dunque non ha un costo computazionale significativo. Vediamo questa procedura illustrata in Figura 10.3.

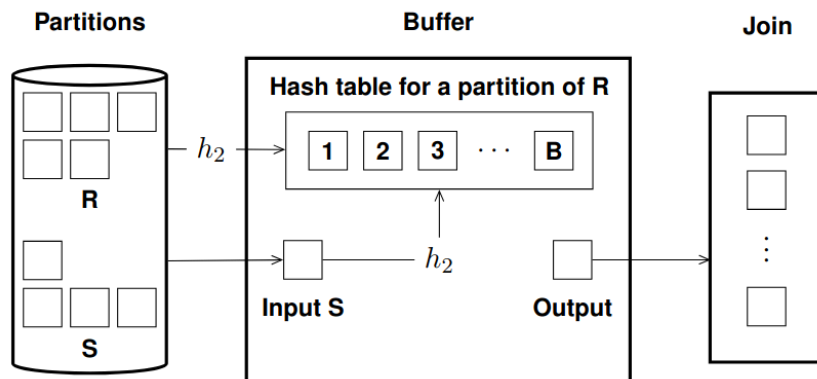


Figura 10.3: Fase di probing delle partizioni nell'hash join

Dal momento che entrambe le sorgenti vengono lette completamente sia per il probing che per il partizionamento, il costo di questo algoritmo è dato da:

$$C_{\text{HashJoin}} = C(O_E) + C(O_I) + 2 \cdot (N_{\text{pag}}(O_E) + N_{\text{pag}}(O_I))$$

Cardinalità di un JOIN

Così come per le operazioni di filtraggio è interessante andare ad analizzare la cardinalità del risultato di un join tra due relazioni. Data infatti la seguente operazione di join:

$$R \bowtie_{\psi_J} S = \sigma_{\psi_J}(R \times S)$$

Possiamo andare a studiarne la cardinalità tramite la seguente formula:

$$E_{\text{rec}} = \lceil s_{f(\psi_J)} \cdot E_{\text{rec}}(O_E) \cdot E_{\text{rec}}(O_I) \rceil$$

In questo caso il fattore di selettività $s_{f(\psi_J)}$ rappresenta la probabilità che una coppia di record soddisfi la condizione di join. È dunque riferito alla distribuzione di probabilità congiunta.

10.2.8. Operatori Fisici per le Operazioni su Set e Multi-Set

Per effettuare operazioni di unione, intersezione e differenza tra due relazioni, rappresentabili per mezzo di multi-set, abbiamo la necessità che gli operatori fisici di nostra implementazione abbiano accesso a dati già **ordinati**. In questo modo possiamo utilizzare tecniche simili a quelle viste per il merge join.

Oltre ai classici operatori abbiamo un operatore speciale che è anche il più semplice dal punto di vista implementativo ossia l'**UnionAll**(O_E, O_I) che prende in input due multi-set di record e restituisce il loro multi-set unione. Il costo di questo operatore è semplicemente dato dalla somma dei costi delle due sorgenti: $C = C(O_E) + C(O_I)$.

10.3. Ottimizzazione delle Query Relazionali

Il motivo per cui abbiamo spiegato in dettaglio le varie implementazioni e costi legati ad ogni operatore fisico illustrato è che, dato un piano di accesso logico, il gestore dei piani di accesso deve essere in grado di tradurre questo piano in un piano di accesso fisico efficiente. Questo processo di traduzione è noto come **ottimizzazione delle query** e viene svolto dal modulo **query optimizer** del DBMS che possiamo vedere in Figura 3.1.

Come abbiamo visto è possibile tradurre una query relazionale in molteplici tipi di piani di accesso fisico, utilizzando operatori diversi a seconda delle strutture dati a disposizione e delle condizioni specificate sulla query. È dunque di fondamentale importanza riuscire ad individuare il piano di accesso fisico più efficiente per svolgere la query richiesta.

Nel processo di ottimizzazione delle query, il query optimizer divide il compito in due fasi principali.:

- **analisi della query**: in questa fase viene analizzata la correttezza sintattica della query e viene generato un *albero logico* basato sull'algebra relazionale estesa.
- **trasformazione della query**: l'albero logico prodotto nella fase precedente viene trasformato in un piano logico equivalente ma più **efficiente**.
- **generazione del piano fisico**: il piano logico ottimizzato viene tradotto in un piano di accesso fisico. In questa fase diverse implementazioni fisiche degli operatori logici vengono prese in considerazione e viene scelto il piano fisico con costo minore.

Solo a questo punto è finalmente possibile andare ad eseguire il piano di accesso fisico generato dallo storage engine.

10.3.1. Analisi della Query

Durante la fase di analisi della query vengono svolte le seguenti operazioni:

- Analisi lessicale e sintattica della query SQL per verificarne la correttezza.
- Analisi **semantica** della query, per verificare che la query sia semanticamente corretta (fa riferimento a tabelle e attributi esistenti, i tipi di dato sono corretti, ...) e che l'utente abbia i permessi necessari per eseguire la query.
- La condizione **WHERE** della query viene riscritta in forma **conjunctive normal form** (CNF) per semplificare le successive fasi di ottimizzazione.
- Una volta riscritta la condizione **WHERE** questa viene ulteriormente semplificata:
 - ▶ vengono applicate le regole di equivalenza delle espressioni booleane
 - ▶ vengono eliminate le condizioni ridondanti
 - ▶ vengono eliminate condizioni contraddittorie (es. $A > 20 \wedge A < 18 \equiv \perp$)
 - ▶ vengono trasformate condizioni in modo tale che il **NOT** non appaia scrivendo il complementare della condizione negata (es. $\neg(A > 20) \equiv A \leq 20$)

Dopo queste trasformazioni viene generato un albero logico basato sull'algebra relazionale estesa precedentemente introdotta. Per esempio consideriamo la query seguente:

```
1 SELECT PkEmp, EName
2 FROM Employee JOIN Department
3 ON Employee.FkDept = Department.PkDept
```

SQL

```

4 WHERE
5 Department.DLocation = 'Pisa' AND Employee.ESalary > 2000;

```

Questa query può essere rappresentata tramite l'albero logico mostrato

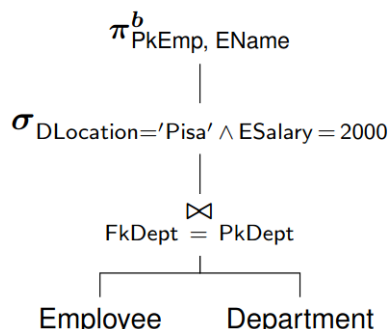


Figura 10.4: Albero logico della query di esempio

Regole di Trasformazione Logica

Andiamo ora a vedere alcune regole che in linea generale possiamo andare a considerare per trasformare un albero logico in un albero equivalente ma più efficiente.

In primo luogo possiamo andare ad utilizzare alcune regole di **semplificazione** delle espressioni algebriche, le elenchiamo di seguito:

- **Eliminazione delle selezioni in cascata:** quando ci troviamo di fronte a più operazioni di selezione concatenate, possiamo andare a fondere queste selezioni in un'unica operazione di selezione con condizione data dalla congiunzione delle condizioni originali: $\sigma_{\psi_X}(\sigma_{\psi_Y}(E)) = \sigma_{\psi_{XY}}(E)$
- **Commutatività di selezione e proiezione:** le operazioni di selezione e proiezione sono commutative tra loro, dunque possiamo scambiare l'ordine in cui queste vengono applicate: $\pi_Y^b(\sigma_{\psi_X}(E)) = \sigma_{\psi_X}(\pi_Y^b(E))$ se $X \subseteq Y$, in caso contrario avremo $\pi_Y^b(\sigma_{\psi_X}(E)) = \pi_Y^b(\sigma_{\psi_X}(\pi_{XY}^b(E)))$.

In linea generale possiamo utilizzare il principio di **selection push-down** che consiste nel portare tutte le operazioni di filtraggio il più possibile verso gli operatori di scansione delle relazioni. In questo modo andremo a ridurre il numero di record che dovranno essere processati dagli operatori successivi.

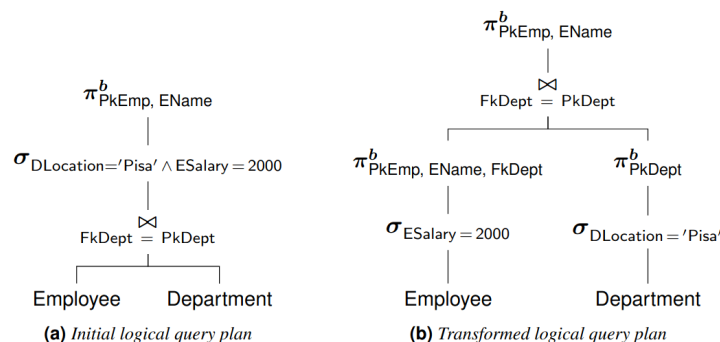


Figura 10.5: Albero logico ottimizzato della query di esempio

Sempre su questo principio, possiamo andare a riposizionare le operazioni di selezione e **proiezione** in maniera tale che queste vengano eseguite **prima dei join**, in questo modo andremo a ridurre il numero di record sui quali effettuare il confronto di uguaglianza tra gli attributi coinvolti nella condizione di join.

Utilizzando le regole appena introdotte possiamo riscrivere l'albero in Figura 10.4 in un albero più efficiente come mostriamo in Figura 10.5.

Eliminazione di Clausole Non Necessarie

Sempre in questo processo di ottimizzazione spesso viene considerato il fatto che la selezione di valori **distinti** richiede un costo computazionale maggiore rispetto alla selezione di tutti i valori che soddisfano una query. Spesso ci troviamo davanti a query che utilizzano tale clausola senza che questa sia effettivamente necessaria. Per questo motivo, durante la fase di analisi della query, il query optimizer può andare a rimuovere questa clausola nel caso in cui sia possibile dimostrare che il risultato della query non conterrà mai dati duplicati.

Lo stesso discorso vale per le operazioni di **raggruppamento**, anche in questo caso è necessario che i dati vengano ordinati o partizionati tramite funzioni di hashing. In questo caso potremo procedere con l'eliminazione in due casi:

- la query risponde con dati in un **singolo gruppo**
- la query restituisce **una tupla per gruppo**

In entrambi questi casi non è necessario effettuare operazioni di raggruppamento sui dati, dunque possiamo andare a rimuovere queste operazioni dall'albero logico della query.

Un altro possibile punto di ottimizzazione è quello di cercare il più possibile di non tenere le **subquery** come delle entità separate, in modo tale da includerle nell'albero principale e poterle ottimizzare insieme al resto della query. Per fare questo ci basta sapere che tutti gli operatori (**IN** , **ANY** , **ALL** , ...) possono essere riscritti in termini di **EXISTS** e **NOT EXISTS** , e che questi ultimi possono a volte essere riscritti tramite **JOIN** . In particolare andremo ad utilizzare un normale **JOIN** per riscrivere le condizioni con **EXISTS** e un **OUTER JOIN** per riscrivere le condizioni con **NOT EXISTS** .

Similmente a subquery, grouping e distinct, possiamo anche andare a eliminare operazioni che vanno a creare **viste** di dati. Consideriamo per esempio la seguente query:

```
1 WITH Technician AS SQL
2 (
3   SELECT PkEmp, EName, Salary
4   FROM Employee
5   WHERE ERole = 'Technician'
6 )
7 SELECT EName, Salary
8 FROM Technician
9 WHERE Salary > 3000;
```

Questa può essere semplificata e riscritta nel modo seguente:

```

1 SELECT EName, Salary
2 FROM Employee
3 WHERE ERole = 'Technician' AND Salary > 3000;

```

SQL

Esistono anche situazioni in cui questo tipo di ottimizzazione non può essere accorpata alla query principale, per esempio quando la vista viene utilizzata per creare valori aggregati.

```

1 WITH NoEmployeeByDept AS
2 (
3     SELECT FkDept, COUNT(*) AS NumEmp
4     FROM Employee
5     GROUP BY FkDept
6 )
7 SELECT AVG(NoEmp), FROM NoEmployeeByDept

```

SQL

In questo caso abbiamo una query che calcola il numero medio di dipendenti in ogni dipartimento. In questo caso non possiamo andare a rimuovere la vista.

Un'ultima operazione possibile è quella di andare a spostare le operazioni di **raggruppamento** successivamente alle operazioni di **join**. In questo modo andremo a rendere l'operazione di join più leggera.

10.3.2. Generazione del Piano Fisico

Dopo aver visto come sia possibile andare a ottimizzare l'albero logico della query, andiamo a vedere come sia possibile tradurre questo albero logico in un **piano di accesso fisico**.

A livello pratico, abbiamo già menzionato il fatto che dato un albero logico è possibile utilizzare molteplici operatori fisici per implementare ogni operazione logica. Per esempio, per effettuare una scansione di una relazione possiamo utilizzare un **table scan**, uno **sort scan** oppure un **index scan**.

In generale quello che accade è che vengono generati diversi piani alternativi e viene fatta una **stima** del costo di ogni piano, per poi scegliere quello con **costo atteso minore**. Per fare questa stima del costo è necessario conoscere:

- il **costo** di ogni **operatore** fisico
- la **cardinalità** dei risultati **intermedi**
- se i risultati **intermedi** sono **ordinati** o meno

Nel caso in cui nella query da considerare vengano utilizzate operazioni di **join**, ossia vengono utilizzate più relazioni, il numero di possibili piani di accesso fisico diventa estremamente grande. Nello specifico, il problema di dare la soluzione ottima è **esponenziale** nel numero di relazioni coinvolte nella query. Per questo motivo, nella pratica vengono utilizzate tecniche **euristiche**, specie nel caso di query molto grandi.

11. Gestione di Transazione e Concorrenza

In questo capitolo andremo a trattare gli ultimi componenti fondamentali dello storage engine di una DBMS: il modulo di gestione della concorrenza e quello di gestione delle transazioni.

Possiamo, anche senza aver visto in dettaglio il loro funzionamento, immaginare come questi due componenti siano strettamente collegati: sappiamo infatti che tra i compiti fondamentali di una base di dati c'è quelli di garantire l'integrità dei dati, e questo è possibile solo se si riesce a gestire correttamente la concorrenza tra le operazioni che vengono eseguite sulla base di dati, e se si riesce a garantire che le transazioni vengano eseguite in modo atomico, consistente, isolato e durevole.

11.1. Gestore delle Transazioni

Andiamo a trattare prima il modulo di gestione delle transazioni dal momento che questo viene utilizzato dal modulo di gestione della concorrenza.

Definizione 11.1 (Transazione)

Una transazione, dal punto di vista di un programmatore, è un'unità di lavoro sequenziale da eseguire sulla base di dati che garantisce le proprietà di atomicità, consistenza, isolamento e durabilità (ACID).

Possiamo vedere ogni transazione come un insieme di operazioni di **lettura** e **scrittura** sulla base di dati, che vengono iniziate tramite un comando `beginTransaction` e che sono concluse tramite un comando `commit` o `rollback`.

Il motivo per cui è possibile che terminino con un `rollback` è che, durante l'esecuzione di una transazione, possono verificarsi degli errori che impediscono un completamento corretto, come ad esempio un errore di sintassi in una query o una violazione di vincoli, che può avvenire a causa dell'esecuzione di operazioni concorrenti. In questi casi, è necessario annullare tutte le modifiche operate dalla transazione corrente.

Osservazione

Il modulo di gestione delle transazioni si occupa di garantire le proprietà di **atomicità** e **durabilità** delle transazioni, mentre il modulo di gestione della concorrenza si occupa di garantire **isolamento**.

11.1.1. Requisiti di funzionamento

In questa sezione andiamo brevemente a descrivere i requisiti di funzionamento di questo modulo. Come già anticipato, sappiamo che ha il compito di garantire **atomicità** e **durabilità** delle transazioni. In senso pratico è richiesto che le transazioni vengano svolte in maniera completa, oppure che non vengano svolte affatto, e che le modifiche operate da una transazione vengano mantenute anche in caso di crash del sistema.

Supponiamo per esempio avere cinque transazioni all'interno del nostro sistema, alcune delle quali vanno a buon fine, mentre altre non fanno in tempo a completarsi a causa di un crash di sistema. Possiamo vedere questa situazione rappresentata in Figura 11.1.

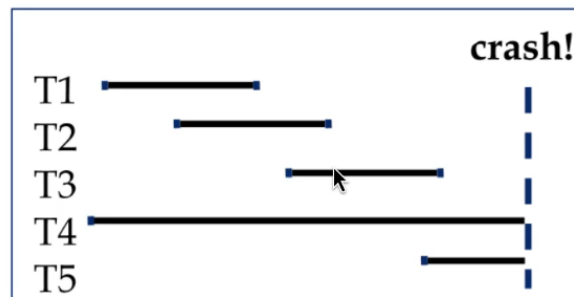


Figura 11.1: Esempio di crash del sistema durante l'esecuzione di alcune transazioni

In questo caso vorremmo che dopo il ripristino del sistema, le transazioni T_1, T_2, T_3 siano visibili e permanenti nel sistema, mentre le transazioni T_4 e T_5 non siano visibili e non abbiano modificato lo stato del sistema.

11.1.2. Ciclo di vita di una transazione

Prima di introdurre come vengono gestite le transazioni all'interno del sistema, andiamo a vedere quali sono gli stati in cui una transazione può trovarsi durante il suo ciclo di vita.

Possiamo vedere una transazione come un oggetto che può trovarsi in uno dei seguenti stati:

- una transazione diventa **attiva** quando viene iniziata tramite il comando **beginTransaction**, mentre la transazione è attiva si susseguono una serie di operazioni di lettura e scrittura sulla base di dati;
- ad un certo punto la transazione potrà essere utilizzato il comando **commit** tramite il quale si indica che non è più possibile effettuare operazioni di lettura o scrittura sulla base di dati, e che la transazione è pronta per essere resa permanente, in questo stato la transazione si trova nello stato di **partially committed**;
- in alternativa al comando **commit** è possibile utilizzare il comando **abort** tramite il quale si indica che la transazione deve essere annullata, in questo caso la transazione passa allo stato di **failed**, questo comando viene tipicamente utilizzato quando si verifica un errore logico durante l'esecuzione di un comando, per esempio una violazione di vincolo, oppure nel caso in cui il sistema venisse spento in maniera "graceful", ovvero senza un crash improvviso, ci potrebbe essere la possibilità di operare un **abort** della transazione;
- dallo stato **partially committed** è soltanto possibile passare allo stato **committed** quando tutte le modifiche sono effettivamente rese permanenti sulla base di dati, oppure allo stato **failed** nel caso in cui si verifichi un errore durante il processo di commit, ad esempio a causa di un crash di sistema.
- mentre la transazione si trova nello stato **failed** viene tipicamente svolto un **rollback** tramite il quale tutte le modifiche operate dalla transazione vengono annullate, e la transazione passa allo stato di **aborted**.

Vediamo il ciclo di vita di una transazione ben rappresentato in Figura 11.2.

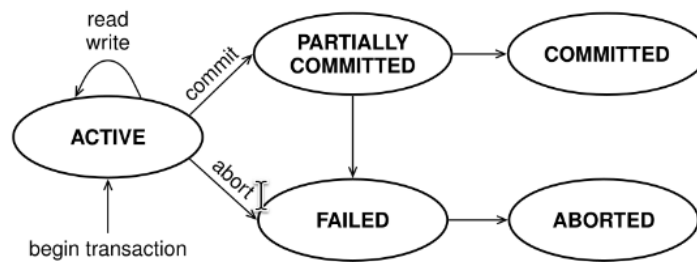


Figura 11.2: Ciclo di vita di una transazione

Abbiamo mostrato come all'interno della vita di una transazione sia possibile che questa venga abortita per diversi motivi. Andiamo a vedere in seguito quali sono le tipologie di failure che possono verificarsi durante l'esecuzione di una transazione.

Transaction Failure

Si tratta di un'interruzione della transazione che non danneggia il contenuto della memoria temporanea e di quella permanente. Questo tipo di failure è tipicamente causato da **errori logici** all'interno delle operazioni svolte dalla transazione, come ad esempio una violazione di vincoli di integrità, oppure un comando **abort** esplicito.

System Failure

Si tratta di interruzioni del sistema, che può essere inteso sia come il DBMS che come l'intero sistema operativo del server ospitante la base di dati. Questo tipo di failure può essere causato da crash di sistema, errori hardware, o errori software critici. In questo caso il contenuto della memoria temporanea viene perso, mentre il contenuto della memoria permanente è tipicamente preservato.

Media Failure

Questo tipo di failure è tipicamente il più grave, anche noto come **catastrophic failure**. Si tratta di un'interruzione del DBMS che va a causare perdita di informazioni salvate in memoria secondaria.

11.1.3. Compiti del modulo

Di seguito andiamo ad elencare quelli che sono i compiti principali a cui deve assolvere il modulo di gestione delle transazioni e del ripristino. Questo nome non è mai stato utilizzato in maniera completa, essendoci sempre riferiti a questo modulo come modulo di gestione delle transazioni, ma è importante sottolineare che questo modulo si occupa anche del ripristino del sistema in caso di failure, ed è evidente che lo sia, dal momento che deve garantire durevolezza e atomicità delle transazioni a fronte di possibili failure.

Esecuzione di operazioni per conto di una transazione

In primo luogo è fondamentale che il modulo sia in grado di gestire l'**esecuzione** di **read, write, commit, abort per conto di una transazione**. In particolare, sappiamo che a livello dello storage engine viene ricevuto un piano di esecuzione, che utilizza il gestore dei metodi di accesso per accedere alla memoria. Queste operazioni non vengono direttamente eseguite, ma vengono invece prima "inviare" al modulo di gestione delle transazioni, in modo

che possa tenere traccia di tutte le operazioni svolte, nel caso in cui sia necessario un rollback o un ripristino del sistema.

Quello specificato nel paragrafo precedente è il requisito sostanziale del modulo, per fare questo però è necessario che questo modulo svolga altri compiti secondari che sono tutti necessari per garantire il corretto funzionamento del workflow di gestione delle transazioni.

Gestione dei log e del ripristino

Come già anticipato, tutte le operazioni che dovrebbero essere svolte da una transazione, prima di passare per il gestore dei metodi di accesso, devono essere “viste” e tracciate da questo modulo. Per operare questo tracciamento viene utilizzato un meccanismo di **logging**, ovvero di registrazione di tutte le operazioni svolte da una transazione, in modo che sia possibile, in caso di failure, risalire a tutte le operazioni svolte e annullarle o ripristinarle se necessario.

Prevenzione di perdita di dati

Come già anticipato, è di fondamentale importanza garantire **durabilità** delle transazioni, e questo è possibile solo se si riesce a prevenire la perdita di dati in caso di failure. Per fare questo è necessario che tutte le modifiche operate da una transazione vengano salvate in modo permanente sulla base di dati, e che queste modifiche vengano mantenute anche in caso di crash del sistema.

Ripristino del sistema in caso di failure

In caso di failure di sistema, è compito di questo modulo operare un comando **restart** per fare in modo che tutto possa essere ripristinato correttamente, e che le transazioni che erano in corso al momento del crash vengano gestite in modo appropriato, ovvero che quelle che erano in stato **partially committed** vengano completate, mentre quelle che erano in stato **active** vengano annullate e/o fatte ripartire.

Di seguito andiamo ad illustrare in dettaglio come ognuno di questi compiti viene svolto all'interno del modulo di gestione delle transazioni e del ripristino, e quali sono le tecniche più comuni utilizzate per garantire il corretto funzionamento di questo modulo.

Tecniche di gestione delle transazioni

In queste sezioni andiamo ad esplorare in quale modo è possibile andare a rendere questo modulo di delegato all'esecuzione di operazioni per conto di una transazione. Come sappiamo l'unico interesse dal punto di vista del DBMS è quello di gestire quali dati sono letti e/o scritti da/su una base di dati. Per semplificare la trattazione *assumiamo di andare a leggere **pagine** non singoli record*.

Un'operazione di **lettura** di una pagina $r_i[x]$, questa necessita che la pagina sia spostata *dal disco al buffer pool* nel caso in cui questa non sia già presente nel buffer.

Un'operazione di **scrittura** di una pagina $w_i[x]$ viene inizialmente modificata una copia in memoria della pagina la quale viene scritta su disco quando il **gestore del buffer** lo decide. Per questo motivo, se una failure avviene, l'effetto delle operazioni di scrittura potrebbe non essere visibile sul disco. In questi casi è necessario l'utilizzo di tecniche di **recovery**.

11.1.4. Prevenzione in caso di Failure

Di seguito elenchiamo alcuni possibili approcci per prevenire la perdita di dati in caso di failure del sistema:

- **database backup**: vengono creati dei *dump* della base di dati su supporti esterni in modo da poter ripristinare la base di dati in caso di perdite di dati. Ovviamente questa soluzione non consente di recuperare le modifiche effettuate dopo l'ultimo backup, ad eccezione di quelle salvate in un log;
- **undo-redo logging**: tramite questa tecnica è possibile tenere traccia di tutte le operazioni svolte da una transazione in maniera da poter annullare o ripristinare le modifiche in caso di failure. Questo meccanismo permette anche il ripristino in caso di crash di sistema: in questo caso infatti, al riavvio del sistema, tutte le transazioni attive vengono abortite, e le modifiche operate dalle transazioni che avevano completato il commit vengono rese permanenti. Ogni record del log ha la seguente struttura:
 - ▶ **(T, begin)** : indica l'inizio della transazione T;
 - ▶ per ogni scrittura **(T, write, oldValue, newValue)** ;
 - ▶ **(T, commit)** o **(T, abort)** : indica la fine della transazione T;

Operation	Data	Information in the log
<i>beginTransaction</i>		<i>(begin, 1)</i>
<i>r[A]</i>	<i>A = 50</i>	No record written to the log
<i>w[A]</i>	<i>A = 20</i>	<i>(W, 1, A, 50, 20)</i> — Old and new value of <i>A</i> are written to the log
<i>r[B]</i>	<i>B = 50</i>	No record written to the log
<i>w[B]</i>	<i>B = 80</i>	<i>(W, 1, B, 50, 80)</i>
<i>commit</i>		<i>(commit, 1)</i>

Figura 11.3: Esempio di log per una transazione

Figura 11.3 mostra un esempio di come questo log possa essere strutturato per una transazione. Il motivo per cui salviamo tutte queste informazioni sta nel fatto che, la strategia di ripristino si basa su due operazioni fondamentali: **undo** e **redo**; l'idea è andare a scorrere il log all'indietro per annullare le modifiche di tutte le transazioni non completate (undo), e andare a scorrere il log in avanti per ripristinare le modifiche relative alle transazioni completate (redo).

- **checkpoint periodici**: per quanto la procedura di undo-redo sembri semplice non è ancora chiaro un punto fondamentale, ossia, *quanto dobbiamo andare all'indietro nel log effettuando undo*. Se infatti guardiamo a Figura 11.1, nel momento in cui T_5 ha un crash, T_3 sta ancora eseguendo, quindi è necessario un undo di T_3 , ma questo ragionamento si propaga a catena anche su T_1 e T_2 . Il rischio concreto in cui incorriamo è quello di dover ogni volta effettuare undo per la gran parte delle transazioni, possiamo notare che questo è causato da **transazioni** che vengono eseguite in maniera **interlacciata**. Il principio alla base dei **checkpoint periodici** è proprio quello di andare a creare dei punti di salvataggio nel log in cui tutte le transazioni attive sono completate, in questo modo, in caso di crash, sarà possibile effettuare l'undo soltanto delle transazioni iniziate dopo l'ultimo checkpoint.

Creazione dei Checkpoint

Come abbiamo citato, i checkpoint sono un componente chiave di tutta la gestione del ripristino; sappiamo come il DBMS crei periodicamente un checkpoint per andare a minimizzare il tempo necessario per il ripristino in caso di crash, che consisterebbe per la gran parte di operazioni di undo. Esistono diversi approcci per la creazione dei checkpoint, andiamo di seguito a vedere i due più comuni.

Commit Consistent Checkpoint

Si tratta dell'approccio più **semplice** per la creazione di un checkpoint. In questo caso quando viene deciso di creare un checkpoint si procede come segue:

- il sistema **non accetta nuove transazioni** fino a completamento del checkpoint
- si **attende il completamento** di tutte le transazioni attive
- viene effettuato un **flush** di tutte le pagine modificate nel buffer pool verso il disco
- viene scritto sul log un **record di checkpoint** che indica che tutte le transazioni sono state completate e che tutte le pagine modificate sono state scritte su disco, tale record sarà utilizzato per fermare la sequenza di operazioni undo

Chiaramente il problema di questo approccio è che il sistema rimane completamente bloccato fino al completamento del checkpoint, il che può essere un problema in sistemi con un alto grado di concorrenza.

Buffer Consistent Checkpoint

Si tratta di un approccio più **complesso** per la creazione di un checkpoint, ma che consente di minimizzare il tempo in cui il sistema rimane bloccato. In questo caso non viene atteso il completamento delle transazioni attive ma viene tenuto traccia delle **transazioni attive** al momento della creazione del checkpoint. In questo modo sarà possibile andare più indietro del checkpoint soltanto per le transazioni attive al momento della creazione del checkpoint, evitando di dover annullare tutte le transazioni completate prima del checkpoint.

11.1.5. Procedure per Undo e Redo

Per quanto sia già stato spiegato a sommi capi il funzionamento di questa tecnica, è importante andare a mostrare in dettaglio il funzionamento di queste procedure.

Osservazione

L'esistenza di queste procedure assume che qualsiasi operazione di **logging** sia fatta in maniera forzata su disco, in maniera tale che sia sempre possibile operare un ripristino qualora fosse necessario.

Gli algoritmi di ripristino differiscono a causa delle informazioni che vengono memorizzate nei log, in come queste informazioni sono strutturate e, ancora più importante, rispetto al **momento in cui le pagine sono scritte nella memoria stabile**.

Diciamo che un algoritmo di ripristino richiede un **undo** nel caso in cui un aggiornamento di qualche transazione non committed sia presente nella memoria stabile. Se si verifica un

errore di transazione o un guasto del sistema, l'algoritmo di ripristino deve annullare gli aggiornamenti copiando l'immagine precedente della pagina dal log al database stabile.

In maniera simile, diciamo che è necessaria un'operazione di **redo** nel caso in cui una transazione è considerata "committed" prima che tutti i suoi aggiornamenti siano presenti nella memoria stabile. Se si verifica un errore di transazione o un guasto di sistema dopo il commit ma prima che tutti gli aggiornamenti siano resi permanenti, l'algoritmo di ripristino dovrà effettuare un redo degli aggiornamenti copiando l'immagine aggiornata della pagina dal log al database stabile.

Gli algoritmi di ripristino si possono classificare in base al modo in cui gestiscono le operazioni di scrittura e di commit delle transazioni. In particolare, possiamo distinguere:

- algoritmi **undo-redo**
- algoritmi **redo-only**
- algoritmi **undo-only**
- algoritmi **no undo-no redo**

Di seguito mostriamo quelle che sono note come **Undo** e **No-Undo** policies, ossia le politiche che regolano il momento in cui le modifiche operate da una transazione vengono effettivamente scritte su disco.

Deferred Update (No-Undo Policy)

Secondo questo approccio, un aggiornamento di una transazione non può essere scritto sulla memoria stabile prima che la transazione sia terminata con successo (committed). In questo modo non è necessario annullare gli aggiornamenti effettuati in caso di fallimento della transazione, dal momento che questi non sono mai stati scritti su disco.

Immediate Update (Undo Policy)

Secondo questo approccio, un aggiornamento potrebbe essere scritto sulla memoria stabile prima che la transazione sia terminata con successo (committed). In questo caso, se la transazione non termina con successo, è necessario annullare gli aggiornamenti effettuati.

Nel caso in cui venga utilizzata questa politica, abbiamo la seguente regola per applicare gli undo (**Write-Ahead Logging** - WAL): se una nuova versione di una pagina è scritta nel DB prima che la transazione termini, allora la vecchia versione della pagina deve essere salvata nel log prima che i cambiamenti della pagina siano scritti su disco. In questo modo, in caso di fallimento della transazione, è possibile recuperare la vecchia versione della pagina dal log.

Deferred Commit (No-Redo Policy)

Secondo questo approccio, una transazione si considera terminata con successo solamente una volta che tutte le modifiche operate da questa sono state scritte sulla memoria stabile. In questo modo non è necessario ripristinare gli aggiornamenti effettuati in caso di fallimento del sistema, dal momento che questi sono già stati resi permanenti.

Immediate Commit (Redo Policy)

Secondo questo approccio, una transazione si considera terminata con successo non appena viene emesso il comando di **commit**, indipendentemente dal fatto che tutte le modifiche operate da questa siano state scritte sulla memoria stabile. In questo caso, se si verifica un

fallimento del sistema prima che tutte le modifiche siano state rese permanenti, è necessario ripristinare gli aggiornamenti effettuati.

Nel caso in cui venga utilizzata questa politica, abbiamo la seguente regola per applicare il redo di una transazione T (**Commit rule**): la nuova versione delle pagine deve essere scritta nel log (presente nello storage stabile) prima che la transazione sia considerata committed.

Shadow Paging

Un algoritmo “no-undo” richiede che tutte le modifiche operate da una transazione siano memorizzate **dopo** che questa è stata committata. Al contrario, un algoritmo “no-redo” richiede che tutte le modifiche siano memorizzate **prima** che la transazione sia committata. Di conseguenza un algoritmo “no-undo no-redo” richiede che tutte le modifiche siano nella memoria stabile né prima, né dopo il commit. L’unica alternativa è dunque che l’operazione di commit scriva in modo atomico tutte le modifiche operate da una transazione e marchi la transazione come committata.

Questo può essere ottenuto tramite la tecnica dello **shadow paging**: l’implementazione si basa su un indice permanente della memoria che mappa ogni identificatore di pagina (PID) ad un indirizzo fisico (questo indice è noto come **page table**). Esiste inoltre un **database descriptor** presente ad un indirizzo fisso della memoria il quale contiene un puntatore alla page table.

Quando una transazione inizia, una copia della page table è creata nella memoria stabile e viene utilizzata dalla transazione. Quando una transazione aggiorna per la prima volta una pagina, succede quanto segue:

- una nuova pagina del database è creata e viene utilizzata come pagina corrente, con indirizzo p . La pagina vecchia rimane inalterata e diventa **shadow page**;
- la nuova page table è aggiornata in modo che il PID della pagina aggiornata punti all’indirizzo p ;

Tutte le operazioni su quella pagina operano sulla nuova copia. Quando la transazione raggiunge il punto di **commit**, il sistema sostituisce tutte le pagine shadow per mezzo di un’azione atomica. È necessaria atomicità in quanto se un fallimento si verificasse durante il commit, la base di dati verrebbe lasciata in uno stato inconsistente.

Figura 11.4 mostra in maniera concreta il funzionamento di questo approccio. È facile immaginare che questa tecnica possa avere diversi svantaggi:

- le pagine occupate dai record di una stessa tabella sono sparse per tutta la memoria, causando alta **frammentazione** e **bassa località**;
- è necessario un meccanismo di **garbage collection** per recuperare lo spazio occupato dalle shadow page non più utilizzate;
- è comunque necessario un meccanismo di logging per tenere traccia delle transazioni in corso in modo da poter ripristinare il sistema in di disastri.
- la complessità aumenta nel momento in cui si vuole garantire **concorrenza**

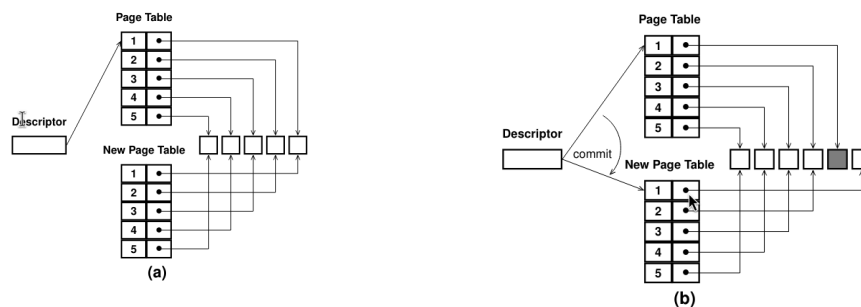


Figura 11.4: Esempio di shadow paging: (a) prima dell'aggiornamento di una pagina; (b) dopo l'aggiornamento e commit di una pagina

In generale ci troviamo di fronte al dover scegliere tra le diverse politiche già presentate. In generale utilizzare una politica di tipo **undo-redo** è più efficiente in casi di utilizzo normale, per quanto richieda più risorse in termini di spazio e tempo per il ripristino in caso di failure. Nonostante questo, si tratta della politica più utilizzata nei DBMS moderni.

11.1.6. Algoritmo di Ripristino

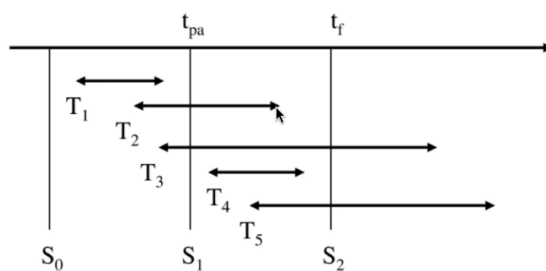
Dopo aver presentato le diverse tecniche di gestione delle transazioni e di prevenzione in caso di failure, andiamo a mostrare un esempio di algoritmo di ripristino basato su undo-redo logging con write-ahead logging (WAL) e checkpoint periodici, che è tra l'altro l'approccio più comunemente utilizzato nei DBMS moderni.

Di seguito mostriamo, a seconda del tipo di failure, cosa si rende necessario:

- in caso di **transaction failure** è necessario scrivere sul log il record (T, abort) e iniziare una procedura di **undo**;
- in caso di un **system failure** è necessario utilizzare un comando di **restart** a partire dall'**ultimo checkpoint**, le transazioni non terminate saranno annullate e le transazioni terminate saranno ripristinate;
- in caso di **media failure** sarà necessario un **cold restart** a partire dall'ultimo backup, seguito da un comando di **restart** per ripristinare le transazioni completate dopo il backup (ipotizzando che il log sia ancora presente).

Esempio: Esecuzione dell'algoritmo di ripristino

Consideriamo la seguente situazione:



dove S_0 è lo stato iniziale del sistema, S_1 è il momento in cui viene eseguito un checkpoint, S_2 è il momento in cui avviene un crash di sistema.

Sappiamo che T_1 termina prima che venga effettuato il checkpoint, dunque non subirà modifiche nel momento in cui il sistema verrà ripristinato. Per quanto riguarda tutte le altre transazioni, queste verranno **annullate** tutte, dal momento che sono tutte attive a partire dal checkpoint, tuttavia T_2 e T_3 avevano completato il commit prima del crash, dunque le loro modifiche devono essere **ripristinate**.

Di seguito mostriamo un il flusso operativo per il ripristino dopo un crash in un sistema che utilizza un **checkpoint** di tipo **buffer consistent**. Nel momento in cui si inizia con il ripristino si inizia a scorrere tutto il log in ordine inverso, partendo dall'ultimo record scritto prima del crash.

Immaginiamo di avere due liste L_r, L_u che conterranno rispettivamente le transazioni che necessitano di un **redo** e quelle che necessitano di un **undo**. A questo punto controlliamo il tipo di ogni record:

- se il record è di tipo **(T_i, "commit")**, allora la transazione T viene aggiunta alla lista L_r , ossia, sarà al termine del rollback necessario effettuare un **redo** delle sue modifiche;
- se il record è di tipo **(T_i, "update")** allora l'operazione viene **annullata** e la transazione è aggiunta a L_u , indicando che sarà effettuare un **undo** delle sue modifiche;
- se il record è di tipo **("begin", T_i)** significa che non dobbiamo più annullare operazioni per una transazione e possiamo rimuovere T_i da L_u ;
- se il record è di tipo **("checkpoint", L)**, dove L è la lista delle transazioni attive al momento del checkpoint, allora possiamo procedere come segue:
 - per ogni transazione $T_i \in L$, se $T_i \notin L_r$ allora aggiungere T_i ad L_u
 - a questo punto il rollback continua come sopra fino a quando L_u non è vuota;
- una volta terminato il rollback si può procedere con il **rollforward**: per ogni record a partire dall'ultimo checkpoint fino al crash, si procede con il redo dei record, se il record è di tipo commit allora la transazione T_i è rimossa da L_r . Nel momento in cui L_r è vuota, il ripristino è completo.

Log					
LSN	Record	LSN	Record	LSN	Record
1	(begin, 1)	6	(begin, 3)	12	(begin, 5)
2	(W, 1, A, 50, 20)	7	(W, 3, A, 20, 30)	13	(W, 5, E, 50, 30)
3	(begin, 2)	8	(CKP, {2, 3})	14	(commit, 2)
4	(W, 2, B, 10, 20)	9	(W, 2, C, 5, 10)	15	(W, 3, B, 20, 30)
5	(commit, 1)	10	(begin, 4)	16	(commit, 4)
		11	(W, 4, D, 5, 30)		

rollback					rollforward			
LSN	Operation	L_r	L_u	action	LSN	Operation	L_r	action
16	(commit, 4)	{4}	{}	no action	9	(W, 2, C, 5, 10)	{4, 2}	redo: C = 10
15	(W, 3, B, 20, 30)	{4}	{}	undo: B = 20	10	(begin, 4)	{4, 2}	no action
14	(commit, 2)	{4, 2}	{}	no action	11	(W, 4, D, 5, 30)	{4, 2}	redo: D = 30
13	(W, 5, E, 50, 30)	{4, 2}	{}	undo: E = 50	12	(begin, 5)	{4, 2}	no action
12	(begin, 5)	{4, 2}	{}	no action	13	(W, 5, E, 50, 30)	{4, 2}	no action
11	(W, 4, D, 5, 30)	{4, 2}	{}	no action	14	(commit, 2)	{4}	no action
10	(begin, 4)	{4, 2}	{}	no action	15	(W, 3, B, 20, 30)	{4}	no action
9	(W, 2, C, 5, 10)	{4, 2}	{}	no action	16	(commit, 4)	{}	L_r is empty, end of procedure
8	(CKP, {2, 3})	{4, 2}	{3}	no action				
7	(W, 3, A, 20, 30)	{4, 2}	{3}	undo: A = 20				
6	(begin, 3)	{4, 2}	{}	L_u is empty, end of procedure				

Indice delle figure

Figura 1.1	Interazioni del DBMS	7
Figura 1.2	Diagramma Entità-Relazione di una libreria	11
Figura 1.3	Due diversi piani di query per la stessa richiesta in algebra relazionale.	13
Figura 2.1	Esempio di applicazione dell'algoritmo MapReduce per contare le occorrenze di ogni parola all'interno di un input	17
Figura 2.2	Applicazione della funzionalità di combine al metodo MapReduce	17
Figura 2.3	Esempio di de-normalizzazione: a sinistra una struttura normalizzata basata su due entità distinte, a destra una rappresentazione de-normalizzata della stessa relazione.	20
Figura 2.4	Esempio di Salvataggio atomico	20
Figura 2.5	Rappresentazione grafica della pipeline di aggregazione specificata nel codice sopra	28
Figura 2.6	Esempio di utilizzo dell'operatore <code>\$lookup</code> per effettuare un left-outer join ..	34
Figura 3.1	Organizzazione interna delle componenti di un R-DBMS	36
Figura 3.2	Esempio di Run-Length Encoding	40
Figura 3.3	Esempio di Bit-vector encoding	40
Figura 3.4	Esempio di dictionary-encoding	41
Figura 3.5	Esempio di dictionary-encoding per sequenze	41
Figura 3.6	Esempio di Frame-of-Reference encoding con valori oltre l'offset massimo . .	42
Figura 3.7	Esempio di encoding differenziale con valori oltre l'offset massimo	42
Figura 4.1	Terminologia degli Extensible Record Stores	45
Figura 4.2	Flusso delle operazioni di scrittura in un Extensible Record Store	47
Figura 4.3	Flusso delle operazioni di lettura in un Extensible Record Store	48
Figura 4.4	Formato di un file in un Extensible Record Store	49
Figura 4.5	Write-Ahead Logging in un Extensible Record Store	49
Figura 4.6	Compattazione dei file in un Extensible Record Store	51
Figura 4.7	Bloom Filter all'interno dei file in un Extensible Record Store	51
Figura 4.8	Due esempi di utilizzo di un Bloom Filter durante la lettura in un Extensible Record Store, <code>query1</code> mostra il caso in cui la chiave cercata è presente, <code>query2</code> mostra il caso in cui la chiave cercata non è presente	53
Figura 5.1	Esempio di grafo diretto (a sinistra) e indiretto (a destra).	54
Figura 5.2	Esempio di multigrafo con più archi tra alcuni nodi. e_3, e_4 sono archi multipli tra i nodi v_2 e v_3	54
Figura 5.3	Rappresentazione delle relazione 'conoscenti di conoscenti' tramite un grafo .	55
Figura 5.4	Esempi di basi di dati a grafo in un social network (a sinistra) e in un sistema geospaziale (a destra).	57
Figura 5.5	Esempio di Property Graph che rappresenta una rete sociale.	57
Figura 5.6	Esempio di Property Graph con tipi e label che rappresenta una rete sociale. .	58
Figura 5.7	Esempio di multi-edge non ammesso dal momento che ha la stessa label di un altro arco già presente per collegare gli stessi nodi.	58
Figura 5.8	Esempio di multigrafo non diretto con cicli (a sinistra) e la sua matrice di adiacenza (a destra).	60

Figura 5.9	Esempio di multigrafo diretto con cicli (a sinistra) e la sua matrice di incidenza (a destra).	61
Figura 5.10	Esempio di multigrafo diretto con cicli (a sinistra) e la sua matrice di incidenza (a destra).	61
Figura 5.11	Esempio di rappresentazione di un grafo indiretto (a sinistra) e diretto (a destra) tramite liste di adiacenza.	62
Figura 5.12	Esempio di rappresentazione di un grafo indiretto (a sinistra) e diretto (a destra) tramite liste di incidenza.	63
Figura 5.13	Esempio di pattern matching in Cypher per trovare nodi e relazioni specifiche all'interno di un grafo.	67
Figura 5.14	Grafo che rappresenta una rete sociale con relazioni di amicizia e ristoranti preferiti.	68
Figura 5.15	Esempio di property graph model che rappresenta persone e film con relazioni	73
Figura 6.1	Illustrazione di un guasto su un nodo singolo in un sistema distribuito.	76
Figura 6.2	Illustrazione di un guasto di rete in un sistema distribuito.	76
Figura 6.3	Illustrazione di un partizionamento di rete in un sistema distribuito.	77
Figura 6.4	Esempio di Hash Tree con 8 nodi foglia data una lista di messaggi A,B,C,D,E,F,G,H.	79
Figura 6.5	Esempio di Consistent Hashing con 3 nodi e 6 frammenti di dati.	81
Figura 6.6	Esempio di Consistent Hashing con rimozione di un nodo.	81
Figura 6.7	Esempio di Consistent Hashing con aggiunta di un nodo.	82
Figura 6.8	Illustrazione dell'utilizzo di nodi virtuali nel Consistent Hashing.	82
Figura 6.9	Illustrazione di un guasto di singolo server con replication factor = 2.	85
Figura 6.10	Illustrazione di un guasto di entrambi i server con replication factor = 2.	85
Figura 6.11	Illustrazione del Two-Phase Commit: a sinistra un commit riuscito, a destra un commit abortito.	87
Figura 7.1	Rappresentazione schematica di un disco magnetico	91
Figura 7.2	Rappresentazione schematica del buffer pool	93
Figura 7.3	Rappresentazione schematica della tabella delle pagine residenti	94
Figura 7.4	a) Rappresentazione di utilizzo di riferimenti diretti per i record all'interno di una pagina. b) Rappresentazione di utilizzo di uno slot array per i record all'interno di una pagina.	97
Figura 7.5	Confronto tra modello seriale e sequenziale	102
Figura 7.6	Rappresentazione schematica dell'algoritmo di external merge-sort con dimensione del buffer di 3 pagine, e pagine di dimensione 2	104
Figura 7.7	Benchmark di external merge-sort in funzione del numero di pagine per due diverse dimensioni del buffer: a sinistra B=3, a destra B=257	105
Figura 8.1	Workflow di un'organizzazione hash-based	106
Figura 8.2	Gestione dell'overflow con chained overflow	108
Figura 8.3	Costo medio delle operazioni di ricerca in funzione del loading factor	109
Figura 8.4	Esempio di gestione di overflow in una situazione di virtual hashing. Nota Bene: il bit-vector viene mostrato ma è da considerare presente.	111
Figura 8.5	Posizione di partenza per linear hashing	112
Figura 8.6	Situazione dopo due split in linear hashing	113

Figura 8.7	Situazione dopo l'inserimento di 3820 in linear hashing	113
Figura 8.8	Esempio di B+ Tree di ordine 4	114
Figura 8.9	Esempio di indice clustered (sinistra) e unclustered (destra) data una tabella .	118
Figura 8.10	Ricerca di un intervallo di valori in un indice unclustered e clustered	120
Figura 8.11	Esempio di tabella con attributo non chiave 'Quantity'	121
Figura 8.12	Esempio di indice a lista invertita per l'attributo 'Quantity'	121
Figura 8.13	Grafico della funzione di Cardenas	123
Figura 8.14	Esempio di indice bitmap per l'attributo 'Quantity'	124
Figura 9.1	Esempio di linearizzazione dello spazio bidimensionale	127
Figura 9.2	Esempio di partizionamento dello spazio bidimensionale	128
Figura 9.3	Esempio di albero di ricerca generalizzato	128
Figura 9.4	Esempio di B+ Tree con ordinamento lessicografico	129
Figura 9.5	Curva di riempimento dello spazio con ordinamento lessicografico	130
Figura 9.6	Curva di riempimento dello spazio con ordinamento diagonale	130
Figura 9.7	Curva di riempimento dello spazio con ordinamento Morton (Z-order)	131
Figura 9.8	Varianti della curva di Morton a diverse risoluzioni	131
Figura 9.9	Rappresentazione binaria dei punti nello spazio secondo la curva di Morton .	132
Figura 9.10	Curva di riempimento dello spazio con ordinamento Peano con diversi livelli di ricorsività	132
Figura 9.11	Curva di riempimento dello spazio con ordinamento Hilbert	133
Figura 9.12	Esempio di Point Region Quadtree dato un dataset	133
Figura 9.13	Esempio di costruzione di un point region octree	134
Figura 9.14	Esempio di KDB-Tree dato un dataset	135
Figura 9.15	Esempio di G-Tree dato un dataset	136
Figura 9.16	B+ Tree associato agli encoding delle partizioni identificate	136
Figura 9.17	Esempio di G-Tree dopo l'inserimento di due punti P1 e P2	138
Figura 9.18	Esempio di R*-Tree dato un dataset	140
Figura 9.19	Esempio di inserimento di una nuova regione in un R*-Tree	142
Figura 10.1	Albero logico della query in Algebra Relazionale	148
Figura 10.2	Piano di accesso fisico della query dopo ottimizzazione	148
Figura 10.3	Fase di probing delle partizioni nell'hash join	159
Figura 10.4	Albero logico della query di esempio	161
Figura 10.5	Albero logico ottimizzato della query di esempio	161
Figura 11.1	Esempio di crash del sistema durante l'esecuzione di alcune transazioni	165
Figura 11.2	Ciclo di vita di una transazione	166
Figura 11.3	Esempio di log per una transazione	168
Figura 11.4	Esempio di shadow paging: (a) prima dell'aggiornamento di una pagina; (b) dopo l'aggiornamento e commit di una pagina	172
Figura 11.5		173